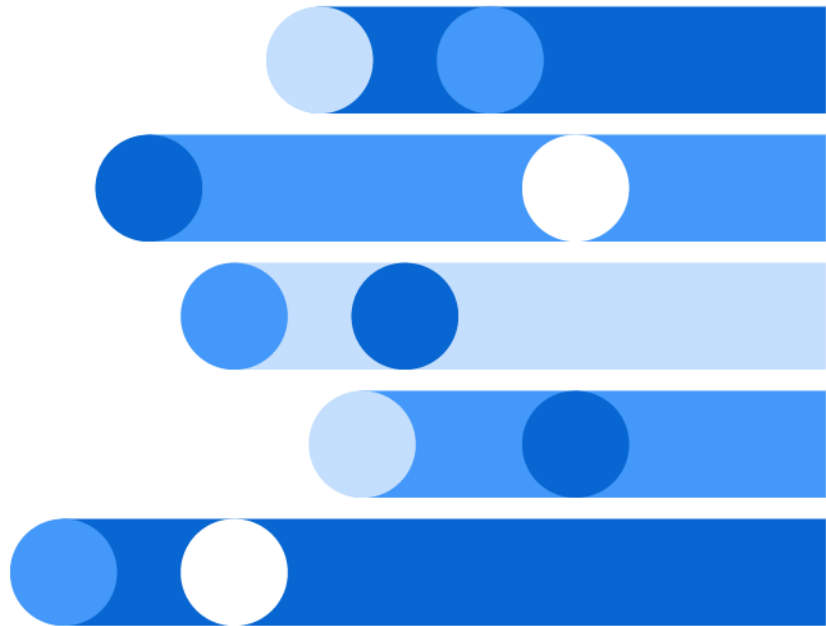




SAS[®] Intelligent Decisioning: User's Guide

2026.06*



* This document might apply to additional versions of the software. Open this document in [SAS Help Center](#) and click on the version in the banner to see all available versions.



The correct bibliographic citation for this manual is as follows: SAS Institute Inc. 2026. *SAS® Intelligent Decisioning: User's Guide*. Cary, NC: SAS Institute Inc.

SAS® Intelligent Decisioning: User's Guide

Copyright © 2026, SAS Institute Inc., Cary, NC, USA

All Rights Reserved. Produced in the United States of America.

For a hard copy book: No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

For a web download or e-book: Your use of this publication shall be governed by the terms established by the vendor at the time you acquire this publication.

The scanning, uploading, and distribution of this book via the Internet or any other means without the permission of the publisher is illegal and punishable by law. Please purchase only authorized electronic editions and do not participate in or encourage electronic piracy of copyrighted materials. Your support of others' rights is appreciated.

U.S. Government License Rights; Restricted Rights: The Software and its documentation is commercial computer software developed at private expense and is provided with RESTRICTED RIGHTS to the United States Government. Use, duplication, or disclosure of the Software by the United States Government is subject to the license terms of this Agreement pursuant to, as applicable, FAR 12.212, DFAR 227.7202-1(a), DFAR 227.7202-3(a), and DFAR 227.7202-4, and, to the extent required under U.S. federal law, the minimum restricted rights as set out in FAR 52.227-19 (DEC 2007). If FAR 52.227-19 is applicable, this provision serves as notice under clause (c) thereof and no other notice is required to be affixed to the Software or documentation. The Government's rights in Software and documentation shall be only those set forth in this Agreement.

SAS Institute Inc., SAS Campus Drive, Cary, NC 27513-2414

June 2026

SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

v_059-P1:edmug

Contents

Chapter 1 / Introduction to SAS Intelligent Decisioning	1
Enterprise Decision Management Systems	2
About Business Rules	2
SAS Intelligent Decisioning Features	3
Support for Models in Decisions	5
Sign in to SAS Intelligent Decisioning	5
SAS Intelligent Decisioning Settings	6
SAS Intelligent Decisioning Home Page	6
Using SAS Intelligent Decisioning as a Progressive Web App	7
Sorting, Searching, and Filtering in Category Views	9
Importing and Loading Data	12
Importing and Exporting Objects Using JSON Files	13
Manage Folders and Folder Content	18
View the Properties of Any Object	19
Manage Comments for Any Object	19
Chapter 2 / Using SAS Viya Copilot for SAS Intelligent Decisioning	21
About SAS Viya Copilot for SAS Intelligent Decisioning	21
How to Use SAS Viya Copilot for SAS Intelligent Decisioning	22
Third-Party Models for SAS Viya Copilot	23
Examples	25
Chapter 3 / Working with Rule Sets	27
About Rules and Rule Set Types	29
Using Filtering Rule Sets	30
About Condition and Action Expressions	31
Create a New Rule Set	32
Managing the Variables in a Rule Set	33
Defining New Rules in a Rule Set	42
Defining Expressions	56
Delete Expressions	64
Managing Rules in a Rule Set	64
Copy a Rule Set URL	66
Compare Rule Set Content	66
Compare Rule Set Code	68
Generate PDF Documentation for a Rule Set	69
Undoing and Redoing Rule Set Changes	70
Managing Rule Sets	70
Managing Versions of Rule Sets	72
Determine Which Objects Use a Rule Set	76
Checking Out and Committing Rule Set Versions	77
Publishing a Rule Set	83
Testing Rule Sets	89
Manage Comments for a Rule Set Test	104

Executing Published Content	104
Importing and Exporting Rule Sets	108
Chapter 4 / Working with Treatments and Treatment Groups	111
About Treatments and Decisions	112
Example: A Decision That Includes a Treatment Group	113
About Attributes, Eligibility Rules, and Effective Dates	116
Arbitrating Treatments	118
About Channels	118
Define a Treatment	119
Define a Treatment Group	120
Activating a Treatment Group	122
Managing Attributes and Aliases	124
Delete Treatments from a Treatment Group	126
Manage Eligibility Rules for a Treatment	127
Copy a Treatment or Treatment Group URL	127
Compare Treatment or Treatment Group Content	128
Compare Treatment Group Code	130
Generate PDF Documentation for a Treatment Group	131
Managing Treatments and Treatment Groups	132
Managing Versions of Treatments and Treatment Groups	134
Determine Which Objects Use a Treatment or Treatment Group	137
Checking Out and Committing Treatment or Treatment Group Versions	137
Chapter 5 / Working with Lookup Tables and Functions	143
About Lookup Tables and Functions	144
About Labels for Lookup Keys and Values	145
Predefined Lookup Tables	146
Create a New Lookup Table	146
Importing and Exporting Lookup Tables	147
Add Lookup Table Entries	151
Edit Lookup Table Entries	151
Delete Lookup Table Entries	152
Edit Lookup Key or Value Labels	152
Copy a Lookup Table URL	152
Compare Lookup Table Content	153
Generate PDF Documentation for a Lookup Table	154
Managing Lookup Tables	155
Managing Versions of Lookup Tables	157
Determine Which Objects Use a Lookup Table	160
Checking Out and Committing Lookup Table Versions	161
Activating a Lookup Table	166
Chapter 6 / Managing Global Variables	173
Using Global Variables	173
Create a New Global Variable	174
Managing Global Variables	175
Restore or Permanently Delete Hidden Global Variables	176
Managing Versions of Global Variables	177
Determine Which Objects Use a Global Variable	179
Activating a Global Variable	179

Chapter 7 / Using Custom Code Files	183
Introduction to Custom Code Files	184
DS2 Code Files	185
Python Code Files	189
Data Query Files	192
Custom Context Files	197
Create a New Code File	200
Managing the Variables in a DS2 or Python Code File	202
Validate SQL, DS2, or Context Files	210
Copy a Code File URL	211
Compare Generated Code for Code Files	211
Managing Code Files	212
Managing Versions of Code Files	213
Determine Which Objects Use a Code File	217
Checking Out and Committing Code File Versions	218
Testing Code Files	222
Manage Comments for a Code File Test	232
Chapter 8 / Using Custom Functions	233
About Custom Functions	234
Using Custom Functions in a Rule Set or Code File	234
Create a Custom Function	235
Using the Custom Function Editor	236
Rules for Writing Custom Function Files	237
Managing the Variables in a Custom Function File	238
Validate a Custom Function File	244
Testing a Custom Function	244
Copy a Custom Function URL	245
Compare Custom Functions	245
Managing Custom Function Categories	245
Managing Custom Functions	247
Restore or Permanently Delete Hidden Functions and Categories	248
Chapter 9 / Using Value Lists	251
About Value Lists	252
Value Lists and Local Value Lists	253
Create a New Value List	254
Manually Add Entries to a Value List	254
Specifying Value List Entries	255
Edit Entries in a Value List	257
Delete Entries in a Value List	257
Importing and Exporting Value Lists	258
Copy a Value List URL	261
Compare Value Lists	261
Generate PDF Documentation for a Global Value List	263
Managing Value Lists	263
Managing Versions of Value Lists	265
Determine Which Objects Use a Value List	268
Checking Out and Committing Value List Versions	269
Enabling Entries in a Value List	272

Chapter 10 / Managing Advanced Lists	273
About Advanced Lists	274
Create a List	279
Managing Lists	281
Managing List Definition	286
Managing List Contents	287
Managing List Revisions	294
Format of a CSV Data File	296
Chapter 11 / Working with Segmentation Trees	299
About Segmentation Trees	301
About Outcomes	301
Branch Element Types in a Segmentation Tree	302
Example: A Segmentation Tree	303
Create a Segmentation Tree	305
Controlling the Tree Display	306
Managing the Variables in a Segmentation Tree	307
Using Local Value Lists	316
Creating and Managing Objects and Branches in a Tree	320
Creating and Managing Outcomes	334
Importing and Managing Outcome Code Files	340
Search for Objects in a Segmentation Tree Diagram	344
Compare Segmentation Tree Code	345
Managing Segmentation Trees	345
Managing Versions of Segmentation Trees	348
Determine Which Objects Use a Segmentation Tree	351
Checking Out and Committing Segmentation Tree Versions	352
Testing Segmentation Trees	356
Manage Comments for a Segmentation Tree Test	365
Chapter 12 / Working with REST API Definitions	367
About REST API Definitions	368
Create a REST API Definition	368
Define the REST API Request	370
Define Response Headers	372
API Authorization Types	372
Managing the Variables in a REST API Definition	373
Managing REST API Definitions	378
Managing Versions of REST API Definitions	380
Determine Which Objects Use a REST API Definition	383
Checking Out and Committing REST API Definition Versions	384
Chapter 13 / Working with Decisions	389
What can I include in a decision?	391
Create a Decision	392
Views for Editing a Decision	394
Searching for Objects in a Decision Diagram	395
Managing the Variables in a Decision	395
Adding Objects to a Decision	405
Display and Export the Included Object Report for a Decision	434
Specifying a Subject ID Variable for A Decision	434
Specifying a Python Version for a Decision	435

Mapping Variables within a Decision	436
Editing the Objects in a Decision	441
Update Decisions to Use New Object Versions	443
Validate a Decision in the Decision Editor	444
Copy a Decision URL	445
Compare Decision Content	445
Compare Decision Code	447
Generate PDF Documentation for a Decision	447
Generate a Report for a Published Decision	448
Managing Decisions	449
Managing Versions of Decisions	452
Determine Which Objects Use a Decision	461
Checking Out and Committing Decision Versions	462
Publishing a Decision	468
Testing Decisions	476
Manage Comments for Decision Nodes and Tests	501
Executing Published Content	501
View All Published Decisions	505
Chapter 14 / Using Custom Workflows with SAS Intelligent Decisioning	507
About Custom Workflows and SAS Intelligent Decisioning	507
Defining Custom Workflows	508
Start a Workflow Instance for a Decision	515
Working with Tasks	516
View the Status Change History	517
Using the Sample Workflow	517
Chapter 15 / Managing and Using Tags	521
About Tags	521
Configuring and Administering Tags	522
Managing the List of Tags	522
Managing Tags That Are Associated with Object Versions	523
Determine Which Objects Are Associated with a Tag	525
Compare the Tags That Are Associated with Objects	525
Chapter 16 / Using the Code Editor and Expression Editor	527
Using the Code Editor	527
Code Editor Settings	528
Using the Expression Editor	535
Appendix 1 / Querying the Subject Contact Database	539
About the Subject Contact Database	539
How to Query the Subject Contacts Database	540
Using Views Defined by SAS Intelligent Decisioning	541
Columns in the dcm_trt_fixed_attr_vw and dcm_trt_dynamic_attr_vw Views	542
Columns in the dcm_treatments_contacts_vw View	545
Appendix 2 / Import File Formats	553
Introduction	553
CSV Format of the Variable Import File	553
JSON Format of the Variable Import File	556

Introduction to SAS Intelligent Decisioning

<i>Enterprise Decision Management Systems</i>	2
<i>About Business Rules</i>	2
<i>SAS Intelligent Decisioning Features</i>	3
<i>Support for Models in Decisions</i>	5
<i>Sign in to SAS Intelligent Decisioning</i>	5
<i>SAS Intelligent Decisioning Settings</i>	6
<i>SAS Intelligent Decisioning Home Page</i>	6
<i>Using SAS Intelligent Decisioning as a Progressive Web App</i>	7
Benefits of Using a Progressive Web App	7
Installing SAS Intelligent Decisioning as a Progressive Web App	8
<i>Sorting, Searching, and Filtering in Category Views</i>	9
Sort Object Lists	9
Search Object Lists	9
Filter Object Lists	9
Create, Apply, and Edit Saved Filters	10
<i>Importing and Loading Data</i>	12
<i>Importing and Exporting Objects Using JSON Files</i>	13
Ways to Export and Import Objects Using JSON Files	13
Export Objects to a JSON File	14
Export All Custom Functions in a Category	15
Import Objects from a JSON File	15
Activating Objects On Import	16
<i>Manage Folders and Folder Content</i>	18
<i>View the Properties of Any Object</i>	19
<i>Manage Comments for Any Object</i>	19

Enterprise Decision Management Systems

Enterprise decision management systems can transform how businesses make decisions. They enable businesses to use the information they already have to make better decisions—decisions that are based on predictive analytics rather than on past history. Decision management systems automate the process of making decisions, particularly day-to-day operational decisions. They improve the speed, efficiency, and accuracy of routine business processes, in part by reducing the need for human intervention. Automating decisions with SAS Intelligent Decisioning provides a streamlined mechanism for controlling and monitoring the rules and processes used by your organization. By automating decisions, organizations in every industry can improve interactions with customers, partners, suppliers, and employees. In addition, organizations that are highly regulated, such as financial services, health care, and insurance, can more easily achieve compliance as a result of repeatable, traceable decisions. Explicitly defining a decision makes your organization's decision-making process transparent, and enables you to monitor the process for accuracy.

SAS Intelligent Decisioning helps organizations leverage data, author and manage business rules, and integrate analytical models with other business logic. It enables them to create and optimize efficient business decisions within a single interface that gives users a consistent experience.

About Business Rules

Business rules capture the logic of business decisions and are a core component of decision management systems. Business rules enable you to codify the decision-making process used by your organization. Business rules make the decision-making process transparent and adaptable, enabling organizations to respond quickly to new information about customers and markets. They enable organizations to identify and deal with fraud, avoid unnecessary risk, and find opportunities hidden in customer data.

SAS Intelligent Decisioning Features

You can use SAS Intelligent Decisioning to create a database of business rules, combine those rules together into decisions, and publish the decisions for use by other applications. SAS Intelligent Decisioning provides the following capabilities:

business rule authoring

A business rule specifies conditions to be evaluated and action to be taken if those conditions are satisfied. For example, you can create a rule that determines whether a particular customer has a mortgage. That same rule can then add the outstanding balance of the mortgage to a running total of the customer's debt. With SAS Intelligent Decisioning, you define the conditions and actions for each rule.

rule set management and publishing

A rule set is a logical collection of rules. A single rule set can have many rules, but it generally corresponds to a single step in a decision. For example, you can have a rule set that determines a customer's asset balance and another rule set that determines a customer's debt level. You can use SAS Intelligent Decisioning to easily create new rule sets, reorder the rules in a rule set, add new rules to existing rule sets, and so on. When a rule set is published, the versioning features of SAS Intelligent Decisioning create a static version of the rule set. This static version helps you enforce integrity and governance over the rules that are put into production.

treatment authoring and management

A treatment is an offer that can be sent to a customer as part of an inbound marketing campaign. For example, when a customer visits your company's website, a customer service application can capture information about the customer. The application can invoke a decision that includes a set of treatments and pass the customer's information to the decision. The decision determines which of the treatments (offers) are suitable to present to that customer at that time. With SAS Intelligent Decisioning, you can author treatments, combine them into treatment groups, and add the groups to decisions.

lookup table authoring and management

Lookup tables are tables of key-value pairs. You can use lookup tables for tasks such as retrieving a part name based on a part number or retrieving a location name based on a ZIP code. SAS Intelligent Decisioning provides two predefined lookup tables for use with treatments: one for treatment channels and one for subject levels.

code file authoring and management

You can define custom code files to do things that are not possible in rules, models, or treatments. For example, you can define a code file that makes HTTP calls to REST APIs, interacts with a database, manipulates files in the file system, or performs custom data transformations. In SAS Intelligent Decisioning, you can easily create, edit, and manage custom data query files,

DS2 files, and Python files. You can create and edit data query files by using the SQL editor or by launching SAS Data and AI Studio.

global variable authoring and management

You can use global variables to define common variables for use in multiple rule sets, models, and decisions. For example, you might want to define global variables for benchmark interest rates, current exchange rates, or inventory levels for specific products. By using global variables, you can set these values in one place and refer to the variables in multiple objects.

custom function authoring and management

You can use custom functions to perform actions that are not available with the standard functions that SAS provides. Custom functions also enable you to encapsulate and reuse business logic. You can use custom function in rule sets and in DS2 code files.

decision authoring and publishing

You can combine rule sets, analytical models, treatment groups, custom code files, record contacts nodes, and branches (conditional logic) into decisions. You can also add a decision to another decision. You can investigate various scenarios, test and refine the decision logic, and then publish the decisions for use in batch applications and online transactions. After a decision has been published, it is available for use by other applications.

segmentation tree authoring and management

A segmentation tree is a hierarchical structure that consists of a root node, branches, and final leaf nodes called outcomes. Outcomes define actions to be taken when all of the branch criteria leading to the outcome are satisfied. Segmentation trees enable you to develop logic that uses simple conditions and to nest those conditions several levels deep. For a large, complicated decision that uses simple conditions, a segmentation tree is an effective way to design the decision and to manage the associated outcomes.

value list authoring and management

A value list is the list of acceptable values for a variable. You use value lists when you define variables and matrices in segmentation trees. When you add a variable to a segmentation tree, SAS Intelligent Decisioning creates new branches in the tree for each value in the variable's value list. When you define a matrix, SAS Intelligent Decisioning uses the values in the value lists associated with each matrix variable to determine the cells in the matrix. When you add the matrix to a segmentation tree, SAS Intelligent Decisioning uses the results of the matrix to determine what branches to add to the tree.

advanced list authoring and management

An advanced list is a tabular data set that can have multiple keys and values and can be shared among SAS applications and workflows. In a rule set, you can define rules that retrieve values from advanced lists. The reference can specify multiple key columns, and it can retrieve multiple column values.

Support for Models in Decisions

Support for a model depends on the tool that you use to create the model and the model's score code type. For example, decisions that use certain model score code types cannot be published or can be published only to SAS Micro Analytic Service destinations. For more information, see [“High-Level Model Support Matrix for Primary Functions” in SAS Model Manager: User's Guide](#).

Note: Do not include multiple versions of the same analytic store model in the same decision if two or more versions of the model use the same ASTORE files.

For more information, see [“Publishing Decisions That Include Analytic Store Models” on page 472](#) and [“Where Can I Publish Decisions?” on page 470](#).

Sign in to SAS Intelligent Decisioning

Note: If you are already signed in to SAS Viya, you can access SAS Intelligent Decisioning by clicking  and selecting **Build Decisions**.

To sign in to SAS Intelligent Decisioning:

- 1 In the address bar of your web browser, enter the URL for SAS Intelligent Decisioning and press **Enter**. The Sign In page appears.

Note: Contact your system administrator if you need the URL for SAS Intelligent Decisioning. The default URL is `http://host_name/SASDecisionManager`.

- 2 Enter a user ID and password.
- 3 Click **Sign In**. If this is your first time signing in to SAS Intelligent Decisioning, the Welcome to SAS window appears.
- 4 Do one of the following:
 - Set up a profile and customize user-specific settings. For more information, see [“Settings” in SAS Viya Platform: General Usage Help for Web Applications](#) under General Usage in the Help Center and [“SAS Intelligent Decisioning Settings” on page 6](#).

- Click **Skip setup** if you do not want to set up a profile. You can edit your profile settings later by clicking the user icon in the upper right.

After you exit the profile setup page, the [SAS Intelligent Decisioning home page](#) appears.

SAS Intelligent Decisioning Settings

In addition to the [global settings](#) that are available with SAS Viya applications, SAS Intelligent Decisioning provides two additional settings. To access these settings, click the user icon in the top right corner of the application window, select **Settings** to open the Settings window, and click **SAS Intelligent Decisioning**.

- By default, if you add an object to a decision and that decision does not already have a variable of the same name and data type as that object's variable, then SAS Intelligent Decisioning displays the following message: Some objects in the decision define variables for which no corresponding decision variables have been created. You can turn on the **Create variables automatically in decisions** setting if you want SAS Intelligent Decisioning to automatically create decision variables. For more information, see ["Mapping Variables within a Decision" on page 436](#).
- In the Check Out Decision Contents window you can check out the objects that are used in a decision at the same time that you check out the decision. This window is disabled by default. To enable this window, select **Enable the Check Out Decision Contents window**. If this option enabled when you check out a decision, then SAS Intelligent Decisioning displays the list of objects in the decision that you can check out. The objects that you select are checked out together with the decision. For more information, see ["Checking Out and Committing Included Objects Together with A Decision" on page 463](#).

Note: In order for changes to settings to take effect, you must sign out and sign back in to SAS Intelligent Decisioning.

SAS Intelligent Decisioning Home Page

By default, the SAS Intelligent Decisioning Home page appears when you sign in to the application. To return to the home page at any time, click  in the left navigation bar.

The Home page displays up to 100 of the most recent objects that you have opened, the date on which each object was last modified, and the user ID of the

user who modified it. You can click  and  to switch between the list view and tile view of the recent objects. In the tile view, you can click  to display additional information for a specific object. To open an object, either click on the object name, or click  and select **Open**.

The **Tasks** section at the bottom of the home page lists the active tasks to which you or a group of which you are a member have been assigned. The  icon appears in the user  column for any task that you have claimed. Tasks that have been claimed by another user or that have been completed are removed from this list when the list is refreshed. Click  to refresh the list.

To claim a task in the task list, select the task and click . To complete a task, select the task and click **Complete**. To release a task, select the task and click .

Alternatively, you can claim, release, and complete a task in the Task window. Click the task name to open the Task window. You can also work with tasks for a specific decision by using the options in the [Tasks menu on page 516](#) that is available when you open a decision that is in a workflow. Click the decision name to open the decision.

You can toggle the display of category labels in the left navigation bar by clicking  and . To create a new object, click  and select the object type. You can sort the list of recent objects by name or by date modified, and you can filter the list by name.

The Stay Connected pane on the right contains links to new topics in SAS Support Communities pages. It also contains links to an overview video and to documentation for SAS Intelligent Decisioning.

Using SAS Intelligent Decisioning as a Progressive Web App

Benefits of Using a Progressive Web App

The benefits of using SAS Intelligent Decisioning as a Progressive Web App (PWA) include the following:

- Application persistence – By default, your session will never time out, so you can restart your work more quickly. Your administrator can enable a timeout for the PWA using a configuration property in SAS Environment Manager. For more information, see [“sas.htmlcommons” in SAS Viya Platform: Configuration Properties](#).
- Performance – When installed as a PWA, SAS Intelligent Decisioning typically starts faster than when accessed in the browser.

- Desktop experience – Installing SAS Intelligent Decisioning as a PWA confers all the benefits of a traditional installation. You can launch SAS Intelligent Decisioning from the Start menu or Taskbar, and you do not need to sort through countless tabs to find the correct instance of SAS Intelligent Decisioning.
- Renaming – You can rename each PWA instance to quickly access different environments, such as development, test, or production servers.

Installing SAS Intelligent Decisioning as a Progressive Web App

SAS Intelligent Decisioning must be deployed with TLS and HTTPS enabled.

To install SAS Intelligent Decisioning as a PWA:

- 1 Open a Chromium-based web browser, and sign in to SAS Intelligent Decisioning.
- 2 Do one of the following:
 - Click the appropriate icon at the end of the address bar, and then select **Install SAS**.
 - Open the web browser's **More** menu, and select **Install SAS**.

Note: The **Install SAS** option might be located under another menu option. For example, in the Microsoft Edge browser, it is located under **Apps**.

SAS Intelligent Decisioning is now installed as a desktop program named **SAS**. After one web application has been installed as a PWA, all other SAS applications are accessible in the PWA.

To uninstall the PWA for SAS Intelligent Decisioning:

- 1 Open the PWA for SAS Intelligent Decisioning.
- 2 In the PWA menu, select **Uninstall SAS**.

Sorting, Searching, and Filtering in Category Views

Sort Object Lists

To sort the object list based on the values of a column, right-click the column heading, and select **Sort** ⇒ **Sort (ascending)** or **Sort (descending)**. If the column is sorted in ascending order, ↑ appears beside the column heading. When the column is sorted in descending order, ↓ appears. To add another column to the sort, right-click the column heading, and select **Add to sort** ⇒ **Sort (ascending)** or **Sort (descending)**. To clear a column filter, right-click the column heading and select **Sort** ⇒ **Remove sort**.

Search Object Lists

To search the **Name** column, enter the string for which you want to search in the search field above the object list, and click 🔍. The search is case insensitive. To clear the search text, click ✕.

TIP See also “Searching for Objects in a Decision Diagram” on page 395.

Filter Object Lists

- 1 Click **Filter**, and select **Create filter**.

.....
Note: The **Create filter** option does not appear if you have applied a filter to the view. [Clear the applied filter](#).
.....

- 2 Enter or select the filter criteria, then click **Filter**.

TIP If you specify a folder location as part of the filter criteria, the filter searches the selected folder and recursively searches all of its subfolders.

The node count for a decision does not include any nodes that are in nested decisions.

SAS Intelligent Decisioning filters the object list and displays a token for each filter criteria. For example:

Name: credit × Created by: sasdemo × Date created: Jul 1, 2025 12:00:00 AM - Jul 4, 2025 12:00:00 AM ×

Create, Apply, and Edit Saved Filters

You can save a filter, and then reapply that filter to a category view later without having to redefine the filter criteria. You can save up to 10 filters.

Define and Save a Filter

To define and save a filter:

- 1 Click **Filter**, and select **Create filter**.

Note: The **Create filter** option does not appear if you have applied a filter to the view. [Clear the applied filter](#).

- 2 Enter or select the filter criteria, then click **Save filter**.

TIP If you specify a folder location as part of the filter criteria, the filter searches the selected folder and recursively searches all of its subfolders.

The node count for a decision does not include any nodes that are in nested decisions.

- 3 In the Save Filter window, enter a name for the filter. Filter names are limited to 100 characters and can contain only alphanumeric characters and underscores (_).
- 4 (Optional) Enter a description for the filter.
- 5 Verify that your filter criteria is correct, and click **Save and filter**.

SAS Intelligent Decisioning filters the object list and displays a token for each filter criteria. For example, the following list of tokens shows a saved filter named NEWOBJ:

NEWOBJ: Name: credit × Date created: Jul 1, 2025 12:00:00 AM - Jul 4, 2025 12:00:00 AM ×

Clear an Applied Filter

Filters remain in effect until you remove them. To remove a specific filter, click × for that filter. To remove all filters, click × at the end of the list of the filter tokens.

Apply a Saved Filter

To apply a saved filter to a category's object list, click ∨ in the **Filter** button, and select the filter that you want to use. If the filter specifies criteria that do not apply to the current object list, those criteria are ignored. For example, if you apply a filter that specifies a criterion for location to the Global Variables category view, that criterion is ignored.

Edit a Saved Filter

To edit a saved filter:

- 1 [Apply the saved filter.](#)
- 2 Click **Filter**, and select **Edit applied filter**.
- 3 In the Filter window, edit the filter criteria, and click **Save filter**.
- 4 (Optional) In the Edit Filter window, modify the filter name and description.
- 5 Verify that your filter criteria is correct, and click **Save and filter**.

Delete a Saved Filter

To delete a saved filter, click ∨ in the **Filter** button, and click × for the filter that you want to delete.

Importing and Loading Data

If the table that you need does not appear in the list of available tables in the Choose Data window, you can import data by using either the **Import data** option in SAS Intelligent Decisioning or SAS Data Hub. You can import data from local files, import data that is stored in SAS folders, and copy data in caslib directories.

From a category view in SAS Intelligent Decisioning, click  and select **Import data**. This option is also available when you are adding variables from a data table or selecting an input table for a [decision test](#), a segmentation tree test, or a rule set test. In the Choose Data window, click **Import data**.

Data imported by using the **Import data** option is available to all objects in SAS Intelligent Decisioning.

To open SAS Data Hub, click  and select **Access and Manage data**. If the table does not appear on the **Data Sources** tab in SAS Data Hub, click the **Import** tab.

SAS Intelligent Decisioning and SAS Data Hub have similar features for importing data. The fields and selections that are available in the SAS Intelligent Decisioning Import Data window are also available from SAS Data Hub. For descriptions of the fields and selections, see following topics:

- [“Importing Local Files” in SAS Data Hub: User’s Guide](#)
- [“Importing Data from SAS Folders” in SAS Data Hub: User’s Guide](#)
- [“Working with Data in CAS” in SAS Data Hub: User’s Guide](#)

The process of importing data loads it into memory, but data tables can also be unloaded from memory.

If the data has already been imported but is not currently loaded into memory, you can use SAS Data Hub to load it. In SAS Data Hub, click the **Data Sources** tab. If the table appears on the **Data Sources** tab, right-click on the table, and select **Load** to load it into memory. If the table does not subsequently appear on the **Available** tab, click .

Importing and Exporting Objects Using JSON Files

Ways to Export and Import Objects Using JSON Files

You can export and import SAS Intelligent Decisioning objects by using SAS Environment Manager, the SAS Viya command-line interface (sas-viya), or the **Export objects to JSON files** and **Import objects from JSON files** options in SAS Intelligent Decisioning. These export methods create a transfer package that is written in JSON.

Depending on how you export an object, the transfer package might include an object's dependencies. A dependency is any object that is included in or referenced by the exported object such as a rule set that is included in a decision or a lookup table that is referenced by a rule set. Dependencies also include any notes or version tags that are associated with the object being exported.

You can use the following methods to export and import SAS Intelligent Decisioning objects:

- **SAS Environment Manager.** You can use SAS Environment Manager to export individual objects that are stored in folders or to export the contents of an entire folder. If you enable the **Include dependencies** option, SAS Environment Manager includes the dependencies of the objects that you are exporting. SAS Environment Manager exports all versions of the selected objects. You can use SAS Environment Manager to export and import all SAS Intelligent Decisioning object types. For information about using SAS Environment Manager, see [“Content Page” in SAS Environment Manager: User's Guide](#) and [“Import Page” in SAS Environment Manager: User's Guide](#).
- **The transfer plug-in to the SAS Viya command-line interface (sas-viya CLI).** You can use the sas-viya CLI to export and import the contents of a folder, individual objects, and objects that are not stored in folders. To include dependencies, specify the `--include-dependencies` option. The CLI exports all versions of the selected objects. You can use the CLI to export and import all SAS Intelligent Decisioning object types. For information about the CLI, see [SAS Viya Platform: Content Migration from SAS Viya 4](#).
- **The **Export objects to JSON file** and **Import objects from JSON file** menu options in SAS Intelligent Decisioning.** You can use these options to export and import an individual object regardless of whether the object is stored in a folder. You can choose to export a single version of the object or to export all versions of the object. Dependencies are automatically included in the transfer package.

If you choose to export a single version of an object, with some exceptions, only the dependencies for the exported version are included in the transfer package. You can use these menu options to export and import all SAS Intelligent Decisioning object types except advanced lists.

Note: If an object uses multiple versions of a dependent object, you cannot export only a single version of the object. For example, if a decision includes multiple versions of rule set or a custom code file, then you cannot export only a single version of the decision.

IMPORTANT If you use the menu option to import only a single version of an object, and if the object already exists in the target environment, only the version in the transfer package is saved. All other previously existing versions of the object are deleted.

IMPORTANT Review your company's governance policies and auditing procedures before choosing to export and import only a single version of an asset. Maintain a development or other non-production environment that always includes all versions of an object and is available if you need to recover a previous version of an object.

There are two special cases for the dependencies that are included in the transfer package:

- Objects that use treatment groups, lookup tables, and global variables use the active version of these items. In the exported object, these dependent items are not referenced by version numbers. When a treatment group, lookup table, or global variable is included in a transfer package as a dependency, all versions of the treatment group, lookup table, or global variable are included, even if the exported object uses only a single version.
- Custom functions are not included as dependencies in a transfer package. If an exported object references a custom function file, the reference is included in the transfer package, but the code for the custom function is not included. You must manually export and import custom function files. It is recommended that you import the custom function file before importing the objects that use the custom function.

Export Objects to a JSON File

Note: When you export a REST API definition that specifies a credential domain, the credential domain is not included in the transfer package. You must recreate the credential domain in the target environment.

- 1 Open the object that you want to export.

- 2 On the **Versions** tab, click , and select **Export objects to JSON file**.

Note: Custom functions are not versioned. Go to [Step 4](#).

- 3 (Optional) In the Export Objects to JSON window, clear the **Export all versions of the object** check box, and select the specific version of that you want to export.
- 4 Enter a name for the export file if you do not want to use the default name, and click **Export**. SAS Intelligent Decisioning exports the selected version or versions and their dependent objects to the JSON file.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Export All Custom Functions in a Category

- 1 In the Custom Functions category view, click  and select **Manage categories**.
- 2 In the Manage Categories window, select the category that you want to export, click , and select **Export**.
- 3 In the Export Objects to JSON window, enter a name for the export file, and click **Export**. SAS Intelligent Decisioning exports all of the functions in the selected category.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Import Objects from a JSON File

When you import an existing object, the contents of the transfer package replace the existing object in the target environment. If the transfer package contains only a single version of an object, and if the target environment already contains multiple versions of the same object, all of the existing versions are overwritten by the single, imported version.

Note: If you are importing a global variable, and a global variable with the same name but with a different ID already exists, the existing variable is not replaced. If you are importing a global variable, and a global variable with the same name and the same ID already exists, the existing variable is replaced if you are importing

only the global variable. However, it is not replaced if you are importing the variable as a dependency of another object.

- 1 In a category view, click  and select **Import objects from JSON file**.
- 2 In the Import window, click , select the JSON file whose content you want to import, and click **OK**. SAS Intelligent Decisioning imports all of the objects that are defined in the JSON file.
- 3 (Optional) If you import a REST API definition that specifies a credential domain, define the credential domain in the target environment. For more information, see [“Define the Authentication Domain for External REST API Requests” in SAS Intelligent Decisioning: Administrator’s Guide](#).

If an error occurs, it is recommended that you use SAS Environment Manager to import the objects, and view the detailed error messages on the results page. For more information, see [“Import Page” in SAS Environment Manager: User’s Guide](#).

Activating Objects On Import

Activation Status and Options for Activation

When you export a global variable, lookup table, or treatment group to a JSON file, the status of the object in the source environment (whether it is activated) is included in the exported data. When you import the object, its status in the target environment is set to the status that is specified in the exported data. However, whether an imported object is automatically activated in the target environment is determined by the settings of the following configuration options:

- [sas.referencedata.activation.activateGlobalVariableOnImport](#)
- [sas.referencedata.activation.activateLookupOnImport](#)
- [sas.treatmentdefinitions.activation.activateTreatmentGroupOnImport](#)
- [sas.treatmentdefinitions.import.resetActiveVersionOfTreatmentGroupOnImport](#)

Note: If an object is activated in the source environment, but the activation option for that object type is turned off in the target environment, the you might see a message in the SAS Intelligent Decisioning interface that indicates that the imported object is activated even though it is not.

Increasing Activation Resources

Turning on an [activation option](#) increases the resources needed by the importing process. If the import process exhausts the resources needed, some of the imported objects might not be activated. If you are importing very large lookup tables, or if you are importing multiple objects from the same import package, it is recommended that you increase the activation timeout limits or the size of the JDBC connection pool. See the documentation for the following SAS Intelligent Decisioning configuration options:

- [sas.treatmentdefinitions.activation.timeout](#) specifies the timeout limit for treatment groups.
- [sas.referencedata.activation.timeout](#) specifies the timeout limit for global variables and lookup tables.

For information about changing the property settings for the JDBC connection pool, see [“Tuning the JDBC Connection Pool” in SAS Viya Platform Administration: Tuning](#).

Ensuring Which Version is Active

To display a list of objects that are used in a decision:

- 1 Open the decision, and click the **Versions** tab.
- 2 Click  next to the name of the decision to open the **Included Objects** window. The information in this window includes lookup tables and treatment groups that are used in the decision, the version of each object, and its location. If a lookup table or treatment group has been activated, the version column displays **Active**. If it is not activated, the window displays **Not Activated**.

Note: This window does not include global variables.

To ensure that objects are activated after import, you can manually activate the imported objects. For more information, see the following topics:

- [“Activating a Global Variable” on page 179](#)
- [“Activating a Lookup Table” on page 166](#)
- [“Activating a Treatment Group” on page 122](#)

Manage Folders and Folder Content

In the Manage Folders window, with the appropriate permissions, you can create new folders, delete, move, and rename existing folders or objects, and restore objects from the recycle bin. Objects that you delete in SAS Intelligent Decisioning are moved to the recycle bin. For information about permissions, see [“Managing Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Note: When you delete a specific version of an object, that version is permanently deleted. It is not moved to the recycle bin, and it cannot be recovered. When you delete an entire object, that object is moved to the recycle bin and can be restored.

To open the Manage Folders window, click , and select **Manage folders**. This menu option is available in the **Decisions** and **Value lists** category views and in any category view listed in the **DECISION ELEMENTS** section of the category navigation pane.

- To create a new folder, click , type the folder name, and press Enter.
- To rename an item, right-click the item, select **Rename**, type the new name, and press Enter.
- To move an item, right-click the item, select **Move to folder**, select the new folder, and press Enter.
- To delete an item, right-click the item, and select **Delete**. The deleted item is moved to the recycle bin.
- To restore an item from the recycle bin, right-click on the item in the recycle bin, and select **Restore**.
- To permanently delete an item, right-click on the item in the recycle bin, and select **Delete**. The item is deleted from the recycle bin and cannot be restored.
- To empty the recycle bin, right-click **Recycle Bin** and select **Empty recycle bin**.

The **Search** field supports both keyword searches and faceted searches. For a keyword search, enter at least three characters of the string for which you are searching. To display a list of all of the available search facets, enter a space into the empty **Search** field. For example, to use the AssetType facet to search for all decisions and rule sets, enter the following string in the **Search** field:

```
AssetType:(Decision Ruleset)
```

For more information about searching in the Manage Folders window, see [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#).

For more information about folders, see [SAS Viya Platform: Folders](#).

View the Properties of Any Object

In the category views, you can view the properties of any object. Select the check box for the object, and click  in the property pane. Click  again or click **>>** to close the properties pane.

Manage Comments for Any Object

In the category views, you can associate comments and attachments with any object.

To open the **Comments** pane for an object, select the check box for the object, and click  in the properties pane.

To add a new comment, enter the comment in the text box and click **Post**.

To add an attachment to a comment, click , select the file that you want to attach, and click **Post**. (The attachment icon appears after you enter text in text box.) You cannot attach executable files such as BAT and EXE files.

To reply to a comment, click **Reply**, enter your reply in the text box, and click **Post**.

To delete a comment, click  for that comment.

Note: You can also add comments to rule set tests, code file tests, decision tests, and to the Start and End nodes within a decision. Comments that you add to tests are not displayed with the comments in the category views. Comments that you add to the Start and End nodes in a decision are displayed in the Decisions category view. For more information, see [“Manage Comments for a Rule Set Test” on page 104](#), [“Manage Comments for a Segmentation Tree Test” on page 365](#), [“Manage Comments for a Code File Test” on page 232](#), and [“Manage Comments for Decision Nodes and Tests” on page 501](#).

Using SAS Viya Copilot for SAS Intelligent Decisioning

<i>About SAS Viya Copilot for SAS Intelligent Decisioning</i>	21
<i>How to Use SAS Viya Copilot for SAS Intelligent Decisioning</i>	22
<i>Third-Party Models for SAS Viya Copilot</i>	23
Model Information	23
<i>Examples:</i>	25
Using Follow-Up Questions	25
Making a Query More Specific	25

About SAS Viya Copilot for SAS Intelligent Decisioning

You can use SAS Viya Copilot for SAS Intelligent Decisioning to get answers to your questions while you are working in the application. The copilot uses SAS Intelligent Decisioning documentation and related documentation to find answers. The related documentation that SAS Viya Copilot for SAS Intelligent Decisioning searches includes the documentation for the DS2 language, and for the SAS Container Runtime, SAS Micro Analytic Service, SAS Model Manager, SAS Workflow Manager, SAS Data Explorer products.

For more information about SAS Viya Copilot, see ["Introduction" in SAS Viya Copilot: Foundation Overview](#).

IMPORTANT It is your responsibility to review any output generated with AI to ensure that the AI-generated content that you select is appropriate and accurate for its intended purpose.

How to Use SAS Viya Copilot for SAS Intelligent Decisioning

SAS Viya Copilot must be enabled in your environment. For more information, see [“Enable SAS Viya Copilot in SAS Environment Manager” in SAS Viya Copilot: Foundation Overview](#).

Note: By enabling GenAI for your organization, you acknowledge and agree that the terms found at [SAS Generative AI Terms](#) apply to your organization's use of SAS Intelligent Decisioning.

To use the copilot, click  in the application bar, enter a question (prompt) in the **SAS Viya Copilot** pane, and click ➤.

Getting the best AI responses can be an iterative process. Your first prompt might not return the best response. If the reply is not what you are looking for, try rewriting your question or asking follow-up questions. Also, because generative AI is non-deterministic, submitting the same question multiple times might generate different answers each time.

For more information entering prompts, see the following topics:

- [“Best Practices” on page 24](#)
- [“Best Practices for SAS Viya Copilot” in SAS Viya Copilot: Foundation Overview](#)

Below each response, the copilot displays a list of links to topics in the documentation that it used to generate the answer to your question.

TIP Click  to clear the current conversation and start a new one with the copilot.

Third-Party Models for SAS Viya Copilot

Model Information

Model Overview

For a list of the Microsoft Azure models that are used by SAS Viya Copilot, see "Models Used by SAS Viya Copilot" in [SAS Viya Copilot: Foundation Overview](#).

Input and Output Modalities

Input modalities and size limit for each modality as used by SAS:

- gpt-5.1: Text, 272,000 tokens
- gpt-5-nano: Text, 272,000 tokens
- gpt-4.1: Text, 1,047,576 tokens
- gpt-4.1-nano: Text, 1,047,576 tokens
- text-embedding-3-large: Text, 8,192 tokens

Output modalities and size limit for each modality as used by SAS:

- gpt-5.1: Text, 128,000 tokens
- gpt-5-nano: Text, 128,000 tokens
- gpt-4.1: Text, 32,768 tokens
- gpt-4.1-nano: Text, 32,768 tokens
- text-embedding-3-large: Embedded, Fixed-size vector (1536 dimensions)

Note: Because embedding models return a fixed-dimension vector rather than text tokens, there is no published output token size.

Prerequisites

- Your site must have a license for SAS Intelligent Decisioning 2026.01 or later.

- Your administrator must set up credentials to manage the copilot. For more information, see [“Set Up the Credential and Secrets for SAS Viya Copilot” in SAS Viya Platform: Licensing](#).

Using the Model

- Acceptable use policy: https://www.sas.com/en_us/legal/acceptable-use.html.
- Intended use: This feature is a tool to help query the documentation for SAS Intelligent Decisioning.

Type and Nature of AI Systems

The models support features in SAS software, but they cannot be more broadly incorporated into customer AI systems.

Best Practices

You interact with SAS Viya Copilot through the SAS Viya Copilot pane. For more information, see [“How to Use SAS Viya Copilot for SAS Intelligent Decisioning” on page 22](#).

When conversing with the assistant, you can follow these best practices for writing your prompts:

- Be as clear and specific as possible in the prompts.
- Be intentional with the tone of your prompts because the AI-generated suggestions often match the tone of the original prompt.
- Ensure that your prompts are safe and respectful.
- Enter your prompts in natural language English only.
- Avoid words or phrases in your prompts that are culturally specific, such as slang or idioms that are not easily understood by others.
- If you are not getting good results, try rephrasing the prompt.
- If you need more specific information, ask follow-up questions.

Known Issues or Limitations

- SAS does not guarantee that the AI responses are correct, or that the generated suggestions are optimal.
- LLMs are known to hallucinate.

- SAS Viya Copilot supports only English inputs and output.
- SAS Viya Copilot in SAS Intelligent Decisioning can provide answers only to questions related to SAS Intelligent Decisioning. Asking questions about unrelated SAS products or features might produce incorrect responses, or the copilot might state that it is unable to answer the question. See [“About SAS Viya Copilot for SAS Intelligent Decisioning” on page 21](#) for the list of documents that the copilot searches.

Examples

Example 1: Using Follow-Up Questions

You are looking for information that compares lookup tables and advanced lists. You enter the following prompt:

What is the difference between a lookup table and an advanced list?

The reply describes the purpose, structure, and other features of both lookup tables and advanced list. The response indicates that lookup tables need to be activated, but it does not indicate whether advanced lists need to be activated. You could follow up with a second question:

Do advanced lists have to be activated?

The response indicates that you do not activate advanced lists. Instead, you can assign a status of either developing or deployed to lists and list items.

Example 2: Making a Query More Specific

In your rule set, you need to test a string for a specific pattern of characters. You enter the following prompt:

How do i test whether the characters in a string follow a specific pattern?

The reply might include information about the PRX functions, PCRX packages, and LIKE expressions.

The packages cannot be used in a rule set. If you make your query more specific, the copilot returns more relevant information:

In a rule set, how do i test whether the characters in a string follow a specific pattern?

In this case, the copilot omits the information about the packages, and it might include information about the LIKE rule operator.

Working with Rule Sets

<i>About Rules and Rule Set Types</i>	29
<i>Using Filtering Rule Sets</i>	30
<i>About Condition and Action Expressions</i>	31
<i>Create a New Rule Set</i>	32
<i>Managing the Variables in a Rule Set</i>	33
About Variables	33
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object ...	36
Add Global Variables to a Rule Set	37
Create a Custom Variable Dynamically	38
Create Custom Variables on the Variables Tab	38
Duplicate a Variable	39
Importing and Exporting Variables	40
Delete Variables	41
Edit Variable Properties	41
Edit Metadata for Data Grid Variables	41
Determine Which Objects Use a Particular Variable	41
<i>Defining New Rules in a Rule Set</i>	42
Add a Standalone Assignment Statement	42
Add an IF-THEN or IF-THEN-ELSE Rule to an Assignment or Common Rule Set ..	43
Add a RETURN Statement	45
Adding a Common Rule Set Reference to an Assignment Rule Set	45
Adding an Advanced List Reference to an Assignment Rule Set	49
Add a New Rule in a Filtering Rule Set	51
Copy Rules From Another Rule Set	52
Controlling Which Conditions Are Evaluated	53
How Rules Are Evaluated and When Rule-Fired Records Are Generated	55
<i>Defining Expressions</i>	56
About Defining Expressions in Rules and Assignment Statements	56
Enter LOOKUP and LOOKUPVALUE Expressions	56
Entering Literal Data Values	57
Operators for Use in Expressions	59
Using the LIKE Operator	60
Using Functions in Expressions	61
Working with Missing Values	62
Examples: Condition Expressions	62
Examples: Action Expressions	63

See Also	64
Delete Expressions	64
Managing Rules in a Rule Set	64
Duplicate a Rule	64
Delete a Rule	64
Edit a Rule Name or Description	65
Reorder Rules	65
Copy a Rule Set URL	66
Compare Rule Set Content	66
Compare Rule Set Code	68
Generate PDF Documentation for a Rule Set	69
Undoing and Redoing Rule Set Changes	70
Managing Rule Sets	70
Duplicating Rule Sets	70
Delete Rule Sets	71
Rename Rule Sets	72
Move Rule Sets to a Different Folder	72
Managing Versions of Rule Sets	72
Set the Displayed Version	72
Create a New Version	72
Create a New Version Automatically When You Publish a Rule Set	74
Copy the Content of a Version	74
Delete a Version	75
Upgrade Decisions to Use a New Version of a Rule Set	76
Determine Which Objects Use a Rule Set	76
Checking Out and Committing Rule Set Versions	77
About Checking Out and Committing Versions	77
Checking Out a Rule Set from within a Decision	78
Checking Out and Committing a Common Rule Set from within an Assignment Rule Set	78
Checking Out and Committing a Lookup Table from within a Rule Set	79
Committing a Lookup Table Together with a Rule Set	80
Concurrently Checking Out and Committing Rule Set Versions	80
Check Out and Commit a Rule Set Version	81
Determine Whether an Object Is Checked Out	82
Determine Who Has a Version Checked Out	82
Opening the Original Object	82
Undoing a Check Out	83
Publishing a Rule Set	83
Introduction to Publishing	83
Where Can I Publish Rule Sets?	84
Publish Rule Sets	85
Testing Rule Sets	89
Ways to Test a Rule Set	89
Test a Rule Set	89
Test a Scenario	93
Run a Rule-Fired Analysis	97
Validate a Published Rule Set	100

Working with Test and Validation Test Output Data	102
Status Icons for Tests	102
Duplicate Tests	103
Manage Comments for a Rule Set Test	104
Executing Published Content	104
Executing Content Published to SAS Micro Analytic Service Destinations	104
Executing Content That Has Been Published to SAS Cloud	
Analytic Services Destinations	105
Content Executed by Published Rule Sets	107
Importing and Exporting Rule Sets	108
About Importing and Exporting Rule Sets	108
Import a Rule Set from a CSV File	109
Export a Rule Set to a CSV File	109

About Rules and Rule Set Types

A rule specifies conditions to be evaluated, and it can also specify actions to be taken if those conditions are satisfied. Rules are grouped together into rule sets. Rule sets are logical collections of rules that are grouped together because of interactions or dependencies between the rules or because they are processed together after they are published.

A rule set can be one of following types:

filtering rule sets

contain rules that correspond to the form:

```
IF condition_expressions
```

These rules contain only IF statements. They have conditional expressions but do not have action expressions. Filtering rule sets enable you to select only certain records for processing. Filtering rule sets are used as the eligibility rule set in treatments, but they can also be added directly to a decision. The results of a filtering rule set that is added directly to a decision differs based on the destination type in which the decision is run. For more information, see [“Using Filtering Rule Sets” on page 30](#).

assignment rule sets

contain rules that correspond to one of the following forms:

```
ASSIGN variable action_expression
```

```
IF condition_expressions THEN action_expressions
```

```
IF condition_expressions THEN action_expressions
ELSE condition_expressions THEN action_expressions
```

These rules are either assignment statements, IF-THEN statements, or IF-THEN-ELSE statements. IF-THEN and IF-THEN-ELSE statements have both conditional expressions and action expressions.

Assignment statements contain only an action expression. Assignment statements always execute unless a RETURN action stops the execution of the rule set before execution reaches the assignment statement. See [Step 3 of “Add an IF-THEN or IF-THEN-ELSE Rule to an Assignment or Common Rule Set” on page 43](#) for information about the RETURN action.

Assignment rule sets can also include references to common rule sets and references to advanced lists.

common rule sets

enable you to share the same rules among different assignment rule sets without redefining the rules in each assignment rule set. Common rule sets can contain assignment statements, IF-THEN statements, and IF-THEN-ELSE statements just like an assignment rule set can. You add common rule sets to assignment rule sets. When the assignment rule set executes, the rules in the common rule set execute as if they were defined directly in the assignment rule set. Any change to a common rule set affects all assignment rule sets that include the common rule set.

Using Filtering Rule Sets

Filtering rule sets are used as the eligibility rule set in treatments, but they can also be added directly to a decision. The output of a filtering rule set differs depending on how it is used and where it is run.

- In a treatment, the eligibility rule set defines who is eligible to receive the offer that is defined in the treatment. When the treatment group that contains the treatment is executed, only the records for which the rule set conditions evaluate to True are processed by the treatment. The output data grid for the treatment group contains only the records that were processed by the treatment.
- When a filtering rule set is added directly to a decision, the result depends on the type of destination in which the decision is run.

When the decision is tested in SAS Intelligent Decisioning or when a published decision is run in a CAS, Teradata, or Hadoop destination, only the records for which the rule set conditions evaluate to True are processed by the remaining objects in the decision. The output of the decision does not include any records that are filtered out by the filtering rule set.

In Micro Analytic Score destinations and in container destinations, all of the input records are processed by the decision and are included in the decision's output. However, SAS Intelligent Decisioning adds a column to the output data set named `_filter_`. The value of this column is 1 if the input record meets the conditions that are specified in the filtering rule set. The value of this column is 0 if the input record does not meet the filter criteria. In other words, if `_filter_=0`, the input record would normally be filtered out of the input. In your decision, you can add a branch that uses the value of the `_filter_` column to control the processing of the input records. However, it is recommended that

you define a branch expression that uses the same logic as your filtering rule set, which produces a decision that is not dependent on the presence of the `_filter_column`. Be aware that the `_filter_column` is created only if the decision is run in a container destination or in a Micro Analytic Score destination such as `maslocal`. If the decision is run in other destination types, that column is not created.

Note: Tests that are run in SAS Intelligent Decisioning are run in CAS, so the `_filter_column` is not added to the output of these tests.

About Condition and Action Expressions

An assignment rule corresponds to this form:

```
IF condition_expressions THEN action_expressions
```

For example, suppose you have the following rule:

```
IF customer_debts > customer_assets THEN approval_status = 'Decline'
```

In this case, `customer_debts > customer_assets` is a condition expression, and `approval_status = "Decline"` is an action expression.

For example, the following figure shows the rule above as it appears in the rule set editor:

IF	customer_debts	>	customer_assets
THEN	ASSIGN	approval_status	'Decline'

Note: Filtering rules do not contain action expressions.

A single rule in an assignment rule set can contain multiple condition expressions and action expressions. Multiple condition expressions within the same rule are joined together with the AND operand. For example, suppose you define the following rule in SAS Intelligent Decisioning:

IF	customer_debts	>	customer_assets
AND	credit_score	<	750
AND	isHomeowner	=	0
THEN	ASSIGN	approval_status	'Decline'

SAS Intelligent Decisioning generates the following rule:

```
IF ((customer_debts > customer_assets) AND (credit_score < 750)
AND (isHomeowner = 0)) THEN approval_status = 'Decline'
```

Create a New Rule Set

- 1 Click  to navigate to the Rule Sets category view.
- 2 Click **New rule set**. The New Rule Set window appears.
- 3 Enter a name for the rule set if you do not want to use the default name. Rule set names are limited to 100 characters and must be unique within a folder. Rule set names are case-sensitive.

Note: Some publishing destinations restrict the characters that can be used in the published name of a rule set. For more information, see [Table 3.6 on page 87](#).

- 4 Select the rule set type. See “[About Rules and Rule Set Types](#)” on page 29 for descriptions of each rule set type.

IMPORTANT You can publish decisions that include filtering rule sets to SAS Micro Analytic Service destinations and to container destinations, but you cannot publish the filtering rule sets themselves to SAS Micro Analytic Service destinations, to Git destinations, or to container destinations.

- 5 (Optional) Enter a description for the new rule set. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 6 Click , and select the folder where you want to save the rule set.
- 7 Click **Save**. SAS Intelligent Decisioning opens the new rule set and displays the **Variables** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 8 (Optional) If the rule set is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the rule set. For more information, see [“Check Out and Commit a Rule Set Version” on page 81](#).
- 9 Add variables and rules to the rule set. For more information, see the following topics:
 - [“Managing the Variables in a Rule Set” on page 33](#)
 - [“Defining New Rules in a Rule Set” on page 42](#)

Managing the Variables in a Rule Set

About Variables

The Properties of a Variable

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a rule set. Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See “Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.
Data type	SAS Intelligent Decisioning supports the following data types: Boolean, character, data grid, date, datetime, decimal, and integer. Integer, decimal, date, and datetime variables are converted to doubles when generated code is run.
Input and Output	A variable can be an input variable, an output variable, both, or neither (a temporary variable). See “Input Variables, Output Variables, and Temporary Variables” on page 35 for more information.

Property	Description
Length	Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run. The length for Boolean and numeric variable types is set automatically. For output-only data grid variables, the length is set to the value that you specify. For character variables and data grid variables that are input-only or input-output variables, the <code>variableLengthOverridden</code> configuration property determines how the length is set. By default, this property is set to Off, and the length is set to the length in the input data. When the <code>variableLengthOverridden</code> property is set to On, the length of input-only and input-output character variables and data grid variables is set to the value that you specify. For more information, see “sas.businessrules.variableLengthOverridden” in SAS Intelligent Decisioning: Administrator's Guide. Note: For scenario tests, the <code>variable.length.honorOutputLengthInMAS</code> and <code>variableLengthOverridden</code> configuration options are ignored. Scenario tests always use the length that you specify. In objects that are published to SAS Micro Analytic Service destinations or to container destinations, the maximum length for character variables (outside of a data grid) and for data grid variables is 50 Mb. For other destination types, the maximum length for these variables is 10 Mb. The maximum length for character variables within a data grid is 32767 characters. For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	You can specify an initial value only for variables of type character, integer, decimal, or boolean. Initial values are used only for output-only variables. Initial values are used when you test a rule set in SAS Intelligent Decisioning and when you publish a rule set independently. When you publish a rule set as part of a decision, the initial value that is specified for the corresponding decision variable is used, and the initial value that is specified in the rule set is ignored. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. When you are entering

Property	Description
	expressions in the expression editor, specify 1 for True and 0 for False. To use the number sign (#) in literal values, specify the percent-encoded value %23. For content that will be published to a SAS Micro Analytic Service destination, the values of date and datetime input variables must be numeric. They cannot be formatted values such as 18jul2019. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.
Maximum row count	The maximum number of rows that are allowed in each column of a data grid that is generated from an input table in which the data grid column names use array syntax. If the input data contains more entries for a column than the specified row count allows, the input data for that column is truncated. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in SAS Intelligent Decisioning: Using Data Grids

Input Variables, Output Variables, and Temporary Variables

For each variable, you must specify whether the variable is an input variable, an output variable, both an input variable and an output variable, or a temporary variable.

- For rule sets that are published independent of a decision, the rule set's input variables receive their value from the input table. When the rule set is deployed in a production system, all of the rule set's input variables must match table columns in input data. When you test a rule set in SAS Intelligent Decisioning, for each input variable, you can either map it to an input table column or specify a constant as its input value.

For rule sets that are published as part of a decision, the rule set's input variables receive their values from the decision variables to which they are mapped. When a decision that uses the rule set is deployed in a production system, all of the decision's input variables must match table columns in input data.

When you create or edit a rule set variable, clear the **Input** check box for any variable that you does not receive its value from either the input table or from a decision variable.

TIP If the `sas.businessrules.inputVariableReadOnly` configuration option is set to Off and the value of an input-only character variable is modified in a rule set, the modified value is passed back to the parent decision and is available to the remaining objects in the decision. For more information, see “[sas.businessrules.inputVariableReadOnly](#)” in *SAS Intelligent Decisioning: Administrator's Guide*.

- Output variables are variables that are written to the output table that is created when a rule set is run. When you create or edit a variable, clear the **Output** check box for any variable that you want to exclude from the output data.

Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.

- Temporary variables are variables that are not present in the input data, and they are not written to the output table. To create a temporary variable for use only while a rule set is executing, clear both the **Input** and **Output** check boxes.

Note: For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.

When you create a new variable, it is created as an input-output variable by default.

Note: Binary and varying-length binary variables are supported in decisions only as input variables or temporary variables in order to support models that require binary data.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click the **Rule Set Variables** subtab, then click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name: "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name: "HMEQ". ["How to Enter Searches for Information Assets in SAS Data Governance" in SAS Data Governance: User's Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see ["Importing and Loading Data" on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➤**. To import all of the variables in the table, click **➤»**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See ["Input Variables, Output Variables, and Temporary Variables" on page 35](#) for more information.

Add Global Variables to a Rule Set

In order to use a global variable in a rule set, the global variable must be activated. Instructions for defining and activating global variables are in ["Create a New Global Variable" on page 174](#) and ["Activate a Global Variable" on page 182](#).

Note: You can add a global variable that has not been activated to a rule set and publish the rule set, but the variable's value is set to missing until the global variable is activated.

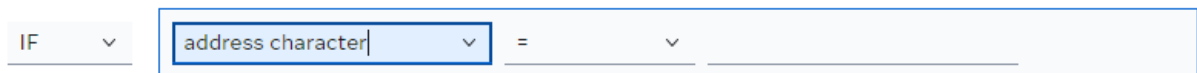
If a global variable has not been activated, the warning icon  is displayed in the **Activated Value** column.

- 1 On the **Variables** tab, click the **Global Variables** subtab.
- 2 Click **Select variables**. The Select Variables window appears.
- 3 Select the check boxes for the variables that you want to add to the rule set, and click **OK**.
- 4 (Optional) Select the **Output** check box if you want the value of the variable to be written to the output table that is generated when the rule set is run.

Note: If the rule set will be included in and run as part of a decision, it is recommended that you do not select the **Output** check box for global variables in the rule set. Instead, select the **Output** check box for the decision variable of the same name after you add the rule set to the decision. For more information, see [“Using Global Variables” on page 173](#).

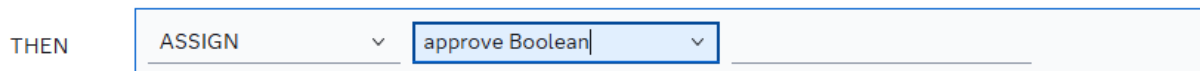
Create a Custom Variable Dynamically

To create a variable in a rule set, you can enter the new variable name in any condition variable field and in the variable field of assignment statements. By default, SAS Intelligent Decisioning creates a variable of type Decimal. To create a variable of a different type, enter the variable name, a space, and then the data type. If you enter the name in a condition variable field, SAS Intelligent Decisioning creates the variable as an input-output variable. If you enter the name in an assignment statement, SAS Intelligent Decisioning creates the variable as an output variable. For example, you can create an input-output variable of type Character that is named **address** by entering it in the condition variable field:



The screenshot shows a rule set editor interface. On the left, there is a label 'IF' followed by a downward arrow. To the right is a dropdown menu currently displaying 'address character'. Further right is an equals sign followed by another downward arrow, indicating an assignment or comparison operation.

You can create an output-only variable of type Boolean named **approve** by entering it in an assignment statement, followed by a space and the data type:



The screenshot shows a rule set editor interface. On the left, there is a label 'THEN'. To its right is a dropdown menu currently displaying 'ASSIGN'. Further right is another dropdown menu displaying 'approve Boolean', followed by a blank space and another downward arrow.

Create Custom Variables on the Variables Tab

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click the **Rule Set Variables** subtab.
- 2 Click **Add variable** and select **Custom variable**. The Add Variables window appears.
- 3 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 33](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Maximum row count**, and **Length** fields.
- c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable's data type. See [“The Properties of a Variable” on page 33](#) for additional information.
- d Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- e Verify that the **Input** and **Output** check boxes are selected correctly for each variable.
 - Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.
 - Clear the **Output** check box for any variable that you want to exclude from the output data.
 - Clear both the **Input** and **Output** check boxes to create a temporary variable.
- 4 (Optional) Modify the variable properties in the table of variables.
- 5 Click **OK** to add the variables and close the Add Variables window.

Duplicate a Variable

- 1 On the **Variables** tab, click the **Rule Set Variables** subtab.
- 2 Select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 3 Enter a new name for the duplicate variable.
- 4 (Optional) Enter a description for the variable.
- 5 Click **Duplicate**.

Importing and Exporting Variables

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, “Import File Formats,” on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the rule set into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.
- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

TIP To add new columns to an existing data grid variable, select **Replace all variables**.

Note: If you replace the list of variables in a rule set and a variable that is used in a locked version of the rule set is not included in the import file, you cannot save the rule set.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Export Variables

Note: If you make changes to the variables, you must save the rule set before exporting variable data.

- 1 Open the rule set from which you want to export variables.
- 2 On the **Rule Set Variables** tab, select the variables that you want to export.
- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default

download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

Note: You cannot delete a variable if it is used in the current version of an object. You can delete variables from the current version if they are used only in locked versions, but those variables are still included in the locked versions.

- 1 On the **Variables** tab, click the **Rule Set Variables** subtab.
- 2 Select the check box for the variables that you want to delete, click **:**, and select **Delete**.

Edit Variable Properties

On the **Variables** tab, click the **Rule Set Variables** subtab, then click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

When you rename a variable, references to that variable within the same object change to use the new name. You cannot change the name or data type of a variable that is used in locked versions.

See [“The Properties of a Variable” on page 33](#) for additional information.

Edit Metadata for Data Grid Variables

For information, see [“Editing Data Grid Variable Metadata” in SAS Intelligent Decisioning: Using Data Grids](#).

Determine Which Objects Use a Particular Variable

On either the **Rule Set Variables** subtab or the **Global Variables** subtab, select the check box for the variable, click **:** on the subtab, and select **View used by report**.

In the report, you can use the **Filter** field to filter the list of objects based on the object names. If the selected variable is a data grid variable, and if you are interested only in a specific column within the data grid, you can select the column

in the **Column name** field, and then click **Apply**. SAS Intelligent Decisioning narrows the search results to only the objects that use the data grid with the selected column.

Click on an object name to open the object. Click ⓘ next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Defining New Rules in a Rule Set

TIP When you add a rule to a rule set, and the rule refers to another object such as a lookup table, common rule set, advanced list, SAS Intelligent Decisioning uses the object's ID, not its name, to define the rule. If you delete the object and create a new object with the same name, you must delete and re-add the rule that uses the object.

Add a Standalone Assignment Statement

Stand-alone assignment statements and references to advanced lists always execute unless a RETURN action stops the execution of the rule set before execution reaches the statements. Rule-fired data is not generated for standalone assignment statements or references to advanced lists.

- 1 Select the **Rule Set** tab.
- 2 If the rule set is empty, click **Add other** to open the Add Other window, select **Assignment**, and click **OK**.
If the rule set contains at least one statement, select **Add** ⇨ **Add assignment**. The application adds an assignment statement immediately after the currently selected statement.
- 3 Import or create any variables that are required for the assignment statement that have not already been added to the rule set. You can add or create the variables on the **Variables** tab, or you can define variables dynamically as you author the statement. See [“Managing the Variables in a Rule Set” on page 33](#) for more information.
- 4 Select the variable to which you want to assign a value.

- 5 Enter the value or expression for the variable in the expression field. For more information, see [“About Defining Expressions in Rules and Assignment Statements” on page 56](#) and [“Using the Expression Editor” on page 535](#).
- 6 (Optional) Move the assignment statement to a different position in the rule set. To move the statement, click  or .
- 7 Click  to save the rule set. SAS Intelligent Decisioning validates the syntax of the expressions. If it does not detect any problems, it saves the rule set. If it detects errors, it highlights the field that contains the error. Hover over the field to display an error message.

Add an IF-THEN or IF-THEN-ELSE Rule to an Assignment or Common Rule Set

- 1 Create or open the rule set. If no variables are defined in the rule set, SAS Intelligent Decisioning displays the **Variables** tab. Otherwise, it displays the **Rule Set** tab.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Import or create any variables that are required for the new rule that have not already been added to the rule set. You can add or create the variables on the **Variables** tab, or you can define variables dynamically as you author the rule. See [“Managing the Variables in a Rule Set” on page 33](#) for more information.
- 3 Click the **Rule Set** tab.
- 4 Click **Add rule** if the rule set is empty. If the rule set contains at least one statement, click **+Add Rule** or select **Add** ⇨ **Add rule**. SAS Intelligent Decisioning adds a new IF-THEN rule to the rule set.
- 5 (Optional) To enter a rule that uses a LOOKUP expression, change the equals operator in the condition expression to **LOOKUP**, and select the lookup table in the expression field. A LOOKUP expression determines whether a key exists in a lookup table. For more information, see [“Enter LOOKUP and LOOKUPVALUE Expressions” on page 56](#).
- 6 (Optional) To enter a rule that uses a LOOKUPVALUE expression, change the keyword in the action expression from **ASSIGN** to **LOOKUPVALUE**, and complete the steps listed in [“Enter LOOKUP and LOOKUPVALUE Expressions”](#). A LOOKUPVALUE expression retrieves a values from a lookup table.

- 7 To define an IF-THEN rule that does not reference a lookup table, select the variable, select the operator, and define the expression for the rule. For more information, see [“About Defining Expressions in Rules and Assignment Statements” on page 56](#) and [“Using the Expression Editor” on page 535](#).

To add additional condition or action expressions to the selected rule, select **Add** ⇒ **Condition** or **Add** ⇒ **Action**.

TIP To move condition or action expressions up or down within an IF, THEN, or ELSE clause, select the expression and click ↑ or ↓.

- 8 (Optional) Change the **IF** keyword to **ELSE** or **OR**. If the rule is the first rule in a rule set, the rule keyword must be IF.

If you change a rule's keyword to ELSE, the rule is moved to the preceding rule as an ELSE clause, and its conditions and actions are preserved. If you change the keyword to OR, the rule's condition expressions are added to the condition expressions for the preceding rule, but the rule's action expressions are deleted. For more information, see [“Controlling Which Conditions Are Evaluated” on page 53](#).

- 9 (Optional) Select **Add** ⇒ **ELSE rule** to add an ELSE clause to the currently selected rule. The ELSE clause does not have a condition, but you can add one by selecting **Add** ⇒ **Condition**.
- 10 (Optional) Define the condition and action expressions for the ELSE clause.
- 11 (Optional) Change the order of the rules. Rules are evaluated sequentially. To move a rule up or down within a rule set, select the rule and click ↑ or ↓.
- 12 (Optional) Change the name of the rule. Rule names are limited to 100 characters and must be unique within a rule set. For instructions, see [“Edit a Rule Name or Description” on page 65](#).

TIP Assigning logical names to the rules makes it easier to determine which rules fired when you review rule-fired data.

- 13 (Optional) Clear the **Record rule-fired data** check box if you do not want a rule-fired record to be written each time this rule fires. See [“How Rules Are Evaluated and When Rule-Fired Records Are Generated” on page 55](#) for more information.
- 14 Click  to save the rule set. SAS Intelligent Decisioning validates the syntax of the expressions. If it does not detect any problems, it saves the rule set. If it detects errors, it highlights the field that contains the error. Hover over the field to display an error message.

Add a RETURN Statement

The RETURN action stops the execution of any additional statements in the rule set. See [“Controlling Which Conditions Are Evaluated” on page 53](#) for more information.

- 1 Select the **Rule Set** tab.
- 2 Do one of the following:
 - If the rule set is empty, click **Add other** to open the Add Other window, select **Assignment**, and click **OK**.
 - If the rule set contains at least one statement, select **Add** ⇒ **Add assignment**.

The application adds an assignment statement immediately after the currently selected statement.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 3 Change the keyword in the action expression from **ASSIGN** to **RETURN**.
- 4 Click  to save the rule set. SAS Intelligent Decisioning validates the syntax of the expressions. If it does not detect any problems, it saves the rule set. If it detects errors, it highlights the field that contains the error. Hover over the field to display an error message.

Adding a Common Rule Set Reference to an Assignment Rule Set

About Rule Set Variables and Mapping

Common rule sets and assignment rule sets each define their own variables. When you add a common rule set to an assignment rule set, you must map the variables that are defined in the common rule set to variables that are used in the assignment rule set. By default, if the assignment rule set defines variables of the same name and data type as the variables in the common rule set, then SAS Intelligent Decisioning automatically maps the common rule set variables to the assignment

rule set variables. If variables of the same name and data type do not exist in the assignment rule set, you must do one of the following:

- create variables in the assignment rule set that have the same name and data type as the variables in the common rule set. You can use the **Add missing variables** option to create these assignment rule set variables and automatically map the variables. This option is available in both the [Add Common Rule Set References wizard](#) and in the [Edit Common Rule Set Reference window](#).
- manually map the common rule set variables to assignment rule set variables that have different names. If the assignment rule set variables do not already exist when you add the common rule set reference in the wizard, you can edit the properties of the reference and map the variables in the [Edit Common Rule Set Reference window](#).

Add a Common Rule Set Reference to an Assignment Rule Set

- 1 Open the assignment rule set. If no variables are defined in the rule set, SAS Intelligent Decisioning displays the **Variables** tab. Otherwise, it displays the **Rule Set** tab.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Import or create any variables that are required for the common rule set that have not already been added to the assignment rule set. See [“Managing the Variables in a Rule Set” on page 33](#) for more information.
- 3 Click the **Rule Set** tab.
- 4 Do one of the following:
 - If the rule set contains at least one rule, click **Add** and select **Reference common rule set**.
 - If the rule set is empty, click **Add other**. In the Add Other window, select **Common rule set reference**, and click **OK**.

SAS Intelligent Decisioning opens the Add Common Rule Set References wizard.

- 5 On the **Select Common Rule Sets** page, select the check boxes for each common rule sets that you want to add to the assignment rule set, and click **Next**.
- 6 On the **Select Common Rule Set Versions** page, select the version of each of the common rule sets, then click **Next**.

TIP To specify the latest version of the common rule set, you can select the version number of the latest version, or you can select **Use latest**. If you select **Use latest**, SAS Intelligent Decisioning always uses the most recently created version regardless of the version number.

- 7 (Optional) On the **Map Inputs** page, for each common rule set variable in the **Input Variable** column, select the assignment rule set variable that you want to map it to in the **Maps To** column. If the assignment rule set contains variables with the same name and data type as the variables in the common rule set, SAS Intelligent Decisioning automatically maps the common rule set variables to the assignment rule set variables.

TIP To create assignment rule set variables that have the same name and data type as the variables in the common rule set, click **Add Missing Variables**. If you want to create new variables with different names in the assignment rule set, you can ignore the error markers, and continue with the next step. After you click **Finish** in [Step 12](#), you can create new variables in the assignment rule set, then edit the properties of the common rule set reference to change the variable mappings. For more information, see [“Edit the Properties of a Common Rule Set Reference” on page 48](#).

For additional information, see [“About Rule Set Variables and Mapping” on page 45](#).

- 8 Click **Next**.
- 9 (Optional) On the **Map Outputs** page, for each common rule set variable in the **Output Variable** column, select the assignment rule set variable that you want to map it to in the **Maps To** column. If the assignment rule set contains variables with the same name and data type as the variables in the common rule set, then SAS Intelligent Decisioning automatically maps the common rule set variables to the assignment rule set variables.

TIP To create assignment rule set variables that have the same name and data type as the variables in the common rule set, click **Add Missing Variables**. If you want to create new variables with different names in the assignment rule set, you can ignore the error markers, and continue with the next step. After you click **Finish** in [Step 12](#), you can create new variables in the assignment rule set, then edit the properties of the common rule set reference to change the variable mappings. For more information, see [“Edit the Properties of a Common Rule Set Reference” on page 48](#).

For more information, see [“About Rule Set Variables and Mapping” on page 45](#).

- 10 Click **Next**.

- 11 (Optional) On the **Order Rules**, page, change the order in which the common rule sets are included in the assignment rule set. To move a common rule set up or down within the assignment rule set, select the common rule set and click **↑** or **↓**.
- 12 Click **Finish**. SAS Intelligent Decisioning adds the common rule set reference to your assignment rule set. You can click **>** to display the content of the common rule set inline in the assignment rule set, but the content is read-only. You can use the rule-specific menu to [edit the reference](#) or to open the common rule set.
- 13 (Optional) Add any global variables that are used by the common rule set to the assignment rule set. For instructions, see [“Add Global Variables to a Rule Set” on page 37](#).

Edit the Properties of a Common Rule Set Reference

- 1 Open the assignment rule set, click **:** for the reference that you want to edit, and select **Edit reference**. The Edit Common Rule Set Reference window appears.
- 2 (Optional) On the **Common Rule Set** tab, select a different common rule set or change the version of the common rule set.
- 3 (Optional) On the **Map Inputs** tab, change the variable mappings for the input variables in the common rule set. For more information, see [Step 7](#) of [“Add a Common Rule Set Reference to an Assignment Rule Set”](#).
- 4 (Optional) On the **Map Outputs** tab, change the variable mappings for the output variables in the common rule set. For more information, see [Step 9](#) of [“Add a Common Rule Set Reference to an Assignment Rule Set”](#).
- 5 Click **OK** to close the Edit Common Rule Set References window and return to the assignment rule set.
- 6 Click  to save your changes to the assignment rule set.

Adding an Advanced List Reference to an Assignment Rule Set

About Advanced List References

Note: You must configure support for advanced lists before you can use references to advanced lists to a rule set. For more information, see [“Configuring Support for Advanced Lists” in SAS Intelligent Decisioning: Administrator’s Guide](#).

An advanced list reference has the following format:

IF list_key_column=rule_set_variable FROM list_name ASSIGN list_and_rule_variable

If the value of *list_key_column* equals the value of *rule_set_variable*, then the reference assigns the value of the list column named *list_and_rule_variable* to the rule set variable of the same name. The reference can specify multiple key columns, and it can retrieve multiple column values. SAS Intelligent Decisioning assigns the retrieved values to rule set variables whose names match the names of the value columns. If rule set variables with the same names do not exist, then SAS Intelligent Decisioning creates the rule set variables for you.

For example, the following reference retrieves the values of the list columns named `salary` and `hireDate` if the values of the key columns `lastname` and `departmentId` match the values of the rule set variables `name2` and `dept`, respectively. SAS Intelligent Decisioning assigns the retrieved values to rule set variables named `salary` and `hireDate`.

```
IF name2=lastName, dept=departmentId FROM frdDeptList ASSIGN salary, hireDate
```

Redis evaluates the IF conditions, retrieves the column values, and assigns the rule set variable values. Advanced list references are always executed. Rule-fired data is not recorded for advanced list references.

Note: You can test rule sets and decisions that reference a single advanced list in SAS Intelligent Decisioning. Rule sets and decisions that are published to SAS Micro Analytic Service destinations or to container destinations can reference multiple advanced lists. Rule sets and decisions that are published to other destination types can reference only one advanced list, and the advanced list must be included inline in the code that is generated for the published object. For more information, see [Table 13.4 on page 470](#).

Add an Advanced List Reference to an Assignment Rule Set

Note: In order to retrieve values from an advanced list, both the list and the list rows from which you want to retrieve data must be deployed. An advanced list reference does not return data from a list or a list item whose status is "developing". For more information, see ["Deploy an Existing List" on page 282](#) and ["Deploy Specific List Items" on page 290](#).

Note: The rule set variables that you want to map to the key columns in the advanced list must exist before you add the advanced list reference.

To add a rule that retrieves values from an advanced list:

- 1 Open the rule set to which you want to add the advanced list reference, and select the **Rule Set** tab.
 - 2 Do one of the following:
 - If the rule set contains at least one rule, click **Add** and select **Reference advanced list**.
 - If the rule set is empty, click **Add other**. In the Add Other window, select **Advanced list reference**, and click **OK**.
- SAS Intelligent Decisioning opens the Reference Advanced List wizard.
- 3 On page 1, **Select an advanced list**, select the advanced list from which you want to retrieve values, and click **Next**.
 - 4 On page 2, **Select value columns**, select the columns in the list whose values you want to retrieve. The values that you select are added to the ASSIGN clause in the reference.

TIP The ASSIGN clause displays the first 25 characters of the list of value columns. On the **Rule Set** tab, you can click  to view the complete list of value columns that are specified in the reference.

Masked columns cannot be selected as value columns.

- 5 On page 3, **Key mapping**, select the rule set variables to which you want to compare the value of each of the list's key columns, and click **Finish**. The variables that you select are specified in the IF clause in the reference.

Edit an Advanced List Reference

- 1 On the **Rule Set** tab, click  for the reference that you want to edit.
- 2 (Optional) In the Edit Reference window, click , and select a different advanced list. The default location for advanced lists is **SAS Content/Products/List Data**.
- 3 (Optional) On the **Value Columns** tab, select the check box for any variable whose value you want to retrieve from the list, and clear the check box for any variable whose value you do not want to retrieve.
- 4 (Optional) On the **Key Mappings** tab, change the rule set variables to which you want to compare the value of each of the list's key columns.
- 5 Click **Save** to save your changes and return to the **Rule Set** tab.

Add a New Rule in a Filtering Rule Set

Note: When SAS Intelligent Decisioning generates code for a filtering rule set, it joins the rules together with the OR operand.

- 1 Create or open the filtering rule set. If no variables are defined in the rule set, SAS Intelligent Decisioning displays the **Variables** tab. Otherwise, it displays the **Rule Set** tab.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Import or create any variables that are required for the new rule that have not already been added to the rule set. You can add or create the variables on the **Variables** tab, or you can define variables dynamically as you author the rule. See [“Managing the Variables in a Rule Set” on page 33](#) for more information.
- 3 Click the **Rule Set** tab.
- 4 Click **Add Rule** if the rule set is empty or, if the rule set contains at least one statement, select **Add** ⇒ **Add rule** or **+Add Rule**. SAS Intelligent Decisioning adds a new IF rule immediately after the currently selected rule.

- 5 Define the condition expression for the rule. For more information, see [“About Defining Expressions in Rules and Assignment Statements” on page 56](#) and [“Using the Expression Editor” on page 535](#).

To add additional condition expressions to the selected rule, select **Add** ⇒ **Condition**.

TIP To move condition expressions up or down within an IF clause, select the expression and click ↑ or ↓.

- 6 (Optional) Change the order of the rules. Rules are evaluated sequentially. To move a rule up or down within a rule set, select the rule and click ↑ or ↓.
- 7 (Optional) Change the name of the rule. Rule names are limited to 100 characters and must be unique within a rule set. For instructions, see [“Edit a Rule Name or Description” on page 65](#).

TIP Assigning logical names to the rules makes it easier to determine which rules fired when you review rule-fired data.

- 8 (Optional) Clear the **Record rule-fired data** check box if you do not want a rule-fired record to be written each time this rule fires. See [“How Rules Are Evaluated and When Rule-Fired Records Are Generated” on page 55](#) for more information.
- 9 Click  to save the rule set. SAS Intelligent Decisioning validates the syntax of the expressions. If it does not detect any problems, it saves the rule set.

Copy Rules From Another Rule Set

The menu option **Copy from rule set** appears in the **Add** menu for a rule set. The rule set must contain at least one rule in order for this menu option to appear. Adding a rule from another rule set does not replace any of the rules in the rule set that you are editing.

You can copy rules only from another rule set of the same type as the rule set you are editing.

- 1 Open the rule set into which you want to add the rule, and click the **Rule Set** tab.
- 2 Select the existing rule after which you want to add IF rules or to which you want to add an ELSE rule. If you do not select a rule, then SAS Intelligent Decisioning adds copied rules to the end of the rule set.
- 3 Click **Add** and select **Copy from rule set**. The **Copy Rules** window appears. This window displays the latest version of every rule in every rule set that you have permission to view.

- 4 Select the rules that you want to copy, and click **Copy**.

SAS Intelligent Decisioning adds the selected rules to the rule set together with any new variables that these rules need.

Note: If the copied rule contains functions or complex expressions, the variables might not be added to the rule set. Add any variables used in the expression to the rule set, and validate the rule set.

Controlling Which Conditions Are Evaluated

By default, rules are assigned the IF rule operator, which means that the rule's conditions are evaluated regardless of the results of previous rules. In assignment rule sets, you can control whether condition expressions are evaluated by using the RETURN action, the ELSE operator, or the OR operator.

The RETURN action stops the execution of any remaining rules in a rule set. If you are executing a single rule set, execution ends. If you are executing a decision, control moves to the next object in the decision. For example, the rule in the following figure stops the execution of any remaining rules in the rule set if the value of the Order_Quantity variable is missing.

▼ Default_rule_1

☒ Record rule fired data

IF

▼

Order_Quantity

▼

=

▼

.

THEN

RETURN

▼

If you set a clause's keyword to ELSE, then the clause's conditions are evaluated only if the previous clause's conditions evaluated to false. For example, given the rule set shown in the following figure, if Order_Quantity is 9, the condition for the IF clause evaluates to false, and the condition for the first ELSE clause evaluates to true. Therefore, the action for first ELSE clause is executed, and the condition for the last ELSE clause is not evaluated. The value of Offer_Percent is set to 5.

▼ Default_rule_1

☒ Record rule fired data

ⓘ

⋮

IF	▼	Order_Quantity	▼	=	▼	5
THEN		ASSIGN	▼	Offer_Percent	▼	0
ELSE	▼	Order_Quantity	▼	>	▼	5
AND		Order_Quantity	▼	<=	▼	10
THEN		ASSIGN	▼	Offer_Percent	▼	5
ELSE	▼	Order_Quantity	▼	>	▼	10
AND		Order_Quantity	▼	<=	▼	20
THEN		ASSIGN	▼	Offer_Percent	▼	10

Use the OR keyword to execute the same action expression for each of the several conditions. If any of the OR conditions evaluate to true, SAS Intelligent Decisioning executes the action. If several consecutive rules are all assigned the OR keyword, the action is executed as soon as a set of conditions evaluates to true. The conditions for the remaining consecutive OR rules are not evaluated.

For example, given the rules in the following figure, REJECT is assigned the value of 0 (False) if any of the following conditions are true:

- (VALUE >= 120000 AND CLNO < 15)
- (DEROG <= 1 AND DELINQ = 0)
- YOJ > 15

v Default_rule_1

☒ Record rule fired data ⓘ

IF

VALUE

>=

120000

AND

CLNO

<

15

OR

DREOG

<=

1

AND

DELINQ

=

0

OR

YOJ

>

15

THEN

ASSIGN

REJECT

0

How Rules Are Evaluated and When Rule-Fired Records Are Generated

By default, the condition expressions for all rules in a rule set are evaluated sequentially regardless of the results of previous rules. However, in assignment rule sets, you can use the ELSE keyword, the OR keyword, or the RETURN action to control whether condition expressions are evaluated. See [“Controlling Which Conditions Are Evaluated” on page 53](#) for more information.

If a rule's condition expressions evaluate to true, the rule is said to have *fired*, and in assignment rule sets, SAS Intelligent Decisioning executes the rule's action expressions.

By default, every time a rule fires, it generates a rule-fired record. You can control when rule-fired records are generated by using the **Record rule-fired data** check boxes. See [Step 13 in “Add an IF-THEN or IF-THEN-ELSE Rule to an Assignment or Common Rule Set” on page 43](#).

Note: Stand-alone assignment statements and references to advanced lists always execute unless a RETURN action stops the execution of the rule set before execution reaches the statements. Rule-fired data is not generated for standalone assignment statements or references to advanced lists.

For additional information, see [“Run a Rule-Fired Analysis” on page 97](#).

Defining Expressions

About Defining Expressions in Rules and Assignment Statements

Expressions can be up to 1024 characters long. They can contain numeric constants, character strings, variables, operators, SAS DS2 functions, SAS Data Quality Server functions, data grid functions, custom functions that you define in the Custom Functions category view, and the SAS Intelligent Decisioning LOOKUP and LOOKUPVALUE functions. You can enter expressions directly into the expression fields, or you can use the Expression Editor to create and edit expressions.

TIP Use caution when you test for equality by using scientific notation. Two numbers that appear to be the same might evaluate to different numbers because of the precision involved in scientific notation.

Enter LOOKUP and LOOKUPVALUE Expressions

Note: You can enter the LOOKUP function only in condition expressions, and you can enter the LOOKUPVALUE function only in action expressions. The LOOKUP function determines whether a lookup key exists in the lookup table, and the LOOKUPVALUE function retrieves the value associated with a key. It is a best practice to use the LOOKUP function to verify that a key exists before you try to retrieve the value associated with the key.

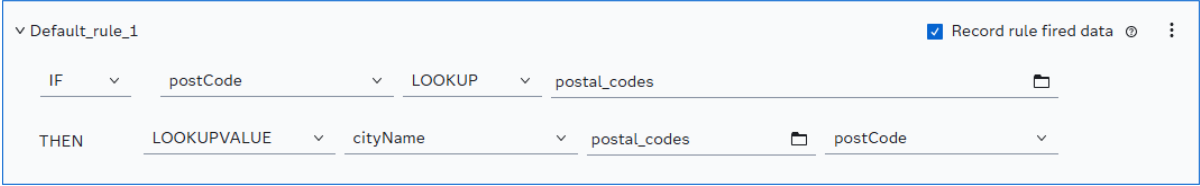
To enter a condition expression that uses the LOOKUP function, select **LOOKUP** as the rule operator, and select the lookup table in the expression field.

To enter an action expression that uses the LOOKUPVALUE function, complete these steps:

- 1 Select **LOOKUPVALUE** as the keyword in the THEN clause.
- 2 In the second field of the THEN clause, select the variable to which you want to assign the value that is retrieved from the lookup table.

- 3 Click , and select the lookup table from which you want to retrieve the lookup value.
- 4 In the last field, select the variable whose value matches the value of the lookup key, or select the actual value of the lookup key. To select the actual value of the lookup key:
 - a Click  in the last field.
 - b Select **Select a lookup key**. SAS Intelligent Decisioning displays the list of lookup entries in the specified table.
 - c Select the record whose key you want to use in the expression, and click **Save**.

For example, suppose you have a lookup table in which the lookup keys are postal codes and the lookup values are city names. You could use the following rule to test whether the lookup table contains an entry for the postal code, and, if the entry exists, retrieve the city name that is associated with the postal code.



▼ Default_rule_1 ☒ Record rule fired data ⓘ ⋮

IF  postCode  LOOKUP  postal_codes 

THEN LOOKUPVALUE  cityName  postal_codes  postCode 

Entering Literal Data Values

Depending on whether you use the Expression Editor or enter expressions directly into the expression fields, you must enter some values differently.

Data Type	How To Enter Values	Example
Character	Enclose character strings in quotation marks. If the character string contains embedded quotation marks, use different quotation marks for the embedded and enclosing quotation marks.	'Gold Account' "d'oscail" 'd"oscail'
Data grid	Enclose data grid JSON strings in single quotation marks. Note: The only time you might need to enter JSON strings manually is when you are using the DATAGRID_CREATE function.	See SAS Intelligent Decisioning: Using Data Grids for an explanation of the JSON syntax and a description of the DATAGRID_CREATE function.

Data Type	How To Enter Values	Example
Date	In the rule set editor, enter Date values by using the format DDMMYYYYY. Enclose each value in single quotation marks followed by d.	'01AUG2017'd
	In the Expression Editor, use the DS2 function TO_DOUBLE and specify the DATE data type in order to cast the Date value to a DOUBLE value so that it can be compared correctly to other variables. See SAS DS2 Programmer's Guide for information about date, time, and timestamp values, and see SAS DS2 Language Reference for information about the TO_DOUBLE function.	to_double(date '2017-11-04')
Datetime	In the rule set editor, enter Datetime values by using the format DDMMYYYYY:HH:MM:SS. Use 24-hour clock notation. Enclose each value in single quotation marks followed by dt.	'31AUG2017:15:00:00'dt
	In the Expression Editor, use the DS2 function TO_DOUBLE and specify the TIMESTAMP data type in order to cast the Datetime value to a DOUBLE value so that it can be compared correctly to other variables. See SAS DS2 Programmer's Guide for information about date, time, and timestamp values, and see SAS DS2 Language Reference for information about the TO_DOUBLE function.	to_double(timestamp '2017-11-04 10:54:34.012')
Boolean	In the rule set editor, Boolean values are not enclosed in quotation marks. Enter only the values.	True False
	In the Expression Editor, use numeric values to indicate True or False. Specify 1 for True and 0 for False.	1 0

Operators for Use in Expressions

The following table lists the operators that you can use in an expression. Do not enter a space between the elements of the operators `<=`, `>=`, or `^=`. Some mnemonic equivalents for these operators cannot be used in SAS Intelligent Decisioning expressions. See *SAS DS2 Programmer's Guide* for more information about specifying operators in expressions.

Table 3.1 Operators for Use in Expressions

Operator	Definition	Example
<code>*</code>	Multiply	<code>0.085 * sales</code>
<code>/</code>	Divide	<code>amount / 5</code>
<code>+</code>	Add	<code>num + 3</code>
<code>-</code>	Subtract	<code>sale - discount</code>
<code>**</code>	Raises the first operand to the power of the second operand	<code>num1**num2</code>
<code>=</code>	Equal to	<code>tries = maxTriesAllowed</code>
<code> </code>	Concatenates the first string and the second string	<code>string1 string2</code>
<code>!=</code>	Not equal to	<code>insufficientFunds != 1</code>
<code>^=</code> # (Expression Editor interface)	Not equal to	<code>balance ^= 'low'</code>
<code>></code>	Greater than	<code>daysLate > 5</code>
<code>>=</code>	Greater than or equal to	<code>balance >= 1000</code>
<code><</code>	Less than	<code>balance < 1000</code>
<code><=</code>	Less than or equal to	<code>balance <= 250</code>
<code><></code>	The maximum of the left and right operands	<code>num1 <> num2</code>

Operator	Definition	Example
IN (<i>value-list</i>)	Equal to an item in <i>value-list</i>	risk IN ('high', 'medium', 'low')
NOT IN (<i>value-list</i>)	Not equal to an item in <i>value-list</i>	offerPercent NOT IN (10, 20, 30)
LIKE ' <i>pattern</i> '	If the variable's value matches the expression <i>pattern</i> in <i>pattern</i> , the result is true.	LIKE 'HS%PP'
<i>expression</i> AND <i>expression</i>	If both expressions are true, the result is true.	dateExpired >= '01AUG2015'd AND dateExpired <= '31AUG2015'd
<i>expression</i> OR <i>expression</i>	If either expression is true, the result is true.	dateEnrolled >= '01JAN2015' OR member = 1

Using the LIKE Operator

The LIKE operator determines whether the value of a variable matches a pattern-matching expression. An expression that uses the LIKE operator has the following syntax:

LIKE '*pattern-matching-expression*'

If a variable's value matches the pattern that is specified by *pattern-matching-expression*, the expression evaluates to true (1). Otherwise, the expression evaluates to false (0).

There are three classes of pattern-matching characters.

Table 3.2 Pattern-Matching Characters

Character	Description
underscore (_)	Matches any single character
percent sign (%)	Matches any sequence of zero or more characters Note: Be aware of the effect of trailing blanks. To match values, you might have to use the TRIM function to remove trailing blanks.
any other character	Matches that character

The LIKE expression is case sensitive. To search for mixed-case strings, use the UPCASE function to create an uppercase version of the variable that you want to search. You can use a temporary variable to store the results of the UPCASE function. Use the LIKE operator to search the uppercase version of the variable. For example, you can search the variable Part_Number for mixed-case strings that begin with HS and end with PP by using the two rules shown in the following figure.

The screenshot shows the SAS rule configuration interface. At the top, there is a rule configuration bar with 'ASSIGN' selected, a dropdown for 'temp', and the expression 'UPCASE(Part_Number)'. Below this, a rule named 'Default_rule_1' is shown. It has a condition 'IF temp LIKE ('HS%PP')' and an action 'THEN ASSIGN Category 'Performance''. A checkbox 'Record rule fired data' is checked.

The following table shows examples of the matches that result if you search a variable that could have these values: Smith, Smooth, Smothers, Smart, Smuggle.

Table 3.3 Examples of LIKE Expressions

LIKE Expression Example	Matching Results
like 'Sm%'	Smith, Smooth, Smothers, Smart, Smuggle
like '%th'	Smith, Smooth
like 'S__gg%'	Smuggle
like 'S_o'	(no matches)
like 'S_o%'	Smooth, Smothers
like 'S%th'	Smith, Smooth

Using Functions in Expressions

SAS Intelligent Decisioning supports the following functions in expressions:

- LOOKUP and LOOKUPVALUE functions. Condition expressions can contain the LOOKUP function, and action expressions can contain the LOOKUPVALUE function. However, if the expression contains the LOOKUP or LOOKUPVALUE function, then the expression cannot contain anything else. For more information, see [“Enter LOOKUP and LOOKUPVALUE Expressions” on page 56](#), [“LOOKUP Function” on page 169](#), and [“LOOKUPVALUE Function” on page 171](#).
- SAS DS2 functions. For additional information about these functions and additional DS2 functions, see [SAS DS2 Language Reference](#).

- SAS Data Quality Server functions. For information about these functions, see [“Functions and CALL Routines” in SAS Data Quality: Language Reference](#).

Note: You cannot publish rule sets that use SAS Data Quality functions to container destinations. Also, rule sets that use these functions fail when they are run inside the database.

- SAS Intelligent Decisioning data grid functions. For more information about these functions, see [“Data Grid Functions” in SAS Intelligent Decisioning: Using Data Grids](#).
- Custom DS2 functions that you define in SAS Intelligent Decisioning or by using the sid-functions CLI. For more information, see [Chapter 8, “Using Custom Functions,” on page 233](#) and [“sid-functions Plug-In” in SAS Intelligent Decisioning: Command-Line Interfaces](#).

To enter a function in an expression, use the expression editor. See [“Using the Expression Editor” on page 535](#) for more information.

Working with Missing Values

You can use the MISSING function to check for missing numeric and character values. This function returns 0 (false) or 1 (true). Missing values have a value of `false` when you use them with logical operators such as AND or OR. You can use the MISSING function to eliminate errors, notes, and warnings in the SAS log that are caused by missing values.

In expressions, you can use the period (.) to denote missing numeric values, and two single quotation marks with no space (the empty string ' ') to denote missing character values.

For more information, see [“How DS2 Processes Nulls and SAS Missing Values” in SAS DS2 Programmer’s Guide](#).

TIP To specify an empty data grid that does not contain data or column metadata, use empty brackets ([]). An empty data grid is not considered missing. You cannot use the MISSING function to determine whether a data grid is missing or empty.

Examples: Condition Expressions

REASON IN ('HomeImp','DebtCon')

determines whether the value of the character variable REASON is in the set of values ('HomeImp' , 'DebtCon'). This expression returns 1 if the expression is true and 0 if it is false.

temp LIKE('Sm%')

determines whether the value of the character variable TEMP begins with the string *Sm*. This expression returns 1 if the expression is true and 0 if it is false.

DEBTINC = .

uses the missing value for numeric variables (a period) to determine whether the value of the numeric variable DEBTINC is missing. This expression returns 1 if the expression is true and 0 if it is false.

MISSING(make)

uses the MISSING function to determine whether the value of the character variable MAKE is missing. This function can also be used with numeric variables. This expression returns 1 if the expression is true and 0 if it is false.

MAKE = "

uses the empty string (two single quotation marks with no space) to determine whether the value of the character variable MAKE is missing. This expression returns 1 if the expression is true and 0 if it is false.

VALUE >= 120000 AND DEBTINC ^= . AND (DEROG= 0 OR DELINQ= 0)

determines whether all of the following conditions are true:

- The value of the variable VALUE is greater than or equal to 120000.
- The value of the variable DEBTINC is not missing.
- The value of either of the variables DEROG or DELINQ is 0.

This expression returns 1 if all of these conditions are true and 0 if any of these conditions are false.

Examples: Action Expressions

address='1234 Fabric Road'

assigns the character string *1234 Fabric Road* to the variable named ADDRESS.

name=UPCASE(name)

uses the UPCASE function to make the value of the variable NAME uppercase.

quote='I said "What did they say?"'

assigns the string *I said "What did they say?"* to the character variable named QUOTE.

randomInt = INT(RAND('UNIFORM'))

uses the RAND function to generate a random number, which returns a decimal number. The decimal number is passed to the INT function. The INT function truncates the fractional portion of the number and returns the whole number portion.

expYear=substr(expDate,length(expDate)-3,4)

extracts the last four characters of the value of the variable EXPDATE and assigns them to the character variable EXPYEAR. This expression works for all date formats that end with the year, such as *04/02/2022*, *April 2, 2022*, or *02APR2022*.

See Also

- “Using the Expression Editor” on page 535
- “Data Grid Functions” in *SAS Intelligent Decisioning: Using Data Grids*

Delete Expressions

To delete a condition or action expression, click  for that expression.

Managing Rules in a Rule Set

Duplicate a Rule

- 1 Click  for the rule that you want to duplicate, and select **Duplicate**. The Duplicate Rule window appears.
- 2 Enter a name for the duplicate rule if you do not want to use the default name.
- 3 Do one of the following:
 - If you are duplicating a rule that contains only an IF statement or only an IF-THEN statement, click **Duplicate**.
 - If you are duplicating a rule that contains an IF-THEN statement and either an ELSE clause or an OR clause, you can choose to duplicate only the IF-THEN statement or to duplicate the entire rule. Click **Duplicate IF** to duplicate only the IF-THEN statement, or click **Duplicate All** to duplicate the entire rule.

Delete a Rule

To delete only a condition or an action for a rule, click  for the condition or action. If you delete all of the condition expressions for a rule, the remaining action expressions are converted to standalone assignment statements.

Note: When you delete an IF clause and the remaining clause becomes an assignment statement, the rule name that was associated with the IF clause is dropped.

To delete an entire IF-THEN rule:

- 1 Click  for the rule, and select **Delete**.
- 2 (Optional) If the rule contains an IF-THEN statement and either an ELSE clause or an OR clause, you can choose to delete only the IF-THEN statement or to delete the entire rule. Click **Delete All** to delete the entire rule, or click **Delete IF** to delete only the IF-THEN statement. If you select **Delete IF**, then the ELSE or OR clause becomes an assignment statement if it does not have any condition expressions. If it has a condition expression, it becomes a new IF rule.

To delete an entire ELSE clause, right-click on the ELSE keyword, and select **Delete the selected object**.

Edit a Rule Name or Description

- 1 Do one of the following:
 - To edit the name or description of an IF-THEN rule, click  for the rule, and select **Edit rule information**.
 - To edit the name or description of an ELSE or OR clause, right-click on the **ELSE** or **OR** operator, and select **Edit rule information**.
- 2 In the Edit Rule window, enter the new name or description, and click **Save**.

TIP Rule names for ELSE and OR clauses do not appear in the rule set editor.

Reorder Rules

You can move individual conditions or actions up or down within an IF clause, an ELSE clause, or an OR clause, but you cannot move individual conditions or actions to a different rule.

The  and  buttons move the selected object. You can select an individual condition or action, or you can select an entire rule, then use the  and  buttons to move the selected item.

Copy a Rule Set URL

To create a link for external documentation that automatically opens a rule set in SAS Intelligent Decisioning, open the rule set, click , and select **Copy rule set URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Rule Set Content

- 1 Select the objects that you want to compare.
 - To compare the contents of two different rule sets, select the rule sets in the category view, click , and select **Compare object contents**.
 - To compare the contents of two versions of the same rule set, open the rule set, click  on the **Versions** tab, and select **Compare object contents**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Object Contents window.

By default, the **Rule Sets** tab displays a simplified version of the rules that are different in each object. To display a simplified version of all of the rules in each object, click **Show All**.

TIP Click  beside a rule set name to display its location.

SAS® Intelligent Decisioning - Build Decisions

< Rule Sets Opened items (1)

Compare Object Contents

Export ×

Rule set Variables

Show differences Show all

High_Ratio (1.0) ⓘ

Approve

```
IF DELINQ <= 1
AND DELINQ ^= .
AND CLAGE >= 290
THEN REJECT=0
```

Else

```
ELSE DELINQ > 1
AND CLAGE < 178
THEN REJECT=1
```

Low_Ratio (1.0) ⓘ

Approve

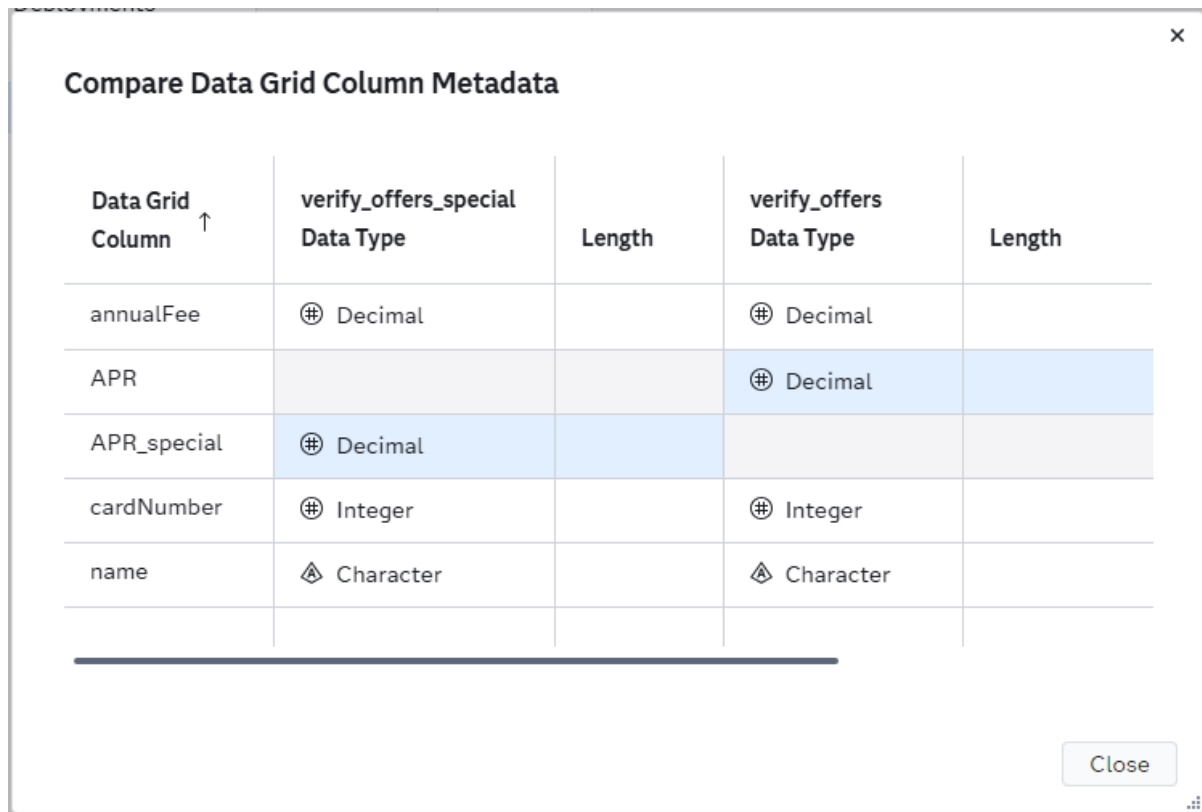
```
IF VALUE >= 120000
AND REASON IN 'HomeImp','DebtCon'
AND CLNO < 15
THEN REJECT=0
```

Reject

```
IF VALUE < 120000
AND DEROG > 2
AND DELINQ >= 2
THEN REJECT=1
```

- 3 (Optional) Click the **Variables** tab to display the properties of the variables in the two objects. This tab displays the name, type, initial value (if a value is defined), and length of each variable in both objects. The tab also indicates whether the variable is an input variable or an output variable. If the variable list for both objects is the same, this tab displays a message stating that the two objects are identical.

If either of the objects that you are comparing contain a data grid variable, you can click ⓘ beside the variable name to display the data type and length of each data grid column.



Data Grid Column ↑	verify_offers_special Data Type	Length	verify_offers Data Type	Length
annualFee	⊕ Decimal		⊕ Decimal	
APR			⊕ Decimal	
APR_special	⊕ Decimal			
cardNumber	⊕ Integer		⊕ Integer	
name	⚠ Character		⚠ Character	

- 4 (Optional) Click **Export** to export the results of the comparison to a PDF file. The Export Comparison Results window appears.
- 5 (Optional) Select the information that you want to export, and click **Export**. To export the information on a specific tab, select the tab name. To export the data type and length of each column in data grid variables, select **Data grid metadata**. If you want the PDF file to display only the differences between the two objects, select **Show differences**. To display all of the objects' information, select **Show all**.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Compare Rule Set Code

You can compare the generated code of two different rule sets, or you can compare the generated code of two different versions of the same rule set.

- 1 Select the objects that you want to compare.
 - To compare the generated code of two different rule sets, select the rule sets in the category view, click **⋮**, and select **Compare code**.

- To compare the generated code of two versions of the same rule set, open the rule set, click  on the **Versions** tab, and select **Compare code**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window and highlights the differences.
- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Generate PDF Documentation for a Rule Set

You can generate detailed documentation for a rule set as a PDF document. The PDF includes the rule set properties, details about each of the rules, and a table of the variables that are used in the rule set. You can download additional documents for lookup tables that are used in the rule set.

- 1 Open the rule set.
- 2 Click , and select **Create document**. The Create Document window appears.

TIP If this option is disabled, there might be unsaved changes. Click .

- 3 Enter a name for the document if you do not want to use the default name.
- 4 (Optional) Select **Show common rule details** to include the properties and rules of any common rule sets that are used in the rule set for which you are generating the PDF document.
- 5 (Optional) Select **Choose additional documents to download** to display a window from which you can download additional documents. You can download documents for any lookup tables that are used in the rule set.
- 6 Click **Create**. SAS Intelligent Decisioning creates the PDF. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

If you selected the option **Choose additional documents to download**, the Download Additional Documents window appears.

- 7 (Optional) Click **Download** for each additional document that you want to download.
- 8 (Optional) Click **Close** to close the Download Additional Documents window.

Undoing and Redoing Rule Set Changes

You can use the Undo and Redo buttons,  and , to undo and redo any change you make to a rule set in an editing session. You can undo saved changes, but you cannot undo any changes that you made in previous editing sessions.

If you click  to refresh the rule set editor, the changes that you have made in the current session are lost and cannot be redone. If you undo some changes and then enter new edits, the changes that were undone are lost and cannot be redone.

Managing Rule Sets

Duplicating Rule Sets

About Duplicating Rule Sets in Folders for Which the Check-Out and Commit Feature Is Enabled

When you duplicate a rule set, SAS Intelligent Decisioning first creates an empty rule set, and then updates the rule set to add the individual rules. The default permissions for the Decision Repository enable users to create new objects, but the permissions do not include the ability to update the objects. In order to duplicate an object that is in the Decision Repository, you must check out the object.

If your site has created additional folders for which the check-out and commit feature is enabled, and if those folders have been assigned the same permissions as those assigned to the Decision Repository, you must also check out objects in those folders before you can duplicate them.

When you duplicate an object that is checked out, the duplicate object is not automatically checked out.

Duplicate Rule Sets

Note: You cannot duplicate a rule set if it is open.

To duplicate a single rule set:

- 1 In the Rule Sets view, select the rule set that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Rule Set window appears.
- 3 Enter a new name for the duplicate rule set.
- 4 (Optional) Enter a description for the rule set.
- 5 Select the version of the rule set that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate rule set.
- 7 Click **Duplicate**.

To duplicate multiple rule sets:

- 1 In the Rule Sets view, select the rule sets that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the rule sets and appends `_Copy_n` to the names of the duplicate copies.

Note: When you duplicate a rule set, SAS Intelligent Decisioning creates a relationship between the original rule set and the duplicate rule set. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the rule set before you paste the new content.

Delete Rule Sets

Note: You cannot delete a rule set if it is open.

In the Rule Sets view, select the rule sets that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the rule sets to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename Rule Sets

Note: You cannot rename a rule set if it is open.

- 1 In the Rule Sets view, select the rule set that you want to rename.
- 2 Click  and select **Rename**. The Rename window appears.
- 3 Enter a new name for the rule set, and click **Rename**.

Move Rule Sets to a Different Folder

- 1 In the Rule Sets view, select the rule sets that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the rule sets, and click **OK**.

Managing Versions of Rule Sets

Set the Displayed Version

The displayed version is the version whose information is displayed on the other tabs, such as the **Properties**, **Rule set**, and **Variables** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You

cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Rule Set Version” on page 81](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.
- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Create a New Version Automatically When You Publish a Rule Set

You can automatically create a new version of a rule set when you publish the current version by selecting **Publish and lock**. For information, see [“Publish Rule Sets” on page 85](#).

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the rule set whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the rule set whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target rule set into which you want to paste the contents of the version. You can paste version contents only into an object of the same type. If the rule set from which you are copying the version is a filtering rule set, then you must paste the contents into another filtering rule set. If it is an assignment rule set, then you must paste the version contents into another assignment rule set.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target rule set. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

TIP The input and output designations for a variable on the **Variables** tab for the new version are removed, and the variable is treated as a temporary variable in the following situations:

- The new version of the target object does not use a variable that is used in an earlier version.
- The source and target object originally had a variable of the same name, and you deleted the variable from the source object before you copied it into the target object.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see “[Default Permissions](#)” in *SAS Intelligent Decisioning: Administrator's Guide* and “[Folder-Based Permissions](#)” in *SAS Intelligent Decisioning: Administrator's Guide*. By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of rule sets, the [sas.businessrules.deleteVersions](#) configuration option must be turned on.

On the object's **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Upgrade Decisions to Use a New Version of a Rule Set

If you create a new version of a rule set that is already used in decisions, you can upgrade the decisions to use the new version of the rule set.

- 1 On the **Versions** tab for the rule set, click , and select **Upgrade decisions**. The Upgrade Decisions window appears. This window lists all of the decisions that include the rule set.
- 2 In the **Version to upgrade to** field, select the version of the rule set to which you want to upgrade the decisions.
- 3 Select **Automatically map variables** if you want SAS Intelligent Decisioning to automatically map new rule set variables in the decisions where the rule set is used.

For information about how SAS Intelligent Decisioning maps variables, see [“About Decision Variables and Mapping” on page 436](#).

- 4 Select the check boxes for the decisions that you want to upgrade, and click **Upgrade decisions**.

TIP To update all of the objects that are used in a decision, see [“Update Decisions to Use New Object Versions” on page 443](#).

Determine Which Objects Use a Rule Set

To list the objects that use a specific rule set:

- 1 On the **Rule sets** category page, select the check box for rule set, click , and select **View used by report**. The All Objects that Use the Selected Item window appears. By default, this window lists the objects that use the latest version of the selected rule set.
- 2 (Optional) Select the specific version for which you want to view the Used By report.

To view the list of all objects that use any version of the selected rule set, clear the value in the **Version** field.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click ⓘ next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Rule Set Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.businessrules.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a rule set version, the **Check Out** button for that rule set is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Rule Set Versions” on page 80](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent

Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Rule Sets category view shows two rule sets with the same name, but the folders listed in the **Location** column differ for each rule set. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

You cannot publish the checked-out version that is in **My Folder**. To publish a version with your changes, you must commit your changes, and publish the committed version.

Checking Out a Rule Set from within a Decision

See “[Checking Out and Committing Objects from within A Decision](#)” on page 464 for information.

Checking Out and Committing a Common Rule Set from within an Assignment Rule Set

You can check out a common rule set from within an assignment rule set if the following conditions are true:

- The assignment rule set is checked out.
- The common rule set is stored in a folder for which the check-out feature is enabled.
- The assignment rule set uses the latest version of the common rule set. (See [Add a Common Rule Set Reference to an Assignment Rule Set on page 46.](#))

To check out or commit a common rule set from within a checked-out assignment rule set, click  for the common rule set reference, and select **Check out common rule set** or **Commit common rule set**.

When you check out the common rule set from within a checked-out assignment rule set, SAS Intelligent Decisioning adds an asterisk in front of the common rule set name, and the assignment rule set is modified to use the checked-out copy of the common rule set.

To open the checked-out common rule set from within the assignment rule set, click  for the common rule set reference, and select **Open common rule set**.

If you make changes to the common rule set and save it, you can click  in the assignment rule set to display the changes in the rule set editor for the assignment rule set. When you commit the assignment rule set, all checked-out common rule sets that are used in the assignment rule set are also committed. The newly committed version of the assignment rule set is modified to use the newly committed versions of the common rule sets. Any additional assignment rule sets that use the same common rule sets and that are still checked out are also modified to use the newly committed versions of the common rule sets.

Checking Out and Committing a Lookup Table from within a Rule Set

You can check out a lookup table from within a rule set if the following conditions are true:

- The rule set is checked out.
- The lookup table is stored in a folder for which the check-out feature is enabled.
- The rule set does not have any unsaved changes. If the rule set contains any unsaved changes, the **Check out lookup** option is disabled.

To check out or commit a lookup table from within a checked-out rule set, right-click on the LOOKUP or LOOKUPVALUE expression, and select **Check out lookup** or **Commit lookup**.

When you check out a lookup table from within a rule set, SAS Intelligent Decisioning adds an asterisk in front of the checked-out lookup table, and rule set is modified to use the checked-out copy of the lookup table.

To open the checked-out lookup table, click  to navigate to the Lookup Tables category view, click , and open the lookup table in My Folder.

Commit the lookup table from within the rule set. When you commit the rule set, all of the checked-out lookup tables that are used in the rule set are also committed. The newly committed versions of the lookup tables are not automatically activated when they are committed. You must activate the newly committed versions of the lookup tables. For more instructions, see [“Activate a Lookup Table” on page 169](#).

Rule sets that are not checked out always use the activated versions of lookup tables. If you do not activate the newly committed versions of the lookup tables, all rule sets that use the lookup tables will continue to use the previously activated versions of each table.

To cancel the check out of a lookup table from within a rule set, right-click on the LOOKUP or LOOKUPVALUE expression, and select **Cancel check out**.

Committing a Lookup Table Together with a Rule Set

If you check out a lookup table from within a rule set and then commit the rule set, the lookup table is also committed. For more information, see [“Checking Out and Committing a Lookup Table from within a Rule Set” on page 79](#).

Alternatively, you can check out a rule set that uses a lookup table, open the lookup table in the **Lookup tables** category view and check it out, then change the reference in the rule set to use the checked-out version of the lookup table in *My Folder*. The lookup table is committed when you commit the rule set.

The newly committed version of the lookup table is not automatically activated when they are committed. You must activate the newly committed version of the lookup table. For more instructions, see [“Activate a Lookup Table” on page 169](#).

Concurrently Checking Out and Committing Rule Set Versions

The ability for multiple users to check out the same rule set at the same time is controlled by the [sas.businessrules.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's *My Folder* location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Rule Set Version

Note: If you are checking out a common rule set or a lookup table from within an assignment rule set, see [“Checking Out and Committing a Common Rule Set from within an Assignment Rule Set” on page 78](#) or [“Checking Out and Committing a Lookup Table from within a Rule Set” on page 79](#) for information.

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 83](#).

- 4 On the **Versions** tab, click **Commit**. The Commit Rule Set Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 7 (Optional) To add the tags in the **Version tags** field to all of the objects that you are committing, select the **Apply tags to all committed components** check box.

- 8 (Optional) Enter information about the new version in the **Notes** field.
- 9 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

TIP If you or SAS Intelligent Decisioning have updated the assignment rule set to use any common rule sets or lookup tables that you have checked out, then SAS Intelligent Decisioning also commits those common rule sets and lookup tables. See [“Checking Out and Committing a Common Rule Set from within an Assignment Rule Set” on page 78](#) and [“Checking Out and Committing a Lookup Table from within a Rule Set” on page 79](#) for more information.

Determine Whether an Object Is Checked Out

The **Checked Out** column in the category view displays a check mark if the object is checked out.

If you are editing an object that is stored in a checkout-enabled folder, the object name in the editor is displayed with either (Read-Only) or (Checked Out) depending on whether it is checked out.

If you are editing a rule set that includes a [common rule set that has been checked out from within the assignment rule set](#), an asterisk appears before the common rule set name in rule set editor.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

How you undo the checkout of an object depends on how the object was checked out.

If you [checked out an object from within a decision](#), click  on the object's node in the decision diagram, and select **Cancel checkout**.

If you [check out a common rule set from within an assignment rule set](#), click  in the assignment rule set editor to open the menu for the common rule set, and select **Cancel check-out**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Publishing a Rule Set

Introduction to Publishing

Publishing content makes it available to other applications. Publishing a rule set creates an entity that can be managed and run in another environment. When you publish content, SAS Intelligent Decisioning generates code for that content and writes it to the destination. The following table describes what form the generated code takes for each destination type.

The publishing destinations that are available to you depend on what is configured at your site. See [SAS Viya Platform: Publishing Destinations](#) for more information.

Table 3.4 What SAS Intelligent Decisioning Publishes to Each Destination Type

Destination Type	What SAS Intelligent Decisioning Does
SAS Cloud Analytic Services (CAS), Teradata, or Apache Hadoop	Adds a row to the model table for the destination. The rows in the model tables are callable REST API endpoints, independent of SAS Intelligent Decisioning.
SingleStore	Publishes the decision as an embedded process with the specified published name. You can run the published decision as a job in SAS Data and AI Studio and SAS Job Execution Web Application. Running the decision invokes SAS Embedded Process, which is installed in SingleStore.
SAS Micro Analytic Service	Writes a Micro Analytic Service module in the service. The Micro Analytic Service modules are callable REST API endpoints, independent of SAS Intelligent Decisioning.
Git	Creates a directory in the remote Git repository with the same name as the published object and writes the generated code to a file named <code>scoreResource.txt</code>. You can use the SAS Intelligent Decisioning Git Deployment CLI to deploy content from the remote repository to destination of the type for which the Git repository was configured. The destination type is either a CAS destination or to a SAS Micro Analytic Service destination. The deployed content is a callable REST API endpoint. For more information, see “Configure Git Publishing Destinations” in SAS Viya Platform: Publishing Destinations.
Container destinations	Creates a SAS Container Runtime container and writes it to the destination. The SAS Container Runtime containers are callable REST API endpoints, independent of SAS Intelligent Decisioning. Note: The container destination types to which you can publish content from SAS Intelligent Decisioning are Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and private Docker repositories.

Where Can I Publish Rule Sets?

The destinations to which you can publish a rule set depend on the destinations that are available, the rule set type, and the rules that are used in the rule set. Your

system administrator determines which publishing destinations are available. The following table shows the destination types to which you can publish a rule set that contains the given rule types.

Table 3.5 Rule Types and the Destinations to Which You Can Publish Rule Sets

Rule Set Contents	SAS Micro Analytic Service and Compatible Git Destinations	SAS Cloud Analytic Services (CAS) and Compatible Git Destinations	Teradata and Apache Hadoop Destinations	Container Destinations
Assignment rule sets that do not use advanced lists ¹	yes	yes	yes	yes
Assignment rule sets that use advanced lists ⁷	yes ⁴	yes ⁶	yes ⁶	yes ⁵
Filtering rule sets	no	yes ³	yes	no
Rule sets that use SAS Data Quality functions	no	yes	yes ²	yes

¹ Rule sets and nested decisions can use lookup tables, custom functions, and global variables.

² Rule sets that use SAS Data Quality functions fail if they are run inside the database.

³ You can publish the rule set to CAS but not to GIT destinations.

⁴ Redis instances that are configured with Transport Layer Security (TLS) are not supported.

⁵ The Redis instance must be configured with Transport Layer Security (TLS).

⁶ The `sas.businessrules.inlineAdvanceListValues` configuration option must be set to Yes. For more information, see [“sas.businessrules” in SAS Intelligent Decisioning: Administrator’s Guide](#).

⁷ Rule sets that are published to SAS Micro Analytic Service destinations or to container destinations can reference multiple advanced lists. Rule sets that are published to other destination types can reference only one advanced list.

Note: You cannot publish rule sets to Databricks destinations.

For more information about publishing destinations, see [“Configuring Publishing Destinations” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Publish Rule Sets

- 1 Open the rule set. By default, the version that is published is the displayed version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 (Optional) On the **Versions** tab, [click the version number](#) to change the displayed version to the version that you want to publish.
- 3 Publish the selected version.

Note: By default, SAS Intelligent Decisioning displays a **Publish** button. If your administrator has turned on the [requirePublishAndLock configuration option](#), the **Publish and Lock** button replaces the **Publish** button, and the **Publish and lock** menu option is removed.

Note: If the decision is being developed in a workflow that controls whether the **Publish** or **Publish and lock** button is enabled, you might need to claim and complete a user task that enables the button before you can publish the decision. For more information, see [“Controlling Publishing from within a Workflow” on page 513](#).

Select the appropriate option:

- Click the **Publish** or **Publish and lock** button, depending on which button is available. The **Publish** button publishes the selected version without locking it or creating a new minor version. The **Publish and lock** button locks and publishes the selected version and creates a new minor version.
 - If the `requirePublishAndLock` configuration option is turned off, you can use the **Publish and lock** menu option to lock and publish the selected version, and simultaneously create a new minor version. Click , and select **Publish and lock**.
- 4 In the Publish Rule Sets window, select the destination to which you want to publish. The publishing destinations that are available to you depend on what is configured at your site. See [SAS Viya Platform: Publishing Destinations](#) for more information.
 - 5 (Optional) Enter any tags that you want to associate with the published object. For each tag, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').
-

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of

existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

-
- 6 (Optional) In the **Items to Publish** section, complete the following steps for each item that you are publishing:
- a Edit the **Published name** if you do not want to use the default name for the published module. The maximum length for all published names is 100 characters. Character restrictions differ depending on your destination. See [Table 3.6](#).

Table 3.6 Requirements and Restrictions for Published Names

Destination	Requirements And Restrictions
Container destinations	For Amazon Web Services (AWS) destinations, the published name can contain only alphanumeric characters. For other container destinations, the published name must start with a letter, a number, or a period. The name can include dashes, a single underscore, or a double underscore. It cannot contain other special characters, spaces, or multi-byte characters. Note: The name that you enter is prepended with a forward slash (/) and assigned to the SAS_SCR_APP_PATH environment variable. The value of the environment variable determines the published module name. For more information, see “Changing the Endpoint Name for a Container” in SAS Container Runtime: Programming and Administration Guide.
Git	The published name cannot contain colons (:), forward slashes (/), single quotation marks ('), or double quotation marks (").
SAS Micro Analytic Service	The published name cannot contain the following characters: ! @ # \$ % ^ & * () = ~ ` \ / . { } " ' ; -
SAS Cloud Analytic Services (CAS)	The published name cannot contain single or double quotation marks.
Apache Hadoop	The published name cannot contain colons (:) or double quotation marks.
Teradata	The published name must start with a letter or an underscore. It cannot contain spaces, multi-byte characters, or special characters other than the underscore.

- b If you have previously published the rule set, click the switch to turn on the **Replace item with the same name** option in order to replace the previously published rule set of the same name in the same destination.
- c Select the **Rule-fired tracking** check box if you want the published rule set to generate rule-fired data.

TIP This rule-fired data is recorded in the ruleFiredFlags column in the output table. The rule-fired data that is recorded when you select **Record rule-fired data** for a record contacts node is recorded in the subject contact history.

This option is disabled if you are publishing content to a SAS Micro Analytic Service destination, a Git destination, or a container destination.

- 7 Publish the rule set.

The button that is available depends on the option that you selected in [Step 3](#).

Publish

publishes the rule set.

Publish and lock

locks and publishes the rule set and creates a new minor version.

The Publishing Results window appears. It displays the names of the published items, their status, and a link to the log that was generated during the publishing process.

Note: In order for content that is published to a SAS Cloud Analytic Services (CAS) destination to be available to other applications, the CAS destination table must be reloaded. For other destinations, the destination table does not need to be reloaded.

If you are publishing content to a SAS Cloud Analytic Services (CAS) destination, SAS Intelligent Decisioning automatically reloads the CAS destination table. If another user is executing the code for an item that was previously published to CAS while the destination table is being reloaded, reloading the table might cause temporary problems with accessing the table content. After the table is reloaded, all authorized users can access all items in the table.

- 8 After the status changes to **Published successfully**, click **Close** to close the Publishing Results window.

TIP To view the publishing history for a rule set, click the **History** tab.

- 9 (Optional) Click **x** to close the rule set.

Testing Rule Sets

Ways to Test a Rule Set

There are three types of tests:

Basic test

executes the rule set in the SAS Cloud Analytic Services (CAS) destination using the input table that you specify. You can also specify a debugging variable. For more information, see [“Create and Run a New Test” on page 89](#).

Scenario test

enables you to enter specific input values and the output values that you expect the test to generate. A scenario test identifies differences between the output that you expect to see and the actual output that is generated when the test is run. You can also compare the test definitions and test results of different scenarios. Scenario tests are also run in CAS. For more information, see [“Create and Run a Scenario Test” on page 93](#).

Publishing validation test

executes the rule set in a publishing destination using the input table that you specify. When you publish the rule set, a validation test is automatically defined for that rule set in that destination. For more information, see [“Validate a Published Rule Set” on page 100](#).

Test a Rule Set

Create and Run a New Test

Testing a rule set is optional, but doing so is a best practice. Testing enables you to discover any problems before the rule set is published and incorporated into a production system.

IMPORTANT If you are testing a rule set that uses functions that are defined in a custom context file, verify that the context file is specified in the **Test custom context** field on the **Properties** tab before running the test.

If you are testing a rule set that uses a lookup table and both the rule set and the lookup table are checked out, the rule set test uses the checked-out

version of the lookup table. If the lookup table is not checked out, the test uses the activated version of the lookup table.

- 1 On the **Scoring** tab, click the **Tests** tab.
- 2 Click **New test**. The New Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 5 Click  for the **Location** field, and select the folder in which you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 6 Click  for the **Input table** field, select the input table for the test, and click **OK**.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name: "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name: "HMEQ". ["How to Enter Searches for Information Assets in SAS Data Governance" in SAS Data Governance: User's Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see ["Importing and Loading Data" on page 12](#).

- 7 Verify or change the variable mappings. To run a full test, map all of the input variables to columns in the input table that you selected for the test. To run a partial test, you can map only the input variables that are needed for the test. SAS Intelligent Decisioning automatically maps the input variables in the rule set to columns in the input table when the names and data types of the variables match those of the table columns.

If any input variables are not mapped to input columns or to static values, the application displays the warning message `Input variables must be mapped`

to table columns. At run time, SAS Intelligent Decisioning assigns missing values to input variables that are not mapped.

You can change the automatic variable mappings in the Variable Mappings window.

To change variable mappings:

- a Click **Variables**. The Variable Mappings window appears. This window displays all of the rule set variables and the input table columns to which the rule set variables are mapped.

TIP Click **Show unmapped variables** to filter the list to show only the variables that have not been mapped to an table column.

- b For each input variable, select the table column to which the variable should be mapped. If the input table contains more than 15 columns, click **More columns** to display additional column names. Alternatively, for Decimal, Integer, and Character variables, you can select **Use value** for the table column, and specify a literal value in the **Value** column. When you are entering literal values, remember these rules:
 - For character variables, input values are limited to 10 Mb. Output values longer than 32767 characters will be truncated.
 - Do not enclose character strings in quotation marks.
 - To specify a missing value for character variables, select **Use value** and leave the **Value** column empty. When SAS Intelligent Decisioning generates code, it generates an empty string (' '). For numeric values, enter a period (.).

For more information about entering variable values, see [“The Properties of a Variable” on page 33](#).

- c Click **OK** to close the Variable Mappings window.
- 8 (Optional) Click **Advanced** to display the advanced options.
 - 9 (Optional) Click  for the **Output data library** field, and select the library to which you want to write the output of the test.
 - 10 (Optional) Enter a name for the test results output table in the **Output table name** field if you do not want to use the default name. The default name is `test-name_object-name_timestamp_output`.
 - 11 (Optional) For the **Target version**, select the version of the rule set that you want to test. The default version is the latest version.
 - 12 (Optional) Select the variable that you want to serve as an input debug variable. You can specify an input-only variable or an input-output variable. The rule set writes the name and value of this variable to the log for each input record that is processed. It writes the value just before the logic of the rule set is executed for the input record.

For more information, see [“Debugging Rule Set Tests” on page 92](#).

- 13 (Optional) Select **Preserve unmapped columns in the output table** if you want columns that are not mapped to an output variable to be written to the output table.
- 14 (Optional) If you are testing a filtering rule set, select **Keep filtered records in output table** to include records in the test output that would normally be excluded. When you select this option, SAS Intelligent Decisioning creates a new column in the output table named `_filter_`. If the value in this column is 0, then the record does not match any of the conditions in the rule set, and the record is filtered out of the output when the rule set is run in a production environment. If the value in this column is 1, then the record matches at least one condition in the rule set, and the record is included in the output.

Note: This option is ignored for assignment rule sets.

- 15 Click **Run** to save and run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 102](#).

- 16 Click  in the **Results** column to view the results of the test.
- 17 In the test results window, click **Test Results** in the navigation pane to display the URIs and other information for the test. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

Debugging Rule Set Tests

When you create a test, you can specify a variable to use as a debugging variable in the **Input debug variable** field. You can specify an input-only variable or an input-output variable. The rule set writes the name and value of this variable to the log

for each input record that is processed. It writes the value just before the logic of the rule set is executed for the input record. For more information, see [Step 12 of “Create and Run a New Test” on page 89](#).

When you specify an input debug variable, you can use the `sas.businessRules.messageOrder` configuration option to control whether the log messages are written as they are produced or after the rule set executes. For more information, see [“sas.businessrules.messageOrder” in SAS Intelligent Decisioning: Administrator's Guide](#).

When you specify an input debug variable, SAS Intelligent Decisioning automatically sets the maximum number of threads that can be allocated for the test to 1. Setting the thread count to 1 ensures that the variable's values are written to the log in the correct order and are not affected by different threads completing at different times.

To capture variable values for input-only or temporary variables after the rule set logic has executed for a specific record, you can specify that the variable is an output variable, and then re-run the test. Before publishing the rule set to a production environment, return the input and output settings for the variable to their previous settings. For more information, see [“Input Variables, Output Variables, and Temporary Variables” on page 35](#) and [“Edit Variable Properties” on page 41](#).

Test a Scenario

Create and Run a Scenario Test

IMPORTANT If you are testing a rule set that uses functions that are defined in a custom context file, verify that the context file is specified in the **Test custom context** field on the **Properties** tab before running the test.

If you are testing a rule set that uses a lookup table and both the rule set and the lookup table are checked out, the rule set test uses the checked-out version of the lookup table. If the lookup table is not checked out, the test uses the activated version of the lookup table.

- 1 On the **Scoring** tab, click the **Scenarios** tab.
- 2 Click **New test**. The New Scenario Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Click  for the **Location** field, and select the folder where you want to save the test definition.
- 5 Click  for the **Output table location** field, and select the data library where you want to save the test results.

- 6 (Optional) Select the version of the rule set that you want to test. The default version is the latest version.

Note: When you change the version, variables for the new version are reloaded and any variable values that were entered for the test are cleared.

- 7 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 8 Enter the values that you want to use for each input variable. You do not have to enter values for every input variable. At run time, SAS Intelligent Decisioning uses missing values for input variables for which you do not specify a value.

Note: For character variables, input and output values longer than 32767 characters will be truncated. You cannot enter input values for variables of type binary or of type varying-length binary.

To enter values for the columns in a data grid variable:

- a Click  in the **Value** field. The Edit Data Grid window appears. One row is automatically added to the input data grid.
- b Enter values for each column in the data grid.
- c (Optional) Click  to add a new row, and then enter the values for each column.

Repeat this step for each row of values that you want to add to the data grid.

TIP By default, data grid column names appear across the top of the data grid view, and row numbers appear down the left side. You can click  to change the view of the data grid so that row numbers appear across the top and data grid column names appear down the left side.

- d Click **OK** to save the data grid values and close the Edit Data Grid window.

For more information about entering variable values, see [“The Properties of a Variable” on page 33](#).

- 9 (Optional) For each output variable, select the **Include** check box and enter the expected output value. The **Include** check box controls whether a variable's expected value is used to determine the status of a scenario test. If you select **Include** for a variable and the test does not return the expected value, the test status is set to Completed with warnings (). If you do not select the check box, the application ignores the expected value of that variable when it determines the status of the test.

To enter expected values for the columns in a data grid variable, click in the **Expected Output** field, and follow the instructions in [Step 8](#) for entering values for the columns in a data grid variable.

Note: A scenario test cannot verify issues with trailing spaces. For example, it cannot distinguish between a string that contains a single space ' ' and a string that contains three spaces ' ' '.

- 10 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 102](#).

- 11 Click  in the **Results** column to view the results of the test.
- 12 On the Test Results page, click **Scenario Test Results** in the navigation pane to display the URIs and other information for the test. Click **Input** to display the values of input variables and dynamic attributes. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

On the **Output** page, click **Show All** to display the expected and actual values of all output variables. Click **Show Differences** to display only the variables whose expected values do not match the actual values that were returned by the test.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Output** page, click **Export** to export the output table as a comma-separated values (CSV) file. On the **Log** page, click  to download the log file.

- 13 Click **×** to close test results and return to the **Scoring** tab.

Import Scenario Test Definitions

You can import scenario test values from a comma-delimited (CSV) file. Each line in the CSV file is imported as one scenario test. In the CSV file, add a column of values for each variable. In the header row, enter the names of the input variables and of the output variables with `_expected` appended to the name.

For example, suppose your scenario test has the input variables `policyholder`, `cscore`, and `claims`, and it has the output variables `eligible` and `policies`. An import file for this test might appear in a spreadsheet application as shown in the following figure.

	A	B	C	D	E	F	G	H	I	J
1	policyholder	cscore	claims	eligible_expected	policies_expected					
2	Smyth,Joe	784	0	TRUE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["4535					
3	Dupree, Marcel	803	2	FALSE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["9836					

The policies output variable in this example is a data grid variable. To enter values for data grid variables, use the JSON string format described in [“Introduction to Data Grids” in SAS Intelligent Decisioning: Using Data Grids](#).

The format for date and datetime variables depends on your locale. Use the same format that is created by the date and datetime pickers when you click  or  to enter initial values for custom variables on the **Variables** tab.

To import scenario test definitions:

- 1 On the **Scoring** tab, click the **Scenarios** subtab, and then click **Import scenarios**. The Import Scenarios window appears.
- 2 In the **Import from** field, click  and select the CSV file that contains the scenario test values.

Note: The import file is limited to 10 MB.

- 3 Select **Select encoding** to select the encoding of the CSV file from the predefined list, or select **Enter encoding** to enter a custom encoding.
- 4 Select or enter the encoding of the CSV file.
- 5 Enter a prefix for the scenario test definitions. SAS Intelligent Decisioning appends a number to this prefix for each test definition. The prefix can include double-byte characters and special characters, including single quotation marks.
- 6 (Optional) Click  for the **Folder location** field, and select the folder where you want to save the test definitions.
- 7 Click  for the **Output table location** field, and select the folder where you want to save the test results.
- 8 Click **Import**.

Compare Different Scenarios

You can display the scenario definitions or results of two or more tests side-by-side. On the **Scenarios** tab, select two or more tests, click , and select one of the following options:

Compare ⇒ **Definitions**

displays the input values and the expected output values that you entered for both tests. To edit the input values for a scenario test definition, click  next to the test name under **Input Table**. To edit the expected output values for a test, click  next to the test name under **Output Table**.

Compare ⇒ **Results**

displays the input values and the actual output values that were generated by the test. To display both the expected values and the actual values in the output table, select **Display expected values**. For each variable for which you selected **Include** in [Step 9 on page 94](#), the application highlights the variable if the actual and expected values do not match.

Click **Export** to export the results comparison as a comma-separated values (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Run a Rule-Fired Analysis

If a rule's conditions evaluate to true, then the rule is said to have *fired*. Rule-fired data includes summary information about how many times each rule fired and detailed information for each time that a rule evaluates to true. See [“How Rules Are Evaluated and When Rule-Fired Records Are Generated” on page 55](#) for more information.

Note: Rule-fired data is recorded only for rule sets, including rule sets in nested decisions and filtering rule sets that are included directly in a decision. Rule-fired data is not recorded for other objects, including filtering rule sets that are used as eligibility rule sets in treatments.

For filtering rule sets that are included directly in a decision, rule-fired data is recorded for an input record if the rule set does not filter out the record. When the rule set filters out an input record, rule-fired data is not recorded for that input record.

For rule sets that iterate over a data grid (in other words, the rule sets score the rows in the data grid), the rule-fired data indicates that the rules in the rule set fire once for the entire data grid instead of firing once for each row in the data grid.

If the `sas.decisions.nodetraces.includeRuleFiredPathTrackInfoInVariableAssignmentLogging` configuration option is turned off, you cannot run rule-fired analyses for scenario tests. For more information, see [“Decisions Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

TIP This rule-fired analysis uses the data that is in the ruleFiredFlags column in the test results output table. To analyze rule-fired data that is in the subject contact history, use the %DCM_GET_SUBJECTCONTACT_HISTORY and %DCM_RULEFIRE_DETAIL macros. For more information, see *SAS Intelligent Decisioning: Macro Guide*.

To run a rule-fired analysis:

- 1 In the test results window, click **Rule-Fired Analysis** in the navigation pane.
- 2 Click **Run Rule-Fired Analysis**. SAS Intelligent Decisioning analyzes the test results to determine which rules fired for each row in the input table, and displays the Analysis page.

The Analysis page displays the number of rules that fired for each output record that was generated by the decision. The number in the **Rules Fired Count** column is a link to more information. You can click on this link to display the rules that fired for that output row.

For example, the following displays shows the rule-fired analysis for the low_ratio rule set.

The screenshot shows the SAS Intelligent Decisioning - Build Decisions interface. The top bar indicates the current view is 'Rule Sets > Low_Ratio (1.0)'. The left navigation pane shows a tree structure with 'Test Results' and 'Rule-Fired Analysis' sections. The 'Rule-Fired Analysis' section is expanded, showing 'Analysis' as the selected option. The main content area displays the 'Analysis' page for 'Low_Ratio_Test'. It features a table titled 'Output Records' with the following data:

Rules Fired Count	REJECT	REVIEW
1	1	
1	0	
1	0	
1	0	
1	0	
1	0	
1	1	
1	1	

Click Rule-Fired Analysis to display information about the tests such as the URIs for the test analyses and the paths to the rule-fired tables.

- 3 Click on a number in the **Rule Fired Count** column. SAS Intelligent Decisioning displays the Rule Fired Count window. This window shows which rules produced the selected output record.

Rule Fired Count

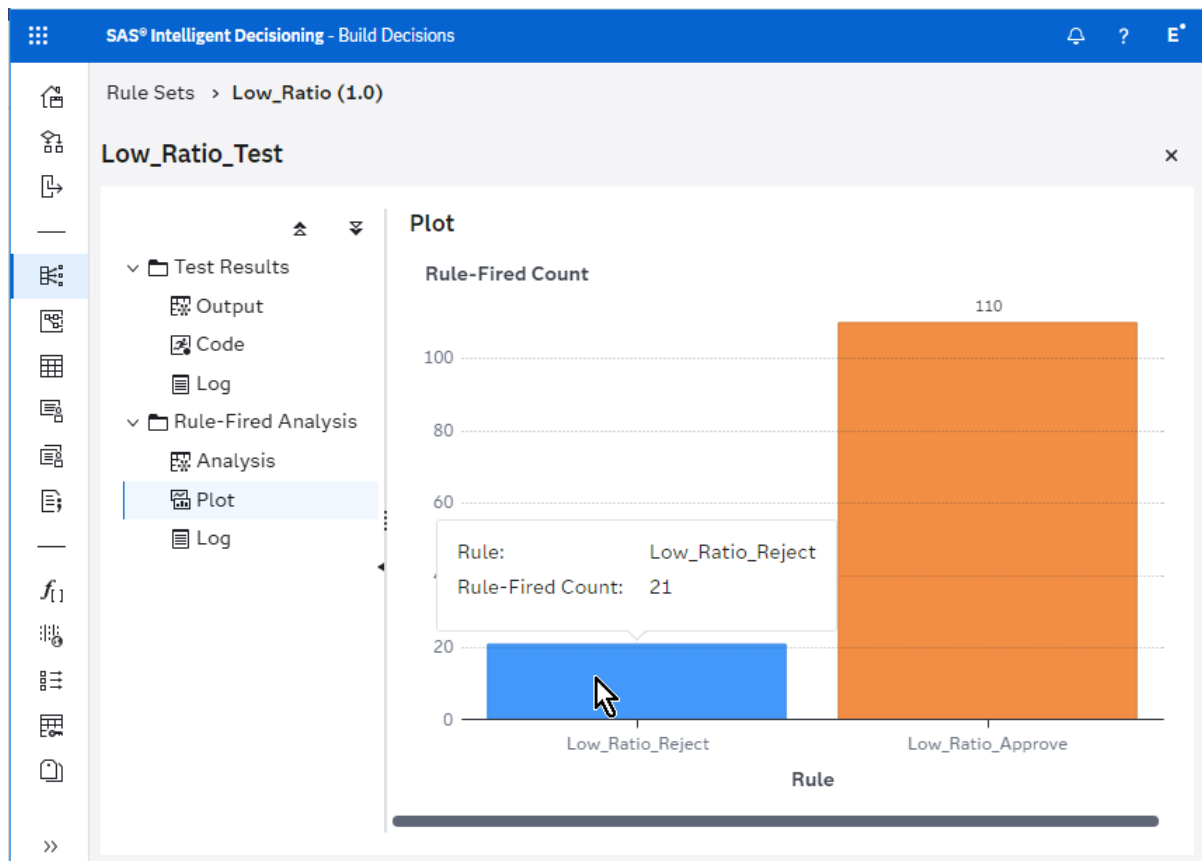
Details Analysis Output Columns

Rule Set Name	Rule Name	Order in Diagram ⓘ	Rule Logic ⌵
Low_Ratio	Approve	1	IF (VALUE >= 120000) AND (REASON in ('Homelmp', 'DebtCon')) AND (CLNO < 15) THEN REJECT = 0

Close

Note: The value in the **Rule Order** column is the ordinal number of the rule as it occurs in the rule set diagram. The values in this column do not indicate the order in which the rules in the rule set fired.

- 4 Click **Close** to close the Rule Fired Count window.
- 5 Click **Plot** in the navigation pane. SAS Intelligent Decisioning displays a bar chart that shows how many times each rule fired. Position your cursor over a bar to display the name of the rule and the number of times that the rule fired.



- 6 Click **Rule-Fired Analysis** in the navigation pane to display the URIs and other information for the rule-fired test.
- 7 Click **x** to close the rule set.

Validate a Published Rule Set

You can test the published rule set in the target publishing destination. When you publish the rule set to any destination type except Git, a validation test is automatically defined for that rule set in that destination. To run the publishing validation test:

- 1 On the **Scoring** tab, click the **Publishing Validation** tab. The  icon in the **Status** column indicates that the test is not ready to run. The  icon indicates that the test is ready to run.
- 2 Click on the test name. The Edit Publishing Validation Test window appears.

Note: To generate the name of the publishing validation test, SAS Intelligent Decisioning appends a timestamp to the rule set name. The timestamp indicates when the rule set was published.

- 3 (Optional) Click  in the **Location** field, and select the folder where you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 4 Click  in the **Input table** field, and select the input table for the test.

Note: If the input table contains a character column, and that column contains control characters in any row, do not use the table as input for publishing validation tests.

For character variables, output values longer than 32767 characters will be truncated.

If you are validating content that was published to SAS Micro Analytic Service, the time required to run the test depends on the number of worker threads on your system, the number of threads in the middle tier, and the network latency between CAS and the middle tier server. It is recommended that you select an input table with as few input records as needed to accurately test the published content. See [SAS Micro Analytic Service: Programming and Administration Guide](#) for more information.

- 5 (Optional) Expand the **Advanced** section, click  in the **Output data library** field, and select a library to store the validation test output data.
- 6 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 102](#).

- 7 Click  in the **Results** column to view the test results.
- 8 In the test results window, click **Test Results** in the navigation pane to display the URIs and other information for the test. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

- 9 Click  to close the test results.

Working with Test and Validation Test Output Data

After you run a test or a publishing validation test, you can work with the output table in other SAS applications to analyze the data, create and compare models, discover relationships hidden in the data, and generate reports based on the data.

Note: The actions available to you depend on the applications that are available at your site.

On the Test Results page, select the **Output** table in the navigation pane, click **Actions**, and select one of the following options:

Explore Lineage

opens SAS Lineage Viewer. SAS Lineage Viewer enables you to better understand the relationships between objects in your SAS Viya platform applications. These objects include data, transformation processes, reports, and visualizations. For more information, see [SAS Lineage: User's Guide](#).

Explore and Visualize

opens the output table in SAS Visual Analytics. SAS Visual Analytics enables you to create, test, and compare models based on the patterns discovered during exploration of the data. You can export the model before or after performing model comparison for use with other SAS products or to put the model into production. SAS Visual Analytics supports a range of visualization, discovery, and reporting features. For more information, see [Welcome to SAS Visual Analytics](#).

Access and Manage Data

opens the output table in SAS Data Hub. SAS Data Hub enables you to import and download tables, load tables into memory, unload tables, and more. For more information, see [SAS Data Hub: User's Guide](#).

Status Icons for Tests

Icon	Status
	The test is not ready to run. The test definition is not complete, or it might contain errors.

Icon	Status
	The test has been defined and can be run. Some input variables have not been mapped or have not been assigned a value, so the test might execute only a subset of the rules in the rule set.
	The test is defined correctly and is ready to run.
	The test is running.
	The test completed successfully.
	The test completed, but warnings were issued in the SAS log. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.
	The test did not run successfully. Check the SAS log for information. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.

Duplicate Tests

To duplicate a single test:

- 1 On the **Scoring** tab, click the **Tests** tab, the **Scenarios** tab, or the **Publishing Validation** tab, depending on the test that you want to duplicate.
- 2 Select the check box for the test that you want to duplicate, click  on the subtab, and select **Duplicate**.
- 3 In the Duplicate Tests window, enter a name for the duplicate test if you do not want to use the default name.
- 4 (Optional) Enter a description for the duplicate test.
- 5 Click **Duplicate**.

To duplicate multiple tests:

- 1 On the **Scoring** tab, click the **Tests** tab, the **Scenarios** tab, or the **Publishing Validation** tab, depending on the test that you want to duplicate.
- 2 Select the check boxes for the tests that you want to duplicate, click  on the subtab, and select **Duplicate**.

SAS Intelligent Decisioning duplicates the tests and adds `_Copy` to each filename.

Manage Comments for a Rule Set Test

You can associate comments and attachments with rule set tests.

To open the Comments properties pane, select the rule set test on the **Scoring** tab, and click  in the property pane.

To add a new comment, enter the comment in the text box and click **Post**.

To add an attachment to a comment, click , select the file that you want to attach, and click **Post**. (The attachment icon appears after you enter text in text box.) You cannot attach executable files such as BAT and EXE files.

To reply to a comment, click **Reply**, enter your reply in the text box, and click **Post**.

To edit an existing comment, click  for the comment, and select **Edit**. Edit the comment, and click **Post**.

To delete a comment, click  for the comment, and select **Delete**.

Executing Published Content

How you execute published content depends on the destination to which the content is published.

Executing Content Published to SAS Micro Analytic Service Destinations

The user who is executing the published content must be authenticated. In SAS Viya, authentication options vary, based on which interface and operating system are used in your environment. External mechanisms include direct LDAP authentication, host authentication, Kerberos, Security Assertion Markup Language (SAML), and OAuth 2.0 with OpenID Connect. For additional information, see [SAS Viya Platform: Authentication](#)

When a rule set is published from SAS Intelligent Decisioning to a SAS Micro Analytic Service destination, an EXECUTE step is created in the published module. For information about the request and response data formats used in this step, see [Execute a step](#) in the REST API documentation for the Micro Analytic Score API.

IMPORTANT If the SAS Micro Analytic Service configuration property `service.removeTrailingUnderscoresFromInputs` is set to **False**, add an underscore to the name of each input variable. If this option is set to **True**, do not add underscores. Your administrator can add this property to the `supplementalProperties` section in the `sas.microanalyticsservice.system` configuration definition in SAS Environment Manager. The default value for this option is **False**. For additional information, see [“sas.microanalyticsservice.system: supplementalProperties” in SAS Micro Analytic Service: Programming and Administration Guide](#).

Executing Content That Has Been Published to SAS Cloud Analytic Services Destinations

To execute content that has been published to SAS Cloud Analytic Services (CAS), use the CAS Model Publishing and Scoring action set. For example, the following code runs a model named `Evaluate_Loans` on the local CAS server.

```
/* Start a CAS session named _mmcas_. */
cas _mmcas_;

/* Add the formats library for lookup */
/* tables to your search path.          */
sessionProp.setFmtSearch / fmtLibNames="userformats3";

/* Create librefs for all existing caslibs so that they */
/* are visible in the SAS Data and AI Studio Libraries tree. */
caslib _all_ assign;

proc cas;
  /* Specify the session to use for the runModelLocal action. */
  session _mmcas_;

  /* Define the parameter list for the runModelLocal action. */
  /* Reload the destination model table ("targetCode") */
  /* before you execute the decision. */
  destination_model_table = "targetCode";
  destination_model_lib = "public";

  destination_model = "Evaluate_Loans";

  dp_inputTable="hmeq_test";
  dp_inputCASLib="public";

  dp_outputTable="hmeq_test_dm";
  dp_outputCASLib="public";

  parmlist = {
    modelTable={
```

```

        name=destination_model_table,
        caslib=destination_model_lib
    },
    modelName=destination_model,
    inTable={
        name=dp_inputTable,
        caslib=dp_inputCASLib
    },
    outTable={
        name=dp_outputTable,
        caslib=dp_outputCASLib
    }
};

/* Load the modelPublishing action set. */
loadactionset "modelPublishing";

/* Set the library search list for lookup tables. */
action setFmtSearch status=status / fmtLibNames="userformats3";
run;

/* (Optional) Display the list of libraries that will be searched.
*/
listFmtSearch result=r;
print "Format library search order:";
if (r.ListFmtSearch.nrows > 0) then
    do i=1 to r.ListFmtSearch.nrows ;
        print r.ListFmtSearch[i];
    end;
run;

/* Run the model locally on the CAS server. */
action runModelLocal submit result=r status=rc / parmlist;

/* Load the table action set. */
loadactionset "table";

/* Promote the output table */
table.promote /
    caslib=dp_outputCASLib
    name=dp_outputTable
    targetLib=dp_outputCASLib;
run;
quit;

/* Terminate the CAS session named _mmcas_. */
cas _mmcas_ terminate;

```

If SAS Data and AI Studio is licensed at your site, you can submit this code in SAS Data and AI Studio. To open SAS Data and AI Studio, click  and select **Develop Code and Flows**. For more information, see [SAS Data and AI Studio: User's Guide](#).

You can view additional examples of using this CAS action set to execute published content by viewing the test results that are generated by publishing validation tests. On the Test Results page for a rule set, click **Code** to display the code that was generated by SAS Intelligent Decisioning . For information about running

publishing validation tests and viewing the results, see [“Validate a Published Rule Set” on page 100](#).

For more information about CAS and the Model Publishing and Scoring action set, see the following documentation:

- [Getting Started with CASL Programming](#)
- [SAS Cloud Analytic Services: CASL Reference](#)
- “Model Publishing and Scoring Action Set” in [SAS Visual Analytics: Programming Guide](#)
- [SAS Cloud Analytic Services: User’s Guide](#)

Content Executed by Published Rule Sets

When you execute a published rule set, the version of the content that is executed depends on the publishing destination.

Destination Type	Content That Is Executed by Published Rule Sets
SAS Micro Analytic Service	For information about the value of global variables that are used in published content, see “Which Value Is Used for a Global Variable?” on page 181. Lookup tables are embedded in decisions if the lookupStaticBinding configuration option is turned on. If this option is off, lookup tables are not embedded, and published modules use newly activated versions of lookup tables. For more information, see “sas.businessrules.lookupStaticBinding” in SAS Intelligent Decisioning: Administrator’s Guide.
SAS Cloud Analytic Services	For information about the value of global variables that are used in published content, see “Which Value Is Used for a Global Variable?” on page 181. Lookup tables are locked if the lookupStaticBinding configuration option is turned on. If this option is off, lookup tables are not locked, and published modules use newly activated versions of lookup tables. For more information, see “sas.businessrules.lookupStaticBinding” in SAS Intelligent Decisioning: Administrator’s Guide.
Teradata, Hadoop, Git, Databricks, SingleStore, and Container Destinations	All of the published rule set’s content, including custom functions, lookup tables, and global variables is included inline in the published module. Updates to the objects used in the rule set are not used by the published module.

Importing and Exporting Rule Sets

About Importing and Exporting Rule Sets

You can import and export rule sets by using either of the following file types:

- Comma-delimited (CSV) files. The ability to import and export a rule set as a CSV file is for modifying an existing rule set. You can export an existing rule set, edit the CSV file, and import the updated CSV file. You can use the **Import** and **Export** buttons in SAS Intelligent Decisioning, or you can use the %DCM_EXPORT_RULESET and %DCM_IMPORT_RULESET macros. For more information, see the following topics:
 - [“Import a Rule Set from a CSV File” on page 109](#)
 - [“Export a Rule Set to a CSV File” on page 109](#)
 - [“%DCM_EXPORT_RULESET Macro” in SAS Intelligent Decisioning: Macro Guide](#)
 - [“%DCM_IMPORT_RULESET Macro” in SAS Intelligent Decisioning: Macro Guide](#)

When you use the **Import** button, the format of the CSV file is the same format that is used by the %DCM_IMPORT_RULESET macro. The rule set ID, rule set name, and folder path that is specified in the input CSV file must match the ID, name, and folder path that is specified in the rule set into which you are importing content. For more information, see [“Format of Rule Set CSV Input File” in SAS Intelligent Decisioning: Macro Guide](#).

- JavaScript Object Notation (JSON) files. If you do not need to edit the content of the CSV file, use SAS Environment Manager, the SAS Viya command-line interface (sas-viya), or the **Export objects to JSON files** and **Import objects from JSON files** options in SAS Intelligent Decisioning.

The SAS Viya CLI and SAS Environment Manager enable you to import new objects. For information about the CLI, see [SAS Viya Platform: Content Migration from SAS Viya 4](#). For information about using SAS Environment Manager, see [“Content Page” in SAS Environment Manager: User’s Guide](#) and [“Import Page” in SAS Environment Manager: User’s Guide](#). For information about using the JSON options in SAS Intelligent Decisioning, see [“Importing and Exporting Objects Using JSON Files” on page 13](#).

Import a Rule Set from a CSV File

IMPORTANT If you import rules from a CSV file into a rule set that already contains rules, the existing rules are replaced with the rules that are defined in the CSV file. To avoid replacing existing rules, create a new empty rule set, and then import the contents of the CSV file into the empty rule set. To append rules to a rule set, you can export the rule set, add content to the CSV file, and then re-import the CSV file.

- 1 Open the rule set into which you want to import the modified content.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click **Import**. If the rule set already contains rules, SAS Intelligent Decisioning asks you if you want to replace the existing rules. Select **Import** in the message window to replace the existing rules and continue the import process.
- 3 In the Import Rule Set window, click , and select the CSV file that you want to import.
- 4 Select or enter the encoding of the CSV file, and click **Import**.

Export a Rule Set to a CSV File

IMPORTANT Do not modify the file structure or the header row in the exported CSV file. You can modify the data values.

To export a rule set, open the rule set and click **Export**. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Working with Treatments and Treatment Groups

<i>About Treatments and Decisions</i>	112
<i>Example: A Decision That Includes a Treatment Group</i>	113
<i>About Attributes, Eligibility Rules, and Effective Dates</i>	116
Attributes and Attribute Aliases	116
Eligibility Rules	117
Effective Dates	117
<i>Arbitrating Treatments</i>	118
<i>About Channels</i>	118
<i>Define a Treatment</i>	119
<i>Define a Treatment Group</i>	120
<i>Activating a Treatment Group</i>	122
What Does Activating a Treatment Group Do?	122
Controlling Where Treatment Groups Are Activated and How They Are Used	123
Activate a Treatment Group	123
<i>Managing Attributes and Aliases</i>	124
About Attributes and Aliases	124
Define Custom Attributes	124
Add Attributes from a Different Treatment	125
Duplicate Attributes	125
Delete Attributes from a Treatment	126
View Attribute Aliases for a Treatment Group	126
Remove an Attribute Alias from a Treatment Group	126
Determine Which Treatments Use an Attribute	126
<i>Delete Treatments from a Treatment Group</i>	126
<i>Manage Eligibility Rules for a Treatment</i>	127
<i>Copy a Treatment or Treatment Group URL</i>	127
<i>Compare Treatment or Treatment Group Content</i>	128
<i>Compare Treatment Group Code</i>	130
<i>Generate PDF Documentation for a Treatment Group</i>	131

Managing Treatments and Treatment Groups	132
Duplicate Treatments or Treatment Groups	132
Delete Treatments or Treatment Groups	133
Rename a Treatment or Treatment Group	133
Move Treatments or Treatment Groups to a Different Folder	133
Managing Versions of Treatments and Treatment Groups	134
Set the Displayed Version	134
Create a New Version	134
Copy the Content of a Version	135
Delete a Version	136
Determine Which Objects Use a Treatment or Treatment Group	137
Checking Out and Committing Treatment or Treatment Group Versions	137
About Checking Out and Committing Versions	137
Checking Out a Treatment Group from within a Decision	139
Checking Out and Committing Treatments from within a Treatment Group	139
Concurrently Checking Out and Committing a Treatment or Treatment Group Versions	139
Check Out and Commit a Treatment or Treatment Group Version	140
Determine Whether an Object Is Checked Out	141
Determine Who Has a Version Checked Out	142
Opening the Original Object	142
Undoing a Check Out	142

About Treatments and Decisions

In SAS Intelligent Decisioning, you use treatments to define offers that you might want to present to subjects that contact your company as a result of an inbound marketing campaign.

In an inbound marketing campaign, a subject initiates contact with a company through a specific channel such as email, a website, or a phone call. A subject can be an individual customer, a company account, a household, or other entity. Depending on the channel, an application or website captures the subject's behavior or a human representative enters the subject's information into a form or a customer service application.

The subject usually has an identification number (a subject ID). The customer service application or other calling application uses the subject ID to send a request to SAS Intelligent Decisioning that invokes a decision. The decision typically performs the following tasks:

- 1 Uses the subject ID to issue a web service call or to query the company database in order to retrieve additional customer data, such as name, address, and household income, if such information exists.
- 2 Determines the set of offers for which the subject qualifies.
- 3 Arbitrates the set of offers to determine which ones the subject is most likely to respond to.

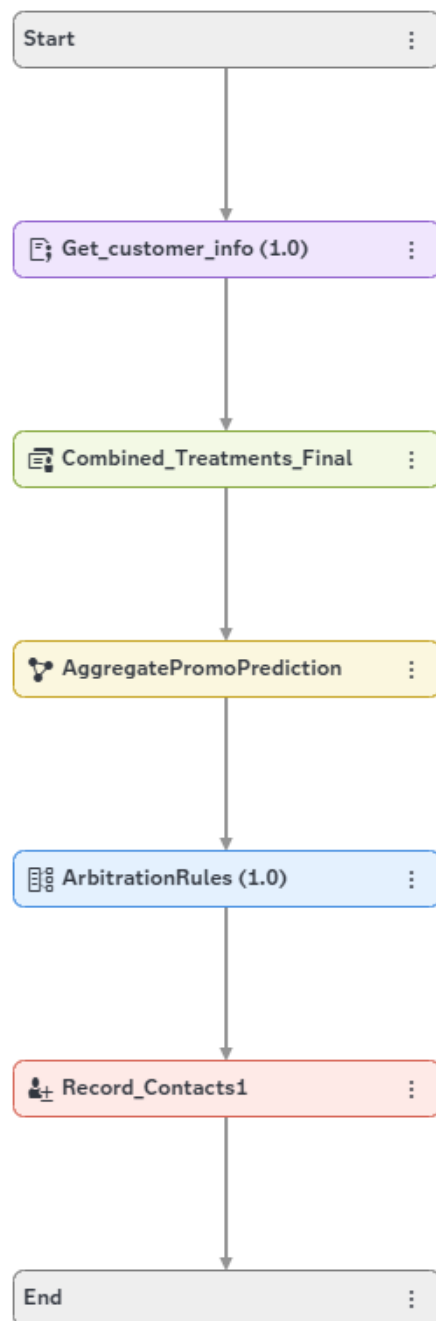
- 4 Generates a response tracking code and subject contact history data. The subject contact history data typically includes the set of offers from step 3.
- 5 Returns to the calling application the response tracking code and the set of offers for which the subject qualifies.

At this point, the calling application or a customer service representative can present the offers to the customer. The calling application can use the response tracking code to update the subject contact history data. For example, it can record which treatments are presented to the customer and the subject's response to the treatments.

[“Example: A Decision That Includes a Treatment Group” on page 113](#) shows how to accomplish the tasks listed above by using a decision in SAS Intelligent Decisioning.

Example: A Decision That Includes a Treatment Group

[Figure 4.1](#) shows a basic decision that defines offers, arbitrates these offers, and updates the subject contact history. This example contains only one treatment group and one record contacts node, but decisions can include multiple treatment groups and record contacts nodes.

Figure 4.1 Example Decision That Uses Treatments

This decision flow has the following elements:

- 1 The data query node named Get_customer_Info queries the company database and retrieves additional information about the subject based on the subject ID. For example, if the treatment is an offer to upgrade the subject's cell phone, the SQL query could retrieve the model number for the subject's current phone, if that information is available. For more information, see ["Data Query Files" on page 192](#).

Nodes that retrieve additional information about the subject are typical but not required.

- 2 The treatment group name `Combined_Treatments_Final` defines a set of treatments (offers) and includes rules that define who is eligible to receive the treatments. The decision uses the information about the subject and the eligibility rules to determine the treatments for which the subject qualifies.

When you add a treatment group to a decision, SAS Intelligent Decisioning defines an output variable of type data grid. After the decision executes, this data grid contains a row for each treatment for which the subject qualifies.

Note: See [“About Decision Variables and Mapping” on page 436](#) for information about when variables are created and how they are mapped.

You can include as many treatment groups in a decision as necessary. You can merge multiple output data grids into a single data grid if needed.

For more information, see the following topics:

- [“About Attributes, Eligibility Rules, and Effective Dates” on page 116](#)
- [“About Channels” on page 118](#)
- [SAS Intelligent Decisioning: Using Data Grids](#)

- 3 The model `AggregatePromoPrediction` and the rule set `ArbitrationRules` arbitrate the treatments. For example, the subject might qualify for several treatments, but you might want to offer them only the two treatments that they are most likely to accept. You might also want to return only the treatments that are appropriate for the channel.

- The model calculates a probability for each treatment (each row in the data grid) that indicates how likely the subject is to respond to the treatment.
- The rule set uses the `DATAGRID_TOPN` function to sort the treatment data grid based on the values of the probability column. The function returns the top two treatments to which the subject is most likely to respond. See [“DATAGRID_TOPN Function” in SAS Intelligent Decisioning: Using Data Grids](#) for more information.

You can use rule sets, scoring models, attribute values, or other methods to arbitrate treatments. For more information, see [“Arbitrating Treatments” on page 118](#).

- 4 The record contacts node records information that you want to track, such as the treatments that are returned to the calling application and whether the record is used in generating aggregate reports for the channel. You can also specify variable values to track.

In real-time destinations (SAS Micro Analytic Service and SAS Container Runtime destinations), this node writes a record to the subject contact history. In all other destination types, this node creates an output variable that contains the information that you specify that you want to track.

Record contacts nodes also create a response tracking code that the calling application can use to add additional information to the subject contact history.

TIP You can use record contacts nodes to track variable values even if you are not using treatments.

For more information, see the following topics:

- [“Adding Record Contacts Nodes” on page 410](#)
- [Subject Contacts REST API](#)
- [“Update Subject Contact History” in *SAS Intelligent Decisioning: Decision Management REST API Examples*](#)

About Attributes, Eligibility Rules, and Effective Dates

A treatment defines the details of an offer that can be sent to a subject, who is eligible for the offer, and the dates during which the offer is available to be sent to a subject. A treatment is a set of attributes, eligibility rules, and effective dates.

Attributes and Attribute Aliases

Attributes are name-value pairs that define the specific details of the offer that you present to a subject. For example, for a treatment that gives subjects a 20% discount, you could define an attribute named DISCOUNT that has the value 20. To give subjects a special deal on a specific cell phone model, you can define an attribute named MODEL whose value is the specific model name. You could use treatment attributes to set values such as profitability, risk, cost, priority, or order. You can use the values of these attributes to arbitrate the treatments. For more information, see [“Arbitrating Treatments” on page 118](#).

An attribute's value can be dynamic or fixed.

Fixed

You specify the value or set of values for fixed attributes when you define the attribute. You cannot customize the values in treatment groups, and the decision cannot set them at run time.

Dynamic

You can set the value or set of values for a dynamic attribute when you add a treatment group to a decision, or the decision can set them at run-time. When you define a dynamic attribute, you can specify default values for the attribute. Within each treatment group in which the attribute appears, you can set static values that are used only within that specific treatment group, or you can specify that the attribute's values are set by the decision at run time.

When you add a treatment group to a decision, SAS Intelligent Decisioning does the following:

- creates an output decision variable of type data grid that contains a column for each attribute in the treatment group.
- creates an input decision variable for each dynamic attribute whose value is set by the decision.

SAS Intelligent Decisioning automatically maps the treatment group's attributes to the decision variables that have the same name and data type. For more information, see ["Mapping Variables within a Decision" on page 436](#).

You can assign aliases to attributes. An alias is an alternative name that you can assign to a treatment attribute after you add the treatment to a treatment group. Aliases are useful when attributes in different treatments within the group represent the same data but are defined with different names. For example, treatment A could define an attribute named DISCOUNT, and treatment B could define an attribute named DISC. Assuming that these two attributes represent the same value, you can assign an alias to one or both of the attributes. An attribute can have only one alias.

For more information, see ["Managing Attributes and Aliases" on page 124](#).

Eligibility Rules

Eligibility rules define who is eligible to receive the offer defined by the treatment. For example, your campaign can target people who already have a specific credit card and who are at least 30 years old, or people who own a home but who do not have a home equity line of credit. You define eligibility rules in the eligibility rule set. An eligibility rule set is a filtering rule set. For more information, see ["About Rules and Rule Set Types" on page 29](#) and ["Manage Eligibility Rules for a Treatment" on page 127](#).

Effective Dates

The effective dates for a treatment are the start and end dates when the treatment is active, that is, when it can be returned to a subject. Responses that are defined by a treatment are not returned to subjects outside of the effective dates. Effective dates are not required. A treatment that does not have effective dates is considered to always be active.

Note: The effective dates are always based on the time zone of the server where the decision is executed. The time zone for SAS Viya 4 servers is set to Coordinated Universal Time (UTC).

Arbitrating Treatments

You can determine which treatments a subject is most likely to respond to by using one or more of the following methods:

- use filtering rule sets to select only certain records for processing. Only the records whose conditions evaluate to true are processed by the remaining objects in the decision. For more information, see [“About Rules and Rule Set Types” on page 29](#).

Note: Filtering rule sets can be included directly in a decision, but they are also used as the eligibility rule set in treatments.

- use models to score treatments. For example, you might have a model that calculates propensity scores for individual treatments.
- use data grid functions to sort or subset the treatments according to the values of their attributes. For example, you could have attributes for value, profitability, or risk. For more information, see [“Data Grid Functions” in SAS Intelligent Decisioning: Using Data Grids](#).
- use DS2 code files to create custom code for arbitrating the treatments. [Chapter 7, “Using Custom Code Files,” on page 183](#).

About Channels

Channels are routes by which your company and a subject are in contact. Typical channels are email, phone call, and website. A default set of channels is defined in the Treatment Channels lookup table. Your administrator can customize this lookup table for your enterprise. You can use this lookup table in rules to narrow the list of offers to ones that are appropriate for the channel. See [“Predefined Lookup Tables” on page 146](#) for more information.

You can create assignment rules to determine the channel through which your company and a subject are in contact if the channel was not specified in the original request.

Define a Treatment

Create a New Treatment and Define Attributes

- 1 Click , and then click **New treatment**. The New Treatment window appears.
- 2 Enter a name for the treatment. Treatment names must be unique within a folder.
- 3 (Optional) Enter a description for the treatment. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 4 Click , and select the folder where you want to save the treatment.
- 5 Click **Save** to save the treatment. SAS Intelligent Decisioning opens the new treatment and displays the **Attributes** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 6 (Optional) If the treatment is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the treatment. For more information, see [“Check Out and Commit a Treatment or Treatment Group Version” on page 140](#).
- 7 (Optional) Define treatment attributes. A treatment can have zero or more attributes. For more information about attributes, see [“About Attributes, Eligibility Rules, and Effective Dates” on page 116](#).

You can add new custom attributes, add attributes from other treatments, or duplicate attributes within the same treatment. For more information, see the following topics:

- [“Define Custom Attributes” on page 124](#)
- [“Add Attributes from a Different Treatment” on page 125](#)
- [“Duplicate Attributes” on page 125](#)

Specify Eligibility Rules

You specify eligibility rules in a filtering rule set. See [“About Rules and Rule Set Types” on page 29](#) for more information.

- 8 On the **Eligibility Rule Set** tab, click **Add Rule Set**. The Choose an Item window appears.

- 9 Select the filtering rule set that you want to use as the eligibility rule set for the treatment, and click **OK**.
- 10 At the top of the **Eligibility Rule Set** tab, select the version of the eligibility rule set that you want to use.

TIP To view or edit an eligibility rule set from the **Eligibility Rule Set** tab, click **Open**.

(Optional) Specify Effective Dates

- 11 On the **Properties** tab, select the **Start date** and **End date** for the period in which the treatment can be sent to a subject. If you do not specify effective dates, the treatment is considered as always active. Specify the time based on the Coordinated Universal Time (UTC) time zone.

Note: The effective dates are always based on the time zone of the server where the decision is executed. See [“Effective Dates” on page 117](#).

Define a Treatment Group

Create a New Treatment Group

- 1 Click  to navigate to the Treatment groups category view, and then click **New treatment group**. The New Treatment Group window appears.
- 2 Enter a name for the treatment group. Treatment group names must be unique within a folder.
- 3 (Optional) Enter a description for the treatment group. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 4 Click , and select the folder where you want to save the treatment group.
- 5 Click **Save** to save the treatment group. SAS Intelligent Decisioning opens the new treatment group and displays the **Treatments** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 6 (Optional) If the treatment group is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the treatment group. For more information, see [“Check Out and Commit a Treatment or Treatment Group Version”](#) on page 140.

Add Treatments

When you add a treatment to a treatment group, SAS Intelligent Decisioning uses the treatment's ID, not its name, to add the treatment. If you delete a treatment and create a new treatment with the same name, you must delete and re-add the treatment to the treatment group.

- 7 On the **Treatments** tab, click **Add treatments**. The Add Treatments window appears.
- 8 Select the treatments that you want to add to the treatment group, and click **OK**.

TIP After you add a treatment to a treatment group, the variables in the eligibility rule set are added to the list of eligibility variables for the treatment group. You can view the eligibility variables for a treatment group on the **Eligibility Variables** tab.

- 9 (Optional) Select the version of each treatment that you want to use in the treatment group.

TIP You can change the version of a treatment at any time on the **Treatments** tab.

Customize Dynamic Treatment Attributes

For information about dynamic and fixed attributes, see [“Define Custom Attributes”](#) on page 124.

- 10 On the **Treatments** tab, click **Set attributes**. The Set Attributes window appears.
- 11 Select the treatment for which you want to customize dynamic attributes.
- 12 For each dynamic variable, verify that its setting is correct for the current treatment group. By default, the switch for **Dynamic** is turned on, and the values of dynamic attributes are set at run-time by the decision flow.
 - If you want the attribute's value to be set by the decision flow, leave the **Dynamic** switch turned on.
 - If you want to enter a static value or list of values for the attribute, turn off the **Dynamic** switch, and then enter one or more values that you want to be used in the current treatment group. To enter new value in a list, press Enter after each new value. To delete a value from a list, click the **×** for that value.
- 13 Click **OK** to return to the **Treatments** tab, and click  to save your changes to the treatment group.

Add Attribute Aliases

For information about aliases, see [“Attributes and Attribute Aliases” on page 116](#).

- 14 On the **Attribute Aliases** tab, click  for the attribute to which you want to assign an alias. The Choose an Alias window appears and displays the list of existing attribute names that you can associate with the selected attribute.
 - To assign an existing attribute name as an alias, click **+** for that attribute name and click **OK**. The list of existing attribute names includes only those attributes of the same type as the attribute for which you are setting the alias.
 - To enter a new alias, type the new alias in the **Alias** field, and click **OK**. When you create a new alias, SAS Intelligent Decisioning [creates a new column in the output data grid](#) for the treatment group.

TIP Aliases are also listed in the Set Attributes window.

Activating a Treatment Group

What Does Activating a Treatment Group Do?

Activating a version of a treatment group makes that version available for use by published decisions. Published decisions can use only activated versions.

In SAS Micro Analytic Service destinations, the [sas.decisions.masInlineTreatmentGroup configuration option on page 123](#) controls what happens when you activate a treatment group. If this configuration option is set to Off, the treatment group is converted into executable code in the form of a SAS Micro Analytic Service module. This module enables decisions that are published to SAS Micro Analytic Service destinations to use the most recently activated version of a treatment group. If this option is set to On, a static version of the treatment group is included inline in the generated code for decisions.

CAS, Teradata, Hadoop, and container destinations do not support SAS Micro Analytic Service modules. Decisions that are published to these destinations always include a static version of each treatment group.

You must activate a version of any treatment group that is used in a decision. When you activate a version of a treatment group, that version is locked and cannot be edited. Each treatment group can have only one active version.

The active version of a treatment group is used in the following ways:

- when you run a test or scenario test prior to publishing an object.

- to generate a static copy of the treatment group when you publish a decision if you publish the decision to CAS, Teradata, Hadoop, or container destinations. Static copies are always generated for these destinations.
- by published decisions in SAS Micro Analytic Service destinations if the configuration `sas.decisions.masInlineTreatmentGroup` property is set to Off. See [“Controlling Whether Global Variables Are Included Inline in Code” on page 180](#) for more information.

Controlling Where Treatment Groups Are Activated and How They Are Used

Static versions of treatment groups are automatically embedded in content that is published to CAS, Teradata, Hadoop, and container destinations. Your administrator can use the following configuration options to control how treatment groups are used in SAS Micro Analytic Service destinations:

`sas.treatmentdefinitions.activation.destinations = destinations`

specifies the SAS Micro Analytic Service destinations in which treatment groups are activated. Only one SAS Micro Analytic Service destination is supported. By default, that destination is named `maslocal`.

`sas.decisions.masInlineTreatmentGroup = On | Off`

specifies whether the code for treatment groups is included inline in generated code when you publish a decision to SAS Micro Analytic Service destinations.

When this property is set to On, a static copy of each treatment group is included in the code that is generated for SAS Micro Analytic Service destinations. Changes to the treatment groups do not affect published objects. When this property is set to Off, the generated code uses a SAS Micro Analytic Service module to retrieve the active version of the treatment group when the decision is run.

Activate a Treatment Group

Activating a version of a treatment group version locks that version. Locked groups cannot be unlocked.

You must activate a treatment group version before you can publish an object that uses that version. By default, when you activate a treatment group, it is activated in all of the publishing destinations that are configured at your site. However, your administrator might specify that groups are activated only in specific SAS Micro Analytic Service destinations. For more information, see [“Controlling Where Treatment Groups Are Activated and How They Are Used” on page 123](#).

To activate a treatment group:

- 1 Open the treatment group that you want to activate.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, select the version that you want to activate, and click **Activate**.

Managing Attributes and Aliases

About Attributes and Aliases

For additional information about attributes and aliases, see the following topics:

- [“Attributes and Attribute Aliases” on page 116](#)
- [“Add Attribute Aliases” on page 122](#)
- [“Customize Dynamic Treatment Attributes” on page 121](#)

Define Custom Attributes

- 1 On the **Attributes** tab of a treatment, click **Add Attribute** and select **Custom attribute**. The Add Attribute window appears.
- 2 Complete these steps for each attribute:
 - a Enter a name for the attribute.
 - b (Optional) Enter a description for the attribute.
 - c Select the data type for the attribute.
 - d Select **Fixed** or **Dynamic**, depending on how the attribute’s value will be set when it is included in a treatment group.

Dynamic

The values of dynamic attributes can be set by the decision at run-time. When you define a dynamic attribute, you can specify a default value or a list of default values for the attribute. Within each treatment group in which the attribute appears, you can specify that the attribute’s value is

set by the decision at run time, or you can set a static value that is used only within that specific treatment group.

Fixed

You define the value or list of values for fixed attributes when you define the attribute. The values cannot be customized in treatment groups.

- e Specify the value or the list of values for the attribute. For dynamic attributes, these values are used as the default values if the attribute's values are not customized in treatment groups.

For attributes that are character, integer, or decimal attributes, you can specify a single value or a list of values. Select the **List of values** check box to specify a list of possible values for the attribute. Character values can be up to 3998 characters long. In the **Value** field, press Enter after each specific value.

For attributes of other data types, you can specify only a single value.

- f Click **Add** to add the attribute to the table.
- 3 Click **OK** to add the attributes to the treatment, then click  to save the changes to the treatment.

Add Attributes from a Different Treatment

- 1 On the **Attributes** tab of a treatment, click **Add Attribute**, and select **Treatment**. The Choose an Item window appears.
- 2 Select the treatment from which you want to copy an attribute, and click **OK**. The Add Attributes window appears.
- 3 Click **➤** to select all of the attributes, or select one or more individual attributes and click **➤**.
- 4 Click **Add** to add the attributes to the treatment, or click **Add and replace** to replace existing attributes that have the same name.

Duplicate Attributes

- 1 On the **Attributes** tab of a treatment, select the attribute that you want to duplicate.
- 2 Click , and select **Duplicate**. The Duplicate Attribute window appears.
- 3 Enter a new name for the duplicate attribute, and click **Duplicate**.

Delete Attributes from a Treatment

On the **Attributes** tab of a treatment, select the check boxes for the attributes that you want to delete, click , and select **Delete**.

View Attribute Aliases for a Treatment Group

You can view attribute aliases in two ways:

- Click the **Attribute Aliases** tab in a treatment group.
- On the **Treatments** tab, click **Set attributes**.

Remove an Attribute Alias from a Treatment Group

On the **Attribute Aliases** tab of a treatment group, select the attributes whose aliases you want to remove, click , and select **Remove alias**. Alternatively, click  for the alias that you want to delete.

Determine Which Treatments Use an Attribute

On the **Attribute Aliases** tab of a treatment group, select the attribute for which you want treatment information, click , and select **Used by**. SAS Intelligent Decisioning opens the Used By Treatments window, which lists the treatments that use the attribute.

Delete Treatments from a Treatment Group

On the **Treatments** tab of the treatment group, select the check box for the treatments that you want to remove from the group, click , and select **Delete**.

Manage Eligibility Rules for a Treatment

You define eligibility rules in the eligibility rule set. An eligibility rule set is a filtering rule set. For additional information, see [“Eligibility Rules” on page 117](#) and [“About Rules and Rule Set Types” on page 29](#).

For instructions about creating a filtering rule set, defining filtering rules, and adding the rule set to a treatment, see the following topics:

- [“Create a New Rule Set”](#)
- [“Add a New Rule in a Filtering Rule Set”](#)
- [“Specify Eligibility Rules”](#)

To remove the eligibility rule set from a treatment, open the treatment, click the **Eligibility Rule Set** tab, and click **Remove**.

To change the eligibility rule set that is specified for a treatment:

- 1 Open the treatment, click the **Eligibility Rule Set** tab, and click **Replace**. The Choose an Item window appears.
- 2 Select the filtering rule set that you want to use as the eligibility rule set for the treatment, and click **OK**.
- 3 At the top of the **Eligibility Rule Set** tab, select the version of the eligibility rule set that you want to use.

TIP To view or edit an eligibility rule set, click the **Eligibility Rule Set** tab, and click **Open**.

Copy a Treatment or Treatment Group URL

To create a link for external documentation that automatically opens a treatment or treatment group in SAS Intelligent Decisioning, open the treatment or treatment group, click , and select either **Copy treatment URL** or **Copy treatment group URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Treatment or Treatment Group Content

You can compare the contents of two different treatments or treatment groups, or you can compare the contents of two different versions of the same treatment or treatment group.

- 1 Select the objects that you want to compare.
 - To compare the contents of two different treatments or treatment groups, select the treatments or treatment groups in the category view, click , and select **Compare object contents**.
 - To compare the contents of two versions of the same treatment or treatment group, open the treatment or treatment group, click  on the **Versions** tab, and select **Compare object contents**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Object Contents window.

For treatment groups, the Compare Object Contents window displays the name and version number of the treatments that differ in each object. To display all of the treatments in each group, click **Show All**. For example, in the following figure, both treatment groups contain the card_preferred treatment and the card_silver treatment. The card_gold treatment appears only in the card_offers group, and the card_gold_special treatment appears only in the card_offers_special treatment group.

Compare Object Contents

Filter treatments

Show differences Show all

Number of Different Entries: 2 Treatment Group Treatment	Total Number of Treatments 3 card_offers_special(1.0) ⓘ Version	Total Number of Treatments 3 card_offers(1.0) ⓘ Version
card_gold		1.0
card_gold_special	1.0	
card_preferred	1.0	1.0
card_silver	1.0	1.0

TIP Click ⓘ beside a treatment name or treatment group name to display its location.

For treatments, the **Attributes** tab displays the attribute names and values that are different between the two objects. For example, in the following figure, the two treatments contain the same attributes, but the values of all of the attributes are different except for the annual_fee attribute.

Compare Object Contents

Attributes Eligibility Rule Set

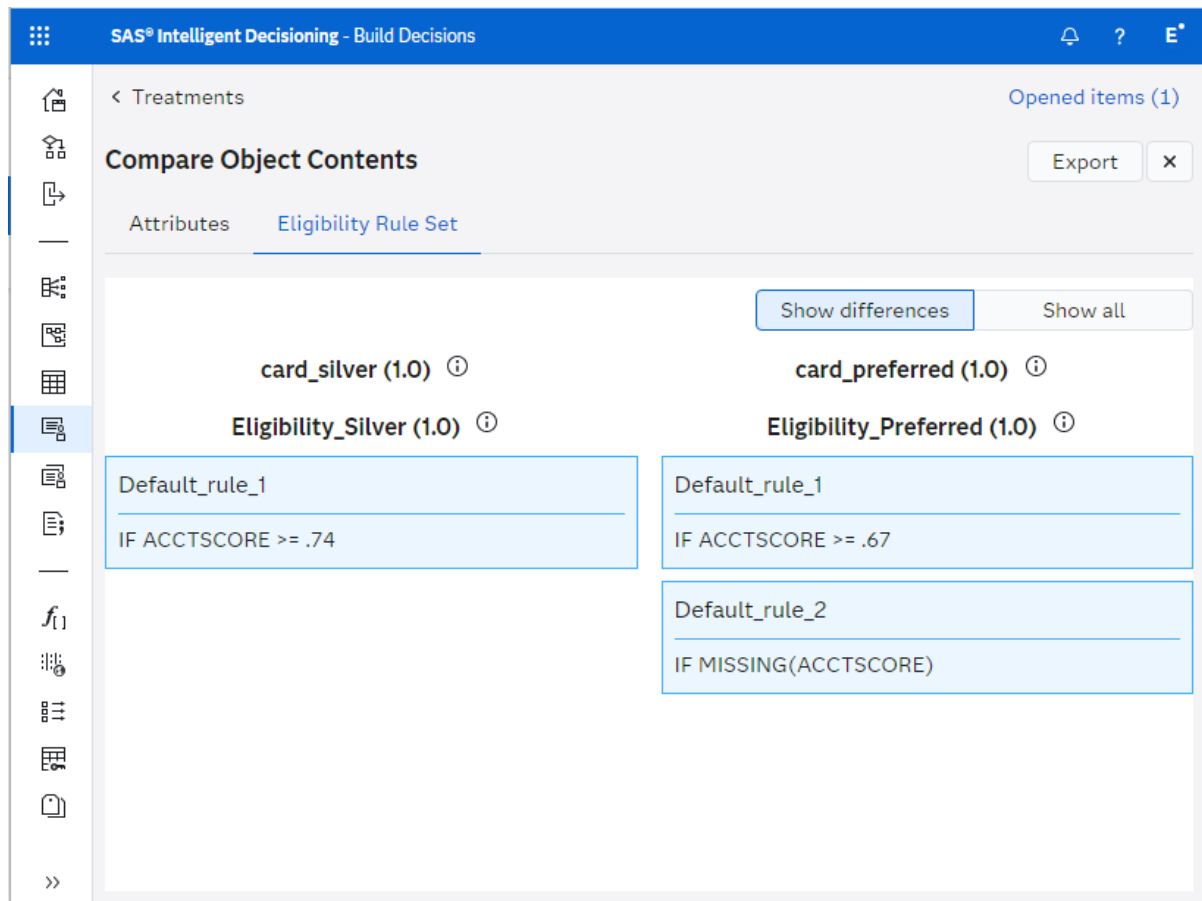
Filter attribute name

Show differences Show all

Number of Different Entries: 5 Treatment Attribute	Total Number of Entries: 6 card_silver(1.0) ⓘ Data Type	Value	Value Type	Total Number of Entries: 6 card_preferred(1.0) ⓘ Data Type	Value	Value Type
annual_fee	Decimal	0	Fixed	Decimal	0	Fixed
APR	Decimal	16.5	Fixed	Decimal	18.7	Fixed
card_type	Character	silver	Fixed	Character	preferred	Fixed
cashback_gas	Decimal	2	Fixed	Decimal	3	Fixed
cashback_stream	Decimal	2	Fixed	Decimal	3	Fixed
intro_bonus	Decimal	0	Fixed	Decimal	200	Fixed

- (Optional) Click the **Eligibility Rule Set** tab to display the differences in the eligibility rule sets. For example, in the following figure, the two treatments use

different eligibility rule sets. The Eligibility_Preferred rule set contains an additional rule.



- 4 (Optional) Click **Export** to export the results of the comparison to a PDF file. The Export Comparison Results window appears.
- 5 (Optional) Select the information that you want to export, and click **Export**. If you want the PDF file to display only the differences between the two objects, select **Show differences**. To display all of the objects' information, select **Show all**.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Compare Treatment Group Code

You can compare the generated code of two different treatment groups, or you can compare the generated code of two different versions of the same treatment group.

- 1 Select the objects that you want to compare.
 - To compare the generated code of two different treatment groups, select the groups in the category view, click , and select **Compare code**.
 - To compare the generated code of two versions of the same treatment group, open the treatment group, click  on the **Versions** tab, and select **Compare code**.

The Select Versions window appears.
- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window and highlights the differences.
- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Generate PDF Documentation for a Treatment Group

You can generate detailed documentation for a treatment group as a PDF document. The PDF includes the treatment group properties, and details about each treatment that is included in the group. Treatment details include treatment properties, the name of the eligibility rule set, and details about each of the treatment's attributes.

- 1 Open the treatment group.
- 2 Click , and select **Create document**. The Create Document window appears.

TIP If this option is disabled, there might be unsaved changes. Click .

- 3 Enter a name for the document if you do not want to use the default name.
- 4 Click **Create**. SAS Intelligent Decisioning creates the PDF. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Treatments and Treatment Groups

Duplicate Treatments or Treatment Groups

.....
Note: You cannot duplicate a treatment or treatment group if it is open.
.....

To duplicate a single treatment or treatment group:

- 1 In the Treatments or Treatment Groups category view, select the treatment or treatment group that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Treatment or Duplicate Treatment Group window appears.
- 3 Enter a new name for the duplicate treatment or treatment group.
- 4 (Optional) Enter a description for the treatment or treatment group.
- 5 Select the version of the treatment or treatment group that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate treatment or treatment group.
- 7 Click **Duplicate**.

To duplicate multiple treatments or treatment groups:

- 1 In the Treatments or Treatment Groups category view, select the treatments or treatment groups that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the latest version of the treatments or treatment groups and appends **_Copy_n** to the names of the duplicate copies.

.....
Note: When you duplicate a treatment or treatment group, SAS Intelligent Decisioning creates a relationship between the original treatment or treatment group and the duplicate object. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the treatment or treatment group before you paste the new content.
.....

Delete Treatments or Treatment Groups

Note: You cannot delete a treatment or treatment group if it is open.

In the Treatments or Treatment Groups category view, select the treatments or treatment groups that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the deleted items to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename a Treatment or Treatment Group

Note: You cannot rename a treatment or treatment group if it is open.

- 1 In the Treatments or Treatment Groups category view, select the treatment or treatment group that you want to rename.
- 2 Click , and select **Rename**.
- 3 In the Rename window, enter a new name for the treatment or treatment group, and click **Rename**.

Move Treatments or Treatment Groups to a Different Folder

- 1 In the Treatments or Treatment Groups category view, select the treatments or treatment groups that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the treatments or treatment groups, and click **OK**.

Managing Versions of Treatments and Treatment Groups

Set the Displayed Version

The displayed version is the version whose information is displayed on the other tabs, such as the **Properties** and **Attributes** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Treatment or Treatment Group Version” on page 140](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.
- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object’s version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the treatment or treatment group whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the treatment or treatment group whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target treatment or treatment group into which you want to paste the contents of the version. You can paste version contents only into an object of the same type.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target treatment or treatment group. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see “[Default Permissions](#)” in *SAS Intelligent Decisioning: Administrator’s Guide* and “[Folder-Based Permissions](#)” in *SAS Intelligent Decisioning: Administrator’s Guide*. By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of treatments or treatment groups, the [sas.treatmentdefinitions.deleteVersions](#) configuration option must be turned on.

On the object’s **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version. You cannot delete an active version.

Determine Which Objects Use a Treatment or Treatment Group

To list the objects that use a specific treatment or treatment group, select the treatment or treatment group on the **Treatments** category page or the **Treatment groups** category page, click , and select **View used by report**. For treatments, the report lists all treatment groups that use any version of the selected treatment. For treatment groups, the report lists all objects that use an active version of the selected group.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Treatment or Treatment Group Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check

out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.treatmentdefinitions.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a treatment version or treatment group version, the **Check Out** button for that treatment or treatment group is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing a Treatment or Treatment Group Versions”](#) on page 139.

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Treatments or Treatment Group category view shows two treatments or treatment groups with the same name, but the folders listed in the **Location** column differ for each object. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

Note: When a decision that uses a treatment group is deployed to a production environment, the decision always uses the active version of a treatment group. When you edit a decision that uses a treatment group that you have checked out, the properties pane for the treatment group shows the location of the treatment group in the original folder. The pane does not list the location of the treatment group that is checked out because that copy is not the active version.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

You cannot activate the checked-out version that is in **My Folder**. To activate a version with your changes, you must commit your changes, and activate the committed version.

Checking Out a Treatment Group from within a Decision

See “[Checking Out and Committing Objects from within A Decision](#)” on page 464 for information.

Checking Out and Committing Treatments from within a Treatment Group

You can check out a treatment from within a treatment group if the following conditions are true:

- The treatment group is checked out.
- The treatment is stored in a folder for which the check-out feature is enabled.

To check out or commit an individual treatment from within a checked-out treatment group, select the check box for the treatment on the **Treatments** tab, click , and select **Check out** or **Commit**.

When you check out a treatment from within a treatment group, SAS Intelligent Decisioning adds an asterisk in front of the name of the treatment, and modifies the treatment group to use the checked-out copy of the treatment.

To open the checked-out treatment, click  to navigate to the Treatments category view, click , and open the treatment in My Folder.

Commit the treatment from within the treatment group. When you commit a treatment group, all of the checked-out treatments that are used in the treatment group are also committed.

To cancel the check out of a treatment, select the check box for the treatment on the **Treatments** tab of the treatment group, click , and select **Cancel check out**.

Concurrently Checking Out and Committing a Treatment or Treatment Group Versions

The ability for multiple users to check out the same treatment or treatment group at the same time is controlled by the [sas.treatmentdefinitions.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's **My Folder** location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Treatment or Treatment Group Version

Note: If you are checking out a treatment or treatment group from within another object instead of from within the Treatments or Treatment groups category views, see [“Checking Out and Committing Treatments from within a Treatment Group”](#) on page 139 or [“Checking Out and Committing Objects from within A Decision”](#) on page 464 for information.

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out”](#) on page 142.

- 4 On the **Versions** tab, click **Commit**. The Commit Treatment Version or Commit Treatment Group Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 7 (Optional) Enter information about the new version in the **Notes** field.
- 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

TIP If you or SAS Intelligent Decisioning have updated the treatment group to use any treatments that you have checked out, then SAS Intelligent Decisioning also commits those treatments. See [“Checking Out and Committing Treatments from within a Treatment Group” on page 139](#) for more information.

Determine Whether an Object Is Checked Out

The **Checked Out** column in the category view displays a check mark if the object is checked out.

If you are editing an object that is stored in a checkout-enabled folder, the object name in the editor is displayed with either (Read-Only) or (Checked Out) depending on whether it is checked out.

If you are editing a treatment group and one of its [treatments was checked out from within the treatment group](#), an asterisk appears before the treatment name in the treatment group editor.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

How you undo the checkout of an object depends on how the object was checked out.

If you [checked out an object from within a decision](#), click  on the object's node in the decision diagram, and select **Cancel checkout**.

If you have [checked out a treatment from within a treatment group](#), select the check box for the treatment, click  in the treatment group edito, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Working with Lookup Tables and Functions

<i>About Lookup Tables and Functions</i>	144
<i>About Labels for Lookup Keys and Values</i>	145
<i>Predefined Lookup Tables</i>	146
<i>Create a New Lookup Table</i>	146
<i>Importing and Exporting Lookup Tables</i>	147
About Importing, Exporting, and Activating Lookup Tables	147
Format of the CSV Import File for a Lookup Table	147
Import Entries or Refresh Lookup Tables from a CSV File	147
Export a Lookup Table to a CSV File	147
<i>Add Lookup Table Entries</i>	151
<i>Edit Lookup Table Entries</i>	151
<i>Delete Lookup Table Entries</i>	152
<i>Edit Lookup Key or Value Labels</i>	152
<i>Copy a Lookup Table URL</i>	152
<i>Compare Lookup Table Content</i>	153
<i>Generate PDF Documentation for a Lookup Table</i>	154
<i>Managing Lookup Tables</i>	155
Duplicating Lookup Tables	155
Delete Lookup Tables	155
Rename a Lookup Table	155
Move Lookup Tables	155
<i>Managing Versions of Lookup Tables</i>	157
Set the Displayed Version	157
Create a New Version	157
Copy the Content of a Version	157
Delete a Version	157
<i>Determine Which Objects Use a Lookup Table</i>	160

Checking Out and Committing Lookup Table Versions	161
About Checking Out and Committing Versions	161
Checking Out and Committing a Lookup Table from within a Rule Set	161
Committing a Lookup Table Together with a Rule Set	161
Concurrently Checking Out and Committing Lookup Table Versions	161
Check Out and Commit a Lookup Table Version	161
Determine Who Has a Version Checked Out	161
Opening the Original Object	161
Undoing a Check Out	161
Activating a Lookup Table	166
What Does Activating a Lookup Table Do?	166
Controlling Where Lookup Tables Are Activated And How They Are Used	166
Activate a Lookup Table	166
LOOKUP Function	169
LOOKUPVALUE Function	171

About Lookup Tables and Functions

SAS Intelligent Decisioning provides the ability to import lookup tables and reference them from rules. Lookup tables are tables of key-value pairs. For example, you can use a lookup table to retrieve a part name based on the code number of the part or to retrieve the full name for a country based on its abbreviation.

You can import lookup data from comma-separated-values (CSV) files, such as those created by spreadsheet applications, into lookup tables in SAS Intelligent Decisioning. You can re-import updated CSV files as needed to refresh the lookup tables.

Note: SAS Intelligent Decisioning does not support CSV files that contain signature lines.

Figure 5.1 CSV File Imported Into SAS Intelligent Decisioning

	A	B		Key	Value
1	AU	Australia	☐	AU	Australia
2	BR	Brazil	☐	BR	Brazil
3	CA	Canada	☐	CA	Canada
4	CR	Costa Rica	☐	CR	Costa Rica
5	DK	Denmark	☐	DK	Denmark
6	EE	Estonia	☐	EE	Estonia
7	ES	Spain	☐	ES	Spain
8	FR	France			
9	GL	Greenland			
10	HK	Hong Kong			
11	IN	India			
12	JP	Japan			
13	KE	Kenya			

In a lookup table, each *lookup key* is associated with a *lookup value*. Lookup keys must be unique within each lookup table.

SAS Intelligent Decisioning provides two functions, LOOKUP and LOOKUPVALUE, that enable you to determine whether a lookup key exists in a lookup table and to retrieve a lookup value from a lookup table.

About Labels for Lookup Keys and Values

You can specify labels to use for lookup keys and lookup values. For example, for the lookup table shown in [Figure 5.2](#), you could specify *Abbreviation* for the key and *Name* for the value. You could specify the names of variables that are used to store the keys or values as the labels.

The labels that you enter are added to the **Properties** tab for the lookup table. They are also included in windows that contain fields where you enter keys or values. For example, if you add labels to the lookup table shown in [Figure 5.2](#) and open the Add Table Entries window, the labels are displayed in parentheses after the existing **Key** and **Value** field labels.

Add Table Entries

Key (Abbreviati... * Remove

Value (Name): *

+ Add an entry

Predefined Lookup Tables

SAS Intelligent Decisioning defines two lookup tables for you:

Treatment Channels

defines several channels for use with treatments. Channels are the route by which your company and a subject are in contact. The predefined channels include ATM, Agent, Call Center, Mail, Phone, and Web.

Subject Level

defines subject levels for use with treatments. A subject level is the type of account that is associated with a subject. For example, a subject might be an individual customer, a company account, or a household. The predefined levels are Account, Customer, and Household.

Your administrator can modify these tables as described in [“Edit Lookup Table Entries” on page 151](#).

Create a New Lookup Table

- 1 Click  to navigate to the Lookup Tables category view.
- 2 Click **New lookup table**. The New Lookup Table window appears.
- 3 Enter a name for the new lookup table. Lookup table names are limited to 250 characters. Lookup table names are case insensitive and must be unique within a folder.
- 4 (Optional) Enter a label to use for the lookup key. Labels are limited to 100 characters. For more information, see [“About Labels for Lookup Keys and Values” on page 145](#).

- 5 (Optional) Enter a label to use for the value. Labels are limited to 100 characters. For more information, see [“About Labels for Lookup Keys and Values” on page 145](#).
- 6 (Optional) Enter a description for the new lookup table. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 7 Click , and select the folder where you want to create the new lookup table.
- 8 Click **Save**. The application opens the new lookup table and displays the **Lookup Table** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 9 (Optional) If the lookup table is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the lookup table. For more information, see [“Check Out and Commit a Lookup Table Version” on page 164](#).
- 10 Add entries to the new table either by importing a CSV file or by adding entries manually. See [“Import Entries or Refresh Lookup Tables from a CSV File” on page 150](#) and [“Add Lookup Table Entries” on page 151](#) for more information.

Importing and Exporting Lookup Tables

About Importing, Exporting, and Activating Lookup Tables

You can import and export lookup tables by using either of the following file types:

- Comma-delimited (CSV) files. The ability to import and export a lookup table as a CSV file is for modifying an existing lookup table. You can export an existing lookup table, edit the CSV file, and import the updated CSV file. You can use the **Import** and **Export** buttons in SAS Intelligent Decisioning, or you can use the %DCM_EXPORT_LOOKUP and %DCM_IMPORT_LOOKUP macros. For more information, see the following topics:
 - [“Format of the CSV Import File for a Lookup Table” on page 148](#)

- “Import Entries or Refresh Lookup Tables from a CSV File” on page 150
- “Export a Lookup Table to a CSV File” on page 150
- “%DCM_EXPORT_LOOKUP Macro” in *SAS Intelligent Decisioning: Macro Guide*
- “%DCM_IMPORT_LOOKUP Macro” in *SAS Intelligent Decisioning: Macro Guide*
- JavaScript Object Notation (JSON) files. If you do not need to edit the content of the CSV file, use SAS Environment Manager, the SAS Viya command-line interface (sas-viya), or the **Export objects to JSON files** and **Import objects from JSON files** options in SAS Intelligent Decisioning.

The recommended approach is to use the transfer plug-in to the sas-viya CLI. The CLI preserves the ID and version history of the objects that are being transferred. The CLI preserves information about which rule sets use the lookup table. For information about the CLI, see [SAS Viya Platform: Content Migration from SAS Viya 4](#). For information about using SAS Environment Manager, see “Content Page” in *SAS Environment Manager: User's Guide* and “Import Page” in *SAS Environment Manager: User's Guide*. For information about using the JSON options in SAS Intelligent Decisioning, see “Importing and Exporting Objects Using JSON Files” on page 13.

IMPORTANT When you export a lookup table by using a JSON file, the status of the table in the source environment (whether or not it is activated) is included in the exported data. When you import the table, its status in the target environment is set to the status that is specified in the exported data. However, whether an imported table is automatically activated in the target environment is determined by the setting of the [sas.referencedata.activation.activateLookupOnImport](#) configuration option. If a table is activated in the source environment but this configuration option is turned off, you might see a message in the SAS Intelligent Decisioning interface that indicates that the imported table is activated even though it is not. To ensure that the imported lookup table is activated, manually [activate the table on page 169](#) after it is imported.

Format of the CSV Import File for a Lookup Table

Note: The format of the CSV file depends on whether you are importing a CSV file by clicking the **Import** button in the user interface or by using the %DCM_IMPORT_LOOKUP macro. For information about the format that the macro uses, see “Format of the CSV Input File for the Macro” in *SAS Intelligent Decisioning: Macro Guide*.

Do not include a header row in the CSV file. Each row of the CSV input file identifies a key-value pair. To create a blank column in the CSV file, specify two

comma separators without any content between them. Each entry in the file must specify a lookup key and a lookup value.

The following example defines a lookup table of three-letter [ISO country codes](#) (keys) and the country names that correspond to each code (values).

```
"BEL", "Belgium"
"CCK", "Cocos Islands"
"ESP", "Spain"
"ISL", "Iceland"
"JOR", "Jordan"
"MLI", "Mali"
"THA", "Thailand"
```

In a spreadsheet application, this lookup table appears as shown in the following figure.

	A	B
1	BEL	Belgium
2	CCK	Cocos Islands
3	ESP	Spain
4	ISL	Iceland
5	JOR	Jordan
6	MLI	Mali
7	THA	Thailand

After you import the CSV file, it appears in SAS Intelligent Decisioning as shown in the following figure.

Key 	Value 
BEL	Belgium
CCK	Cocos Islands
ESP	Spain
ISL	Iceland
JOR	Jordan
MLI	Mali
THA	Thailand

Import Entries or Refresh Lookup Tables from a CSV File

Note: It is recommended that a single lookup table contains no more than 10,000 entries.

Note: Lookup key names and values that contain two hash signs (##) are not supported.

You can import entries into an empty table, and you can refresh an existing lookup table by re-importing the same table.

- 1 Open the lookup table to which you want to import entries.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click **Import**. The Import Lookup Table window appears.
- 3 Click , and select the CSV file that contains the lookup table entries.
- 4 Select or enter the encoding for the lookup table, and click **Import**.

Export a Lookup Table to a CSV File

IMPORTANT Do not modify the file structure or the first row in the exported CSV file. You can modify the data values.

To export a lookup table, open the lookup table and click **Export**. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Add Lookup Table Entries

Note: It is recommended that a single lookup table contains no more than 10,000 entries.

TIP You cannot add new entries to a lookup table version that has been activated. In order to edit the table, you must create a new version.

- 1 Open the lookup table to which you want to add entries.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click **New Entries** if the lookup table is empty, or click **+** if the lookup table already contains entries. The Add Table Entries window appears.
- 3 Enter the lookup key name and value for the new entry. Key names are limited to 256 characters. Values are limited to 512 characters. Key names must be unique within the same lookup table.

Note: Lookup key names and values that contain two hash signs (##) are not supported.

- 4 (Optional) For each additional entry that you want to add, click **Add an entry**, and enter the new key name and value.
- 5 Click **Save** to save the new entries and close the Add Table Entries window.

Edit Lookup Table Entries

TIP You cannot edit entries in a lookup table version that has been activated. In order to edit the table, you must create a new version.

- 1 Open the lookup table.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Select the entries that you want to edit, and click  in the toolbar above the table. The Edit Table Entries window appears.
- 3 Edit the exiting entries, and click **Save**.

TIP To remove an entry from the list shown in the Edit Table Entries window, click .

Delete Lookup Table Entries

Open the lookup table, select the entries that you want to delete, and click .

Edit Lookup Key or Value Labels

- 1 In the Lookups category view, open the lookup table.
- 2 Click  at the right of the **Key** or **Value** column heading in the lookup table. The Rename window appears.
- 3 Enter the label that you want to use for the key or value, and click **OK**.

Copy a Lookup Table URL

To create a link for external documentation that automatically opens a lookup table in SAS Intelligent Decisioning, open the lookup table, click , and select **Copy**

lookup table URL. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Lookup Table Content

You can compare the contents of two different lookup tables, or you can compare the contents of two different versions of the same lookup table.

- 1 Select the objects that you want to compare.
 - To compare the contents of two different lookup tables, select the lookup tables in the category view, click , and select **Compare object contents**.
 - To compare the contents of two versions of the same lookup table, open the lookup table, click  on the **Versions** tab, and select **Compare object contents**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Object Contents window.

By default, the Compare Object Contents window displays the names and values in each entry in both objects and highlights the differences. To display all of the entries in each object, click **Show All**. For example, in the following figure, only version 2.0 of the Country_Codes lookup table contains entries for Algeria, and Antigua and Barbuda.

TIP Click  beside a table name to display its location.

Lookup Tables > Country_Codes (2.0) Opened items (1)

Compare Object Contents Export Close

Filter keys Show differences Show all

Number of Different Key-Value Pairs: 2 Lookup Table Key	Total Number of Key-Value Pairs: 254 Country_Codes(2.0) ⓘ Value	Total Number of Key-Value Pairs: 252 Country_Codes(1.0) ⓘ Value
AA	Aruba	Aruba
AC	Antigua and Barbuda	
AE	United Arab Emirates	United Arab Emirates
AF	Afghanistan	Afghanistan
AG	Algeria	
AJ	Azerbaijan	Azerbaijan
AL	Albania	Albania
AM	Armenia	Armenia

- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The Export Comparison Results window appears.
- 4 (Optional) Select the information that you want to export, and click **Export**. If you want the PDF file to display only the differences between the two objects, select **Show differences**. To display all of the objects' information, select **Show all**.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Generate PDF Documentation for a Lookup Table

You can generate detailed documentation for a lookup table as a PDF document. The PDF includes the lookup table properties and a table of the current key-value pairs that are contained in the lookup table.

- 1 Open the lookup table.
- 2 Click , and select **Create document**. The Create Document window appears.

TIP If this option is disabled, there might be unsaved changes. Click .

- 3 Enter a name for the document if you do not want to use the default name.
- 4 Click **Create**. SAS Intelligent Decisioning creates the PDF. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Lookup Tables

Duplicating Lookup Tables

About Duplicating Lookup Tables in Folders for Which the Check-Out and Commit Feature Is Enabled

When you duplicate a lookup table, SAS Intelligent Decisioning first creates an empty lookup table, and then updates the lookup table to add the individual lookup entries. The default permissions for the Decision Repository enable users to create new objects, but the permissions do not include the ability to update the objects. In order to duplicate an object that is in the Decision Repository, you must check out the object.

If your site has created additional folders for which the check-out and commit feature is enabled, and if those folders have been assigned the same permissions as those assigned to the Decision Repository, you must also check out objects in those folders before you can duplicate them.

When you duplicate an object that is checked out, the duplicate object is not automatically checked out.

Duplicate Lookup Tables

Note: You can duplicate the predefined Treatment Channels or Subject Level lookup tables, but the duplicates will not be used by SAS Intelligent Decisioning to determine channels or subject levels. You cannot delete the duplicates by using the

SAS Intelligent Decisioning user interface because these tables are not stored in a folder. You must use the REST API to delete the duplicate tables.

To duplicate a single lookup table:

- 1 In the Lookup tables category view, select the tables that you want to duplicate, click , and select **Duplicate**.
- 2 Enter a new name for the duplicate lookup table if you do not want to use the default name. Names are limited to 250 characters. Lookup table names are case insensitive and must be unique within the database.
- 3 (Optional) Enter a description for the duplicate table. Descriptions are limited to 1000 characters.
- 4 Select the version of the lookup table that you want to duplicate.
- 5 Click  and select the location where you want to save the duplicate lookup table.
- 6 Click **Duplicate**.

To duplicate multiple lookup tables, select the tables that you want to duplicate, click , and select **Duplicate**. SAS Intelligent Decisioning duplicates the latest version of each lookup table and appends `_copy_n` to the names of the duplicate copies.

Note: When you duplicate a lookup table, SAS Intelligent Decisioning creates a relationship between the original lookup table and the duplicate lookup table. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the lookup table before you paste the new content.

Delete Lookup Tables

Note: You cannot delete a lookup table if it is open.

Select the tables that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the lookup table to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename a Lookup Table

- 1 Select the table that you want to rename, click , and select **Rename**. The Rename window appears.
- 2 Enter a new name for the table, and click **Rename**.

Move Lookup Tables

Note: You cannot move the predefined Treatment Channels or Subject Level lookup tables.

- 1 Select the tables that you want to move, click , and select **Move**. The Choose a Location window appears.
- 2 Select the folder where you want to move the tables, and click **OK**.

Managing Versions of Lookup Tables

Set the Displayed Version

The displayed version is the version whose information is displayed on the **Properties** and **Lookup Table** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

Note: If you activate the only unlocked version of a lookup table, SAS Intelligent Decisioning locks that version and automatically creates a new unlocked version. When you manually create a new version of a lookup table, previous versions are

not locked. The only way to lock a version of a lookup table is to activate it. For more information, see [“Activate a Lookup Table” on page 169](#).

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Lookup Table Version” on page 164](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, click **New version**. The New Version window appears.
- 3 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 4 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 5 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 6 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the lookup table whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the lookup table whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target lookup table into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target lookup table. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see “Default Permissions” in *SAS Intelligent Decisioning: Administrator’s Guide* and “Folder-Based Permissions” in *SAS Intelligent Decisioning: Administrator’s Guide*. By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of lookup tables or value lists, the `sas.referencedata.deleteVersions` configuration option must be turned on.

On the object’s **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version. You cannot delete an active version.

Determine Which Objects Use a Lookup Table

On the **Lookup tables** category page, select the lookup table, click , and select **View used by report**. The report lists all objects that use an active version of the selected lookup table.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Lookup Table Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.referencedata.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a lookup table version, the **Check Out** button for that lookup table is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Lookup Table Versions” on page 163](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Lookup Tables category view shows two lookup tables with the same name, but the folders listed in the **Location** column differ for each lookup table. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

You cannot activate the checked-out version that is in **My Folder**. To activate a version with your changes, you must commit your changes, and activate the committed version.

Checking Out and Committing a Lookup Table from within a Rule Set

You can check out a lookup table from within a rule set if the following conditions are true:

- The rule set is checked out.
- The lookup table is stored in a folder for which the check-out feature is enabled.
- The rule set does not have any unsaved changes. If the rule set contains any unsaved changes, the **Check out lookup** option is disabled.

To check out or commit a lookup table from within a checked-out rule set, right-click on the LOOKUP or LOOKUPVALUE expression, and select **Check out lookup** or **Commit lookup**.

When you check out a lookup table from within a rule set, SAS Intelligent Decisioning adds an asterisk in front of the checked-out lookup table, and rule set is modified to use the checked-out copy of the lookup table.

To open the checked-out lookup table, click  to navigate to the Lookup Tables category view, click , and open the lookup table in **My Folder**.

Commit the lookup table from within the rule set. When you commit the rule set, all of the checked-out lookup tables that are used in the rule set are also committed. The newly committed versions of the lookup tables are not automatically activated when they are committed. You must activate the newly committed versions of the lookup tables. For more instructions, see [“Activate a Lookup Table” on page 169](#).

Rule sets that are not checked out always use the activated versions of lookup tables. If you do not activate the newly committed versions of the lookup tables, all rule sets that use the lookup tables will continue to use the previously activated versions of each table.

To cancel the check out of a lookup table from within a rule set, right-click on the LOOKUP or LOOKUPVALUE expression, and select **Cancel check out**.

Committing a Lookup Table Together with a Rule Set

If you check out a lookup table from within a rule set and then commit the rule set, the lookup table is also committed. For more information, see [“Checking Out and Committing a Lookup Table from within a Rule Set” on page 162](#).

Alternatively, you can check out a rule set that uses a lookup table, open the lookup table in the **Lookup tables** category view and check it out, then change the reference in the rule set to use the checked-out version of the lookup table in `My Folder`. The lookup table is committed when you commit the rule set.

The newly committed version of the lookup table is not automatically activated when they are committed. You must activate the newly committed version of the lookup table. For more instructions, see [“Activate a Lookup Table” on page 169](#).

Concurrently Checking Out and Committing Lookup Table Versions

The ability for multiple users to check out the same lookup table at the same time is controlled by the `sas.referencedata.checkout.allowConcurrentCheckout` configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's `My Folder` location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Lookup Table Version

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 165](#).

- 4 On the **Versions** tab, click **Commit**. The Commit Lookup Table Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 7 (Optional) Enter information about the new version in the **Notes** field.
- 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

How you undo the checkout of an object depends on how the object was checked out.

If you have [checked out a lookup table from within a rule set](#) right-click the LOOKUP or LOOKUPVALUE expression, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a rule set that uses the lookup table.
- The lookup table was not [checked out from within a rule set](#) after the rule set was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Activating a Lookup Table

What Does Activating a Lookup Table Do?

Activating a lookup table makes the table available for use by published objects. By default, when you activate a lookup table, the lookup table is converted into executable code in the form of a SAS Cloud Analytic Services (CAS) format. If your site has a SAS Micro Analytic Service publishing destination, then a SAS Micro Analytic Service module is also created for the table. The CAS format is used when you create and run tests and scenario tests (pre-publishing tests) for the rule sets and decisions that use the lookup tables. The CAS format and SAS Micro Analytic Service module also enable objects that are published to those destination types to use the most recently activated version of a lookup table instead of a static copy.

However, your administrator can set configuration options to specify that SAS Intelligent Decisioning includes lookup tables inline in generated code instead of using formats and modules for CAS and SAS Micro Analytic Service destinations. For more information, see [“Controlling Where Lookup Tables Are Activated And How They Are Used” on page 167](#).

Teradata, Hadoop, Git, and container destinations do not support the format or module that is created when you activate a lookup table. Objects that are published to these destinations must include a static version of each lookup table.

You must activate a version of any lookup table that is used in a rule set or decision. When you activate a version of a lookup table, that version is locked and cannot be edited. Each lookup table can have only one active version.

The active version of a lookup table is used in the following ways:

- when you run a test or scenario test prior to publishing an object.
- to generate a static copy of the table when you publish an object if a static copy is needed, such as when you publish the object to Teradata, Hadoop, Git, or container destinations. In order to generate the static copy for Teradata or Hadoop destinations, the `lookupStaticBinding` configuration property must be set to On. Static copies are automatically generated for Git and container destinations.
- by published objects in CAS and SAS Micro Analytic Service destinations if the [lookupStaticBinding configuration property](#) is set to Off.

Controlling Where Lookup Tables Are Activated And How They Are Used

Your administrator can use the following configuration options to control where lookup tables are activated and how they are used:

`sas.referencedata.activation.destinations = destinations`

specifies the SAS Micro Analytic Service publishing destinations in which lookup tables are activated. Only one SAS Micro Analytic Service destination is supported on `localhost`. By default, that destination is named `maslocal`. For more information, see [“sas.referencedata.activation.destinations” in SAS Intelligent Decisioning: Administrator’s Guide](#).

`sas.referencedata.activation.masOnly = On | Off`

specifies whether lookup tables and global variables are activated for SAS Micro Analytic Service destinations only.

`sas.referencedata.activation.skipOfflineCAS = On | Off`

enables lookup tables and global variables to be activated if at least one CAS server is online. Objects are not activated on servers that are offline.

`sas.referencedata.publish.lookupDisableMasPublish = On | Off`

controls whether lookup tables are activated in your SAS Micro Analytic Service publishing destination. When this option is set to `Off` and you activate a lookup table, it is activated in this destination. If this option is set to `On`, then lookup tables are not activated in this destination, and your administrator must set the `sas.businessrules.lookupStaticBinding` option to `On` in order to embed static copies of lookup tables in generated code and ensure that published content executes correctly. For more information, see [“sas.referencedata.publish.lookupDisableMasPublish” in SAS Intelligent Decisioning: Administrator’s Guide](#).

`sas.businessrules.lookupStaticBinding = On | Off`

controls whether a static copy of the active version of a lookup table is embedded in the generated code for an object when that object is published. When this option is set to `On`, a static copy of the active version of any lookup table that is used in an object is embedded in the generated code when those objects are published. When this option is set to `Off`, a published object’s use of lookup tables depends on the destination to which they were published:

- Objects that are published to SAS Cloud Analytic Services (CAS) and SAS Micro Analytic Service destinations use the most recently activated version of lookup tables.
- Objects that are published to Teradata and Hadoop destinations cannot use lookup tables.

Note: Static versions of lookup tables are automatically embedded in content published to Git and container destinations. This option is ignored for those destination types.

For more information, see “[sas.businessrules.lookupStaticBinding](#)” in *SAS Intelligent Decisioning: Administrator’s Guide*.

TIP To publish very large lookup tables to CAS destinations, set both of the following options to On:

- `sas.referencedata.publish.lookupDisableMasPublish`
- `sas.businessrules.lookupStaticBinding`

Table 5.1 Interaction of Lookup Table Configuration Settings

<code>sas.referencedata.publish.lookupDisableMasPublish</code>	<code>sas.businessrules.lookupStaticBinding</code>	SAS Micro Analytic Service	CAS	Teradata and Hadoop	Git Destinations and Container Destinations
Off	Off	Published objects use the most recently activated versions of lookup tables.	Published objects use the most recently activated versions of lookup tables.	Published objects cannot use lookup tables.	Published objects use the embedded versions of lookup tables.
Off	On	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.
On	Off	Do not use lookup tables. These option settings might produce incorrect results if a lookup table was previously activated in a SAS Micro Analytic Service destination.	Published objects use the most recently activated versions of lookup tables.	Published objects cannot use lookup tables.	Published objects use the embedded versions of lookup tables.
On	On	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.	Published objects use the embedded versions of lookup tables.

Activate a Lookup Table

Activating a lookup table locks it. If you activate the only unlocked version of a lookup table, SAS Intelligent Decisioning automatically creates a new unlocked version.

You cannot activate a lookup table that has been checked out. You must commit the changes to the lookup table before you can activate it.

To activate the latest version of a lookup table that is not in a folder for which the check-out feature has been enabled, open the lookup table, and click **Activate** at the top of the window.

To activate a specific version of the lookup table, or to activate a lookup table that is in a folder for which the check-out feature is enabled, you must activate the lookup table on the **Versions** tab:

- 1 Open the lookup table that you want to activate.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, click the version number of the version that you want to activate. See [“Set the Displayed Version” on page 157](#) for more information.
- 3 Click **Activate**.

LOOKUP Function

Determines whether a lookup key exists in a lookup table.

Restrictions:	You can specify the LOOKUP function only in condition expressions. If an expression contains the LOOKUP function, then the expression cannot contain anything else.
Returned data type:	Boolean

Syntax

LOOKUP ('lookup_table_name', variable_or_value)

Required Arguments

lookup_table_name

specifies the name of the lookup table that you want to search.

variable_or_value

specifies either the literal key value or a variable that contains a lookup key value.

Example

Suppose you have a Country_Codes lookup table that uses two-letter abbreviations for countries as the lookup key and country names as the lookup values.

	Key 	Value 
☐	AU	Australia
☐	BR	Brazil
☐	CA	Canada
☐	CR	Costa Rica

To verify that the value of the variable Country_Key exists as a lookup key in the table Country_Codes, you can use the following expression:

```
LOOKUP ('Country_Codes', Country_Key)
```

If the value of Country_Key exists as a lookup key, the LOOKUP function returns the value **True**.

In the following rule, if the key specified by the variable Country_Key exists in the lookup table Country_Codes, then the value that is associated with that key is assigned to the variable Country_Name.

IF	Country_Key	LOOKUP	Country_Codes	
THEN	LOOKUPVALUE	Country_Name	Country_Codes	Country_Key

LOOKUPVALUE Function

Retrieves a lookup value from a lookup table.

Restrictions: You can specify the LOOKUPVALUE function only in action expressions. If an expression contains the LOOKUPVALUE function, then the expression cannot contain anything else.

Returned data type: CHARACTER, INTEGER, DECIMAL, DATE, DATETIME, BOOLEAN

Tip: Lookup tables are stored as character data. However, you can assign the results of the LOOKUPVALUE function to any of the returned data types. The LOOKUPVALUE function converts the results to match the type of the variable.

Syntax

LOOKUPVALUE ('lookup_table_name', variable_or_value)

Required Arguments

lookup_table_name

specifies the name of the lookup table that you want to search.

variable_or_value

specifies either the literal key value or a variable that contains the lookup key value.

Example

Suppose you have a Country_Codes lookup table that uses two-letter abbreviations for countries as the lookup key and country names as the lookup values. The Country_Codes lookup table contains the lookup key **CA**, and the lookup value that corresponds to that key is **Canada**.

	Key 	Value 
☐	AU	Australia
☐	BR	Brazil
☐	CA	Canada
☐	CR	Costa Rica

If the Country_Key variable in the current input record contains the value **CA**, you can use the following expression to retrieve the lookup value that is associated with that key from the table Country_Codes:

```
LOOKUPVALUE('Country_Codes',Country_Key)
```

In the following rule, if the key specified by the variable Country_Key exists in the lookup table Country_Codes, then the value that is associated with that key is assigned to the variable Country_Name.

IF	Country_Key	LOOKUP	Country_Codes
THEN	LOOKUPVALUE	Country_Name	Country_Codes Country_Key

Managing Global Variables

<i>Using Global Variables</i>	173
<i>Create a New Global Variable</i>	174
<i>Managing Global Variables</i>	175
Duplicate a Global Variable	175
Hide a Global Variable	175
Edit a Global Variable	176
<i>Restore or Permanently Delete Hidden Global Variables</i>	176
<i>Managing Versions of Global Variables</i>	177
Set the Displayed Version	177
Create a New Version	177
Delete a Version	178
<i>Determine Which Objects Use a Global Variable</i>	179
<i>Activating a Global Variable</i>	179
What Does Activating a Global Variable Do?	179
Controlling Whether Global Variables Are Included Inline in Code	180
Which Value Is Used for a Global Variable?	181
Activate a Global Variable	182

Using Global Variables

You can use global variables to define common variables for use in multiple rule sets and decisions. For example, you might want to define global variables for benchmark interest rates, current exchange rates, or inventory levels for specific products. By using global variables, you can set these values in one place and refer to the variables in multiple objects.

TIP You cannot use global variables in models in SAS Model Manager. You can pass the value of a global variable into a model by mapping a model input parameter to a global variable. For more information, see [“Mapping Variables within a Decision” on page 436](#).

After you create a global variable, you can add it to the **Global Variables** tab of a rule set or a decision. You can use global variables in conditional expressions, but you cannot modify the value of a global variable in a rule set or in a decision.

When you add a rule set that uses a global variable to a decision, SAS Intelligent Decisioning creates a decision variable with the same name as the global variable. When you add a common rule set to an assignment rule set and the common rule set uses global variables, you must manually add the global variables to the assignment rule set. For more information, see [“Add Global Variables to a Rule Set” on page 37](#) and [“Add Global Variables to a Decision” on page 400](#).

Note: The decision variable is not a global variable, but it has the same name and value as the global variable. For more information about decision variables, see [“About Decision Variables and Mapping” on page 436](#).

You must activate a global variable in order to use it in a rule set or decision. For more information, see [“Activating a Global Variable” on page 179](#).

Create a New Global Variable

- 1 Click  to navigate to the Global Variables category view.
- 2 Click **New global variable**.
- 3 In the New Global Variable window, enter a name for the variable. The name is limited to 32 characters.
- 4 (Optional) Enter a description for the new global variable. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 5 Select the variable's data type.
- 6 Enter a value for the variable. To enter a value for a date or datetime variable, click  or , and select the value in the date or datetime picker.

Note: The value of a character variable is limited to 1998 characters. Values greater than 1998 characters produce an error.

- 7 Click **Save** to create the variable.

Managing Global Variables

Duplicate a Global Variable

To duplicate a single global variable:

- 1 In the Global variables category view, select the variable that you want to duplicate, click , and select **Duplicate**.
- 2 In the Duplicate Global Variable window, enter a new name for the duplicate global variable. Names are limited to 250 characters. Global variable names are case insensitive and must be unique within the database.
- 3 (Optional) Enter a description for the duplicate variable. Descriptions are limited to 1000 characters.
- 4 Select the version of the global variable that you want to duplicate.
- 5 Click **Duplicate**.

To duplicate multiple global variables, select the variables that you want to duplicate, click , and select **Duplicate**. SAS Intelligent Decisioning duplicates the latest version of the global variables and appends **_Copy_n** to the names of the duplicate copies.

Hide a Global Variable

Hiding a global variable removes it from the list that is shown in the Global variables category view. The variable is moved to the list of [hidden variables](#).

Note: A global variable that is hidden might be referenced in objects that have already been published. To remove references to global variables from a published object, you must edit the object, remove the references, and republish the object.

In the Global variables category view, select the variables that you want to hide, click , and select **Hide**.

Edit a Global Variable

You cannot rename a global variable. You cannot edit the description or value of a global variable version that has been activated.

To edit the description or value of the current version of a global variable, click on the variable name in the Global Variables category view. On the **Properties** tab, modify the description or the current value, and click  to save your changes.

Note: On the **Properties** tab for the latest version of the variable, you can change the value of the variable by entering a new value in the **Current value** field. The **Activated value** field, and the **Latest version value** field are read-only. After you save your changes, the value of the **Latest version value** field changes to match the value that you entered in the **Current value** field.

On the **Properties** tab for previous locked versions, the value of the **Current value** field is the value of that locked version and cannot be changed.

Restore or Permanently Delete Hidden Global Variables

- 1 In the Global variables category view, if there are no global variables visible, click **Manage hidden global variables**. If global variables are visible, click , and select **Manage hidden global variables**.
- 2 In the Manage Hidden Global Variables window, select the global variable that you want to restore or delete.
- 3 Click  to permanently delete the selected variable, or click  to unhide (restore) it.

Managing Versions of Global Variables

Set the Displayed Version

The displayed version is the version whose information is displayed on the **Properties** tab. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Lookup Table Version” on page 164](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.

- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see [“Default Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#) and [“Folder-Based Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#). By default, users have permission to delete objects that they create and any versions of those objects.

On the object’s **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version. You cannot delete an active version.

Determine Which Objects Use a Global Variable

On the **Global variables** category page, select the global variable, click , and select **View used by report**. The report lists all objects that use an active version of the selected global variable.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Activating a Global Variable

What Does Activating a Global Variable Do?

Activating a global variable makes that variable available for use by published objects.

You must activate a version of any global variable that is used in a rule set or in a decision. When you activate a version of a global variable, that version is locked and cannot be edited. Each global variable can have only one active version.

The active version of a global variable is used in the following ways:

- to generate a static copy of the variable when needed. Static copies are used when you publish an object to CAS or SAS Micro Analytic Service destinations and the `inlineGlobalVariableValues` configuration properties are set to On. Static copies are also used when you publish the object to Teradata, Hadoop, Databricks, Git, or container destinations.

- by published objects in CAS and SAS Micro Analytic Service destinations if the `inlineGlobalVariableValues` configuration properties are set to Off. See [“Controlling Whether Global Variables Are Included Inline in Code”](#) on page 180 for more information.

Controlling Whether Global Variables Are Included Inline in Code

Your administrator can use the following configuration options to control whether the values of global variables are included inline in generated code.

`sas.businessrules.inlineGlobalVariableValues = On | Off`

specifies whether the active value of each global variable is included inline in generated code when you publish a rule set to SAS Cloud Analytic Services (CAS) or SAS Micro Analytic Service destinations.

`sas.decisions.inlineGlobalVariableValues = On | Off`

specifies whether the active value of each global variable is included inline in generated code when you publish a decision to CAS or SAS Micro Analytic Service destinations.

Note: Static versions of global variables are automatically included inline in content that is published to Teradata, Hadoop, Databricks, and container destinations. These options are ignored for those destinations.

When either of these configuration options are set to On, the active value of each global variable is included inline in the rule set code or decision code that is generated for CAS and for SAS Micro Analytic Service destinations. When the value of a variable is included inline in generated code, changes to the values of the variables do not affect published objects.

If either of these configuration options are set to Off and you activate a global variable, then the global variable is converted into executable code in the form of a CAS format. If your site has a SAS Micro Analytic Service publishing destination, then a SAS Micro Analytic Service module is also created for the variable. The CAS format and SAS Micro Analytic Service module enable objects that are published to those destination types to retrieve the most recently activated version of a global variable instead of using a static inline copy. Teradata, Hadoop, Databricks, Git, and container destinations do not support the CAS format or SAS Micro Analytic Service module.

If these options are not set to the same value, the value of the `businessrules` option takes precedence the value of the `decisions` option.

Which Value Is Used for a Global Variable?

A global variable can have three different values at the same time.

active value

The value that you most recently activated. For more information, see [“What Does Activating a Global Variable Do?” on page 179](#) and [“Activate a Global Variable” on page 182](#).

latest value

the value of the latest version of the global variable. The latest value might or might not be activated.

inline value

The value that was included in the generated code for a rule set or for a decision. When the code is generated, the active value can be included inline in the generated code. Whether the active value is included inline in the generated code is determined by the settings of the `inlineGlobalVariableValues` configuration options and by the publishing destination. For more information, see [“Controlling Whether Global Variables Are Included Inline in Code” on page 180](#).

The value that is used for a global variable depends on whether you are running a test or executing a published object, and on the settings of the `inlineGlobalVariableValues` configuration options.

Table 6.1 Global Variable Values

decisions.inlineGlobalVariableValues= businessrules.inlineGlobalVariableValues=	On On	Off Off	On Off	Off On
Published Code and Publishing Validation Tests				
decision that is published to CAS or to SAS Micro Analytic Service destinations	inline	active	inline	active
standalone rule set that is published to CAS or to SAS Micro Analytic Service destinations	inline	active	active	inline
rule set that is included in a decision that was published to CAS or to SAS Micro Analytic Service destinations	inline	active	active	inline
decision or rule set that is published to container destinations, Teradata, Hadoop, Git, or Databricks	inline	inline	inline	inline
Basic Tests and Scenario Tests				
decision	active	latest	active	latest
standalone rule set	active	latest	latest	latest

decisions.inlineGlobalVariableValues=	On	Off	On	Off
businessrules.inlineGlobalVariableValues=	On	Off	Off	On
rule set that is contained in a decision	active	latest	latest	latest

Activate a Global Variable

Activating a version of a global variable locks that version. Locked variables cannot be unlocked.

You must activate a global variable before you can publish an object that uses the variable. When you activate a global variable, it is activated in all of the publishing destinations that are configured at your site.

- 1 Open the global variable that you want to activate.
- 2 On the **Versions** tab, select the version that you want to activate, and click **Activate**. If you activate the latest version, SAS Intelligent Decisioning creates a new minor version before it locks the current latest version.

Using Custom Code Files

Introduction to Custom Code Files	184
DS2 Code Files	185
Rules for Creating DS2 Code Files	185
Example: Custom DS2 Code File	187
Testing DS2 Code Files in SAS Data and AI Studio	187
Python Code Files	189
About Publishing Destinations and Python Distributions	189
Rules For Developing Python Code Files	189
Required Dependencies For Python Files in SAS Container	
Runtime Destinations	191
Data Query Files	192
Using Data Query Files	192
Query Output Types and Editors	193
Developing SQL Code	193
Decision Variables for Data Query Files	194
Using Macro Variables in SAS Data and AI Studio	195
Handling Table and Column Names in Data Query Files	196
Custom Context Files	197
Using Custom Context Files	197
Adding an INIT Method to a Custom Context File	198
Associating a Custom Context File with a Decision	198
Testing or Validating an Object That Uses a Custom Context Functions	199
Referencing a Function That Is Defined in a Context File	199
Referencing a Variable That Is Defined in a Context File	200
Create a New Code File	200
Managing the Variables in a DS2 or Python Code File	202
About Variables	202
Syncing Variables and Code	205
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object	206
Create Custom Variables	207
Duplicate a Variable	208
Importing and Exporting Variables	208
Delete Variables	209
Edit Variable Properties	209
Edit Metadata for Data Grid Variables	210
Determine Which Objects Use a Particular Variable	210

<i>Validate SQL, DS2, or Context Files</i>	210
<i>Copy a Code File URL</i>	211
<i>Compare Generated Code for Code Files</i>	211
<i>Managing Code Files</i>	212
Duplicate Code Files	212
Delete Code Files	213
Rename a Code File	213
Move Code Files to a Different Folder	213
<i>Managing Versions of Code Files</i>	213
Set the Displayed Version	213
Create a New Version	214
Copy the Content of a Version	215
Delete a Version	216
Upgrade Decisions to Use a New Version of a Code File	216
<i>Determine Which Objects Use a Code File</i>	217
<i>Checking Out and Committing Code File Versions</i>	218
About Checking Out and Committing Versions	218
Checking Out a Code File from within a Decision	219
Concurrently Checking Out and Committing Code File Versions	219
Check Out and Commit a Custom Code File Version	220
Determine Who Has a Version Checked Out	221
Opening the Original Object	221
Undoing a Check Out	221
<i>Testing Code Files</i>	222
Ways to Test a Code File	222
Test a Code File	222
Test a Scenario	226
Working with Test Output Data	230
Status Icons for Tests	230
Duplicate Tests	231
<i>Manage Comments for a Code File Test</i>	232

Introduction to Custom Code Files

You can define custom code files to do things that are not possible in rules, models, or treatments. For example, you can define a code file that makes HTTP calls to REST APIs, interacts with a database, manipulates files in the file system, or performs custom data transformations. You can use custom context files to define functions that you can use in rule sets, in custom DS2 code files, and in custom functions that are defined in the Custom Functions category.

IMPORTANT You are responsible for ensuring that only users who are authorized to do so can install Python libraries, develop, and test code nodes that use SQL or Python, and execute decisions that use custom code nodes.

IMPORTANT All changes that you make to a code file affect all unpublished decisions that use that file. For more information, see [“Content That Is Used by Tests and Scenarios for Decisions” on page 496](#) and [“Content Executed by Published Decisions” on page 504](#).

Note: Not all of the code that you can write in a custom code file will work in all publishing destinations or in decision tests.

For information about adding a custom code file to a decision, see [“Adding Objects to a Decision” on page 405](#).

DS2 Code Files

Rules for Creating DS2 Code Files

When you are developing your DS2 package, follow these rules:

- Do not change the package name in the PACKAGE statement:

```
package "${PACKAGE_NAME}" /inline;
```

This line must appear exactly as shown and must begin in column one of the code file. The token is replaced with a package name that SAS Intelligent Decisioning uses to maintain the relationship between the code file and the decisions that use it.

- If your custom DS2 code file defines multiple packages, the packages that are used by the `${PACKAGE_NAME}` package must be defined before the `${PACKAGE_NAME}` package.
- You can specify DS2 options in your DS2 code file by adding the [DS2_OPTIONS statement](#) before the PACKAGE statement.
- Custom DS2 code files in SAS Intelligent Decisioning support only three data types: double, varchar, and package datagrid. These data type names are case sensitive. List input-only parameters first. List input-output and output-only parameters next, using the `in_out` modifier. Do not specify a length for input-output and output-only parameters. The length for these parameters is derived from the variables that are passed into the method.

For example, the following lines define the variables LOAN, REASON, and ASSETS.

```
package datagrid "assets"
double "loan"
IN_OUT varchar "reason"
```

IMPORTANT Before you validate a DS2 code file or custom context file that uses a data grid package, verify that the declaration statements for the package specify `datagrid` instead of `dcm_datagrid`. For example:

```
in_out package datagrid variable;

dcl package datagrid variable;
```

Note: Packages are input-output data types.

- Your package must define an EXECUTE method.
- The variables that you use as parameters for the EXECUTE method in your code must be mapped to input variables in your decision.
- Do not enter comments within the signature of the method parameters in your code file. The comment characters cause the signature to be parsed incorrectly.
- The DS2 [SQLSTMT package](#) is not supported on the CAS Server. To query a database, use a custom data query file instead of using a SQLSTMT package in a DS2 code file. For information, see [“Data Query Files” on page 192](#).
- Custom DS2 code files are executed by using the runDS2 CAS action. This action supports varying-length binary variables, but it does not support binary variables.

For information about developing DS2 packages, processing data grids, and the SAS APIs, see the following:

- [SAS DS2 Programmer's Guide](#)
- [SAS DS2 Language Reference](#)
- [SAS Intelligent Decisioning: Using Data Grids](#)
- [“Reserved Words in the DS2 Language” in SAS DS2 Programmer's Guide](#)
- <https://developer.sas.com>
- [“DS2 Programming for SAS Micro Analytic Service” in SAS Micro Analytic Service: Programming and Administration Guide](#)
- [“Best Practices for DS2 Programming in SAS Intelligent Decisioning” in SAS Micro Analytic Service: Programming and Administration Guide](#)

Example: Custom DS2 Code File

The following DS2 package sends a request to the external API <http://helloacm.com/api/fortune>. This API returns a character string that contains escaped characters. The GETFORTUNE method uses the DS2 function TRANSTRN to modify these characters.

```
package "${PACKAGE_NAME}" /inline;
  dcl package http fortune_pkg();
  dcl varchar(1048576) character set utf8 http_response;
  dcl int rc;
  dcl int getMethodDefined;

  method execute(in_out varchar "fortune");
    if missing(getMethodDefined) then do; /* establish GET method */
      fortune_pkg.createGetMethod('http://helloacm.com/api/fortune');
      "getMethodDefined" = 1;
    end;

    "fortune" = 'Gloom and Doom'; /* default fortune */
    fortune_pkg.executeMethod();
    fortune_pkg.getResponseBodyAsString(http_response, rc);
    if (rc = 0) then do; /* clean up escaped characters in the response */
      "fortune" = transtrn(http_response, '\n', ' ');
      "fortune" = transtrn("fortune", '\t', ' ');
      "fortune" = transtrn("fortune", '\"', '\"');
    end;
  end;
endpackage;
```

Testing DS2 Code Files in SAS Data and AI Studio

Note: If your custom code uses data grid variables, see [“Working with Data Grids in SAS Data and AI Studio” in SAS Intelligent Decisioning: Using Data Grids](#).

If your DS2 custom code file uses functions that are defined in a custom context file, insert the packages that are defined in the custom context file before the DS2 code file package.

Replace the placeholder name `${PACKAGE_NAME}` with a valid DS2 package name.

To test your package in a separate DS2 invocation from where the calling program is running, replace the inline modifier with `overwrite=yes`.

```
proc ds2;
  package "testCustomCode" /overwrite=yes;
  method execute(double l, double w, double h, double d,
    in_out double vol, in_out double wgt);
```

```

        vol = l * w * h;
        wgt = vol * d;
    end;
endpackage;
run;
quit;

```

In the following example, the package is compiled in the same PROC DS2 invocation as the DS2 data program that instantiates the package, so the code specifies the `/inline` package modifier.

```

/* Create test data. */
data work.testdata;
    length material $13;
    long=40; wide=20; high=10; density=0.098; material = 'aluminum'; output;
    long=20; wide=10; high=4; density=0.284; material = 'iron'; output;
run;

proc ds2;

    /* Replace the placeholder name with a package name. */
    package "testCustomCode" /inline;
        method execute(double l,
                        double w,
                        double h,
                        double d,
                        in_out double vol,
                        in_out double wgt);

            vol = l * w * h;
            wgt = vol * d;
        end;
    endpackage;

/* Use a DS2 data program to execute the custom code. */
data _null_;
    dcl package testCustomCode myCustomCode(); /* Instantiate the package. */
    dcl double volume;
    dcl double weight;

    method run();
        /* Read in the variables long, wide, high, and density. */
        set work.testdata(drop=(material));
        volume = .;
        weight = .;
        myCustomCode.execute(long, wide, high, density, volume, weight);
        put _all_;
    end;
enddata;
run;
quit;

```

Python Code Files

About Publishing Destinations and Python Distributions

IMPORTANT You can publish decisions that contain Python code files only to SAS Micro Analytic Service and compatible Git destinations, to SAS Cloud Analytic Services (CAS) and compatible Git destinations, and to container destinations.

When you publish a decision that contains a Python code file, SAS Intelligent Decisioning generates a private DS2 PyMAS package. The package is assigned a random name. Your Python program is encapsulated inside a DS2 EXECUTE method. When the decision is executed, the DS2 process sends the Python program to a Python process to be executed.

In SAS Micro Analytic Service and CAS destinations, an executing decision uses the Python distribution that is installed at your site. In container destinations, executing decisions use the Python base image that is included in the published container. You can specify the Python version that you want to use for a decision in the decision's properties. For more information, see [“Specifying a Python Version for a Decision” on page 435](#). For information about the Python base image that is included in published containers, see [“Understanding the SAS Container Runtime Base Image” in SAS Container Runtime: Programming and Administration Guide](#). For information about the standard libraries that are distributed with specific versions of Python, see <https://docs.python.org>.

Rules For Developing Python Code Files

When you are developing your Python code, follow these rules:

- Your Python code must define an EXECUTE function. This function is the only public function allowed.
- When you [sync the variable definitions and the Python code](#) SAS Intelligent Decisioning syncs the **Variables** tab and the last RETURN statement in the file. If your code file defines multiple functions, ensure that the EXECUTE function is the last function in the file.
- An `Output : docstring` is required. This string must immediately follow the Python EXECUTE function declaration, and it must be indented within the

EXECUTE function definition. In the docstring, list all of the output variables produced by the program. For example, if your program has two output variables named `prediction` and `probability`, your docstring would appear as follows:

```
'''Output: prediction, probability'''
```

- If your Python code uses packages that are not built-in packages, list these packages in the `DependentPackages: docstring`. For more information, see [“Required Dependencies For Python Files in SAS Container Runtime Destinations” on page 191](#).
- Optional input arguments to the Python EXECUTE function are not supported.
- You must specify the data types for the input and output variables for the EXECUTE function in your Python code on the **Variables** tab of the decision. The DS2 package code uses this information to resolve the data types for the variables. These variables are specified in the signature of the DS2 EXECUTE method. If you add a Python code file to a decision and you have not specified the variables on the **Variables** tab, SAS Intelligent Decisioning might display the following message: `Some objects in the decision define variables for which no corresponding decision variables have been created`. For more information, see [“About Decision Variables and Mapping” on page 436](#).
- In order to use a data grid variable in Python code, you must serialize the data grid into a JSON string. You can use the [DATAGRID_TOSTRING](#) function to create the JSON string for a data grid. For more information, see the article [How to Access a Data Grid inside a Python Code Node in SAS Intelligent Decisioning](#) on the SAS Communities site.
- The Python EXECUTE function must return standard Python data types. For more information, see [“Return Values” in SAS Micro Analytic Service: Programming and Administration Guide](#).
- You can import modules and define other functions and classes that are used by the Python EXECUTE function.
- Test your Python code by using a Python interpreter that is outside of SAS Intelligent Decisioning before you incorporate your code into a decision.

IMPORTANT After you test your code outside of SAS Intelligent Decisioning, incorporate it into a decision and test the decision. In order to run a test for a decision that contains a Python code file, you or an administrator must configure support for Python. See [“Configuring Support for Python Code Files” in SAS Intelligent Decisioning: Administrator’s Guide](#) for more information.

For more information about using Python with DS2, see the following:

- [“Python Support in SAS Micro Analytic Service” in SAS Micro Analytic Service: Programming and Administration Guide](#)
- [“Executing Python Modules in DS2 Modules” in SAS Micro Analytic Service: Programming and Administration Guide](#)

TIP If you encounter errors that are unrelated to syntax or logic, check with your administrator to verify that Python is configured correctly. For additional information, see “[Configuring Support for Python Code Files](#)” in *SAS Intelligent Decisioning: Administrator’s Guide*.

Required Dependencies For Python Files in SAS Container Runtime Destinations

If you plan to publish a decision that includes a Python code file to a SAS Container Runtime destination, and if your Python file uses packages that are not built-in packages, list these packages in the `DependentPackages`: docstring. List all of the package names on a single line. Use the official package names. Separate the package names with commas. For example:

```
'''DependentPackages: sklearn, requests, dateutil'''
```

When you publish the decision, SAS Intelligent Decisioning passes the name and `pip3` install command for each package to the model publish service. The model publish service uses this information to install the packages in the container that is published to the destination.

For information about the Python base image that is included in published containers, see “[Understanding the SAS Container Runtime Base Image](#)” in *SAS Container Runtime: Programming and Administration Guide*.

Note: Do not specify package aliases or the names of built-in packages in this docstring. If you include the name of a built-in package or of a package alias, SAS Intelligent Decisioning writes the following message to the log when you publish the decision:

```
ERROR: No matching distribution found for name
```

Verify whether a package is built in to the Python distribution that will be used by the executing decision before you add a package name to this docstring. Verify that the docstring is correct by running a publishing validation test for your decision before you deploy your decision in a production environment. For information about the standard libraries that are distributed with specific versions of Python, see <https://docs.python.org>.

Data Query Files

Using Data Query Files

A query enables you to extract data from one or more tables according to criteria that you specify. When you create the query, you can choose which editor you want to use to create the file. Your choice of editor affects whether the data query can return scalar variables. For more information, see [“Query Output Types and Editors”](#) on page 193.

IMPORTANT You can publish decisions that contain data query files to SAS Micro Analytic Service destinations or to container destinations. For SAS Micro Analytic Service destinations, you can query only the data sources that are supported by SAS Micro Analytic Service. For more information, see [“Data Sources Supported for Use with SAS Micro Analytic Service”](#) in *SAS Micro Analytic Service: Programming and Administration Guide*.

In container destinations, only certain database types can be queried by using data query files. For information, see [“Configuring a Database Connection”](#) in *SAS Container Runtime: Programming and Administration Guide*.

In order to test decisions that contain data query files, an administrator must configure support for SQL query files. See [“Configuring Support for Querying Databases”](#) in *SAS Intelligent Decisioning: Administrator's Guide*.

IMPORTANT Tests that are running in SAS Intelligent Decisioning might encounter a significant performance impact. Pre-publish testing requires SAS Intelligent Decisioning to convert the SQL code to use HTTP protocol and instantiate a MAS module for each call to the SQL package. These actions incur a significant performance impact and create a limitation on the size of the results table. When you test a decision that uses a data query node, use an input data set that is as small as possible.

Query Output Types and Editors

When you [create a new data query file](#), you can choose to create the file in the SQL editor or in SAS Data and AI Studio.

Files that are created in the SQL editor can return either a data grid or a single row of scalar variables. You can select the output type on the **Properties** tab of the data query file.

Files that are created in SAS Data and AI Studio are created as stand-alone queries. These files return a data grid. Do not change the output type for these files. For more information, see [“Understanding the Differences between a Standalone Query and a Flow Query” in SAS Data and AI Studio: User’s Guide](#).

For information about data grids, see [SAS Intelligent Decisioning: Using Data Grids](#).

Developing SQL Code

When you are developing your SQL code, follow these rules:

- Data query nodes support SELECT, INSERT, UPDATE, and DELETE statements. They do not support any data definition language (DDL) statements such as ALTER or DROP that alter the structure of the table.
- Data query nodes support only the following data types: decimal, string, date, datetime, and integer. You must specify a length for string variables. You cannot specify a length for other variable types. Specify input variables with a question mark (?). Use the AS keyword to specify the input and output variables in the decision as aliases for the database column names. Enclose the decision variable specifications in braces ({}). For example, in the following SELECT statement, DEBTINC, CAUSE, and BADLOAN are decision variables. The variable BADLOAN is an input variable in the decision. This statement retrieves the values of the DEBTINC, REASON, and BADLOAN columns from the database and assigns their values to the decision variables.

```
SELECT debtinc AS { :debtRatio:decimal },
       reason AS { :cause:string:8 }
FROM hmeq_test WHERE bad = {?:badloan:decimal}
```

The following INSERT statement adds a row that contains the columns NAME and AMT to the table MYTABLE:

```
INSERT INTO mytable (name, amt)
values ({?:name:string:1000},{?:amt:integer})
```

Note: You can read data from native SQL date and datetime variables. However, do not rely on the accuracy of data that contains fractional seconds.

- Do not use an asterisk (*) to select database columns. SAS Intelligent Decisioning does not have any metadata about the table, so it cannot determine what columns are in the table.
- When you create a new data query file, SAS Intelligent Decisioning adds the comment `/* include sqlReturnInfo */` to the file. This comment enables SAS Intelligent Decisioning to define metadata for the query's variables. Do not delete this line if you want SAS Intelligent Decisioning to create all of the metadata. For more information, see [“Decision Variables for Data Query Files” on page 194](#).
- When you create data query files in SAS Data and AI Studio, you must specify the **PROC FEDSQL** output option on the **Output Options** tab in SAS Data and AI Studio. For more information, see [“Generating a FedSQL Query” in SAS Data and AI Studio: User's Guide](#).

For additional information about developing SQL code, see the following topics:

- For information about the data sources that are supported by SAS Micro Analytic Service, see [“Data Sources Supported for Use with SAS Micro Analytic Service” in SAS Micro Analytic Service: Programming and Administration Guide](#).
- For syntax information about SQL statements, see [“FedSQL Statements” in SAS FedSQL Language Reference](#).
- For the list of reserved words in FedSQL, see [“FedSQL Reserved Words” in SAS Viya Platform: FedSQL Programming for SAS Cloud Analytic Services](#).
- For the list of reserved words in DS2, see [“Reserved Words in the DS2 Language” in SAS DS2 Programmer's Guide](#).
- For information about the functions that are supported in SQL code, see [“FedSQL Functions” in SAS FedSQL Language Reference](#).
- For information about using SAS Data and AI Studio to create queries, see [“Creating a Standalone Query” in SAS Data and AI Studio: User's Guide](#) and [“Using Macro Variables in SAS Data and AI Studio” on page 195](#).

Decision Variables for Data Query Files

When you create a data query file, SAS Intelligent Decisioning automatically adds the comment `/* include sqlReturnInfo */`. When this comment is present and you add the data query file to a decision, SAS Intelligent Decisioning defines the metadata for decision variables for the return code, row count, and query output. If this comment is not present, SAS Intelligent Decisioning defines metadata for only the query output. You can use the **Add missing variables** option to add these variables to the decision and map these decision variables to the query's variables. You can set the **Create variables automatically in decisions** setting to automatically add these variables to the decision. For more information, see [“About Decision Variables and Mapping” on page 436](#).

The variables are mapped as shown in [Table 7.1](#). When the decision is run, the variables are created only if the query selects data.

Table 7.1 Variables for Query Files

Query File Variable	Decision Variable	Description
<code>file_name_returnCode_out</code>	<code>returnCode</code>	The return code that is generated by the query file.
<code>file_name_rowCount_out</code>	<code>rowCount</code>	The number of rows returned by the query.
<code>file_name_out</code>	<code>dgo</code>	The data grid variable that contains the output of the data query. The lengths of character columns in the data grid are determined by the lengths of the character variables that are specified in the SELECT statement. Note: This variable is created only if the query selects data.

If the comment is present but the query does not select data, SAS Intelligent Decisioning creates only the return code variables.

If you remove the comment and the query selects data, only the output variables are created.

If you remove the comment and the query does not select data, no variables are created.

Using Macro Variables in SAS Data and AI Studio

When you author a data query file in SAS Data and AI Studio, you can pass input data to the decision by using macro variables. When you add the data query file to a decision, SAS Intelligent Decisioning creates decision variables that have the same name as the macro variables. When the decision executes, the values of the decision variables are set to the same values as the macro variables.

To define the macro variable in SAS Data and AI Studio, open a **SAS Program** tab and submit the %LET statement. For example, the following statement defines the macro variable &myName:

```
%let myname = 'Arinya'
```

To use the macro variable in the **Query** tab, reference it with an ampersand followed by the macro variable name:

```
SELECT name FROM table WHERE name = &myname
```

When you create a filter expression, set the appropriate options so that SAS Data and AI Studio recognizes the macro variable. The settings depend on whether the variable is character or numeric.

- If the macro variable is numeric, check the **Allow macros** check box.

- If the macro variable is character, check the **Match Case** check box, and clear the **Quote Strings** check box.

You can use custom SQL code for your data query instead of using a quick filter. However, SAS Intelligent Decisioning cannot determine the variable's data type when it is creating the decision variable. SAS Intelligent Decisioning assumes that the variable is a numeric unless its name ends with `char` or `string`. The `char` and `string` suffixes are not case sensitive.

For more information, see [“Filtering Data” in SAS Data and AI Studio: User's Guide](#) and [“Macro Variables Defined by Users” in SAS Macro Language: Reference](#).

Handling Table and Column Names in Data Query Files

FedSQL performs case folding to make all table and column names uppercase if they are not enclosed in quotation marks. This case folding might cause issues with databases that perform case folding on unquoted references to make them lowercase. To avoid issues with running your query code in both SAS Data and AI Studio and in SAS Intelligent Decisioning, do the following

- In SAS Data and AI Studio, if you refer to a table column by using a table alias in a JOIN, WHERE, or HAVING clause, do not enclose the table alias in quotation marks. Alternatively, you can refer to the column by using only the column name or by using the full table name instead of the alias.
- When you define a libref in SAS Data and AI Studio to connect to your PostgreSQL database, set the PRESERVE_COL_NAMES and PRESERVE_TAB_NAMES options on the LIBNAME statement to Yes. For additional information, see [“PRESERVE_COL_NAMES= LIBNAME Statement Option” in SAS/ACCESS for Relational Databases: Reference](#) and [“PRESERVE_TAB_NAMES= LIBNAME Statement Option” in SAS/ACCESS for Relational Databases: Reference](#).
- Set the SAS Intelligent Decisioning configuration option `quoteStudioQueryIdentifiers` to On. For additional information, see [“sas.decisions.codefiles.quoteStudioQueryIdentifiers” in SAS Intelligent Decisioning: Administrator's Guide](#).

Custom Context Files

Using Custom Context Files

A custom context file is a custom DS2 package file in which you can define context functions that you can use in rule sets, in DS2 code files, and in custom functions that are defined in the Custom Functions category. When SAS Intelligent Decisioning generates the code for a decision, each context function is defined only once in the generated code.

TIP Custom context files define variables and functions that can be referenced by any rule set, DS2 code file, or custom function in any decision for which the custom context file is specified in the decision's properties. When SAS Intelligent Decisioning generates the code for the decision, each context function is defined only once. For example, you can use variables in a custom context file to hold information about the current input record and use functions in the custom context file to share this information among the nodes in the decision.

Custom function files define functions that can be referenced in rule sets and DS2 code files. When you include these rule sets and DS2 code files in a decision, the code for each custom function is regenerated for each rule set and DS2 code file that uses the custom function. Other objects in the decision cannot reference the variables or functions in the custom function files. Custom function files are typically used to encapsulate logic that is reused multiple times.

You can test or validate rule sets, DS2 code files, and custom functions that use context functions before you publish them. In a production environment, you can reference the functions that are defined in a custom context file only from decisions. Rule sets that are published independently (outside of a decision) cannot use context functions.

At run time, the custom context can hold information in internal variables about the record that is being processed. This information can be shared with other nodes by calling functions that are defined in the context file.

Custom context files that are specified in nested decisions are ignored. You must specify the custom context file in the parent decision. A nested decision cannot use a custom context file that is different from the custom context file that is used by the parent decision. The parent decision's context file is used to resolve all references to context functions.

To create a custom context file, select **Custom context** for the file type in Step 4 of [“Create a New Code File” on page 200](#).

Note: In container destinations, Oracle is the only database that can be queried by using custom context files that contain SQL queries.

Adding an INIT Method to a Custom Context File

You can add an INIT method to your custom context files. By adding an INIT method, you can initialize variables before the first row of data is processed by the decision. The INIT method is executed the first time the custom context file is executed for the decision. The method is executed only once per thread. For example, the following code creates a custom context file with an INIT method that initializes a hash:

```
package CustomContext;
    dcl varchar(1024) ctxtKey;
    dcl varchar(1024000) ctxtValue;
    dcl package hash context;
    method init();
        context = _new_ hash([ctxtKey],[ctxtValue],0, '', '', '', '', 'MULTIDATA');
    end;
    ...
endpackage;
```

Associating a Custom Context File with a Decision

You can specify a custom context file for decisions that are published to SAS Micro Analytic Service destinations and specify a different context file for decisions that are published to other destinations. You do not need to specify a file for both destination types unless the content differs between the two files.

- 1 Open the decision, and click the **Properties** tab.
- 2 Specify a custom context file in one or both of the following fields:
 - **Custom context** specifies the context file to use when the decision is published to SAS Cloud Analytic Services (CAS), Git destinations that are compatible with CAS, Teradata, Apache Hadoop, or container destinations. You must specify the custom context file in this field in order to run scoring tests, scenario tests, and publishing validation tests on the decision.
 - **Custom context for SAS Micro Analytic Service** specifies the context file to use when the decision is published to SAS Micro Analytic Service destinations and Git destinations that are compatible with SAS Micro Analytic Service. Specify the context file in this field if you plan to publish the decision to one of these destinations.
- 3 Select the version of the custom context file.

Testing or Validating an Object That Uses a Custom Context Functions

To test or validate rule sets, DS2 code files, or custom functions that use context functions, you must specify the custom context file in the **Custom context** field on the **Properties** tab before you test or validate the object. For instructions, see [“Validate SQL, DS2, or Context Files” on page 210](#) and [“Testing Code Files” on page 222](#).

Referencing a Function That Is Defined in a Context File

When you add a custom context file to an object’s properties, SAS Intelligent Decisioning defines a variable in the generated code for the object that is named `application`. This variable appears only in the generated code. It does not appear in the **Variables** tab for the object.

You must use the `application` variable to refer to functions that are defined in the custom context file. To call a custom context function from within a rule set, DS2 code file, or custom function, use the following format:

```
application.method_name(arguments);
```

For example, you could create a custom context file that defines a custom function named `squareIt`:

```
package CustomContext;
  method squareIt(in_out double i) returns double;
    dcl double squared;
    squared=i*i;
    return squared;
  end;
endpackage;
```

In a rule set, you could call the `squareIt` function as follows:

```
mySquare = application.squareIt(i)
```

In the Custom Functions category, you can define a custom function that calls the `squareIt` context function as follows:

```
method addSquares() returns double;
  dcl double tally;
  dcl double squared;
  dcl double i;
  tally = 0;
  do i= 1 to 5;
    squared = application.squareIt(i);
    tally + squared;
```

```

        end;
    return tally;
end;

```

Referencing a Variable That Is Defined in a Context File

When you add a custom context file to an object's properties, SAS Intelligent Decisioning defines a variable in the generated code for the object that is named `application`. This variable appears only in the generated code. It does not appear in the **Variables** tab for the object.

You must use the `application` variable to refer to variables that are defined in the custom context file. To refer to a custom context variable from within a rule set, DS2 code file, or custom function, use the following format:

```
application.variable_name
```

For example, you could create a custom context file that sets the value of a variable named `grade` based on the value of the variable `score`:

```

package CustomContext;
  dcl varchar(20) grade;
  method execute(in_out double score);
    if score >= 90 then grade='A';
    else if score >= 80 then grade='B';
    else if score >= 70 then grade='C';
    else if score >= 60 then grade='D';
    else grade='F';
  end;
endpackage;

```

In a rule set, you can execute the custom context file and set the value of the rule set variable `finalGrade` to the value of the custom context variable `grade` as follows:

```

application.execute(score);
finalGrade=application.grade;

```

Create a New Code File

- 1 Click  to navigate to the Code Files category view.
- 2 Click **New code file**. The New Code File window appears.
- 3 Enter a name for the new code file. Code file names must be unique within a folder.

Note: When a data query file generates output, the file name can be up to 28 characters long. When the data query file is added to a decision, SAS Intelligent Decisioning creates an output-only decision variable. The name of the decision variable is the data query file name plus `_out`.

- 4 Select the code file type.
- 5 (Optional) For data query files, select the editor with which you want to edit the file.

TIP For data query files that you choose to edit in SAS Data and AI Studio, SAS Intelligent Decisioning adds `.cqy.dqf` to the file name, and the file type is displayed as **Data Query**. These files are created as stand-alone queries and return a data grid.

For data query files that you choose to edit in the SQL editor, the file type is displayed as **SQL**. These files can return either a data grid or a single row of scalar variables. You can select the output type on the **Properties** tab.

For more information, see [“Query Output Types and Editors” on page 193](#).

- 6 (Optional) Enter a description for the file. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 7 Click , and select the folder where you want to save the file.

- 8 Click **Save**. SAS Intelligent Decisioning opens the new code file.

For data query files that you choose to edit with SAS Data and AI Studio, SAS Intelligent Decisioning displays the **Properties** tab, and you can click **Open Data Query** to open the file in SAS Data and AI Studio.

For all other file types, SAS Intelligent Decisioning uses the appropriate code editor to open the file on the **Code** tab. For more information, see [“Using the Code Editor” on page 527](#).

TIP Except for query files that you edit in SAS Data and AI Studio, you must check out objects that are saved in a folder for which the check-out and commit feature is enabled before you can edit them. The check-out and commit feature is always enabled for the **Decision Repository** folder. See [“Checking Out and Committing Decision Versions” on page 462](#) for more information.

- 9 If the code file is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the code file.

For more information, see [“Check Out and Commit a Custom Code File Version” on page 220](#).

TIP

In the autocomplete list, SAS Intelligent Decisioning displays all function names and variable names that match the text that you enter. Double-click an item to add it to the code field. Single-click an item to display information about the item, such as whether it is a function or a variable. For functions, the information includes the returned data type and category. For variables, the information includes the data type and direction (input, output, or both).

The autocomplete list does not include functions that are defined in a custom context file.

Managing the Variables in a DS2 or Python Code File

About Variables

The Properties of a Variable

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a code file. Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See “Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.
Data type	SAS Intelligent Decisioning supports the following data types: Boolean, character, data grid, date, datetime, decimal, and integer.

Property	Description
	Integer, decimal, date, and datetime variables are converted to doubles when generated code is run.
Input and Output	A variable in a DS2 code file can be an input variable, an output variable, or both, but it cannot be a temporary variable. A variable in a Python code file can be an input variable or an output variable, but not both, and it cannot be a temporary variable. See “Input Variables and Output Variables” on page 204 for more information. Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.
Length	The length for Boolean and numeric variable types is set automatically. For output-only data grid variables, the length is set to the value that you specify. For output-only character variables in decisions that are published to container destinations or to SAS Micro Analytic Service destinations, the length that you specify is ignored unless the <code>sas.decisions.variable.length.honorOutputLengthInMAS</code> configuration option is turned on. For more information, see “sas.decisions.variable.length” in SAS Intelligent Decisioning: Administrator’s Guide. For character variables and data grid variables that are input-only or input-output variables, the <code>variableLengthOverridden</code> configuration property determines how the length is set. By default, this property is set to Off, and the length is set to the length in the input data. When the <code>variableLengthOverridden</code> property is set to On, the length of input-only and input-output character variables and data grid variables is set to the value that you specify. For more information, see “sas.decisions.variableLengthOverridden” in SAS Intelligent Decisioning: Administrator’s Guide. Note: For scenario tests, the <code>variable.length.honorOutputLengthInMAS</code> and <code>variableLengthOverridden</code> configuration options are ignored. Scenario tests always use the length that you specify. In objects that are published to SAS Micro Analytic Service destinations or to container destinations, the maximum length for character variables (outside of a data grid) and for data grid variables is 50 Mb. For other destination types, the maximum length for these variables is 10 Mb. The maximum length for character variables within a data grid is 32767 characters. For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters

Property	Description
	are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	You can specify an initial value only for variables of type character, integer, decimal, or boolean. Initial values are used only for output-only variables. Initial values are used only when you test a code file in SAS Intelligent Decisioning. When you publish a code file as part of a decision, the initial value that is specified for the corresponding decision variable is used, and the initial value that is specified in the code file is ignored. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. When you are entering expressions in the expression editor, specify 1 for True and 0 for False. To use the number sign (#) in literal values, specify the percent-encoded value %23. For content that will be published to a SAS Micro Analytic Service destination, the values of date and datetime input variables must be numeric. They cannot be formatted values such as 18jul2019. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.
Maximum row count	The maximum number of rows that are allowed in each column of a data grid that is generated from an input table in which the data grid column names use array syntax. If the input data contains more entries for a column than the specified row count allows, the input data for that column is truncated. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in <i>SAS Intelligent Decisioning: Using Data Grids</i>

Input Variables and Output Variables

For each variable in an SQL query, a DS2 method signature, or a Python function signature, you must specify whether the variable is an input variable, an output variable, or, in the case of DS2 files, both an input variable and an output variable.

- Input variables for code files receive their value from the decision variable to which the code file variable is mapped. When a decision that uses a code file is deployed in a production system, all of the decision's input variables must be mapped to table columns in input data. When you test a code file in SAS

Intelligent Decisioning, for each input variable, you can either map it to a table column or specify a constant as its input value.

When you create or edit a code file variable, clear the **Input** check box for any variable that does not receive its value from a decision variable.

- Output variables are variables that are written to the output table that is created when a decision that uses the code file is run. When you create or edit a variable, clear the **Output** check box for any variable that you want to exclude from the output data.

Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.

When you create a new variable, it is created as an output-only variable by default.

Note: You cannot manage local variables on the **Variables** tab. Local variables are variables for which neither the **Input** nor the **Output** checkboxes are selected on the **Variables** tab. If local variables are listed on the **Variables** tab, SAS Intelligent Decisioning displays an error message when you save the code file or click a sync button. The message is `The direction of some variables is not selected.` You must delete local variables from the **Variables** tab.

Syncing Variables and Code

To propagate changes that you enter on the **Variables** tab to the code that is shown on the **Code** tab, click **Sync to Code** on the **Variables** tab. For Python code files, changes to input variables are propagated to the signature of the EXECUTE function, and changes to output variables are propagated both to the docString and to the RETURN statement. For DS2 files, changes to input-only, output-only, and input-and-output variables are propagated to the signature of the EXECUTE method. Both output-only and input-and-output variables are added with the `in_out` keyword.

To propagate changes that you enter on the **Code** tab to the **Variables** tab, save the code file or click **Sync Variables** on the **Code** tab. For Python code files, changes to the signature of the EXECUTE function are propagated to the **Variables** tab as changes to input variables. Changes to the docString and to the RETURN statement are propagated as changes to output variables. New variables are added as character variables, but you change the data type on the **Variables** tab. For DS2 files, variables that you add to the signature of the EXECUTE method with the `in_out` keyword are added to the **Variables** tab as output-only variables. Variables that are not specified with the `in_out` keyword are added to the **Variables** tab as input-only variables. You can specify that any variable is both an input and output variable by selecting both the **Input** and **Output** check box on the **Variables** tab.

If you make conflicting changes to both the **Code** tab and the **Variables** tab and then save the file, SAS Intelligent Decisioning prompts you to select which tab contains the changes that you want to retain. If you make conflicting changes on the two tabs and then click a sync button, only the changes on the tab on which you click the sync button are retained. Those changes are then propagated to the other tab.

Note: You cannot manage local variables on the **Variables** tab. Local variables are variables for which neither the **Input** nor the **Output** checkboxes are selected on the **Variables** tab. If local variables are listed on the **Variables** tab, SAS Intelligent Decisioning displays an error message when you save the code file or click a sync button. The message is `The direction of some variables is not selected.` You must delete local variables from the **Variables** tab.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name: `"string"` in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name: `"HMEQ"`. [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➤**. To import all of the variables in the table, click **➤➤**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See [“Input Variables and Output Variables” on page 204](#) for more information.

- 7 Save the code file or click **Sync to Code** to add the variables to the code that is on the **Code** tab. For more information, see [“Syncing Variables and Code” on page 239](#).

Create Custom Variables

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click **Add variable** and select **Custom variable**. The Add Variables window appears.
- 2 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 202](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Maximum row count**, and **Length** fields.
- c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable’s data type. See [“The Properties of a Variable” on page 202](#) for additional information.
- d Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- e Verify that the **Input** and **Output** check boxes are selected correctly for each variable.
 - Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.
 - Clear the **Output** check box for any variable that you want to exclude from the output data.
 - Clear both the **Input** and **Output** check boxes to create a temporary variable.

TIP Variables in DS2 files can be both input and output variables but Python variables cannot.

- 3 (Optional) Modify the variable properties in the table of variables.
- 4 Click **OK** to add the variables and close the Add Variables window.
- 5 Save the code file or click **Sync to Code** to add the variables to the code that is on the **Code** tab. For more information, see [“Syncing Variables and Code” on page 239](#).

Duplicate a Variable

- 1 On the **Variables** tab, select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 2 Enter a new name for the duplicate variable.
- 3 (Optional) Enter a description for the variable.
- 4 Click **Duplicate**.
- 5 Save the code file or click **Sync to Code** to add the variables to the code that is on the **Code** tab. For more information, see [“Syncing Variables and Code” on page 239](#).

Importing and Exporting Variables

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, “Import File Formats,” on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the code file into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.

- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

TIP To add new columns to an existing data grid variable, select **Replace all variables**.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Export Variables

Note: If you make changes to the variables, you must save the code file before exporting variable data.

- 1 Open the code file from which you want to export variables.
- 2 On the **Variables** tab, select the variables that you want to export.
- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

- 1 On the **Variables** tab, select the check box for the variables that you want to delete, click , and select **Delete**.
- 2 Click **Sync to Code** to delete the variables from the code that is on the **Code** tab.

Edit Variable Properties

On the **Variables** tab, click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

Note: When you rename a variable, click **Sync to Code** to change references to that variable within the same object change to use the new name.

See “The Properties of a Variable” on page 202 for additional information.

Edit Metadata for Data Grid Variables

For information, see “Editing Data Grid Variable Metadata” in *SAS Intelligent Decisioning: Using Data Grids*.

Determine Which Objects Use a Particular Variable

On the **Variables** tab, select the check box for the variable, click , and select **View used by report**.

In the report, you can use the **Filter** field to filter the list of objects based on the object names. If the selected variable is a data grid variable, and if you are interested only in a specific column within the data grid, you can select the column in the **Column name** field, and then click **Apply**. SAS Intelligent Decisioning narrows the search results to only the objects that use the data grid with the selected column.

Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Validate SQL, DS2, or Context Files

To validate a code file of type SQL, DS2, or custom context, open the file, click , and select **Validate**. For DS2 code files that use context functions, specify the custom context file in the **Custom context** field on the **Properties** tab.

For DS2 code files and custom context files, SAS Intelligent Decisioning compiles the code.

IMPORTANT Before you validate a DS2 code file or custom context file that uses a data grid package, verify that the declaration statements for the package specify `datagrid` instead of `dcm_datagrid`. For example:

```
in_out package datagrid variable;
```

```
dcl package datagrid variable;
```

For SQL query files, SAS Intelligent Decisioning runs the query in order to verify the SAS Micro Analytic Service connection string, scans the SQL statements for syntax errors, and looks for references to non-existent tables and columns.

IMPORTANT Do not validate a query that uses dynamic parameters to alter data. In order to validate a query, SAS Intelligent Decisioning generates values to use for the query parameters and executes the query. These parameter values might not match the values that are passed to the query during normal execution and can produce unintended changes in the database. For example, data might be deleted that should not be deleted.

If your query does not use dynamic parameters but does alter the data, be aware that validating the query will alter the data.

Copy a Code File URL

To create a link for external documentation that automatically opens a code file in SAS Intelligent Decisioning, open the code file, click , and select **Copy code file URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Generated Code for Code Files

You can compare the generated code of two different code files, or you can compare the generated code of two different versions of the same code file.

- 1 Select the objects that you want to compare.
 - To compare the generated code of two different code files, select the code files in the category view, click , and select **Compare code**.
 - To compare the generated code of two versions of the same code file, open the code file, click  on the **Versions** tab, and select **Compare code**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window and highlights the differences.
- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Code Files

Duplicate Code Files

To duplicate a single code file:

- 1 In the Code Files view, select the code file that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Code Files window appears.
- 3 Enter a new name for the duplicate code file.
- 4 (Optional) Enter a description for the code file.
- 5 Click , and select the folder where you want to save the duplicate file.
- 6 Click **Duplicate**.

To duplicate multiple code files:

- 1 In the Code files view, select the code files that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the code files and appends `_copy_n` to the names of the duplicate copies.

Note: When you duplicate a code file, SAS Intelligent Decisioning creates a relationship between the original code file and the duplicate code file. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the code file before you paste the new content.

Delete Code Files

In the Code Files category view, select the code files that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the files to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename a Code File

.....
Note: You cannot rename a code file if it is open.
.....

- 1 In the Code Files category view, select the code file that you want to rename.
- 2 Click  and select **Rename**. The Rename window appears.
- 3 Enter a new name for the code file, and click **Rename**.

Move Code Files to a Different Folder

- 1 In the Code Files category view, select the code files that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the code files, and click **OK**.

Managing Versions of Code Files

Set the Displayed Version

The displayed version is the version whose information is displayed on the other tabs, such as the **Code**, **Properties**, and **Variables** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Custom Code File Version” on page 220](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.
- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of

existing tags. For more information, see “[SAS Intelligent Decisioning Service Properties](#)” in *SAS Intelligent Decisioning: Administrator's Guide*.

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the code file whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the code file whose contents you want to copy:

- a Select the version whose contents you want to copy.
- b Click , and select **Copy version**. The Copy Version window appears.

- 2 Click , and select the target code file into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target code file. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a

warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

.....

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see “Default Permissions” in *SAS Intelligent Decisioning: Administrator’s Guide* and “Folder-Based Permissions” in *SAS Intelligent Decisioning: Administrator’s Guide*. By default, users have permission to delete objects that they create and any versions of those objects.

.....

Note: To be able to delete specific versions of custom code files, the [sas.decisions.codefiles.deleteVersions](#) configuration option must be turned on.

.....

On the object’s **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Upgrade Decisions to Use a New Version of a Code File

If you create a new version of a code file that is already used in decisions, you can upgrade the decisions to use the new version of the code file.

- 1 On the **Versions** tab for the code file, click , and select **Upgrade decisions**. The Upgrade Decisions window appears. This window lists all of the decisions that include the code file.
- 2 In the **Version to upgrade to** field, select the version of the code file to which you want to upgrade the decisions.
- 3 Select **Automatically map variables** if you want SAS Intelligent Decisioning to automatically map new code file variables in the decisions where the code file is used.

For information about how SAS Intelligent Decisioning maps variables, see “About Decision Variables and Mapping” on page 436.

- 4 Select the check boxes for the decisions that you want to upgrade, and click **Upgrade decisions**.

TIP To update all of the objects that are used in a decision, see [“Update Decisions to Use New Object Versions”](#) on page 443.

Determine Which Objects Use a Code File

To list the objects that use a specific code file:

- 1 On the **Code files** category view, select the check box for code file, click **:**, and select **View used by report**. The All Objects that Use the Selected Item window appears. By default, this window lists the objects that use the latest version of the selected code file.
- 2 (Optional) Select the specific version for which you want to view the Used By report.

To view the list of all objects that use any version of the selected code file, clear the value in the **Version** field.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names.

Click on an object name to open the object. Click ⓘ next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Code File Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

IMPORTANT When you create a custom code file in a folder for which the check-out feature is enabled, you must check out the code file before you can edit it in the SAS Intelligent Decisioning user interface. However, custom code files are stored in the Files service. The files service grants the file creator full access to the file. The file creator can use the REST API to edit the file without checking it out. The file creator can do this even when the file is in a folder for which the check-out feature is enabled (including the **Decision Repository** folder). In order to prevent the file from being edited without being checked out, the user ID of the file creator must be different from any user ID that is used to edit the file. For example, an administrator can create an empty code file that other users can then edit if they check it out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.decisions.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a code file version, the **Check Out** button for that code file is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Code File Versions” on page 219](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Code Files category view shows two code files with the same name, but the folders listed in the **Location** column differ for each file. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

Checking Out a Code File from within a Decision

See “[Checking Out and Committing Objects from within A Decision](#)” on page 464 for information.

Concurrently Checking Out and Committing Code File Versions

The ability for multiple users to check out the same code file at the same time is controlled by the [sas.decisions.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user’s **My Folder** location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user’s changes are preserved in a new version when they commit their changes. One user’s changes do not overwrite another user’s changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Custom Code File Version

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 221](#).

- 4 On the **Versions** tab, click **Commit**. The Commit Code File Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 7 (Optional) Enter information about the new version in the **Notes** field.
- 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

How you undo the checkout of an object depends on how the object was checked out.

If you [checked out an object from within a decision](#), click  on the object's node in the decision diagram, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Testing Code Files

Ways to Test a Code File

There are two types of tests:

Basic test

executes the code file in the SAS Cloud Analytic Services (CAS) destination using the input table that you specify. You can also specify a debugging variable. For more information, see [“Create and Run a New Test” on page 222](#).

Scenario test

enables you to enter specific input values and the output values that you expect the test to generate. A scenario test identifies differences between the output that you expect to see and the actual output that is generated when the test is run. You can also compare the test definitions and test results of different scenarios. Scenario tests are also run in CAS. For more information, see [“Create and Run a Scenario Test” on page 226](#).

Note: You can create and run basic tests and scenario tests for DS2, SQL, and Python custom code files, but you cannot run these tests directly on custom context files. You can test the functions that are defined in a custom context file by testing the rule set or DS2 file that uses the context functions. Validating a custom context file determines whether the file is syntactically correct and whether it includes declaration statements for the packages that are used in the file. For more information, see [“Validate SQL, DS2, or Context Files” on page 210](#).

Test a Code File

Create and Run a New Test

Testing a code file is optional, but doing so is a best practice. Testing enables you to discover any problems before the code file is incorporated into a production system.

IMPORTANT If you are testing a DS2 or SQL code file that uses functions that are defined in a custom context file, verify that the context file is

specified in the **Test custom context** field on the **Properties** before running the test.

- 1 On the **Scoring** tab, click the **Tests** tab.
- 2 Click **New test**. The New Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 5 Click  for the **Location** field, and select the folder in which you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 6 Click  for the **Input table** field, select the input table for the test, and click **OK**.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name : "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name : "HMEQ". ["How to Enter Searches for Information Assets in SAS Data Governance" in SAS Data Governance: User's Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see ["Importing and Loading Data" on page 12](#).

- 7 Verify or change the variable mappings. To run a full test, map all of the input variables to columns in the input table that you selected for the test. To run a partial test, you can map only the input variables that are needed for the test.

SAS Intelligent Decisioning automatically maps the input variables in the code file to columns in the input table when the names and data types of the variables match those of the table columns.

If any input variables are not mapped to input columns or to static values, the application displays the warning message `Input variables must be mapped to table columns`. At run time, SAS Intelligent Decisioning assigns missing values to input variables that are not mapped.

You can change the automatic variable mappings in the Variable Mappings window.

To change variable mappings:

- a Click **Variables**. The Variable Mappings window appears.

TIP Click **Show unmapped variables** to filter the list to show only the variables that have not been mapped to an table column.

- b For each input variable, select the table column to which the variable should be mapped. If the input table contains more than 15 columns, click **More columns** to display additional column names. Alternatively, for Decimal, Integer, and Character variables, you can select **Use value** for the table column, and specify a literal value in the **Value** column. When you are entering literal values, remember these rules:
 - For character variables, input values are limited to 10 Mb. Output values longer than 32767 characters will be truncated.
 - Do not enclose character strings in quotation marks.
 - To specify a missing value for character variables, select **Use value** and leave the **Value** column empty. When SAS Intelligent Decisioning generates code, it generates an empty string (' '). For numeric values, enter a period (.).
 - c Click **OK** to close the Variable Mappings window.
- 8 (Optional) Click **Advanced** to display the advanced options.
 - 9 (Optional) Click  for the **Output data library** field, and select the library to which you want to write the output of the test.
 - 10 (Optional) Enter a name for the test results output table in the **Output table name** field if you do not want to use the default name. The default name is `test-name_object-name_timestamp_output`.
 - 11 (Optional) For the **Target version**, select the version of the code file that you want to test. The default version is the latest version.
 - 12 (Optional) Select the variable that you want to serve as an input debug variable. You can specify an input-only variable or an input-output variable.

For more information, see [“Debugging Code File Tests” on page 225](#).

- 13 (Optional) Select **Preserve unmapped columns in the output table** if you want columns that are not mapped to an output variable to be written to the output table.
- 14 Click **Run** to save and run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 230](#).

- 15 Click  in the **Results** column to view the results of the test.
- 16 In the test results window, click **Test Results** in the navigation pane to display the URLs and other information for the test. Click **Output**, **Code**, or **Log** to

display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

Debugging Code File Tests

When you create a test, you can specify a variable to use as a debugging variable in the **Input debug variable** field. You can specify an input-only variable or an input-output variable. For more information, see [Step 12 of “Create and Run a New Test” on page 222](#).

When you specify an input debug variable, SAS Intelligent Decisioning automatically sets the maximum number of threads that can be allocated for the test to 1. Setting the thread count to 1 ensures that the variable's values are written to the log in the correct order and are not affected by different threads completing at different times.

To capture variable values for input-only or temporary variables after the code file logic has executed for a specific record, you can specify that the variable is an output variable, and then re-run the test. Before incorporating the code file into a production system, return the input and output settings for the variable to their previous settings. For more information, see [“Input Variables and Output Variables” on page 204](#) and [“Edit Variable Properties” on page 209](#).

Test a Scenario

Create and Run a Scenario Test

IMPORTANT If you are testing a DS2 or SQL code file that uses functions that are defined in a custom context file, verify that the context file is specified in the **Test custom context** field on the **Properties** before running the test.

- 1 On the **Scoring** tab, click the **Scenarios** tab.
- 2 Click **New test**. The New Scenario Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Click  for the **Location** field, and select the folder where you want to save the test definition.
- 5 Click  for the **Output table location** field, and select the data library where you want to save the test results.
- 6 (Optional) Select the version of the code file that you want to test. The default version is the latest version.

Note: When you change the version, variables for the new version are reloaded and any variable values that were entered for the test are cleared.

- 7 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 8 Enter the values that you want to use for each input variable. You do not have to enter values for every input variable. At run time, SAS Intelligent Decisioning uses missing values for input variables for which you do not specify a value.

Note: For character variables, input and output values longer than 32767 characters will be truncated. You cannot enter input values for variables of type binary or of type varying-length binary.

To enter values for the columns in a data grid variable:

- a Click  in the **Value** field. The Edit Data Grid window appears. One row is automatically added to the input data grid.
- b Enter values for each column in the data grid.

- c (Optional) Click **+** to add a new row, and then enter the values for each column.

Repeat this step for each row of values that you want to add to the data grid.

TIP By default, data grid column names appear across the top of the data grid view, and row numbers appear down the left side. You can click  to change the view of the data grid so that row numbers appear across the top and data grid column names appear down the left side.

- d Click **OK** to save the data grid values and close the Edit Data Grid window.

For more information about entering variable values, see [“The Properties of a Variable” on page 202](#).

- 9 (Optional) For each output variable, select the **Include** check box and enter the expected output value. The **Include** check box controls whether a variable's expected value is used to determine the status of a scenario test. If you select **Include** for a variable and the test does not return the expected value, the test status is set to Completed with warnings (). If you do not select the check box, the application ignores the expected value of that variable when it determines the status of the test.

To enter expected values for the columns in a data grid variable, click in the **Expected Output** field, and follow the instructions in [Step 8](#) for entering values for the columns in a data grid variable.

Note: A scenario test cannot verify issues with trailing spaces. For example, it cannot distinguish between a string that contains a single space ' ' and a string that contains three spaces ' ' '.

- 10 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 230](#).

- 11 Click  in the **Results** column to view the results of the test.
- 12 On the Test Results page, click **Scenario Test Results** in the navigation pane to display the URLs and other information for the test. Click **Input** to display the values of input variables and dynamic attributes. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

On the **Output** page, click **Show All** to display the expected and actual values of all output variables. Click **Show Differences** to display only the variables whose expected values do not match the actual values that were returned by the test.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Output** page, click **Export** to export the output table as a comma-separated values (CSV) file. On the **Log** page, click  to download the log file.

- 13 Click **x** to close test results and return to the **Scoring** tab.

Import Scenario Test Definitions

You can import scenario test values from a comma-delimited (CSV) file. Each line in the CSV file is imported as one scenario test. In the CSV file, add a column of values for each variable. In the header row, enter the names of the input variables and of the output variables with `_expected` appended to the name.

For example, suppose your scenario test has the input variables `policyholder`, `cscore`, and `claims`, and it has the output variables `eligible` and `policies`. An import file for this test might appear in a spreadsheet application as shown in the following figure.

	A	B	C	D	E	F	G	H	I	J
1	policyholder	cscore	claims	eligible_expected	policies_expected					
2	Smyth,Joe	784	0	TRUE	[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["4535					
3	Dupree, Marcel	803	2	FALSE	[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["9836					

The `policies` output variable in this example is a data grid variable. To enter values for data grid variables, use the JSON string format described in [“Introduction to Data Grids” in SAS Intelligent Decisioning: Using Data Grids](#).

The format for date and datetime variables depends on your locale. Use the same format that is created by the date and datetime pickers when you click  or  to enter initial values for custom variables on the **Variables** tab.

To import scenario test definitions:

- 1 On the **Scoring** tab, click the **Scenarios** subtab, and then click **Import scenarios**. The Import Scenarios window appears.

- 2 In the **Import from** field, click  and select the CSV file that contains the scenario test values.

Note: The import file is limited to 10 MB.

- 3 Select **Select encoding** to select the encoding of the CSV file from the predefined list, or select **Enter encoding** to enter a custom encoding.
- 4 Select or enter the encoding of the CSV file.
- 5 Enter a prefix for the scenario test definitions. SAS Intelligent Decisioning appends a number to this prefix for each test definition. The prefix can include double-byte characters and special characters, including single quotation marks.
- 6 (Optional) Click  for the **Folder location** field, and select the folder where you want to save the test definitions.
- 7 Click  for the **Output table location** field, and select the folder where you want to save the test results.
- 8 Click **Import**.

Compare Different Scenarios

You can display the scenario definitions or results of two or more tests side-by-side. On the **Scenarios** tab, select two or more tests, click , and select one of the following options:

Compare ⇨ **Definitions**

displays the input values and the expected output values that you entered for both tests. To edit the input values for a scenario test definition, click  next to the test name under **Input Table**. To edit the expected output values for a test, click  next to the test name under **Output Table**.

Compare ⇨ **Results**

displays the input values and the actual output values that were generated by the test. To display both the expected values and the actual values in the output table, select **Display expected values**. For each variable for which you selected **Include** in [Step 9 on page 227](#), the application highlights the variable if the actual and expected values do not match.

Click **Export** to export the results comparison as a comma-separated values (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Working with Test Output Data

After you run a test, you can work with the output table in other SAS applications to analyze the data, create and compare models, discover relationships hidden in the data, and generate reports based on the data.

Note: The actions available to you depend on the applications that are available at your site.

On the Test Results page, select the **Output** table in the navigation pane, click **Actions**, and select one of the following options:

Explore Lineage

opens SAS Lineage Viewer. SAS Lineage Viewer enables you to better understand the relationships between objects in your SAS Viya platform applications. These objects include data, transformation processes, reports, and visualizations. For more information, see [SAS Lineage: User's Guide](#).

Explore and Visualize

opens the output table in SAS Visual Analytics. SAS Visual Analytics enables you to create, test, and compare models based on the patterns discovered during exploration of the data. You can export the model before or after performing model comparison for use with other SAS products or to put the model into production. SAS Visual Analytics supports a range of visualization, discovery, and reporting features. For more information, see [Welcome to SAS Visual Analytics](#).

Access and Manage Data

opens the output table in SAS Data Hub. SAS Data Hub enables you to import and download tables, load tables into memory, unload tables, and more. For more information, see [SAS Data Hub: User's Guide](#).

Status Icons for Tests

Icon	Status
	The test is not ready to run. The test definition is not complete, or it might contain errors.
	The test has been defined and can be run. Some input variables have not been mapped or have not been assigned a value, so the test might execute only a subset of the code in the custom file.
	The test is defined correctly and is ready to run.

Icon	Status
	The test is running.
	The test completed successfully.
	The test completed, but warnings were issued in the SAS log. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.
	The test did not run successfully. Check the SAS log for information. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.

Duplicate Tests

To duplicate a single test:

- 1 On the **Scoring** tab, click the **Tests** tab or the **Scenarios** tab, depending on the test that you want to duplicate.
- 2 Select the check box for the test that you want to duplicate, click  on the subtab, and select **Duplicate**.
- 3 In the Duplicate Tests window, enter a name for the duplicate test if you do not want to use the default name.
- 4 (Optional) Enter a description for the duplicate test.
- 5 Click **Duplicate**.

To duplicate multiple tests:

- 1 On the **Scoring** tab, click the **Tests** tab or the **Scenarios** tab, depending on the tests that you want to duplicate.
- 2 Select the check boxes for the tests that you want to duplicate, click  on the subtab, and select **Duplicate**.

SAS Intelligent Decisioning duplicates the tests and adds `_Copy` to each filename.

Manage Comments for a Code File Test

You can associate comments and attachments with a code file test.

To open the Comments properties pane, select the code file test on the **Scoring** tab, and click  in the property pane.

To add a new comment, enter the comment in the text box and click **Post**.

To add an attachment to a comment, click , select the file that you want to attach, and click **Post**. (The attachment icon appears after you enter text in text box.) You cannot attach executable files such as BAT and EXE files.

To reply to a comment, click **Reply**, enter your reply in the text box, and click **Post**.

To edit an existing comment, click  for the comment, and select **Edit**. Edit the comment, and click **Post**.

To delete a comment, click  for the comment, and select **Delete**.

Using Custom Functions

<i>About Custom Functions</i>	234
<i>Using Custom Functions in a Rule Set or Code File</i>	234
<i>Create a Custom Function</i>	235
<i>Using the Custom Function Editor</i>	236
<i>Rules for Writing Custom Function Files</i>	237
<i>Managing the Variables in a Custom Function File</i>	238
The Properties of a Variable	238
Syncing Variables and Code	239
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object ...	240
Create Custom Variables	241
Duplicate a Variable	242
Importing and Exporting Variables	242
Delete Variables	243
Edit Variable Properties	243
Edit Metadata for Data Grid Variables	244
<i>Validate a Custom Function File</i>	244
<i>Testing a Custom Function</i>	244
<i>Copy a Custom Function URL</i>	245
<i>Compare Custom Functions</i>	245
<i>Managing Custom Function Categories</i>	245
Create a Custom Function Category	245
Edit a Custom Function Category	246
Hide a Custom Function Category	246
<i>Managing Custom Functions</i>	247
Duplicate Custom Functions	247
Hide Custom Functions	247
Rename a Custom Function	248
Change a Function's Category	248
<i>Restore or Permanently Delete Hidden Functions and Categories</i>	248

About Custom Functions

You can use custom functions to perform actions that are not available with the standard functions that SAS provides. Custom functions also enable you to encapsulate and reuse business logic. You can use custom functions in rule sets and in DS2 code files.

When SAS Intelligent Decisioning generates the code for a published object, the definitions for custom functions are duplicated each time the function is referenced.

TIP Custom function files define functions that can be referenced in rule sets and DS2 code files. When you include these rule sets and DS2 code files in a decision, and SAS Intelligent Decisioning generates the code for the decision, the code for each custom function is regenerated for each rule set and DS2 code file that uses the custom function. Other objects in the decision cannot reference the variables or functions in the custom function files. Custom function files are typically used to encapsulate logic that is reused multiple times.

Custom context files define variables and functions that can be referenced by any rule set, DS2 code file, or custom function in any decision for which the custom context file is specified in the decision's properties. When SAS Intelligent Decisioning generates the code for the decision, each context function is defined only once. For example, you can use variables in a custom context file to hold information about the current input record and use functions in the custom context file to share this information among the nodes in the decision.

Using Custom Functions in a Rule Set or Code File

SAS Intelligent Decisioning adds your custom categories and functions to **Functions** tab in the [custom function editor](#), the [code editor for custom DS2 files](#), and the [expression editor](#). Functions provided by SAS are listed first, under the heading **SAS**. Custom categories and functions are listed second, under the heading **Custom**.

IMPORTANT A delay of up to one minute can occur between the time when you define or modify (replace) a function and the time at which the function is available to be executed.

When SAS Intelligent Decisioning generates code, it inserts the custom function definition ahead of the code that it generates for the rule set logic or code file logic. In the rule set or code file, you can reference custom functions in the same way that you reference functions that are provided by SAS.

Note: You can nest custom function calls. For example, if A and B are custom functions and DIVIDE is the DS2 function provided by SAS, you can nest functions as follows:

```
method B() returns double;
    return 5;
end;
method A() returns double;
    return DIVIDE(10,B());
end;
```

Create a Custom Function

- 1 In the Custom Functions category view, click **New Custom Function**. The New Custom Function window appears.
- 2 Enter a name for the function if you do not want to use the default name. The name cannot be the same as the name of an existing SAS function. Function names must be unique within an environment and are limited to 250 characters. Function names must be valid DS2 regular identifiers. The DS2 language supports only Latin characters for regular identifiers. For more information, see [“DS2 Identifiers” in SAS DS2 Programmer’s Guide](#).
- 3 (Optional) Enter a description for the new function. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 4 Select the category to which the function belongs.

TIP Custom functions must be associated with a function category. A category named **Category 1** is defined by default. You can change this category name, or create new a new category. For instructions about creating new categories, see [“Create a Custom Function Category” on page 245](#).

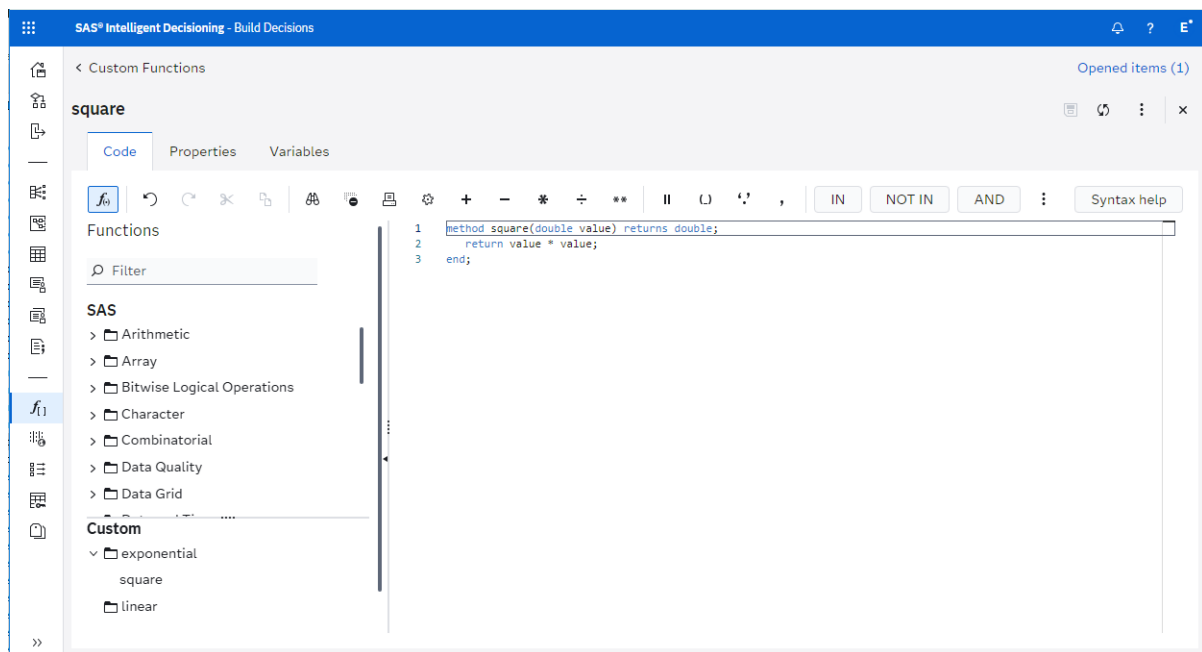
- 5 Click **Save**. SAS Intelligent Decisioning opens the custom function editor.
- 6 Enter the DS2 code for the function. For more information, see the following:
 - [“Using the Custom Function Editor” on page 236](#)
 - [“Rules for Writing Custom Function Files” on page 237](#)
 - [“METHOD Statement” in SAS DS2 Language Reference](#)
 - [SAS DS2 Programmer’s Guide](#)

Using the Custom Function Editor

Use the custom function editor to enter the code for custom functions.

To open the custom function editor, click the function name in the Custom Function category view.

Figure 8.1 Custom Function Editor



You can enter code directly into the code field, or you can use the lists of operators and functions to add them to the code for the custom function.

- To add an operator, click the operator in the rows above the code pane. Click  to select additional operators that are not visible in the rows above the code pane.
- To add a function call, expand the appropriate function category on the **Functions** tab, and double-click the function name. The custom function editor adds a basic syntax template for the function at the cursor's position in the code field.

The icon beside each function name indicates the return type of the function. Functions that return character data are displayed with the  icon, and functions that return numeric data are displayed with the  icon.

You can click  and select **Validate** at any time to check the syntax of the function that you are building. If the code does not compile correctly, SAS Intelligent Decisioning displays detailed information in the **Validation Results** window.

When you are finished editing the function, click **Save**.

TIP

In the autocomplete list, SAS Intelligent Decisioning displays all function names and variable names that match the text that you enter. Double-click an item to add it to the code field. Single-click an item to display information about the item, such as whether it is a function or a variable. For functions, the information includes the returned data type and category. For variables, the information includes the data type and direction (input, output, or both).

The autocomplete list does not include functions that are defined in a custom context file.

Rules for Writing Custom Function Files

When you are developing your DS2 custom function, follow these rules:

- Do not change the name of the method. The method name must match the function name.
- The method signature can include variables that are both input and output variables. Specify these variables with the `in_out` keyword. Custom functions support data grid packages and the DS2 data types that are listed in [“Data Types for SAS Data Sets” in SAS DS2 Language Reference](#).

In the method signature, list input-only parameters first. List input-output and output-only parameters next, using the `in_out` modifier. Do not specify a length for input-output and output-only parameters. The length for these parameters is derived from the variables that are passed into the method.

Note: Packages are input-output data types.

- The variables that you use as function parameters must be mapped to input variables in your decision.

Managing the Variables in a Custom Function File

The Properties of a Variable

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a custom function. Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See “Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.
Data type	Custom functions support data grid packages and the DS2 data types that are listed in “Data Types for SAS Data Sets” in SAS DS2 Language Reference. Integer, decimal, date, and datetime variables are converted to doubles when generated code is run.
Input and Output	A variable in a custom function signature can be an input variable, an output variable, or both, but it cannot be a temporary variable. Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.
Length	The length for numeric variable types is set automatically.

Property	Description
	Note: For scenario tests, the <code>variable.length.honorOutputLengthInMAS</code> and <code>variableLengthOverridden</code> configuration options are ignored. Scenario tests always use the length that you specify. The maximum length for character variables is 10485760 characters. For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	You can specify an initial value only for variables of type character, integer, decimal, or boolean. Initial values are not used when a custom function is run. Initial values are only for documentation purposes. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. When you are entering expressions in the expression editor, specify 1 for True and 0 for False. To use the number sign (#) in literal values, specify the percent-encoded value %23. For content that will be published to a SAS Micro Analytic Service destination, the values of date and datetime input variables must be numeric. They cannot be formatted values such as 18jul2019. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.
Maximum row count	The maximum number of rows that are allowed in each column of a data grid that is generated from an input table in which the data grid column names use array syntax. If the input data contains more entries for a column than the specified row count allows, the input data for that column is truncated. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in <i>SAS Intelligent Decisioning: Using Data Grids</i>

Syncing Variables and Code

To propagate changes that you enter on the **Variables** tab to the code that is shown on the **Code** tab, click **Sync to Code** on the **Variables** tab. Changes to variables are

propagated only to the signature of the custom function. The rest of the custom function code is unchanged. Both output-only and input-and-output variables are added with the `in_out` keyword.

To propagate changes that you enter in the method signature on the **Code** tab to the **Variables** tab, click  and select **Sync Variables** on the **Code** tab. Variables that you add to the signature with the `in_out` keyword are added to the **Variables** tab as both input and output variables. Variables that are not specified with the `in_out` keyword are added to the **Variables** tab as input-only variables. You can specify that any variable is both an input and output variable by selecting both the **Input** and **Output** check box on the **Variables** tab.

If you make conflicting changes to both the **Code** tab and the **Variables** tab and then save the file, SAS Intelligent Decisioning prompts you to select which tab contains the changes that you want to retain. If you make conflicting changes on the two tabs and then click a sync button, only the changes on the tab on which you click the sync button are retained. Those changes are then propagated to the other tab.

Note: You cannot manage local variables on the **Variables** tab. Local variables are variables for which neither the **Input** nor the **Output** checkboxes are selected on the **Variables** tab. If local variables are listed on the **Variables** tab, SAS Intelligent Decisioning displays an error message when you save the code file or click a sync button. The message is `The direction of some variables is not selected.` You must delete local variables from the **Variables** tab.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name: `"string"` in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name: `"HMEQ"`. [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➤**. To import all of the variables in the table, click **➤➤**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See [“Input Variables and Output Variables” on page 204](#) for more information.
- 7 Click **Sync to Code** to add the variables to the code that is on the **Code** tab, and then save the file. For more information, see [“Syncing Variables and Code” on page 239](#).

Create Custom Variables

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click **Add variable** and select **Custom variable**. The Add Variables window appears.
- 2 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 238](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Maximum row count**, and **Length** fields.
- c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable's data type. See [“The Properties of a Variable” on page 238](#) for additional information.
- d Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- e Verify that the **Input** and **Output** check boxes are selected correctly for each variable.

- Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.
 - Clear the **Output** check box for any variable that you want to exclude from the output data.
- 3 (Optional) Modify the variable properties in the table of variables.
 - 4 Click **OK** to add the variables and close the Add Variables window.
 - 5 Click **Sync to Code** to add the variables to the code that is on the **Code** tab, and then save the file. For more information, see [“Syncing Variables and Code” on page 239](#).

Duplicate a Variable

- 1 On the **Variables** tab, select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 2 Enter a new name for the duplicate variable.
- 3 (Optional) Enter a description for the variable.
- 4 Click **Duplicate**.
- 5 Click **Sync to Code** to add the variables to the code that is on the **Code** tab, and then save the file. For more information, see [“Syncing Variables and Code” on page 239](#).

Importing and Exporting Variables

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, “Import File Formats,” on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the custom function into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.

- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

TIP To add new columns to an existing data grid variable, select **Replace all variables**.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Export Variables

Note: If you make changes to the variables, you must save the custom function before exporting variable data.

- 1 Open the custom function from which you want to export variables.
- 2 On the **Variables** tab, select the variables that you want to export.
- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

- 1 On the **Variables** tab, select the check box for the variables that you want to delete, click , and select **Delete**.
- 2 Click **Sync to Code** to delete the variables from the code that is on the **Code** tab.

Edit Variable Properties

On the **Variables** tab, click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

Note: When you rename a variable, click **Sync to Code** to change references to that variable within the same object change to use the new name.

See “The Properties of a Variable” on page 238 for additional information.

Edit Metadata for Data Grid Variables

For information, see “Editing Data Grid Variable Metadata” in *SAS Intelligent Decisioning: Using Data Grids*.

Validate a Custom Function File

To validate a custom function file, open the file, click , and select **Validate**. For custom function files that use a custom context file, specify the custom context file in the **Test custom context** field on the **Properties** tab before validating the custom function file.

Testing a Custom Function

Before you use a custom function in a rule set, test the function code by including it in a DS2 custom code file and running a scenario test on the code file. For example, you can test the SQUARE custom function with the following custom code file:

```
package "${PACKAGE_NAME}" /inline;
  method square(double value) returns double;
    return value * value;
  end;
  method execute(double value, in_out double result);
    result = square(value);
  end;
endpackage;
```

If your custom function uses a custom context file, specify the custom context file in the **Test custom context** field on the **Properties** tab.

For more information, see “Testing Code Files” on page 222 and “Testing Rule Sets” on page 89.

Copy a Custom Function URL

To create a link for external documentation that automatically opens a custom function in SAS Intelligent Decisioning, open the custom function, click , and select **Copy custom function URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Custom Functions

- 1 In the Custom Functions category view, select the two custom functions that you want to compare, click , and select **Compare code**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window.

- 2 (Optional) Click **Export** to export the results of the comparison to a PDF file.
The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Custom Function Categories

Create a Custom Function Category

- 1 In the Custom functions category view, if there are no custom functions visible, click **Manage categories**. If custom functions are visible, click , and select **Manage categories**.
- 2 In the Manage Categories window, click **New category**.
- 3 In the New Category window, enter a name for the category if you do not want to use the default name. Category names are limited to 250 characters and cannot contain forward slashes (/).

- 4 (Optional) Enter a description for the new category. Descriptions are limited to 1000 characters.
- 5 Click **Save**.

Edit a Custom Function Category

- 1 In the Custom functions category view, if there are no custom functions visible, click **Manage categories**. If custom functions are visible, click , and select **Manage categories**.
- 2 In the Manage Categories window, select the category that you want to edit, click , and select **Edit**.
- 3 In the Edit Custom Function Category window, modify the category name and description as needed, and click **Save**.

Hide a Custom Function Category

When you hide a custom function category, the category and all of its custom functions are removed from the category list. These items are hidden from view. They are not moved to the recycle bin, and they are not deleted. You cannot create a new category or new functions that have the same names as the hidden items unless you permanently delete the hidden items. You can [unhide \(restore\)](#) or [permanently delete the hidden items](#) by using the **Manage hidden items** menu option.

To hide a custom function category:

- 1 In the Custom functions category view, if there are no custom functions visible, click **Manage categories**. If custom functions are visible, click , and select **Manage categories**.
- 2 In the Manage Categories window, select the category that you want to hide, click , and select **Hide**. SAS Intelligent Decisioning prompts you to confirm whether you want to hide the category and all of its functions.
- 3 In the Hide Category window, click **Hide**.

Managing Custom Functions

Duplicate Custom Functions

To duplicate a single custom function:

- 1 In the Custom Functions category view, select the custom function that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Custom Function window appears.
- 3 In the Duplicate Custom Function window, enter a new name for the duplicate custom function.
- 4 (Optional) Enter a description for the custom function.
- 5 Select the category for the function, and click **Duplicate**.

To duplicate multiple custom functions:

- 1 In the Custom Functions view, select the custom functions that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the custom functions and appends `_Copy` and a number to the names of the duplicate copies.

Hide Custom Functions

When you hide a custom function, it is removed from the category list. It is hidden from view, but it is not moved to the recycle bin or deleted. You cannot create new functions that have the same names as the hidden functions unless you delete the hidden functions. You can [unhide \(restore\)](#) or [permanently delete the hidden functions](#) by using the **Manage hidden items** menu option.

To hide custom functions, select the functions in the Custom Functions category view, click , and select **Hide**.

Rename a Custom Function

Note: You cannot rename a custom function if it is open. Click **×** to close the custom function.

- 1 In the Custom Functions category view, select the custom function that you want to rename.
- 2 Click **:** and select **Rename**.
- 3 In the Rename window, enter a new name for the custom function, and click **Rename**.

Change a Function's Category

- 1 In the Custom Functions category view, select the custom function, click **:**, and select **Change category**.
- 2 In the Change Category window, select the new category, and click **OK**.

Restore or Permanently Delete Hidden Functions and Categories

- 1 In the Custom functions category view, if there are no custom functions visible, click **Manage hidden items**. If custom functions are visible, click **:**, and select **Manage hidden items**.
- 2 In the Manage Hidden Items window, select the function or category that you want to restore or delete.
- 3 Click  to permanently delete the selected function, or click  to unhide (restore) it.
- 4 If you are restoring a function that was originally stored in a category that has also been hidden, you must select the category to which you want to restore the function.
 - If you restore a category that contained functions that are also hidden, SAS Intelligent Decisioning displays the list of hidden functions in that category.

All of the functions are selected by default. Click **Yes** to restore all of the functions. Select **No** if you do not want to restore any of the functions. Alternatively, change which functions are selected, and click either **Yes** to restore the selected functions or **No** if you do not want to restore the selected functions.

- If you are restoring a function that was originally stored in a category that has also been hidden, and if you are not restoring the category, you must select the category to which you want to restore the function. In the Select Category window, select the category, and click **Apply**.

Using Value Lists

About Value Lists	252
Value Lists and Local Value Lists	253
Create a New Value List	254
Manually Add Entries to a Value List	254
Specifying Value List Entries	255
Edit Entries in a Value List	257
Delete Entries in a Value List	257
Importing and Exporting Value Lists	258
About Importing, Exporting, and Enabling Value Lists	258
Format of the CSV Import File for a Value List	259
Import Entries Into a Value List from a CSV File	260
Export Entries from a Value List to a CSV File	261
Copy a Value List URL	261
Compare Value Lists	261
Generate PDF Documentation for a Global Value List	263
Managing Value Lists	263
Duplicating Value Lists	263
Delete Value Lists	264
Rename a Value List	265
Move Value Lists to a Different Folder	265
Managing Versions of Value Lists	265
Set the Displayed Version	265
Create a New Version	265
Copy the Content of a Version	267
Delete a Version	267
Determine Which Objects Use a Value List	268
Checking Out and Committing Value List Versions	269
About Checking Out and Committing Versions	269
Concurrently Checking Out and Committing Value List Versions	270
Check Out and Commit a Value List Version	270
Determine Who Has a Version Checked Out	271
Opening the Original Object	272
Undoing a Check Out	272

About Value Lists

A value list is the list of acceptable values for a variable. A value list can be a numeric list or a character list. The entries in a numeric value list can be discrete values or ranges.

You use value lists when you are designing segmentation trees. Value lists are required for variables that are used to create variable branches or matrices in [segmentation trees](#). [To use a value list for a variable's values, you must](#) associate the value list with a variable when you define the variable. You can enable or disable individual entries within a list. Only the individual entries that are enabled are available for use.

You can associate labels with the individual entries in a value list. If an entry has a label, the label is displayed in the SAS Intelligent Decisioning interface instead of the value.

For example, the value list shown in [Figure 9.1](#) is a character value list. All of the entries are enabled except entry three, "spindle knuckle". So, for example, when a variable that is associated with this list is used to create a variable branch in a segmentation tree, four outgoing paths are created for the variable branch.

Figure 9.1 Example Character Value List

Order	Enabled	Value
1		motor
2		radiator
3		"spindle knuckle"
4		transmission
5		wheel

The value list shown in [Figure 9.2](#) is a numeric value list that defines four ranges for credit scores. Each range has been assigned a label, so when a variable that is associated with this value list is used, the labels are used in SAS Intelligent Decisioning interface instead of the actual ranges. This example value list is used to create the segmentation tree matrix shown in [Figure 11.3 on page 305](#).

Figure 9.2 Example Numeric Value List with Labels

Order	Enabled	Value	Label
1		$v < 620$	poor
2		$620 \leq v \leq 719$	fair
3		$720 \leq v \leq 775$	good
4		$v > 775$	excellent

Values entered for variables that are associated with a value list are not validated against the value list. Value lists are not published to a destination, and they are not deployed to production environments.

TIP Value lists are for use in designing SAS Intelligent Decisioning content. They cannot be used by other SAS solutions. An advanced list is a tabular data set that can contain multiple keys and values and that can be shared among SAS applications and workflows. For information about advanced lists, see [Chapter 10, “Managing Advanced Lists,”](#) on page 273.

Value Lists and Local Value Lists

There are two types of value lists:

- Value lists that are defined in the Value Lists category are value lists that can be used in any segmentation tree. These value lists support versioning, and you can enable or disable specific entries in these lists.
- Local value lists can be used only within the segmentation tree in which the value list is defined. You create and manage a local value list from within the segmentation tree where you want to use the list. For more information, see [“Using Local Value Lists”](#) on page 316.

In both types of value lists, you specify the individual entries in the same way. For more information, see [“Specifying Value List Entries”](#) on page 255.

Create a New Value List

- 1 Click  in the navigation pane in SAS Intelligent Decisioning to navigate to the Value Lists category view.
- 2 Click **New value list**. The New Value List window appears.
- 3 Enter a name for the new value list. Value list names are limited to 100 characters. The names are case insensitive and must be unique within the same folder.
- 4 Select **Character** or **Numeric**, depending on whether the values in the list are character or numeric.
- 5 (Optional) Enter a description for the new value list. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 6 Click , and select the folder in which you want to create the new value list.
- 7 Click **Save**. SAS Intelligent Decisioning opens the new list.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 8 Add entries to the new table either by importing entries from a CSV file, or by adding entries manually. See [“Import Entries Into a Value List from a CSV File” on page 260](#) and [“Manually Add Entries to a Value List” on page 254](#) for more information.

Manually Add Entries to a Value List

.....
Note: It is recommended that a single value list contain no more than 1000 entries.
.....

- 1 Open the value list to which you want to add entries.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 (Optional) If the value list is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the value list. For more information, see [“Check Out and Commit a Value List Version” on page 270](#).
- 3 For each value that you want to add to the list:
 - a Click **+ Add value**. SAS Intelligent Decisioning adds a new blank entry to the list.
 - b Enter the value for the new entry, and press Enter. Each entry is saved when you press Enter. See [“Specifying Value List Entries” on page 255](#) for more information and examples.

TIP When you use variables to create a matrix in a segmentation tree, the values in the variables' value lists are used to create rows and columns in the matrix. SAS Intelligent Decisioning automatically includes one column and one row for the value **other** in addition to the values that are included in the value lists.

- c (Optional) Enter a label for the value, and press Enter. Labels are each limited to 512 characters. If you enter a label, the label is displayed in the SAS Intelligent Decisioning interface instead of the value.
 - d (Optional) Disable the entry. By default, each new entry is automatically enabled. To disable an entry, click the switch in the **Enabled** column.
- 4 (Optional) Reorder the entries in the list. To move a list entry, select the check box for the entry, and either drag the entry to a new position in the table or click **↓**, **↑**, **↕**, or **↗**. When you are selecting values in a value list in the SAS Intelligent Decisioning interface, the items are displayed in the order in which they are defined in the list.

Specifying Value List Entries

In numeric lists, a single entry can specify multiple discrete values, multiple ranges, or both. In character lists, a single entry can specify multiple discrete values. To specify multiple specifications in one entry, separate the specifications with a semicolon (;). To specify a range of numeric values, use **v** to indicate the value of

the variable that is associated with the list. You can use an exclamation mark in numeric entries to indicate NOT.

When you enter decimal numbers, use a dot for the decimal point. Decimal formats that use commas are not supported.

All numeric values are treated as decimal numbers regardless of whether they contain a decimal point. For example, the list entry 15 is treated as 15.0. This entry matches the input values 15 and 15.0.

Multiple entries within the same list cannot specify the same value. Numeric ranges cannot overlap.

Gaps are allowed in range specifications. However, you must ensure that any gaps in numeric ranges are intentional. For example, because list entries are treated as decimal values, the following two entries produce a numeric gap that excludes the values 20.1 through 20.9.

```
10<=v<=20
21<=v<=30
```

To eliminate the gap, specify the same value for the end of the first range and start of the second range. To avoid overlapping ranges, change the first comparison operator in the second range to greater than (<) instead of greater than or equal to (<=).

```
10<=v<=20
20<v<=30
```

To determine if there are numeric gaps in a list, open the list, click , and select **View Summarize Gaps**. If gaps are present, SAS Intelligent Decisioning displays a message with an **Action** button. To display the list of entries that are associated with gaps, click **Action**, and select **View details**. In the Summarize Gaps window, click the order number of an entry to display more information.

Character values are each limited to 1000 characters. Character values longer than 1000 characters are truncated. If a character string contains either a space or a semicolon, enclose the string in quotation marks. For examples, see the following table.

Table 9.1 Examples of Value List Entries

Value Specification	Meaning
50;100;150	The associated variable's value equals 50, 100, or 150.
150;v>=200	The associated variable's value is either equal to 150 or greater than or equal to 200.
50<v<100;150<=v<=200	The associated variable's value is either between 50 and 100, exclusive, or it is between 150 and 200, inclusive.

Value Specification	Meaning
!50	The associated variable's value does not equal 50.
cat;mouse;weasel	The associated variable's value either equals cat, mouse, or weasel.
'my cat';"your hamster"	The associated variable's value equals either my cat or your hamster.

Edit Entries in a Value List

Open the value list, and change the values as needed on the **Value List** tab. The changes for each entry are saved when you press Enter for that entry.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

Delete Entries in a Value List

Open the value list, select the entries that you want to delete, and click **Delete**.

Importing and Exporting Value Lists

About Importing, Exporting, and Enabling Value Lists

You can import and export value lists by using either of the following file types:

- Comma-delimited (CSV) files. The ability to import and export a value list as a CSV file is for modifying an existing value list. You can export an existing value list, edit the CSV file, and import the updated CSV file. You can use the **Import** and **Export** buttons in SAS Intelligent Decisioning, or you can use the %DCM_EXPORT_LOOKUP and %DCM_IMPORT_LOOKUP macros. (The LOOKUP macros import and export value lists in addition to lookup tables.) For more information, see the following topics:
 - [“Format of the CSV Import File for a Value List” on page 259](#)
 - [“Import Entries Into a Value List from a CSV File” on page 260](#)
 - [“Export Entries from a Value List to a CSV File” on page 261](#)
 - [“%DCM_EXPORT_LOOKUP Macro” in SAS Intelligent Decisioning: Macro Guide](#)
 - [“%DCM_IMPORT_LOOKUP Macro” in SAS Intelligent Decisioning: Macro Guide](#)
- JavaScript Object Notation (JSON) files. If you do not need to edit the content of the CSV file, use SAS Environment Manager, the SAS Viya command-line interface (sas-viya), or the **Export objects to JSON files** and **Import objects from JSON files** options in SAS Intelligent Decisioning.

The recommended approach is to use the transfer plug-in to the sas-viya CLI. The CLI preserves the ID and version history of the objects that are being transferred. For information about the CLI, see [SAS Viya Platform: Content Migration from SAS Viya 4](#). For information about using SAS Environment Manager, see [“Content Page” in SAS Environment Manager: User's Guide](#) and [“Import Page” in SAS Environment Manager: User's Guide](#). For information about using the JSON options in SAS Intelligent Decisioning, see [“Importing and Exporting Objects Using JSON Files” on page 13](#).

Format of the CSV Import File for a Value List

Note: The format of the CSV file depends on whether you are importing the value list by using the %DCM_IMPORT_LOOKUP macro or by clicking the **Import** button that is available in the user interface when you open a value list. For information about the format used by macro, see [“Format of the CSV Input File for the Macro” in SAS Intelligent Decisioning: Macro Guide](#).

To import a value list by clicking the **Import** button, the first line of the CSV must be a header row that defines the three columns:

value

specifies the value or range of values for the entry.

In numeric lists, a single entry can specify multiple discrete values, multiple ranges, or both. In character lists, a single entry can specify multiple discrete values. To specify multiple specifications in one entry, separate the specifications with a semicolon (;). To specify a range of numeric values, use `v` to indicate the value of the variable that is associated with the list. Character values are each limited to 1000 characters.

Character values longer than 1000 characters are truncated. If a character string contains either a space or a semicolon, enclose the string in quotation marks. For examples, see [Table 9.1 on page 256](#).

label

specifies a label to be used in the SAS Intelligent Decisioning interface in place of the actual value. To create a blank column for the `label` column in the CSV file, specify two comma separators without any content between them.

enabled

specifies True if the entry is enabled and False if the entry is disabled. If you leave this column blank, the imported entry is disabled.

Each subsequent row of the CSV import file defines an entry in the value list. To create a blank column for label in the CSV file, specify two comma separators without any content between them.

The following CSV file example defines a value list of credit score levels. This value list defines five credit score levels that can be used to help determine the risk associated with a loan application. All of the entries in this value list are enabled.

```
value,label,enabled
"v >= 800","Exceptional",true
"740 <= v <= 799","Very Good",true
"670 <= v <= 739","Good",true
"580 <= v <= 669","Fair",true
"v <= 579","Poor",true
```

In a spreadsheet application, this value list appears as shown in the following figure.

	A	B	C
1	value	label	enabled
2	v >= 800	Exceptional	TRUE
3	740 <= v <= 799	Very Good	TRUE
4	670 <= v <= 739	Good	TRUE
5	580 <= v <= 669	Fair	TRUE
6	v <= 579	Poor	TRUE

After you import the CSV file, it appears in SAS Intelligent Decisioning as shown in the following figure.

Order	Enabled	Value	Label
1		v >= 800	Exceptional
2		740 <= v <= 799	Very Good
3		670 <= v <= 739	Good
4		580 <= v <= 669	Fair
5		v <= 579	Poor

Import Entries Into a Value List from a CSV File

Note: It is recommended that a single value list contains no more than 1000 entries.

- 1 Open the value list to which you want to import entries.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click **Import**. The Import Entries from CSV File window appears.
- 3 Click , and select the CSV file that contains the value list entries that you want to import.
- 4 Select or enter the encoding of the CSV file

- 5 Select **Add entries** to add the newly imported entries to the existing list, or select **Replace all entries** to replace all of the existing entries with the newly imported entries.
- 6 Click **Import**.

Export Entries from a Value List to a CSV File

IMPORTANT Do not modify the file structure or the header row in the exported CSV file. You can modify the data values.

To export a value list, open the value list and click **Export**. The table is exported into a CSV file, and a download notification appears in the browser window. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Copy a Value List URL

To create a link for external documentation that automatically opens a value list in SAS Intelligent Decisioning, open the value list, click , and select **Copy value list URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Value Lists

You can compare the contents of two different value lists, or you can compare the contents of two different versions of the same value list.

- 1 Select the objects that you want to compare.
 - To compare the contents of two different value lists, select the value lists in the category view, click , and select **Compare object contents**.
 - To compare the contents of two versions of the same value list, open the value list, click  on the **Versions** tab, and select **Compare object contents**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Object Contents window.

By default, the Compare Object Contents window displays the values and their associated labels, and indicates whether each list item is enabled. For example, in the following figure compares version 1 and version 2 of the same value list. These two versions have only one common entry, $720 \leq v \leq 775$. In version 1, this entry has an associated label, `good`, but the label has been removed from version 2. The remaining value list entries are unique to one of the two versions of the value list.

TIP The information icon ⓘ displays the location of the value list.

SAS® Intelligent Decisioning - Build Decisions

Value Lists > credit_score_levels (2.0) Opened items (1)

Compare Object Contents Export Close

Filter values Show differences Show all

Number of Different Entries: 8 Value List Value	Total Number of Entries: 5 credit_score_levels(2.0) ⓘ Label	Enabled	Total Number of Entries: 4 credit_score_levels(1.0) ⓘ Label	Enabled
$v < 599$		True		
$600 \leq v \leq 719$		True		
$720 \leq v \leq 775$		True	good	True
$776 \leq v \leq 799$		True		
$v > 800$		True		
$v < 620$			poor	True
$620 \leq v \leq 719$			fair	True
$v > 775$			excellent	True

- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The Export Comparison Results window appears.
- 4 (Optional) Select the information that you want to export, and click **Export**. If you want the PDF file to display only the differences between the two objects, select **Show differences**. To display all of the objects' information, select **Show all**.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Generate PDF Documentation for a Global Value List

You can generate detailed documentation for a value list as a PDF document. The PDF includes the value list properties, and a table of the current values that are in the list. The table includes any labels defined for each entry, and it identifies whether the entries are enabled or disabled.

- 1 Open the value list.
- 2 Click , and select **Create document**. The Create Document window appears.

TIP If this option is disabled, there might be unsaved changes. Click .

- 3 Enter a name for the document if you do not want to use the default name.
- 4 Click **Create**. SAS Intelligent Decisioning creates the PDF. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Value Lists

Duplicating Value Lists

About Duplicating Value Lists in Folders for Which the Check-Out and Commit Feature Is Enabled

When you duplicate a value list, SAS Intelligent Decisioning first creates an empty value list, and then updates the list to add the individual list entries. The default permissions for the Decision Repository enable users to create new objects, but the permissions do not include the ability to update the objects. In order to duplicate an object that is in the Decision Repository, you must check out the object.

If your site has created additional folders for which the check-out and commit feature is enabled, and if those folders have been assigned the same permissions as those assigned to the Decision Repository, you must also check out objects in those folders before you can duplicate them.

When you duplicate an object that is checked out, the duplicate object is not automatically checked out.

Duplicate Value Lists

To duplicate a single value list:

- 1 In the Value Lists category view, select the value list that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Value List window appears.
- 3 Enter a new name for the duplicate value list.
- 4 (Optional) Enter a description for the value list.
- 5 Select the version of the value list that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate value list.
- 7 Click **Duplicate**.

To duplicate multiple value lists:

- 1 In the Value Lists view, select the value lists that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the value lists and appends **_Copy** to the names of the duplicate copies. If needed, a number is also appended to the names of the duplicate copies.

Note: When you duplicate a value list, SAS Intelligent Decisioning creates a relationship between the original value list and the duplicate value list. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the value list before you paste the new content.

Delete Value Lists

Note: You cannot delete a value list if it is open. Click  to close the list.

In the Value Lists category view, select the value lists that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the value lists to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename a Value List

Note: You cannot rename a value list if it is open. Click **×** to close the list.

- 1 In the Value Lists category view, select the value list that you want to rename.
- 2 Click **:** and select **Rename**. The Rename window appears.
- 3 Enter a new name for the value list, and click **Rename**.

Move Value Lists to a Different Folder

- 1 In the Value Lists category view, select the value lists that you want to move.
- 2 Click **:** and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the value lists, and click **OK**.

Managing Versions of Value Lists

Set the Displayed Version

The displayed version is the version whose information is displayed on the **Properties** and **Value List** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You

cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Value List Version” on page 270](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, click **New version**. The New Version window appears.
- 3 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 4 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 5 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 6 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the value list whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the value list whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target value list into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target value list. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see [“Default Permissions” in SAS Intelligent Decisioning: Administrator's Guide](#) and [“Folder-Based Permissions” in SAS Intelligent Decisioning: Administrator's Guide](#).

Guide. By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of lookup tables or value lists, the [sas.referencedata.deleteVersions](#) configuration option must be turned on.

On the object's **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Determine Which Objects Use a Value List

To list the objects that use a specific value list:

- 1 On the **Value lists** category page, select the check box for value list, click , and select **View used by report**. The All Objects that Use the Selected Item window appears. By default, this window lists the objects that use the latest version of the selected value list.
- 2 (Optional) Select the specific version for which you want to view the Used By report.

To view the list of all objects that use any version of the selected value list, clear the value in the **Version** field.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Value List Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.referenceData.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a value list version, the **Check Out** button for that value list is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Value List Versions” on page 270](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Value List category view shows two value lists with the same name, but the folders listed in the **Location** column differ for each object. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

You cannot enable the checked-out version that is in **My Folder**. To enable a version with your changes, you must commit your changes, and enable the committed version.

Concurrently Checking Out and Committing Value List Versions

The ability for multiple users to check out the same value list at the same time is controlled by the [sas.referencedata.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's **My Folder** location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Value List Version

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into

your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.

- 3 Modify the entries in the checked-out version as needed.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 272](#).

- 4 On the **Versions** tab, click **Commit**. The Commit Value List window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 7 (Optional) Enter information about the new version in the **Notes** field.
- 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Enabling Entries in a Value List

Enabling an entry in a value list makes the list available for use in the SAS Intelligent Decisioning interface when you are editing objects that use variables that are associated with a value list.

To enable or disable specific entries in a list, open the value list, select the **Value List** tab, and click the switch in the **Enabled** column.

Managing Advanced Lists

About Advanced Lists	274
Configuration	274
List Creation	274
List Revisions	275
List Definition Modifications	276
List Contents Modifications	276
Data Masks	277
Immutable Lists	278
Column Name Restrictions	278
Create a List	279
Managing Lists	281
Open a List	281
Preview a List	282
Deploy an Existing List	282
Move a Deployed List to Developing	283
Search Lists	283
Sort Lists	284
Delete a List	284
Refresh Lists	285
Determine Which Objects Use a List	285
Managing List Definition	286
Modify List Properties	286
Modify List Columns	286
Refresh List Columns	287
Managing List Contents	287
Manually Add Data to an Existing List	287
Import Data to an Existing List	288
Modify List Contents	288
Duplicate a Row of List Data	289
Deploy Specific List Items	290
Move Deployed List Items to Developing	291
Add an Expiry to List Items	291
Modify the Expiry for List Items	292
Remove an Expiry from List Items	293
Refresh List Contents	293
Remove List Contents	293
Export List Contents	294

Managing List Revisions	294
Create a List Revision	294
View the Contents of a List Revision	295
Compare the Contents of Revisions	295
Modify a List Revision	296
Set the Deployed Version	296
Format of a CSV Data File	296

About Advanced Lists

An advanced list is a tabular data set that can have multiple keys and values and can be shared among SAS applications and workflows. The advanced lists features enable you to manage tabular data in your environment and also publish and query that data in a low-latency, key-value data store.

Configuration

Advanced lists are stored in Redis. You must provide, configure, and maintain your own Redis instances. Beginning with 2025.10, you have the ability to configure multiple instances of Redis for advanced lists. For information about configuring Redis for your environment, see [“List Data Service” in SAS Viya Platform: Configuration Properties](#).

List Creation

When you create a list, you define its functionality, state, location, and structure.

The list’s functionality is defined in the list properties (for example, name, label, and description). You are required to provide a name for a list. A label and description are optional.

A list’s status is either deployed or developing. A deployed list is available for use by other SAS applications and workflows. A developing list is not available for use by other SAS applications and workflows. When you create a list, its default status is developing, unless you specify to deploy the list as you create it.

You can specify a list to be immutable. This means that once data is added to the list, the contents cannot be changed. For more information about immutable lists, see [“Immutable Lists” on page 278](#).

You must specify the location where you want to save each list. The default location is `/SAS Content/Products/List Data`. For information about folders, see [“Folders: Overview” in SAS Viya Platform: Folders](#).

The list's structure is the columns that you define to represent the data that the list contains. You can either manually define a list's structure and import the data later or upload a CSV file that defines the list structure and also imports the data that is in that file.

IMPORTANT If column name restrictions are enabled in your environment, column names must conform to DS2 variable name conventions when you create a list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).

Note: For information about the format of an imported CSV file, see [“Format of a CSV Data File” on page 296](#).

You can specify that items in a list have an expiry when the items are deployed. An expiry enables you to set a date and time at which a specified list item is no longer used by consumers. The option to set an expiry for list items must be enabled when you create a new list. You cannot enable this option on existing lists.

A list is not valid unless you specify at least one column to be a key column. This column is used to construct a lookup key to retrieve the record from the contents data store. You can create complex lookup keys by identifying more than one column as a key.

List Revisions

You can create revisions of an advanced list, modify contents of the most recent revision, view the contents of a revision, set which list revision is displayed and deployed, and compare the contents of multiple revisions.

Each new list that you create becomes the first revision of the list. This is Version 1 on the list's **Versions** tab. If you create subsequent list revisions, Version 1 is the version that is displayed on the **List Contents** and **Properties** tabs until you select to display a different version.

You can modify only the latest revision. (For more information, see [“List Contents Modifications” on page 276](#).) You can view the contents of all other revisions.

When you deploy a list, you deploy the displayed version, unless you specify to deploy a different version. Only one version of a list can be deployed. A  in the Deployed column on the **Versions** tab indicates the revision that is deployed. When you move a deployed list to developing, the list and the revision are no longer deployed. For information about the deployed and developing states of a list, see [“List Creation” on page 274](#).

You can select to compare the contents of multiple list revisions. There is no unmasked version of a comparison. Only privileged users can see the masked contents in a version. Values that changed between versions are highlighted in the comparison.

List Definition Modifications

You can modify the structure and properties of existing lists based on these rules:

- If a list is empty, you can modify specific properties, such as the label and description, and modify column information, such as the column name, data type, and data mask, and key columns.

IMPORTANT If column name restrictions are enabled in your environment, column names must conform to DS2 variable name conventions when you modify a list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).

- If a list contains data, you can modify specific properties, such as the label and description. You cannot modify list column information or key-columns.

List Contents Modifications

You can modify the data in existing lists based on these rules:

- If a list is empty, you can manually add data or import data from a CSV file.
- If a list contains data, you can do the following:

Note: Some features related to managing an immutable list's content are restricted. These restrictions are discussed in [“Immutable Lists” on page 278](#).

- ☐ manually add or import data from a CSV file.
- ☐ modify the contents of the most recent list revision. You cannot modify the status and key-column values and a privileged user cannot modify masked values if redactions are applied. For more information about list revisions, see [“List Revisions” on page 275](#).
- ☐ change the status of specific list items. List items that are developing are not available for use by other SAS applications and workflows. List items that are deployed are available for use by other SAS applications and workflows.

Note: Changing the status of specified list items does not change the status of a list. For example, if you deploy items in a list that is developing, the list status remains developing. In this case, the list rows are marked to become deployed when you deploy the list.

- ☐ set or remove an expiry for deployed items.

Note: The option to set an expiry for list items must be enabled when you create a new list. (See [“Create a List” on page 279](#) for more information.) You cannot enable this option on existing lists. When a list item expires, its status becomes archived. Archived items are still available in Redis and can be redeployed.

- ❑ change data masks and view redacted data if you are a privileged user. For more information about data masks, see [“Data Masks” on page 277](#).

Certain tasks, such as importing a CSV file, removing a list's entire contents, or updating a list's contents, can involve large data sets. As a result, these processes can take a while to complete. Long-running tasks are recorded on the **Jobs** tab for an advanced list. On this tab, you can check the status of an initiated job as you continue to work in the application, see details about previous jobs, and gain insights to a failed job by opening the job's log. Tasks that complete quickly are not added to the historical record on this tab.

Data Masks

Advanced lists provide the ability to protect sensitive data in your lists through data masks. When you mask data in a list, all or a portion of the data values in an entire list column are redacted in the output. For example, in the following figure, an entire password string is redacted in the list view, whereas, all but the last 4 digits of a phone number are redacted.

Users

 userid	 password	 email	 division	 department	 group	 phone#
user5	*****	user5@sas.com	Division2	Department1	Marketing	*****7894
user10	*****	user1@sas.com	Division1	Department1	R&D	*****7899
user6	*****	user6@sas.com	Division2	Department2	HR	*****7895
user1	*****	user1@sas.com	Division1	Department1	R&D	*****7890

The following rules apply to masking data in a list:

- Any list creator can specify a data mask for list columns when the list is created.
- Any list modifier can change the data mask for columns in an existing list that does not contain data.
- Only a privileged user can modify the data masks in a list that contains data. This permission is granted to a user by their administrator.

Note: For information about providing access to functionality, see [“Access to Functionality: Overview” in SAS Viya Platform: Access to Functionality](#).

By default, masked data is redacted in a list view. Only privilege users can toggle redactions on and off in a list view. Masked values cannot be modified when

redactions are on. For more information about list modifications, see [“List Contents Modifications” on page 276](#).

Immutable Lists

When you specify a list to be immutable, you are specifying that the list's content cannot be changed once the list is created. You can specify an immutable list only when you create a new list by importing a CSV file. This file defines the list and populates its content. Once an immutable list is created with contents, you cannot update or clear the list's contents. You can deploy an immutable list and move a deployed list to developing, but you cannot perform the same actions on specific items in an immutable list. If you are a privileged user, you can change data masks and delete an immutable list. For information about data masks, see [“Data Masks” on page 277](#).

Column Name Restrictions

Starting with 2026.03, advanced lists enforces column name restrictions that are based on DS2 variable name conventions. For information about the DS2 language, see [“Introduction to the DS2 Language” in SAS DS2 Programmer's Guide](#). The `useColumnNameDS2Restrictions` property is enabled in your environment configuration by default to ensure that advanced lists can be referenced in rule sets without error. For information about enabling or disabling column name restrictions, see [“List Data Service” in SAS Viya Platform: Configuration Properties](#).

When restrictions are enabled, column names must conform to the following rules when you create a new list or modify the column names in an existing list:

Note: Column names in existing lists that do not conform to these restrictions are not affected unless you modify them.

Note: Column names in advanced lists that are used in a rule set must conform to the following restrictions regardless of whether column name restrictions are enabled.

- The minimum length is 1 character and the maximum length is 32 characters.
- The name must begin with a single underscore (`_`) or a letter (`a-z`, `A-Z`). A name cannot begin with more than one consecutive underscore (for example, `__abc` is not valid).
- The name can contain only letters (`a-z`, `A-Z`), digits (`0-9`), and underscores (`_`).
- The name cannot be a DS2 reserved word (either a keyword or a data type). For a list of reserved words, see [“Reserved Words in the DS2 Language” in SAS DS2 Programmer's Guide](#).

Note: In addition to the reserved words listed in [“Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide](#), you cannot use TRUE, FALSE, or _N_ as column names.

- Column names are not case-sensitive and must be unique within a list.
- Column names cannot contain non-Latin characters.

Create a List

For more information about creating a list, see [“About Advanced Lists” on page 274](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Click **New list**. The New List wizard opens.
- 3 On the Properties page, enter a name for the list.

IMPORTANT Each list name must be unique within your system. Once you create a list, you cannot modify its name.

- 4 (Optional) Enter a label and description. Labels are limited to 100 characters. Descriptions are limited to 500 characters.
- 5 (Optional) If you want the list to be available for use by other SAS applications and workflows when you create it, select **Deploy list on creation**.

TIP You can deploy the list later. See [“Deploy an Existing List” on page 282](#).

- 6 (Optional) If you want the list to be immutable, meaning that the list’s contents cannot be changed after you add data, select **Is immutable**. For more information about immutable lists, see [“Immutable Lists” on page 278](#).
- 7 (Optional) If you want to allow duplicate keys, select **Allow duplicate keys**.
- 8 (Optional) If you do not want to use the default location to store the new list, click , and select the folder where you want to save the list.

IMPORTANT You can associate a list with a specific storage configuration only at creation. You cannot retroactively update the storage location for an existing list.

- 9 Specify how you want to define the columns that map to the list data.

IMPORTANT If column name restrictions are enabled in your environment, column names must conform to DS2 variable name conventions when you create an advanced list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).

- a To manually define the list columns, select **Define an empty list**.
- b To upload predefined list columns from a CSV file and also import the data from the same CSV file, select **Import a CSV file that defines the list and populates content**. Click  and select the CSV file that you want to import.

Note: If a CSV file does not have a header row, the values in the first row of data in the file are treated as the header row. For information about the format of an imported CSV file, see [“Format of a CSV Data File” on page 296](#).

- 10 (Optional) If you want items in your list to have an expiry when they are deployed, select **Row content has an expiry**.

Note: After you create the list, you can customize the expiry for the list's items on the **List Contents** tab. For more information about setting content expiry, see [“Add an Expiry to List Items” on page 291](#).

- 11 Click **Next**. On the Columns page, specify the column information that maps to the list's data.
 - a (Optional) For each column, specify a name and select the appropriate data type and data mask from the drop-down menus. For more information about data masks, see [“Data Masks” on page 277](#).

IMPORTANT If the `useColumnNameDS2Restrictions` configuration property is enabled in your environment, column names must conform to DS2 variable name conventions when you create an advanced list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).

Regardless of whether column name restrictions are enabled in your environment, column names in advanced lists that are used in a rule set must conform to the restrictions listed in [“Column Name Restrictions” on page 278](#).

IMPORTANT Once you add data to the list, only a privileged user can modify the data masks in the list. This permission is granted to a user by their administrator.

- b (Optional) To add a column, click **+**.

- c (Optional) To delete a column, select the column row, and click .
- 12 Click **Next**. On the Key columns page, select one or more columns to be a key column and click **➔**.

IMPORTANT A list is not valid unless you specify at least one key column.

Note: The order in which you add columns to the **Key columns** field is the order in which the columns appear on the list's **List Contents** tab when you first create the list.

- 13 Click **Finish**. The new list opens and displays the **List Contents** tab.

Managing Lists

Open a List

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select one or more lists to view.
 - a To open a single list, hover over the list that you want to open and click the list name. The list opens in the list details view and displays the **List Contents** tab.
 - b To open multiple lists, select the lists that you want to open and click . The first list that you selected in the Advanced Lists category view opens and displays the **List Contents** tab.

TIP The number of opened items is displayed in the top right corner of the selected list window. To view the contents of another list that you opened, click **Opened items (n)**, and then click the list that you want to view.

- 3 Click **x** to close the list and return to the Advanced Lists category view.

Preview a List

To quickly preview the contents of a list:

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list that you want to preview.
- 3 Right-click in the selected list row and select **Preview**.

The Preview List windows opens and displays the first 50 rows of list data.

- 4 Click **Open** to open the full list of data in the list details view, or click **Close** to close the preview window.

Deploy an Existing List

For information about the deployed and developing states of a list, see [“List Creation” on page 274](#).

For information about how to deploy a list as you create it, see [“Create a List” on page 279](#) or for information about how to deploy specific list items, see [“Deploy Specific List Items” on page 290](#).

Note: When you deploy a list, you deploy the most recent version of a list, unless you specify a different version. For more information about list revisions, see [“List Revisions” on page 275](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select one or more lists to deploy.
- 3 Click , and select **Deploy**.

TIP In the Advanced Lists category view, you can also right-click the name of a selected list and select **Deploy**. If you are viewing list data in the list's details view, click **Deploy list** from any tab in that view.

A  is added to the Status column of the list row on the **List Contents** tab to indicate that the list is deployed.

Move a Deployed List to Developing

For information about the deployed and developing states of a list, see [“List Creation” on page 274](#). For information about how to move specific list items from deployed to developing, see [“Move Deployed List Items to Developing” on page 291](#).

Note: When you move a deployed list to developing, the revision that was deployed with the list also is no longer deployed. For more information about list revisions, see [“List Revisions” on page 275](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select one or more deployed lists to move to developing.
- 3 Click , select **Move to developing**, and click **Move to developing**.

TIP In the Advanced Lists category view, you can also right-click the name of a selected list and select **Move to developing**. If you are viewing data in a list's details view, click **Move list to Developing**. If you are viewing list data in the list's details view, click **Move list to developing** from any tab in that view.

The  is removed from the Status column of a list row, which indicates that a list is developing.

Search Lists

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Enter the full or partial string for your search criteria in the search field above the lists, and click .

The search is not case-sensitive. You can search on values in these list columns:

- Name
- Label
- Created By
- Modified By

TIP If your search returns multiple lists, you can quickly preview the contents of a list to help you further narrow your search. For more information about how to preview a list, see [“Preview a List” on page 282](#).

- 3 To clear the search field and see all lists, click .

Sort Lists

- 1 Click  to navigate to the Advanced Lists category view.
- 2 To sort lists based on the values of a column, right-click the column heading, and select **Sort → Sort (ascending)** or **Sort (descending)**.

When the column is sorted in ascending order, ↑ appears beside the column heading. When the column is sorted in descending order, ↓ appears beside the column heading.

- 3 To clear the column sort, right-click the column heading, and select **Sort → Remove sorting**.

Delete a List

When you delete a list, you delete the list structure and its contents. You can delete only one list at a time.

Note: You cannot delete a list that is used by another object or a list that is deployed. You must move a deployed list to developing before you can delete it. For information about how to move a deployed list to developing a list, see [“Move a Deployed List to Developing” on page 283](#).

- 1 Click  to navigate to the Advanced Lists category view.
 - 2 Select the list that you want to delete.
 - 3 Right-click in the selected list row and select **Delete**.
 - 4 Click **Delete** to confirm that you want to delete the list.
- The list is removed from the Advanced Lists category view.

Refresh Lists

A good practice is to periodically refresh the lists that are displayed in the Advanced Lists category view. Doing so ensures that you see all the changes made to the lists database since you last loaded or refreshed the category view. In the Advanced Lists category view, click  to update the information that is displayed on the page. The changes made to the database since you last loaded or refreshed the page are visible.

Determine Which Objects Use a List

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list that you want to preview.
- 3 Click , and select **View used by report**. The All Objects that Use the Selected Item window lists all objects that use the selected list, the type of object, and the object location.

Note: Objects that reference an advanced list and are in the recycle bin are included in the Used By report. To remove the object from the Used By report, you must permanently delete the object from the recycle bin.

TIP If you are viewing a list's data in the details view, click **View used by report** to see the objects that use the list.

You can use the **Filter** field to filter the list of objects based on the object names. Click an object name to open the object. Click  next to an object name to display information such as whether the item is checked out and to whom, the date on which the object was last modified, and the ID of the user who modified the object.

To export all objects that use the selected item to PDF, click **Export**. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification in the bar at the bottom of the browser window. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing List Definition

Modify List Properties

For more information about modifying list properties, see [“List Definition Modifications” on page 276](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list that has the properties that you want to modify.
- 3 Right-click in the row of the selected list, and select **Edit properties**.

TIP You can also click the name of the selected list and open the list in the list details view. In the **Properties** pane on the list's **Properties** tab, click .

- 4 In the Edit Properties window, you can modify the label and description.
- 5 Click **Save** to save your changes and close the Edit Properties window.

Modify List Columns

For more information about modifying list columns, see [“List Definition Modifications” on page 276](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens in the list details view and displays the **List Contents** tab.
- 3 On the **Properties** tab, click  in the **Columns** section.
- 4 In the Edit Columns window, modify the columns.

- To change a column value, click in the column and specify a new value.

Note: If a list has data, you can change only the data mask for each row of data.

- To add a column, click **+** and enter values for the column.

IMPORTANT If column name restrictions are enabled in your environment, column names must conform to DS2 variable name conventions when you modify an existing advanced list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).

IMPORTANT All columns must map to the list’s source data.

- To delete a column, select the column that you want to delete and click .

Refresh List Columns

A good practice is to periodically refresh the data that is displayed in a list’s details view. Doing so ensures that you see changes that were made by other users of the list since you last loaded or refreshed the list. In a list’s details view, click  on the **Properties** tab to update the list’s columns. The changes made to the database since you last loaded or refreshed the page are visible.

Managing List Contents

Manually Add Data to an Existing List

After you create a list as described in [“Create a List” on page 279](#), you can manually add single rows of data to the list.

Note: You can add multiple rows of data to an existing list by importing a CSV file. For information about how to import data, see [“Import Data to an Existing List” on page 288](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list to which you want to add data and click . The list opens and displays the **List Contents** tab.
- 3 Click **+**.
- 4 In the Add List Item window, enter values for the list’s column fields.

Note: A column value can be empty. The key columns or a combination of values for each key must be unique.

- 5 Click **Save**. The new data row is added on the **List Contents** tab.

Import Data to an Existing List

After you create a list as described in “[Create a List](#)” on page 279, you can import data to an empty list or to a list that already contains data.

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list to which you want to import data.
- 3 Click . The list opens and displays the **List Contents** tab.
- 4 Click **Import**.
- 5 In the **Import List Items** window, click  and select the source data file. For information about the format of an imported CSV file, see “[Format of a CSV Data File](#)” on page 296.

Note: You cannot import a file that is more than 2 GB.

Note: A column value can be empty. The key columns or a combination of values for each key must be unique.

- 6 Click **Import**. The application performs an upsert operation. If keys from the CSV file match existing keys, the data is updated. If keys do not match, the data is appended to the list. The imported data is added to the **List Contents** tab.

TIP Importing a CSV file that includes a large data set can take a while to complete. You can check the status of a long-running job on the **Jobs** tab for an advanced list. A  indicates the job is running. A  indicates the job completed successfully. A  indicates the job failed due to an error. When a job fails, a log is added to the details for the job to assist with troubleshooting the error. Click  for that job to open the log.

Modify List Contents

For more information about modifying list contents, see “[List Contents Modifications](#)” on page 276.

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens and displays the **List Contents** tab.
- 3 Modify the list's contents. You can select one or more rows of data to modify at one time or you can modify a specific value in an advanced list table.

To modify one or more rows of data at one time:

- a Select one or more rows of data that you want to modify and click .

TIP To select a block of rows, click the first row in the block, press Shift, and then click the last row that you want to select.

- b In the Edit List Item window, modify the existing values. You cannot modify the status and key-column values. You also cannot modify masked values when **Apply redactions** is on.

If you are editing multiple rows of data at once, the values in the first column for each selected row are listed in the left side of the Edit List Items window. Click a value to display the data for that row in the right side of the window.

To modify a specific value, double-click the value and modify it, and then press Enter. The value that you changed is bold and the row that contains the modified value is highlighted light blue. An asterisk (*) is displayed next to the list name to indicate that the list is being modified. You cannot modify the status and key-column values. You also cannot modify masked values when **Apply redactions** is on.

TIP If you modify several individual values, you can toggle **Show only unsaved changes** to easily review the modified values before you save them.

- 4 Click **Save** to update the list items.

Duplicate a Row of List Data

If you are adding repetitive data, the duplicate-row feature combines the abilities to add a row of data and then modify the row's values. You can duplicate only one row of data at a time.

To duplicate a row of data and modify the values:

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list to which you want to add data, and click . The list opens and displays the **List Contents** tab.

- 3 Select the row that you want to duplicate, right-click, and select **Duplicate row**.
- 4 (Optional) In the **Duplicate Row** window, change the desired values. In this window, key-column values are not displayed. You have the option to enter different key-column values.

Deploy Specific List Items

For information about the deployed and developing states of list items, see “[List Contents Modifications](#)” on page 276.

Note: Changing the status of specified list items does not change the status of a list. For example, if you deploy items in a list that is developing, the list status remains developing. In this case, the list rows are marked to become deployed when you deploy the list. For information about how to deploy an existing list, see “[Deploy an Existing List](#)” on page 282.

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens and displays the **List Contents** tab.
- 3 Deploy list items.
 - To deploy specific rows of data, select the rows that you want to deploy. Click . Select **Deploy selected rows** and then click **Deploy**.
 - To deploy all items in the selected list, select **Deploy all rows** and then click **Deploy**.

Note: Only privileged users can deploy list items that contain masked data. To do so, first toggle off **Apply redactions**. Click . Select **Deploy selected rows** or **Deploy all rows** and then click **Deploy**.

The  is added to the Status column on the **List Contents** tab to indicate which list items are deployed. The  indicates which list items are developing.

TIP Changing the status of a large data set can take a while to complete. You can check the status of a long-running job on the **Jobs** tab for an advanced list. A  indicates the job is running. A  indicates the job completed successfully. A  indicates the job failed due to an error. When a job fails, a log is added to the details for the job to assist with troubleshooting the error. Click  for that job to open the log.

Move Deployed List Items to Developing

For information about the deployed and developing states of list items, see [“List Contents Modifications” on page 276](#). For information about how to move a deployed list to developing, see [“Move a Deployed List to Developing” on page 283](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens and displays the **List Contents** tab.
- 3 Move list items to developing.
 - To move specific rows of data to developing, select the desired rows, click , select **Move selected rows to developing**, and then click **Move**.
 - To move all deployed items in the selected list to developing, click , select **Move all rows to developing**, and then click **Move**.

Note: Only privileged users can move list items that contain masked data to developing. To do so, first toggle off **Apply redactions**. Click . Select **Move selected rows to developing** or **Move all rows to developing** and then click **Deploy**.

The  is replaced with the  in the Status column of a list row on the **List Contents** tab, which indicates that a list item is developing.

TIP Changing the status of a large data set can take a while to complete. You can check the status of a long-running job on the **Jobs** tab for an advanced list. A  indicates the job is running. A  indicates the job completed successfully. A  indicates the job failed due to an error. When a job fails, a log is added to the details for the job to assist with troubleshooting the error. Click  for that job to open the log.

Add an Expiry to List Items

You can set an expiry for specific list items. This option is enabled only when you create a list. (See [“Create a List” on page 279](#).) For more information about expiry for list items, see [“List Creation” on page 274](#).

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list that contains the items to which you want to add an expiry and click . The list opens and displays the **List Contents** tab.

TIP If you open a list and an Expiry column is included on the **List Contents** tab, you are able to set an expiry for items in that list.

- 3 Deploy one or more list items as described in “[Deploy Specific List Items](#)” on [page 290](#).

Note: If you want to deploy and set an expiry for all list items, go to [Step 4b](#) on [page 292](#).

- 4 Set an expiry for one or more deployed list items.
 - a To set an expiry for specific rows of data, select the desired rows, click , and select **Customize row expiry**. In the Customize Row Expiry window, use the picker to select the expiry date and time, and click **OK**.
 - b To set an expiry for all rows of data, click , and select **Deploy all rows**. In the Deploy All List Items window, select **Customize expiry time with deploy** and then click **Deploy**. In the Customize Row Expiry window, use the picker to select the expiry date and time, and click **OK**.

IMPORTANT If you have list items that are deployed with an expiry and then set an expiry for all list items, the expiry for the previously deployed item is not overwritten.

The date and time are added in the Expiry column for the selected rows.

Note: When a list item reaches its expiry, the item is archived and the  is added to the row's Status column on the **List Contents** tab.

Modify the Expiry for List Items

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens and displays the **List Contents** tab.
- 3 Select the rows with an expiry that you want to modify.
- 4 Click , and select **Customize row expiry**.
- 5 In the Customize Row Expiry window, use the picker to select the expiry date and time, and click **OK**.

The new expiry date and time is added to the Expiry column for the selected rows.

Remove an Expiry from List Items

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens and displays the **List Contents** tab.
- 3 Select the rows with an expiry that you want to remove.
- 4 Click , and select **Remove row expiry**.

The expiry date and time is removed from the Expiry column for the selected rows.

Refresh List Contents

A good practice is to periodically refresh the data that is displayed in a list's details view. Doing so ensures that you see changes that were made by other users of the list since you last loaded or refreshed the list. In a list's details view, click  on the **List Contents** tab to update the list's contents. The changes made to the database since you last loaded or refreshed the page are visible.

Remove List Contents

When you remove the contents of a list, you remove list items but not the list structure.

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Select the list that includes the contents that you want to remove.
- 3 Click . The list opens and displays the **List Contents** tab.
- 4 Remove the list's contents.
 - To remove specific rows of list data, select one or more rows. Click **Clear** and select **Remove selected items**. Click **Remove** to verify the action. The **List Contents** tab refreshes and the selected items are removed.
 - To remove all of the list's contents, click **Clear** and select **Clear list contents**. Click **Clear** to verify the action. The **List Contents** tab refreshes and all contents are removed.

TIP In the Advanced Lists category view, you can also right-click the name of a selected list, select **Clear list contents**, and click **Clear** to verify the action.

TIP Removing an entire list's contents can take a while to complete. You can check the status of a long-running job on the **Jobs** tab for an advanced list. A  indicates the job is running. A  indicates the job completed successfully. A  indicates the job failed due to an error. When a job fails, a log is added to the details for the job to assist with troubleshooting the error. Click  for that job to open the log.

Export List Contents

IMPORTANT You can export list contents only if you are the owner of a list or an administrator for the application. Masked data that is redacted in the user interface is not masked in the exported file.

- 1 Click  to navigate to the Advanced Lists category view.
- 2 Hover over the list that you want to open and click the list name. The list opens in the list details view and displays the **List Contents** tab.
- 3 Click **Export**. The file downloads to your browser's default download location. From there, you can open the file or save it to a specific location
- 4 Click **x** to close the list and return to the Advanced Lists category view.

Managing List Revisions

For more information about advanced list revisions, see [“List Revisions” on page 275](#).

Create a List Revision

- 1 Click  to navigate to the Advanced Lists category view.

- 2 Select the list for which you want to create a new revision. The list opens and displays the **List Contents** tab.
- 3 On the **Versions** tab, select the version from which to create the new revision.
- 4 Click **New version**.

The next incremental revision is added on the **Versions** tab.

Note: The contents that are displayed on the list's **List Contents** and **Properties** tabs are from the version that is selected in the Displayed Version column on the **Versions** tab. For information about how to view the contents of a different revision, see [“View the Contents of a List Revision” on page 295](#).

View the Contents of a List Revision

To view the contents of a specific list revision, on the list's **Versions** tab, click the Displayed Version column in the row of the version that you want to view. A ✓ is added to the Displayed Version column. Select the **List Contents** or **Properties** tab to see the contents for the list revision.

Note: The contents that are displayed on the **List Contents** tab might not be deployed. For more information about deploying a revision, see [“Set the Deployed Version” on page 296](#).

Compare the Contents of Revisions

On the list's **Versions** tab, select the versions that you want to compare. Click ⋮, and select **Compare object contents**. In the comparison view, the key columns are displayed to the left. Changed values are highlighted.

TIP To see only the differences between versions, toggle on **Show differences only**.

Note: There is no masked view of a comparison. Only privileged users can see a comparison of list versions that include masked data.

To close the comparison view, click ✕.

Modify a List Revision

You can make changes only to the list definition and contents of the most recent revision. For information about how to modify the revision's properties and columns, see [“Managing List Definition” on page 286](#). For information about how to modify the revision's contents, see [“Managing List Contents” on page 287](#).

Set the Deployed Version

The most recent version of a list is always deployed unless you specify to deploy a different version. To change the deployed version, on the list's **Versions** tab, select the version that you want to deploy, and click **Deploy version**. The Deployed column is added on the **Versions** tab. A  in the Deployed column indicates the revision that is deployed. If no version is deployed, the Deployed column is not visible on the **Versions** tab.

Format of a CSV Data File

A Character Separated Values (CSV) file is a data file that provides either the structure and source data for an advanced list or imported data that maps to an existing list definition.

The CSV data file must contain a header record as the first row, and the header record must contain a name for each column. If a CSV file does not have a header row, the values in the first row of data in the file are treated as the header row.

Column values can be empty.

The character delimiter that is used in the CSV data file must be defined in the listData service configuration. The character delimiter is typically a comma. Newline characters (for example, \r) cannot be used as a delimiter.

The following data types require specific formats:

- If column name restrictions are enabled in your environment, column names must conform to DS2 variable name conventions when you create or modify a list. For more information about column name restrictions, see [“Column Name Restrictions” on page 278](#).
- Currency must be a numeric value. The application truncates values at four decimal places. You do not receive an error if you enter a value with more than four decimal places.

- Currency_code must be a valid 3-character numeric code based on the [ISO 4217 Currency codes](#) standard.
- Datetime must be in the format YYYY-MM-DD HH:MM:SS (for example, 2023-03-22 10:43:00) based on the [ISO 8601 Date and time format](#) standard. The space between DD and HH is required in the format

Here is an example of a CSV file:

```
employeeId,firstName,lastName,email,phoneNumber,hireDate,jobId,salary,commissionPct,managerId,departmentId
100,Steven,King,SKING,515-123-4567,17-JUN-03,AD_PRES,24000,0,0,90
101,Neena,Kochhar,NKOCHHAR,515-123-4568,21-SEP-05,AD_VP,17000,0,100,90
102,Lex,De Haan,LDEHAAN,515-123-4569,13-JAN-01,AD_VP,17000,0,100,90
```

Note: The header line in this example is shown on two lines for presentation only. The header line in your CSV file should all be on one line.

Working with Segmentation Trees

<i>About Segmentation Trees</i>	301
<i>About Outcomes</i>	301
<i>Branch Element Types in a Segmentation Tree</i>	302
<i>Example: A Segmentation Tree</i>	303
<i>Create a Segmentation Tree</i>	305
<i>Controlling the Tree Display</i>	306
<i>Managing the Variables in a Segmentation Tree</i>	307
About Variables	307
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object ...	310
Create Custom Variables	311
Associating a Value List with a Variable	312
Duplicate a Variable	313
Importing and Exporting Variables	313
Delete Variables	314
Edit Variable Properties	314
Edit Metadata for Data Grid Variables	315
Determine Which Objects Use a Particular Variable	315
<i>Using Local Value Lists</i>	316
What is a value list?	316
Value Lists and Local Value Lists	316
Create a Local Value List	316
Manually Add New Entries to a Local Value List	317
Import Entries Into a Local Value List	318
Edit Entries in a Local Value List	319
Delete Entries from a Local Value List	319
Delete a Local Value List	320
<i>Creating and Managing Objects and Branches in a Tree</i>	320
Create a New Boolean Expression	320
Create a New Matrix	321
Define a New Variable	323
Add a Node to an Empty Tree	324
Add a Node to the End of an Existing Branch	325

Insert a New Node or Replace an Existing Node	326
Copy and Paste a Branch Path	328
Managing the Value Lists Associated with Variable Branches	329
Edit or Rename a Boolean Expression	332
Edit a Matrix	332
Rename a Matrix	333
Remove a Branch	334
Delete a Branch Object from a Tree	334
Creating and Managing Outcomes	334
Create a New Outcome	334
Edit an Existing Outcome	335
Add an Assignment Statement to an Outcome	336
Import and Add a Code File to an Outcome	337
Add an Existing Outcome Code File to an Outcome	337
Duplicate an Outcome	338
Add or Replace an Outcome in The Tree	338
Remove an Outcome from a Tree Branch	339
Replace or Delete All Occurrences of an Outcome	339
Importing and Managing Outcome Code Files	340
About DS2 Code Files and Outcome Code Files	340
Import a New Outcome Code File	340
Update Outcome Code File Versions	341
Edit an Outcome Code File	342
Reimport or Replace an Outcome Code File	342
Edit the Original DS2 Code File	343
Duplicate an Outcome Code File	344
Delete an Outcome Code File	344
Search for Objects in a Segmentation Tree Diagram	344
Compare Segmentation Tree Code	345
Managing Segmentation Trees	345
Duplicating Segmentation Trees	345
Delete Segmentation Trees	347
Rename Segmentation Trees	347
Move Segmentation Trees to a Different Folder	347
Managing Versions of Segmentation Trees	348
Set the Displayed Version	348
Create a New Version	348
Copy the Content of a Version	349
Delete a Version	350
Upgrade Decisions to Use a New Version of a Segmentation Tree	351
Determine Which Objects Use a Segmentation Tree	351
Checking Out and Committing Segmentation Tree Versions	352
About Checking Out and Committing Versions	352
Checking Out a Segmentation Tree from within a Decision	353
Concurrently Checking Out and Committing Segmentation Tree Versions	353
Check Out and Commit a Segmentation Tree Version	354
Determine Who Has a Version Checked Out	355
Opening the Original Object	355
Undoing a Check Out	355
Testing Segmentation Trees	356

Ways to Test a Segmentation Tree	356
Test a Segmentation Tree	356
Test a Scenario	359
Working with Test and Validation Test Output Data	363
Status Icons for Tests	364
Duplicate Tests	365
<i>Manage Comments for a Segmentation Tree Test</i>	365

About Segmentation Trees

A segmentation tree is a hierarchical structure that consists of a root node, branches, and final leaf nodes called outcomes. Any branch can lead to additional branches or to an outcome. Each branch in the tree specifies criteria that determine whether that particular branch is executed. The criteria might be individual variable values, combinations of variable values, or Boolean expressions. Outcomes define actions to be taken when all of the branch criteria leading to the outcome are satisfied.

Segmentation trees can be easier to design than using multiple rule sets or other objects to define the same strategy. Segmentation trees enable you to develop a decision model that uses simple conditions and to nest those conditions several levels deep. A segmentation tree starts with a single node and branches out from that node. In contrast, rule sets and models tend to contain more complicated logic, and it is not as easy to branch between them. SAS Intelligent Decisioning provides a low-code interface for designing segmentation trees, and it displays the segmentation tree in a simplified, compact format.

For a large, complicated decision that uses simple conditions, a segmentation tree can be an effective way to visually represent the decision's model and to manage the associated outcomes.

About Outcomes

An outcome is the last leaf in a segmentation tree branch. An outcome is a named set of actions to be taken when all of the branch conditions leading to the outcome are satisfied. An outcome can be one or more assignment statements, one or more [DS2 code files](#), or both. You can reuse the same outcome in multiple places in a tree. You can customize each instance of the same outcome as needed.

Branch Element Types in a Segmentation Tree

There are three types of elements that you can use to define the branches of a segmentation tree. When you add one of these elements to a segmentation tree, SAS Intelligent Decisioning creates branches for each of the possible values or result strings that can be produced by the element. For each value or result string, you can either add an outcome to be executed, or you can add additional branch elements to the tree.

Boolean expression

specifies an expression that must evaluate to True or False. When you add a Boolean expression to a segmentation tree, SAS Intelligent Decisioning adds two outgoing paths: one for True and one for False.

matrix

is a table in which each cell identifies a different combination of the values of two different variables. Both of the variables that are used to build the matrix must be associated with a [value list](#). The values in the lists are used to determine the rows and columns in the matrix. SAS Intelligent Decisioning also adds a row and a column labeled **other**.

For each cell in the table, you specify a label called a result string. SAS Intelligent Decisioning automatically defines the result string **Other**. When you add the matrix to a segmentation tree, SAS Intelligent Decisioning creates a branch for each unique result string, including **Other**.

For each branch in the segmentation tree, you can specify an outcome to be executed or add another object that creates additional branches.

Matrices are particularly useful when each variable has a large number of possible values.

TIP A result string is a label that identifies a specific combination of two variable values in a matrix. An outcome is a named set of actions to be taken in a segmentation tree branch. [Figure 11.2 on page 304](#) shows how these two elements appear in a segmentation tree diagram.

variable

creates a branch for each value in the value list or local value list that is associated with the variable. All variables that are used to create branches in a segmentation tree must be associated with a value list or a local value list. SAS Intelligent Decisioning also adds a branch labeled **Other** for any values that are not included in the list that is associated with the variable.

For each True or False path, each result string in a matrix, and each value of a variable, you can either add an outcome to be executed, or you can add additional branch elements to the tree.

Example: A Segmentation Tree

The segmentation tree shown in the following two figures processes a loan application for which a mortgaged property is used as collateral. The segmentation tree determines whether the amount owed on the property is more than the current value of the property. If the amount owed is greatest, it rejects the application. If it is not, the tree determines whether the applicant has any derogatory credit reports. If there are no such credit reports, or if the applicant has only one derogatory report but it has been resolved, the tree assigns an interest rate for the loan based on the applicant's credit score, income, and potentially other factors. Otherwise, the segmentation tree rejects the loan application.

You can view the segmentation tree in a graphical view or a tabular view. [Figure 11.1](#) shows a graphical view. [Figure 11.2 on page 304](#) shows the tabular view of the same tree. For information about switching between the views and customizing the view, see [“Controlling the Tree Display” on page 306](#).

Figure 11.1 Example Segmentation Tree (Graphical View)

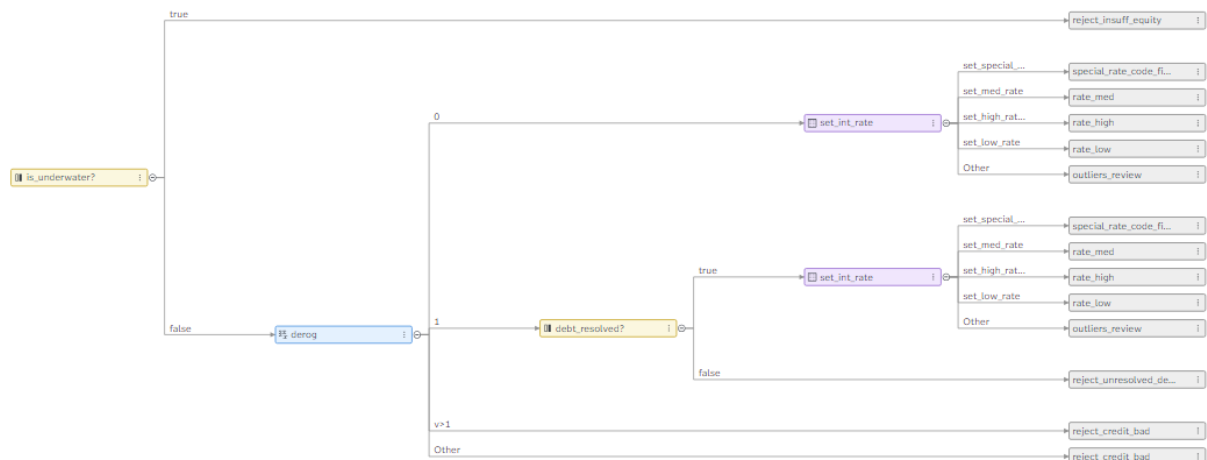


Figure 11.2 Example Segmentation Tree (Table View)

Name	Leaf ID	Outcome
Boolean expression → v is_underwater? ⓘ		
true	Leaf 1	reject_insuff_equity
v false		
Variable → v derog ⓘ		
v 0		
Matrix → v set_int_rate ⓘ		
set_special_rate	Leaf 2	special_rate_code_file
set_med_rate	Leaf 3	rate_med
Result strings → set_high_rate	Leaf 4	rate_high
set_low_rate	Leaf 5	rate_low
Other	Leaf 6	outliers_review
v 1		
v debt_resolv... ⓘ		
v true		

Note: The example matrix in the table view includes additional branches and outcomes that are not shown in [Figure 11.2 on page 304](#).

This segmentation tree contains the following branch elements:

Boolean expression **is_underwater?**

This Boolean expression determines whether the amount owed on the current mortgage is more than the current value of the collateral property. The expression is `mortgage_due > current value`. When you add a Boolean expression to a segmentation tree, SAS Intelligent Decisioning adds True and False branches to the tree. In this example, if this Boolean expression evaluates to True, the outcome named **reject_insuff_equity** is executed. This outcome consists of the following two assignment statements:

```
reject=true
reason = 'insufficient equity'
```

variable branch **derog**

The DEROG variable is the number of derogatory incidents that appear on the applicant's credit report. This variable is associated with a value list that contains the following entries: **0**, **1**, and **v>1**. When you add a variable to a segmentation tree, SAS Intelligent Decisioning creates a branch for each value in the variable's value list. In this example, branch **0** sends execution to the **set_int_rate** matrix. Branch **1** sends execution to the **debt_resolved?** Boolean expression. The tree also contains a branch for the value **v>1** (not shown in the figure).

matrix set_int_rate

The matrix in this example determines a suitable interest rate for the loan. The matrix is built from the values in the value lists that are associated with the two variables `cscore` and `income`. The matrix is shown in [Figure 11.3 on page 305](#). Each cell in the matrix represents a combination of two variable values. Each cell is associated with a result string. A result string is a name that you associate with one or more cells in the matrix. When you add a matrix to a segmentation tree, SAS Intelligent Decisioning adds a branch to the tree for each unique result string. You can then associate an outcome or an additional object with each result string. [Figure 11.2 on page 304](#) illustrates the difference between a result string and an outcome.

In this example, each result string is associated with an outcome. For example, if the applicant has a high income and a fair credit score, the segmentation tree executes the outcome that is associated with the `set_med_rate` result string, which is `med_rate`. This outcome consists of the following two statements:

```
reject=false
interest_rate = 6.3
```

Boolean expression debt_resolved?

This Boolean expression determines whether the debt that appears in the derogatory credit report has been resolved. The expression is `debt_balance = 0`. If this expression evaluates to True, execution moves to the **set_int_rate** matrix. In this example, the same outcomes are assigned to the same branches for both instances of the matrix, but you can use different outcomes each time you reuse a branch element.

Figure 11.3 Example Segmentation Tree Matrix

income	cscore				
	poor	fair	good	excellent	Other
low	set_special... ▾	set_special... ▾	set_med_rate ▾	set_med_rate ▾	Other ▾
medium	set_high_rate ▾	set_med_ra... ▾	set_med_rate ▾	set_low_rate ▾	Other ▾
high	set_high_rate ▾	set_med_ra... ▾	set_low_rate ▾	set_low_rate ▾	Other ▾
Other	Other ▾	Other ▾	Other ▾	Other ▾	Other ▾

Create a Segmentation Tree

- 1 Click  to navigate to the Segmentation trees category.
- 2 Click **New segmentation tree**. The New Segmentation Tree window appears.
- 3 (Optional) Enter a name for the segmentation tree if you do not want to use the default name. Names are limited to 100 characters and must be unique within a folder.

- 4 (Optional) Enter a description for the new segmentation tree. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 5 Click , and select the folder where you want to save the segmentation tree.
- 6 Click **Save**. SAS Intelligent Decisioning opens the new segmentation tree and displays the **Variables** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 7 If the segmentation tree is in a folder for which the checkout and commit feature is enabled, click the **Versions** tab, and check out the latest version of the tree. For more information, see [“Managing Versions of Segmentation Trees” on page 348](#).
- 8 Add variables, branch elements, and outcomes to the segmentation tree. For more information, see the following topics:
 - [“Managing the Variables in a Segmentation Tree” on page 307](#)
 - [“Creating and Managing Objects and Branches in a Tree” on page 320](#)
 - [“Creating and Managing Outcomes” on page 334](#)
- 9 Click **Save**.

Controlling the Tree Display

You can view the segmentation tree in a graphical view or a tabular view. [“Example: A Segmentation Tree” on page 303](#) shows an example of each type of view.

To switch to the tabular view of the segmentation tree, click . To switch to the graphical view of the tree, click .

You can customize the graphical view as follows:

- Click  to open or close the diagram overview. The diagram overview is a scaled-down version of the entire diagram. You can pan across the diagram and position the view over different sections of the diagram. The section that is currently visible on the screen is outlined. The overview is useful when a decision diagram is too large to display all of the nodes on one screen.

- You can click and drag the diagram to change the portion of the diagram that is visible. Depending on how your mouse is configured, you can use the wheel to zoom or scroll the diagram.
- Click  to magnify the diagram. Click  to reduce the size of the diagram. Click  to scale the diagram to fit your browser window.
- For a specific node in the tree, click  or  to collapse or expand the subtree for the node.
- To highlight a path from an outcome back to the root node of the tree, select the outcome and click . The path back to the root node is highlighted for all outcomes with the same name as the selected outcome except for outcomes that were edited after they were added to the tree.

In the tabular view, click  and  to expand and collapse the display for a single branch.

In both the tabular view and the graphical view, click  or  to collapse or expand the branch nodes in a decision.

Managing the Variables in a Segmentation Tree

About Variables

The Properties of a Variable

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a segmentation tree. Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See “Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.

Property	Description
Data type	SAS Intelligent Decisioning supports the following data types: Boolean, character, data grid, date, datetime, decimal, and integer. A variable that is used to create a matrix or variable branch in a segmentation tree must be a decimal, integer, or character variable. Integer, decimal, date, and datetime variables are converted to doubles when generated code is run.
Input and Output	A variable can be an input variable, an output variable, both, or neither (a temporary variable). See “Input Variables, Output Variables, and Temporary Variables” on page 309 for more information. Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.
Length	The length for Boolean and numeric variable types is set automatically. Note: For scenario tests, the <code>variable.length.honorOutputLengthInMAS</code> and <code>variableLengthOverridden</code> configuration options are ignored. Scenario tests always use the length that you specify. In objects that are published to SAS Micro Analytic Service destinations or to container destinations, the maximum length for character variables (outside of a data grid) and for data grid variables is 50 Mb. For other destination types, the maximum length for these variables is 10 Mb. The maximum length for character variables within a data grid is 32767 characters. For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	You can specify an initial value only for variables of type character, integer, decimal, or boolean. Initial values are used only for output-only variables. Initial values are used only when you test a segmentation tree in SAS Intelligent Decisioning. When you publish a segmentation tree as part of a decision, the initial value that is specified for the corresponding decision variable is used, and the initial value that is specified in the segmentation tree is ignored. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. When you are entering

Property	Description
	expressions in the expression editor, specify 1 for True and 0 for False. To use the number sign (#) in literal values, specify the percent-encoded value %23. For content that will be published to a SAS Micro Analytic Service destination, the values of date and datetime input variables must be numeric. They cannot be formatted values such as 18jul2019. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.
Associated value list	A value list is the list of acceptable values for a variable. A variable that is used to create a matrix or variable node in a segmentation tree must be associated with a value list. The value list can be a global value list or a local value list .

Input Variables, Output Variables, and Temporary Variables

For each variable, you must specify whether the variable is an input variable, an output variable, both an input variable and an output variable, or a temporary variable.

- Input variables for segmentation trees receive their value from the decision variable to which the segmentation tree variable is mapped. When a decision that uses a segmentation tree is deployed in a production system, all of the decision's input variables must be mapped to table columns in input data. When you test a segmentation tree in SAS Intelligent Decisioning, for each input variable, you can either map it to a table column or specify a constant as its input value.

When you create or edit a segmentation tree variable, clear the **Input** check box for any variable that does not receive its value from a decision variable.

- Output variables are variables that are written to the output table that is created when a decision that uses the segmentation tree is run. When you create or edit a variable, clear the **Output** check box for any variable that you want to exclude from the output data.

Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.

- Temporary variables are variables that are not present in the input data, and they are not written to the output table. To create a temporary variable for use only while a segmentation tree is executing, clear both the **Input** and **Output** check boxes.

Note: For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.

When you create a new variable, it is created as an input-output variable by default.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name : "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name : "HMEQ". [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➔**. To import all of the variables in the table, click **➔»**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See [“Input Variables, Output Variables, and Temporary Variables” on page 309](#) for more information.

Create Custom Variables

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click **Add variable** and select **Custom variable**. Alternatively, click **Variables** in the **Tree Elements** pane, then click **+**. The Add Variables window appears.
- 2 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 307](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Associated value list**, and **Length** fields.
 - c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable’s data type. See [“The Properties of a Variable” on page 307](#) for additional information.

Note: A variable that is used to create a matrix or variable node in a segmentation tree must be associated with a value list.

- d (Optional) Associate a value list with the variable.
 - 1 Click  for the **Associated value list** field.
 - 2 In the Select or Create a Value List window, select an option for choosing or creating a value list:
 - Select **Select a value list** to select a value list that has been defined in the Value Lists category view. In the Select a Value List window, select the list, and click **OK**. In the Choose Value List Version window, select the version of the list that you want to use, and click **OK**.
 - Select **Select a local value list** to select a value list that has been defined on the **Value list** tab in the segmentation tree. In the Select a Value List window, select the list, and click **Select**.
 - Select **Create a local value list** to create a new local value list. The Add a Local Value List window appears. To create the list, complete [Step 3](#) through [Step 8](#) of [“Create a New Value List” on page 254](#).

- e Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- f Verify that the **Input** and **Output** check boxes are selected correctly for each variable.
 - Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.
 - Clear the **Output** check box for any variable that you want to exclude from the output data.

.....

Note: Any variable that is assigned a value in an outcome must be an output variable.

.....

 - Clear both the **Input** and **Output** check boxes to create a temporary variable.
- 3 (Optional) Modify the variable properties in the table of variables.
- 4 Click **OK** to add the variables and close the Add Variables window.

Associating a Value List with a Variable

To associate a value list that was defined in the Value Lists category or a local value list with a variable that is used to create a variable branch or a matrix in a segmentation tree, select the list in the **Associated value list** field when you create or edit the variable on the **Variables** tab. For more information, see Step 2d of [“Create Custom Variables” on page 311](#).

TIP To disassociate a value list from a variable, click . This icon appears in the Add Variables and Edit Variable windows only if a value list is associated with the variable.

The list that you associate with a variable's definition is considered the variable's default value list. In a segmentation tree, you can change the value list that is associated with a variable in any variable branch or matrix in the tree. For more information, see the following topics:

- [“Change the Value List for a Variable Branch” on page 331](#)
- [“Edit a Matrix” on page 332](#)

Duplicate a Variable

- 1 On the **Variables** tab, select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 2 Enter a new name for the duplicate variable.
- 3 (Optional) Enter a description for the variable.
- 4 Click **Duplicate**.

Importing and Exporting Variables

Note: Data grid variables and reference variables that are created from those data grid variables are exported and imported together. If you choose to export or import one of them, the other is also exported or imported.

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, “Import File Formats,” on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the segmentation tree into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.
- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

TIP To add new columns to an existing data grid variable, select **Replace all variables**.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Note: If a variable is associated with a value list a value list with the same name and version exists in the environment, then the variable is re-associated with the value list. If the value list does not exist, or if a different version of the value list exists, then the variable is imported, but the association with the value list is removed.

Export Variables

Note: If you make changes to the variables, you must save the segmentation tree before exporting variable data.

When a variable is associated with a value list that is defined in the Value Lists category, a reference to the value list is included in the exported data. When a variable is associated with a local value list, the reference to the local value list is not included in the exported data.

- 1 Open the segmentation tree from which you want to export variables.
- 2 On the **Variables** tab, select the variables that you want to export.
- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

Note: You cannot delete a variable if it is used in the current version of an object. You can delete variables from the current version if they are used only in locked versions, but those variables are still included in the locked versions.

On the **Variables** tab, select the check box for the variables that you want to delete, click , and select **Delete**.

Edit Variable Properties

On the **Variables** tab, click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

When you rename a variable, references to that variable within the same object change to use the new name. You cannot change the name or data type of a variable that is used in locked versions.

For information about editing or changing value lists that are associated with an existing variable branch in the tree diagram, see [“About Value Lists and Variable Branches” on page 329](#) and [“Change the Value List for a Variable Branch” on page 331](#).

See [“The Properties of a Variable” on page 307](#) for additional information.

Edit Metadata for Data Grid Variables

For information, see [“Editing Data Grid Variable Metadata” in SAS Intelligent Decisioning: Using Data Grids](#).

Determine Which Objects Use a Particular Variable

On the **Variables** tab, select the check box for the variable, click , and select **View used by report**.

In the report, you can use the **Filter** field to filter the list of objects based on the object names. If the selected variable is a data grid variable, and if you are interested only in a specific column within the data grid, you can select the column in the **Column name** field, and then click **Apply**. SAS Intelligent Decisioning narrows the search results to only the objects that use the data grid with the selected column.

Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Using Local Value Lists

What is a value list?

A value list is the list of acceptable values for a variable. A value list can be a numeric list or a character list. The entries in a numeric value list can be discrete values or ranges. Value lists are required for variables that are used to create variable branches or matrices in segmentation trees. For a complete description of using value lists, see [“About Value Lists” on page 252](#).

Value Lists and Local Value Lists

There are two types of value lists:

- Value lists that are defined in the Value Lists category are value lists that can be used in any segmentation tree. These value lists support versioning, and you can enable or disable specific entries in these lists. For more information, see [Chapter 9, “Using Value Lists,” on page 251](#).
- Local value lists can be used only within the segmentation tree in which the value list is defined. You create and manage a local value list from within the segmentation tree where you want to use the list.

In both types of value lists, you specify the individual entries in the same way. For more information, see [“Specifying Value List Entries” on page 255](#).

Create a Local Value List

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab, and then click **+**. The Add a Local Value List window appears.
- 3 Enter a name for the new value list. Value list names are limited to 100 characters. The names are case insensitive and must be unique within the same segmentation tree.
- 4 Select whether you want to copy a value list that was defined in the Value Lists category or create a completely new local value list.

- Select **Copy a value list** to create a local version of a value list that was defined in the Value Lists category view. Continue with [Step 5](#).
 - Select **Create a local value list** to create a new local value list. Continue with [Step 6](#).
- 5 If you selected **Copy a value list** in the previous step:
 - a Click **Next**.
 - b Click , and select the value list that you want to copy, and click **OK**.
 - c In the Choose Value List Version window, select the version of the value list that you want to copy, and click **OK**.
 - d Click **Finish** to create a local copy of the value list.
 - 6 If you selected **Create a local value list** in [Step 4](#):
 - a Select **Character** or **Numeric**, depending on whether the values in the new local list are character or numeric, and click **Next**.
 - b Add entries to the new table either by importing entries from a CSV file, or by adding entries manually.
 - To manually add new entries, complete [Step 5](#) and [Step 6](#) of “[Manually Add New Entries to a Local Value List](#)” on page 317.
 - To add entries by importing them from a CSV file, complete [Step 5](#) through [Step 9 on page 318](#) of “[Import Entries Into a Local Value List](#)” on page 318.
 - c Click **Finish** to create the new local value list.
 - 7 Click **Save** to save the segmentation tree.

Manually Add New Entries to a Local Value List

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab.
- 3 Hover over the value list that you want to edit, and click .
- 4 In the Local Value List window, click **Edit** to open the Edit Local Value List window.
- 5 For each value that you want to add to the list:
 - a Click **+ Add value**. SAS Intelligent Decisioning adds a new blank entry to the list.
 - b Enter the value for the new entry, and press Enter. See “[Specifying Value List Entries](#)” on page 255 for more information and examples.

TIP When you use variables to create a matrix in a segmentation tree, the values in the variables' value lists are used to create rows and columns in the matrix. SAS Intelligent Decisioning automatically includes one column and one row for the value **other** in addition to the values that are included in the value lists.

- c (Optional) Enter a label for the value, and press Enter. Labels are each limited to 512 characters. If you enter a label, the label is displayed in the SAS Intelligent Decisioning interface instead of the value.
- 6 (Optional) Reorder the entries in the list. To move a list entry, select the check box for the entry, and either drag the entry to a new position in the table or click , , , or . When you are selecting values in a value list in the SAS Intelligent Decisioning interface, the items are displayed in the order in which they are defined in the list.
- 7 Click **Save**.

Import Entries Into a Local Value List

Note: The CSV file must contain the columns that are described in [“Format of the CSV Import File for a Value List” on page 259](#).

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab.
- 3 Hover over the value list that you want to edit, and click .
- 4 In the Local Value List window, click **Edit** to open the Edit Local Value List window.
- 5 Click , and select **Import from CSV**.
- 6 In the Import Entries From CSV window, click , and select the CSV file.
- 7 Select or enter the encoding of the CSV file.
- 8 Select **Add entries** to add the imported entries to the end of the existing list, or select **Replace all entries** to replace the existing entries with the imported entries.
- 9 Click **Import** to import the entries.

Edit Entries in a Local Value List

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab.
- 3 Hover over the value list that you want to edit, and click .
- 4 In the Local Value List window, click **Edit** to open the Edit Local Value List window.
- 5 Add, edit, or import entries as needed.
 - To manually edit existing entries, edit the values and labels in the table. For each change, press Enter or click outside of the table to accept the change.
 - To manually add new entries, complete [Step 5](#) and [Step 6](#) of “[Manually Add New Entries to a Local Value List](#)” on page 317.
 - To add or replace entries by importing them from a CSV file, complete [Step 5](#) through [Step 9](#) on page 318 of “[Import Entries Into a Local Value List](#)” on page 318.
 - To delete an entry, complete the steps in “[Delete Entries from a Local Value List](#)” on page 319.
- 6 Click **Save** to save your changes.

Delete Entries from a Local Value List

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab.
- 3 Hover over the value list that you want to edit, and click .
- 4 In the Local Value List window, click **Edit** to open the Edit Local Value List window.
- 5 In the Edit Local Value List window, select the check box for the entries that you want to delete, click , and select **Delete**.
- 6 Click **Save**.

Delete a Local Value List

Note: You cannot delete a local value list that is associated with a variable or used in a matrix.

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.
- 2 Click  to open the **Local Value Lists** tab.
- 3 Hover over the value list that you want to delete, and click .
- 4 In the Local Value List window, click **Delete**.
- 5 Click **Save**.

Creating and Managing Objects and Branches in a Tree

Create a New Boolean Expression

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click  to open the **Boolean Expressions** tab, and then click **+**.
- 3 In the New Boolean Expression window, enter a name for the new expression, and click **OK**. SAS Intelligent Decisioning opens the expression editor.
- 4 Enter the Boolean expression in the expression editor. Boolean expressions can be up to 4000 characters long. For information about the expression editor, see [“Using the Expression Editor” on page 535](#).

Note: To refer to the value of a cell in a data grid, you can use data grid functions such as DATAGRID_GET to retrieve the value of the cell. For more information, see [“Data Grid Functions” in SAS Intelligent Decisioning: Using Data Grids](#). Alternatively, if you have created a data grid reference variable, you can use the reference variable without using data grid functions. (You can create a data grid reference variable [in the matrix wizard](#) or when you [add a data grid variable branch](#) to the tree.)

- 5 Click **Save** to save the expression, close the expression editor, and return to the segmentation tree.
- 6 Click  to save the changes to the segmentation tree.

Create a New Matrix

TIP You can create a new matrix by editing an existing matrix and using the **Save as** option. For more information, see [“Edit a Matrix” on page 332](#).

- 1 Open the segmentation tree, and click the **Segmentation Tree** tab.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 Click  to open the **Matrices** pane, and then click **+**.
- 3 In the Add Matrix window, enter a name for the new matrix, and click **OK**. SAS Intelligent Decisioning opens the new matrix wizard.

TIP Click **Previous** at any point in the wizard to return to a previous step to make changes. After you click **Finish** on the last page of the wizard, you cannot change the contents of the matrix.

- 4 On the **Select variables** page, select the row and column variables for the matrix. The first variable that you select defines the column variable, and its values or labels are listed across the top of the matrix. The second value defines the row variable, and its values or labels are listed on the left side of the matrix.

For both the row variable and the column variable, click  to open the Select Variable window. This window shows the variables in the segmentation tree from which you can create a matrix.

Note: The Select Variable window does not list data grid variables. To process the values of a cell in a data grid from within a matrix, you must first create a data grid reference variable.

- 5 For both the row variable and the column variable, do one of the following:
 - To use a non-data grid variable as the row or column variable, select the variable in the Select Variable window, and click **Select**.
 - To process the values of a cell in a data grid from within a matrix, you must create a data grid reference variable for that cell, and then select the reference variable as the row or column variable for the matrix. To create a data grid reference variable:
 - 1 In the Select Variable window, click **Add** and select **Add a data grid reference variable**.
 - 2 In the Add Data Grid Variable window, select the data grid variable from which you want to create the reference variable, and click **OK**.
 - 3 Enter a name for the reference variable.
 - 4 Select the column and row in the data grid from which you want the reference variable to get its values.
 - 5 Select the value list that you want to associate with the reference variable, and click **OK** to create the reference variable.
 - 6 Select the reference variable as the column or row variable for the matrix, and click **Select**.

Note: The reference variable is also added to the **Variables** tab for the segmentation tree.

- 6 Click **Next**.
- 7 On the **Specify variable values** page, select the value lists that you want to use for each variable. By default, the matrix uses the value list that was associated with each variable when the variable was defined. To use a different value list, click , select the list, select the version of the list, and click **Select**.

TIP The row and column variables for the matrix must both be associated with [value lists](#). The values in the lists are used to determine the rows and columns in the matrix. SAS Intelligent Decisioning adds the value **Other** to the values in the value lists, so the matrix includes one column and one row for the value **Other**.

- 8 Click **Next**.
- 9 On the **Add result strings** page, define the list of result strings that you want to associate with the cells in the matrix. A result string is a name that identifies a combination of two variable values in a matrix (a cell).

- To add a new result string, enter the string, and either press Enter or click **Add**.
- To delete an existing result string, hover the pointer over the result string, and click  at the right of the result string.

TIP The application adds the result string **Other** to the matrix. This result string is the default result string for each cell unless you change the result strings on the **Define results matrix** page.

- 10 Click **Next**.
- 11 On the **Define results matrix** page, select a result string for each cell in the matrix.
- 12 Click **Finish** to save the matrix and return to the segmentation tree diagram.
- 13 Click  to save the changes to the segmentation tree.

Define a New Variable

Any variable that you use to create a variable branch in a segmentation tree must have an associated value list. When you define a new variable, you must select the default value list for the variable. For more information, see [Chapter 9, “Using Value Lists,” on page 251](#).

To display information about the default variable list that is associated with a variable in the segmentation tree, click  to display the **Variables** pane, hover over the variable, and click .

You can define a new variable in a segmentation tree in either of the following ways:

- Add new variables on the **Variables** tab. On the **Variables** tab, you can add variables from data tables and from other SAS Intelligent Decisioning objects, or you can create new custom variables. For more information, see [“Managing the Variables in a Segmentation Tree” on page 307](#).
- Create new custom variables in the **Variables** pane on the **Segmentation Tree** tab.
 - 1 Open the segmentation tree, click  to display the **Variables** pane, and click **+**.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 In the Add Variables window, define the new custom variable, and click **OK**. For information about the fields in this window, see [“Create Custom Variables” on page 311](#).
- 3 Click  to save the changes to the segmentation tree.

TIP You can use the **Select value list** option to change the value list that is associated with a specific variable branch after you add the variable to the segmentation tree. See [“Change the Value List for a Variable Branch” on page 331](#) for more information.

Add a Node to an Empty Tree

- 1 In a new tree that does not have existing nodes, drag an existing Boolean expression, matrix, or variable from the corresponding pane onto the segmentation tree diagram.

If you select a data grid variable, SAS Intelligent Decisioning opens the Add Data Grid Variable window. You must create a data grid reference variable that identifies the column and row in the data grid that contains the values that you want to process. In the Add Data Grid Variable window:

- a Enter a name for the reference variable.
- b Select the column and row in the data grid from which you want the reference variable to get its values.
- c Select the value list that you want to associate with the reference variable, and click **OK**.

SAS Intelligent Decisioning creates the reference variable and adds it to the **Variables** tab for the segmentation tree.

SAS Intelligent Decisioning adds a new node to the tree, depending on the type of object that you add to the tree.

- For a Boolean expression, it adds a new node with branches for True and False.
- For a matrix, it adds a new node with branches for each result string that is defined in the matrix.
- For a variable, it adds a new node with branches for each value in the default value list that is associated with the variable.

Note: The values for each new branch in the tree are static. Changes to a variable's value list, changes to a matrix's result strings, or changes to a Boolean expression do not affect existing branches in the tree.

- 2 Click  to save the changes to the segmentation tree.

Add a Node to the End of an Existing Branch

- 1 Right-click an outcome node, and select , **Add boolean expression**, **Add matrix**, or **Add variable**. Depending on the option that you select, a window appears in which you can select or create the variable, Boolean expression, or matrix that you want to add to the tree.

TIP Alternatively, you can drag a Boolean expression, matrix, or variable onto the diagram and drop it onto an existing node. The Add window described in [Step 3](#) appears.

- 2 Select the existing object that you want to add to the tree. Alternatively, for Boolean expressions and matrices, you can choose to create a new object of the type that you selected in [Step 1](#).

- In the Add a Boolean Expression window, do one of the following:
 - ☐ To add an existing expression, select **Select an expression**, select the expression that you want to add, and click **OK**.
 - ☐ To create a new Boolean expression, select **Create an expression**, enter the expression name, and click **OK**. In the [expression editor](#), enter the new expression, and click **Save**.
- In the Add a Matrix window, do one of the following:
 - ☐ To add an existing matrix, select **Select a matrix**, select the matrix that you want to add, and click **OK**.
 - ☐ To create a new matrix, select **Create a matrix**, enter the matrix name, and click **OK**. SAS Intelligent Decisioning opens the new matrix wizard. To define the new matrix, complete [Step 4](#) through [Step 12](#) in “[Create a New Matrix](#)” on page 321.
- In the Select a Variable window, select the variable that you want to add, and click **OK**.

If you select a data grid variable, SAS Intelligent Decisioning opens the Add Data Grid Variable window. You must create a data grid reference variable that identifies the column and row in the data grid that contains the values that you want to process. In the Add Data Grid Variable window:

- 1 Enter a name for the reference variable.
- 2 Select the column and row in the data grid from which you want the reference variable to get its values.
- 3 Select the value list that you want to associate with the reference variable, and click **OK**.

SAS Intelligent Decisioning creates the reference variable and adds it to the **Variables** tab for the segmentation tree.

If the outcome node that you selected in [Step 1](#) is an empty outcome node, SAS Intelligent Decisioning inserts the new node in front of the empty outcome. Go to [Step 4](#).

If the outcome node that you selected in [Step 1](#) is not empty, SAS Intelligent Decisioning opens the Add window in which you can choose keep or replace the existing outcome node. Continue with [Step 3](#).

- 3 In the Add window, do one of the following:
 - Select **Insert the new item in front of the existing node and map the existing downstream content to the new node**, and click **Add**. In the Select Branch For Insert window, select the new branch to which you want to move the existing outcome node, and click **OK**. SAS Intelligent Decisioning adds additional empty outcome nodes for any additional new branches.

TIP In the Add and Select Branch For Insert windows, click **View change details** to display a table of current and new branches.

- Select **Replace the existing node and delete the existing downstream content**, and click **Add**. SAS Intelligent Decisioning deletes the existing outcome node, adds the new node, and adds empty outcome nodes for each of the new node's branches.

Note: The values for each new branch in the tree are static. Changes to a variable's value list, changes to a matrix's result strings, or changes to a Boolean expression do not affect existing branches in the tree.

- 4 Click  to save the changes to the segmentation tree.

Insert a New Node or Replace an Existing Node

- 1 Right-click the existing node in front of which you want to insert a new node or that you want to replace, and select **Add boolean expression**, **Add matrix**, or **Add variable**. Depending on the option that you select, a window appears in which you can select or create the variable, Boolean expression, or matrix that you want to add to the tree.

TIP Alternatively, you can drag a Boolean expression, matrix, or variable onto the diagram and drop it onto an existing node. The Add window described in [Step 3](#) appears.

- 2 Select the existing object that you want to add to the tree. Alternatively, for Boolean expressions and matrices, you can choose to create a new object of the type that you selected in [Step 1](#).
 - In the Add a Boolean Expression window, do one of the following:

- To add an existing expression, select **Select an expression**, select the expression that you want to add, and click **OK**.
- To create a new Boolean expression, select **Create an expression**, enter the expression name, and click **OK**. In the [expression editor](#), enter the new expression, and click **Save**.
- In the Add a Matrix window, do one of the following:
 - To add an existing matrix, select **Select a matrix**, select the matrix that you want to add, and click **OK**.
 - To create a new matrix, select **Create a matrix**, enter the matrix name, and click **OK**. SAS Intelligent Decisioning opens the new matrix wizard. To define the new matrix, complete [Step 4](#) through [Step 12](#) in “[Create a New Matrix](#)” on page 321.
- In the Select a Variable window, select the variable that you want to add, and click **OK**.

If you select a data grid variable, SAS Intelligent Decisioning opens the Add Data Grid Variable window. You must create a data grid reference variable that identifies the column and row in the data grid that contains the values that you want to process. In the Add Data Grid Variable window:

- 1 Enter a name for the reference variable.
- 2 Select the column and row in the data grid from which you want the reference variable to get its values.
- 3 Select the value list that you want to associate with the reference variable, and click **OK**.

SAS Intelligent Decisioning creates the reference variable and adds it to the **Variables** tab for the segmentation tree.

- 3 In the Add window, do one of the following:
 - Select **Insert the new item in front of the existing node and map the existing downstream content to the new node**, and click **Add**. The Select Branch For Insert window appears. Select the new branch to which you want to move the existing node's downstream content, and click **OK**.
 - Select **Replace the existing node and map the existing downstream content to the new item**, and click **Add**. The Edit Mappings window appears. For each existing path in the **Branch Path** column, select the new branch path in the **Value** column, and click **OK**.
 - Select **Replace the existing node and delete the existing downstream content**, and click **Add**. SAS Intelligent Decisioning deletes the existing node, adds the new node, and adds empty outcome nodes for each of the new node's branches.

TIP In the Add, Select Branch For Insert, and Edit Mappings windows, click **View change details** to display a table of current and new branches. In the Edit Mappings window, click **View** in the **Branch Path** column to display that specific branch in the tree.

Note: The values for each new branch in the tree are static. Changes to a variable's value list, changes to a matrix's result strings, or changes to a Boolean expression do not affect existing branches in the tree.

- 4 Click  to save the changes to the segmentation tree.

Copy and Paste a Branch Path

Note: You cannot paste a copied branch into itself.

- 1 Right-click the first node of the branch that you want to copy, and select **Copy branch path**. SAS Intelligent Decisioning copies the entire branch from the node that you select to the outcome nodes at the end of the branch.
- 2 Right-click the first node of the branch where you want to paste the copied path, and select **Paste branch path**.
- 3 In the Add window, do one of the following:
 - Select **Insert the new item in front of the existing node and map the existing downstream content to the new node**, and click **Add**. The Select Branch For Insert window appears. Select the value of the path in the copied branch to which you want to move the existing node's downstream content, and click **OK**.
 - Select **Replace the existing node and map the existing downstream content to the new item**, and click **Add**. The Edit Mappings window appears. For each existing path listed in the **Branch Path** column, select the new branch path in the **Value** column.
 - Select **Replace the existing node and delete the existing downstream content**, and click **Add**. SAS Intelligent Decisioning replaces the existing node and its downstream content with the copied branch path.

TIP In the Add, Select Branch For Insert, and Edit Mappings windows, click **View change details** to display a table of current and new branches. In the Edit Mappings window, click **View** in the **Branch Path** column to display that specific branch in the tree.

- 4 Click  to save the changes to the segmentation tree.

Managing the Value Lists Associated with Variable Branches

About Value Lists and Variable Branches

Any variable that you use to create a variable branch in a segmentation tree must have an associated value list. When you define a new variable, you must select the default value list for the variable. When you add the variable to the segmentation tree diagram, the values in the default list are used as the values of the variable branch's downstream branches. The values for the its downstream branches are static. However, you can change the value list that is associated with a variable branch after the branch is created.

For additional information, see the following topics:

- [“Edit Values in a Variable List” on page 330](#)
- [“Edit Variable Properties” on page 314](#)
- [Chapter 9, “Using Value Lists,” on page 251](#)
- [“Using Local Value Lists” on page 316](#)

This flexibility in assigning value lists to variables and branches means that you can have multiple variable branches for the same variable in the same tree, but the downstream branches that were created for each of these variable branches might have different values. For example, suppose your segmentation tree defines a variable named myVar that is associated with a value list named myList. There are three versions of the list myList, and each version has different values. Suppose you created a segmentation tree as follows:

- 1 Add the variable myVar to the tree diagram. SAS Intelligent Decisioning uses the values in the variable's default value list to determine the values of the downstream branches.
- 2 Change the version of the myList value list that is used as the default value list for the myVar variable, and add the variable to the diagram again. SAS Intelligent Decisioning creates a second variable branch for the same variable, but the downstream branches for the second branch use the values in the new version of the myList value list.
- 3 Add the same variable to the tree a third time, and use the [Select a value list](#) option or the [Select a local value list](#) option to choose a different value list or to choose the third version of the same value list for the third variable branch.

The resulting tree would have three different branches for the same variable, but each set of downstream branches uses different values.

On the **Segmentation Tree** tab, you can view value list information for a variable or for a tree branch as follows:

- To display list information for a variable, click  to open the **Variables** pane, hover over the variable name, and click the ⓘ information icon.

If the variable is associated with a value list that was defined in the Value Lists category view, the Default Associated Value List window appears. This window displays the name and version of the variable's default value list and the name and version of the value list that is currently associated with the variable. If you add the variable to the tree, the values in this default list are used as the values of the variable branch's downstream branches.

If the variable is associated with a local value list, the Local Value List window appears. This window displays the name of and values in the local value list. If you add the variable to the tree, the values in this local list are used as the values of the variable branch's downstream branches.

- To display list information for a variable branch in the tree diagram, right-click the variable node, and select **Show information**.

If the tree branch is currently associated with a list that was defined in the Value List category view, SAS Intelligent Decisioning opens the Associated Value List window for the branch. This window displays the name and version of the both the default value list that is associated with the variable and the current value list that is associated with the branch. If the value list that is associated with the branch has been changed by using either the **Select a value list** or the **Select a local value list** option, then these two value lists are different.

If the tree branch is associated with a local value list, SAS Intelligent Decisioning opens the Local Value List window for the variable. This window displays the name of and values in the local value list.

Edit Values in a Variable List

You can edit the values in a value list that was defined in the Value Lists category is associated with a variable by editing the list in the Value Lists category view, or you can edit the value list from within the segmentation tree. For information about the Value Lists category view, see [Chapter 9, "Using Value Lists," on page 251](#).

To edit a value list from within a segmentation tree:

- 1 Click  to display the **Variables** pane if it is not already open.
- 2 Hover over the variable whose value list you want to edit, and click ⓘ.
- 3 In the Default Associated Value List window, click the name of the value list. SAS Intelligent Decisioning opens the value list.
- 4 Add or edit the value list entries. Additions and changes are automatically saved when you press Enter. For more information, see ["Specifying Value List Entries" on page 255](#).

Change the Value List for a Variable Branch

You can change the value list that is associated with a variable branch without changing the variable for the branch.

- 1 Right-click the variable node whose value list you want to replace, and do one of the following:
 - Select **Select a value list** to associate a value list that was defined in the Value Lists category with the variable branch. In the Select a Value List window, select the value list that you want to use for the branch, and click **OK**. In the Choose Value List Version window, choose the version of the value list that you want to use, and click **OK**.
 - Select **Select local value list** to associate a local value list with the variable branch. In the Select a Local Value List window, select the local value list that you want to use for the branch, and click **Select**.
- 2 In the Replace the Value List window, choose whether you want to delete or remap the current variable branch's downstream branches.
 - Select **Delete the existing downstream content**, and click **Replace** to replace the value list and delete the current downstream branches. SAS Intelligent Decisioning replaces the value list and returns to the segmentation tree diagram.
 - Select **Map the existing downstream content to the values in the new value list**, and click **Replace** to replace the value list, keep the existing branch content, and assign that content to the new paths that are created by the replacement value list. The Edit Mappings window appears.
 - 1 For each existing path listed in the **Branch Path** column, select the new branch path in the **Value** column. SAS Intelligent Decisioning assigns the existing downstream content to the replacement branch paths. Any existing content that is not mapped to a replacement path is deleted.

TIP In the Edit Mappings window, click **View** in the **Branch Path** column to display that specific branch in the tree. Click **View change details** to display a table of current and new branches.
- 2 Click **OK** to replace the value list in the tree, update the branch mappings, and return to the segmentation tree diagram.
- 3 Click  to save the changes to the segmentation tree.

Edit or Rename a Boolean Expression

- 1 Click  to open the **Boolean Expressions** pane if it is not already open.
- 2 Hover over the expression that you want to edit or rename, and click .
- 3 In the Expression Information window, click **Edit** to edit the expression, or click  and select **Rename** to rename the expression.

If you click **Edit**, SAS Intelligent Decisioning opens the [expression editor on page 535](#). If you click **Rename**, the Rename window appears.

- 4 Edit the expression or enter a new expression name, and then click **Save**.

Edit a Matrix

If a matrix is used in a tree, then you cannot rename the matrix. You can change the variables and other content in the matrix, but you must save the changed matrix with a new name. If the matrix is not used in the tree, you can edit the content of the matrix and save the changes using the existing name.

- 1 Click  to open the **Matrices** pane if it is not already open.
- 2 Hover over the matrix that you want to edit, and click .
- 3 In the Matrix Information window, click **Edit** to open the Edit Matrix window.
- 4 (Optional) On the **General** page, click  in the **Name** field to rename the matrix. In Rename Matrix window, enter a new name, and click **Save**. The name is limited to 100 characters and must be unique within the folder.
- 5 (Optional) Select the new row or column variables for the matrix. The first variable that you select defines the column variable, and its values or labels are listed across the top of the matrix. The second value defines the row, and its values or labels are listed on the left side of the matrix.

TIP Both variables must be associated with either a [value list that was defined in the Value Lists category view](#), or a [local value list](#). The values in the lists are used to determine the rows and columns in the matrix. SAS Intelligent Decisioning adds the value **other** to the values in the value lists, so the matrix includes one column and one row for the value **other**.

- 6 (Optional) On the **Values** page, select the value lists that you want to use for each variable. By default, the matrix uses the value list that was associated with

each variable when the variable was defined. To use a different value list for a variable:

- a Click  for the variable.
 - b In the Select or Create a Value List window, select an option for choosing or creating a value list:
 - Select **Select a value list** to select a value list that has been defined in the Value Lists category view. In the Select a Value List window, select the list, and click **OK**. In the Choose Value List Version window, select the version of the list that you want to use, and click **OK**.
 - Select **Select a local value list** to select a value list that has been defined on the **Value list** tab in the segmentation tree. In the Select a Value List window, select the list, and click **Select**.
 - Select **Create a local value list** to create a new local value list. The Add a Local Value List window appears. To create the list, complete [Step 3](#) through [Step 8](#) of “Create a New Value List” on page 254.
 - 7 (Optional) On the **Results** page, modify the list of result strings that you want to associate with the cells in the matrix. A result string is a name that identifies a combination of two variable values in a matrix (a cell).
 - To add a new result string, enter the string, and either press Enter or click **Add**.
 - To delete an existing result string, hover the pointer over the result string, and click  at the right of the result string.
- TIP** The application adds the result string **Other** to the matrix. This result string is the default result string for each cell unless you change the result strings on the **Results** page.
- 8 (Optional) On the **Matrix** page, modify the result strings for the cells in the matrix.
 - 9 Click **Save** to save the changes to the existing matrix. Select **Save as** and enter a new matrix name to save the changed matrix as a new matrix.
 - 10 Click  to save the changes to the segmentation tree.

Rename a Matrix

If a matrix is used in a tree, then you cannot rename the matrix. For more information, see [“Edit a Matrix” on page 332](#).

To rename a matrix that is not used in a tree:

- 1 Click  to open the **Matrices** pane if it is not already open.

- 2 Hover over the matrix that you want to edit, and click ⓘ.
- 3 In the Matrix Information window, click ∨ and select **Rename**.
- 4 In Rename Matrix window, enter a new name, and click **Save**. The name is limited to 100 characters and must be unique within the folder.
- 5 Click  to save the changes to the segmentation tree.

Remove a Branch

To remove a tree branch, right-click the branch node and select **Remove**. SAS Intelligent Decisioning removes the node and its branches from the segmentation tree.

Delete a Branch Object from a Tree

Note: If you delete a Boolean expression or a matrix that is used the segmentation tree, SAS Intelligent Decisioning removes any branches that were created with that expression or matrix. You cannot delete a variable that is used in the tree.

To delete a Boolean expression or a matrix:

- 1 Click  or  to open the **Boolean Expressions** pane or the **Matrices** pane if it is not already open.
- 2 Hover over the expression or matrix that you want to delete, click ⓘ and select **Delete**.

For information about deleting variables, see [“Delete Variables” on page 314](#).

Creating and Managing Outcomes

Create a New Outcome

- 1 In the Segmentation trees category view, open the segmentation tree, and do one of the following:
 - Click , and select **Manage outcomes**. In the Manage Outcomes window, click .

- Select an existing empty outcome node, then select **Add** ⇒ **Outcome**, or right-click an existing empty outcome node, and choose **Add outcome**. In the Add an Outcome window, select **Create an outcome**, and click **OK**.

Note: If you select or right-click a non-empty outcome node, the new outcome will replace the existing outcome.

- 2 In the Add Outcome window, enter a name for the outcome. Names are limited to 100 characters and must be unique within a segmentation tree.
- 3 (Optional) Enter a description for the new outcome. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 4 Add assignment statements and outcome code files to the outcome. For instructions, see the following topics:
 - [“Add an Assignment Statement to an Outcome” on page 336](#)
 - [“Import and Add a Code File to an Outcome” on page 337](#)
 - [“Add an Existing Outcome Code File to an Outcome” on page 337](#)
- 5 Click **OK** to create the outcome.
- 6 Click **Save** to save your changes and return to the segmentation tree diagram.
- 7 Click  to save the changes to the segmentation tree.

Edit an Existing Outcome

To modify existing assignment statements:

- 1 Click the outcome to open the **Outcome** pane, and click .

Note: The pane closes if you select anything other than a named outcome.

- 2 Modify the statements as needed, and click **Apply**.

To add, delete, or modify statements or outcome code files:

- 1 Click  and select **Manage outcomes**. The Manage Outcomes window appears.
- 2 Select the outcome that you want to edit. SAS Intelligent Decisioning displays the contents of the outcome.

TIP In the list of outcomes, hover over an outcome, and click ⓘ to display the properties of an outcome.

3 Click . The Edit Outcome window appears.

4 Modify the outcome as needed.

To add statements or code files, follow the instructions in the following topics:

- “Add an Assignment Statement to an Outcome” on page 336
- “Import and Add a Code File to an Outcome” on page 337
- “Add an Existing Outcome Code File to an Outcome” on page 337

To delete a statement or code file, hover the pointer over the item that you want to delete, and click .

5 Click **OK** to return to the Manage Outcomes window.

6 Click **Save** to save your changes and return to the segmentation tree diagram.

7 Click  to save the changes to the segmentation tree.

Add an Assignment Statement to an Outcome

1 [Create a new outcome](#), or [edit an existing outcome](#).

2 In the Add Outcome window or the Edit Outcome window, select **Add** ⇒ **Add assignment**. The application adds an assignment statement to the **Outcome Content** section in the window.

3 In the first field, select the variable to which you want to assign a value. This variable must be an [output variable](#).

4 In the second field, select one of the following:

Value

assigns the literal value that is specified in the last field to the variable that is specified in the first field.

Variable

assigns the value of the variable that is specified in the last field to the variable that is specified in the first field.

5 In the last field, either enter a literal value or select a variable name, depending on which option you selected in [Step 4](#).

Note: You cannot assign an empty character string to a character variable.

For example, the following statements assign the literal value 9.5 to the variable `interest_rate` and assign the value of the `rcode2` variable to the variable `reason`.

interest_rate	▼	=	Value	▼	9.5
reason	▼	=	Variable	▼	rcode2

Import and Add a Code File to an Outcome

In order to add a DS2 file to an outcome, you must import the code file. Importing the file enables segmentation trees to use them.

- 1 [Create a new outcome](#), or [edit an existing outcome](#).
- 2 In the Add Outcome window or the Edit Outcome window, select **Add** ⇒ **Add outcome code file**. The Add Outcome Code File window appears.
- 3 Enter the name that you want to use for the outcome code file, and click **Import**. The Select a Code File window appears.

TIP The outcome code file name is a name for the imported outcome code file. It does not have to match the name of the DS2 code file.

- 4 Navigate to and select the code file that you want to import, and click **OK**.
- 5 In the Choose Code File Version window, select the version number of the code file that you want to import, and click **OK**. SAS Intelligent Decisioning imports the file and displays the code in the Add Outcome Code File window.
- 6 Verify that the displayed code is the code that you want to import, and click **OK**. The application adds the outcome code file to the **Outcome Content** section in the Add Outcome window.

Note: Any changes that are made to the original DS2 code file are not automatically propagated to the imported outcome code file. For more information, see [“About DS2 Code Files and Outcome Code Files” on page 340](#) and [“Replace or Delete All Occurrences of an Outcome” on page 339](#).

- 7 Click **OK** to add the outcome to the segmentation tree.

Add an Existing Outcome Code File to an Outcome

- 1 [Create a new outcome](#), or [edit an existing outcome](#).
- 2 In the Add Outcome window or the Edit Outcome window, select **Add** ⇒ **Select outcome code file**. The Select Code File window appears. This window displays

the list of code files that have already been imported for use as outcome code files.

- 3 Select the outcome code file that you want to add, and select **OK**.

The application adds the outcome code file to the **Outcome Content** section of the Add Outcome window.

- 4 Click **OK** to add the outcome code file to the outcome.

Duplicate an Outcome

- 1 Open the segmentation tree, and click  and select **Manage outcomes**. The Manage Outcomes window appears.
- 2 In the list of outcomes, select the outcome that you want to duplicate. SAS Intelligent Decisioning displays the contents of the outcome.
- 3 Click . The Edit Outcome window appears.
- 4 Click **Duplicate**. The Save As window appears.
- 5 Enter a name for the duplicate outcome file, and click **OK** to return to the Manage Outcomes window.
- 6 Click **Save** to save your changes and return to the segmentation tree diagram.
- 7 Click  to save the changes to the segmentation tree.

Add or Replace an Outcome in The Tree

Note: Adding an outcome to a branch that already has an outcome replaces the existing outcome.

- 1 Open the segmentation tree, right-click the outcome node for which you want to add or replace the outcome, and select **Add outcome**.
- 2 If the outcome node is not empty, the Replace window appears. To replace the existing outcome with a new one, click **Replace**.
- 3 In the Add an Outcome window, do one of the following:
 - To add an existing outcome, select **Select an outcome**, select the outcome that you want to add, and click **OK**.
 - To create a new outcome, select **Create an outcome**, and click **OK**.
- 1 In the Add Outcome window, enter a name for the outcome. Names are limited to 100 characters and must be unique within a segmentation tree.

- 2 (Optional) Enter a description for the new outcome. Descriptions are limited to 1000 characters.
- 3 Add assignment statements and outcome code files to the outcome. For instructions, see the following topics:
 - [“Add an Assignment Statement to an Outcome” on page 336](#)
 - [“Import and Add a Code File to an Outcome” on page 337](#)
 - [“Add an Existing Outcome Code File to an Outcome” on page 337](#)
- 4 Click **OK** to create the outcome.

Remove an Outcome from a Tree Branch

When you remove an outcome from a tree branch, it is removed from only that specific branch. Other branches are not affected, and the outcome remains in the list of outcomes in the Manage Outcomes window.

To remove an outcome from a specific tree branch, right-click the branch and select **Clear outcome**.

Replace or Delete All Occurrences of an Outcome

If the same outcome is used in multiple places in a tree, but you want to replace only a single occurrence of the outcome, add the replacement outcome as described in [“Add or Replace an Outcome in The Tree” on page 338](#). Other occurrences are not affected, and the outcome remains in the list shown in the Manage Outcomes window.

Note: Deleting all occurrences of an outcome from a tree removes the outcome from the list of outcomes that are displayed in the Manage Outcomes window.

To replace or delete all occurrences of an outcome in a segmentation tree:

- 1 Open the segmentation tree, click , and select **Manage outcomes**. The Manage Outcomes window appears.
- 2 Select the outcome that you want to replace or delete, and click  at the top right of the window.
- 3 In the message that is displayed, choose one of the following options:
 - Click **Delete** to delete all occurrences of the outcome from the tree.
 - Click **Replace** to replace all occurrences of the outcome in the tree. The Select an Outcome window appears. Select the replacement outcome, and click **Select**. SAS Intelligent Decisioning replaces all occurrences of the

original outcome, and removes the original outcome from the list of outcomes that are defined for the tree.

- 4 Click **Save** to save your changes and return to the segmentation tree diagram.
- 5 Click  to save the changes to the segmentation tree.

Importing and Managing Outcome Code Files

About DS2 Code Files and Outcome Code Files

You must import a DS2 code file in order to make it available to a segmentation tree as an outcome code file.

Subsequent changes to a previously imported DS2 code file are not propagated to the outcome code file. Changes that you make to the outcome code file are not propagated back to the original DS2 code file. See [“Edit an Outcome Code File” on page 342](#) for additional information.

Import a New Outcome Code File

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Click **+**. The Add Outcome Code File window appears.
- 3 Enter the name that you want to use for the outcome code file, and click **Import**. The Select a Code File window appears.

TIP The outcome code file name is a name for the imported outcome code file. It does not have to match the name of the DS2 code file.

- 4 In the Choose Code File Version window, select the version number of the code file that you want to import, and click **OK**. SAS Intelligent Decisioning imports the file and displays the code in the Add Outcome Code File window.
- 5 Verify that the displayed code is the code that you want to import, and click **OK**.

The application adds the outcome code file to the **Outcome Content** section in the Add Outcome window.

Note: Any changes that are made to the original DS2 code file are not automatically propagated to the imported outcome code file. For more information, see [“About DS2 Code Files and Outcome Code Files”](#) on page 340 and [“Replace or Delete All Occurrences of an Outcome”](#) on page 339.

Update Outcome Code File Versions

TIP In the Manage Outcome Code Files window, a bullet ● appears next to each outcome code file for which a newer version of the original code file is available.

To update the version of a single outcome code file:

- 1 Open the segmentation tree, click , and select **Manage outcome code files**.
- 2 In the Manage Outcome Code Files window, select the code file that you want to update, and click .
- 3 In the Edit Outcome Code File window, click **Update Version**. (This button is disabled if there are no additional versions of the selected code file.)
- 4 In the New Code File Versions window, select the new version that you want to use for this code file, and click **OK** to return to the Edit Outcome Code File window.
- 5 Select **OK** to update the outcome code file version, and then click **Save** to save the change to the segmentation tree.

To update the versions of all of the outcome code files that are used in a segmentation tree:

- 1 Open the segmentation tree, click , and select **Update code file versions**. The Update Versions window appears. This window lists each outcome code file that is used in the segmentation tree for which multiple versions of the code file exist.
- 2 In the Update Versions window, select the check box for each outcome code file that you want to update, and select the new version of the file that you want to use as the outcome code file.
- 3 Select **Update** to update the outcome code files.
- 4 Click **Save** to save the changes to the segmentation tree.

Edit an Outcome Code File

Note: The ability to edit an outcome code file is controlled by the [readOnlyOutcomeCodeFiles](#) configuration option. If this option is turned on, you cannot edit an existing outcome code file. To change the outcome code file, you must reimport the original DS2 code file or import a different DS2 code file. For more information, see [“Reimport or Replace an Outcome Code File” on page 342](#).

TIP When you edit the existing code in an imported outcome code file, the name of the outcome code file remains the same. Only the DS2 code that is associated with the outcome code file changes.

See also [“Update Outcome Code File Versions” on page 341](#).

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Select the outcome code file that you want to edit, and click . SAS Intelligent Decisioning opens the file in the Edit Outcome Code File window.
- 3 Manually edit the code as needed, and click **OK**.
The application replaces the code that is displayed in the Manage Outcome Code Files window.
- 4 Click **Save** to save your changes to the outcome code file and return to the segmentation tree diagram.
- 5 Click  to save the changes to the segmentation tree.

Reimport or Replace an Outcome Code File

TIP You can reimport a previously imported code file, or you can import a different DS2 code file. In both cases, the name of the outcome code file remains the same. Only the DS2 code that is associated with the outcome code file changes.

See also [“Update Outcome Code File Versions” on page 341](#).

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Select the outcome code file that you want to edit, and click . SAS Intelligent Decisioning opens the file in the Edit Outcome Code File window.

- 3 Click **Import**. The Select a Code File window appears.
- 4 Select the DS2 code file that you want to import, and click **OK**. The Choose Code File Version window appears.
- 5 Select the version that you want to import, and click **OK**. SAS Intelligent Decisioning imports the file and displays the code in the Edit Outcome Code File window.
- 6 Verify that the displayed code is the code that you want to import, and click **OK**. The application replaces the code that is displayed in the Manage Outcome Code Files window.
- 7 Click **Save** to save your changes to the outcome code file and return to the segmentation tree diagram.
- 8 Click  to save the changes to the segmentation tree.

Edit the Original DS2 Code File

You can edit the original DS2 code file that was imported for use as an outcome code file.

Note: If a new version of the DS2 code file has been created since the original file was imported, then the original DS2 code file is locked and cannot be edited. To make changes to the DS2 code file, you must edit the newest version. To use these changes in the segmentation tree, you must [reimport the code file](#).

To open the original DS2 code file in the code file editor:

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Select the outcome code file for which you want to edit the original DS2 code file. The **Referenced code file** field at the bottom of the window displays a link to the original DS2 code file.
- 3 Click the link to open the DS2 code file in the code file editor.
- 4 Edit the DS2 code file as needed.
- 5 Click  to save the changes to the original code file, and then click **x** to close the code file.

Note: To use these changes in the segmentation tree, you must [reimport the code file](#).

Duplicate an Outcome Code File

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Select the code file that you want to duplicate, and click . The Edit Outcome Code File window appears.
- 3 Click **Duplicate**. The Save As window appears.
- 4 Enter a name for the duplicate outcome code file, and click **OK** to return to the Manage Outcome Code Files window.
- 5 Click **Save** to save your changes and return to the segmentation tree diagram.
- 6 Click  to save the changes to the segmentation tree.

Delete an Outcome Code File

- 1 Click  and select **Manage outcome code files**. The Manage Outcome Code Files window appears.
- 2 Select the outcome code file that you want to delete, and click .

Search for Objects in a Segmentation Tree Diagram

You can use the **Search** field in the segmentation tree diagram to search for a specific variable, matrix, Boolean expression, or outcome. To search for a specific object, enter the object name in the **Search** field, and click .

For any object that matches the search criteria, SAS Intelligent Decisioning highlights the object in the diagram. Click  and  to navigate between highlighted objects. Click  to clear the search field and remove the highlighting.

Compare Segmentation Tree Code

You can compare the generated code of two different segmentation trees, or you can compare the generated code of two different versions of the same segmentation tree.

- 1 Select the objects that you want to compare.
 - To compare the generated code of two different segmentation trees, select the segmentation trees in the category view, click , and select **Compare code**.
 - To compare the generated code of two versions of the same segmentation tree, open the segmentation tree, click  on the **Versions** tab, and select **Compare code**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window and highlights the differences.
- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Segmentation Trees

Duplicating Segmentation Trees

About Duplicating Segmentation Trees in Folders for Which the Check-Out and Commit Feature Is Enabled

When you duplicate a segmentation tree, SAS Intelligent Decisioning first creates an empty segmentation tree, and then updates the tree to add variables and

branches. The default permissions for the Decision Repository enable users to create new objects, but the permissions do not include the ability to update the objects. In order to duplicate an object that is in the Decision Repository, you must check out the object.

If your site has created additional folders for which the check-out and commit feature is enabled, and if those folders have been assigned the same permissions as those assigned to the Decision Repository, you must also check out objects in those folders before you can duplicate them.

When you duplicate an object that is checked out, the duplicate object is not automatically checked out.

Duplicate Segmentation Trees

To duplicate a single segmentation tree:

- 1 Click  to navigate to the Segmentation trees category view, and select the check box for the segmentation tree that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Segmentation Tree window appears.
- 3 Enter a new name for the duplicate segmentation tree.
- 4 (Optional) Enter a description for the segmentation tree.
- 5 Select the version of the segmentation tree that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate segmentation tree.
- 7 Click **Duplicate**.

To duplicate multiple segmentation trees:

- 1 Click  to navigate to the Segmentation trees category view, and, select the check boxes for the segmentation trees that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the segmentation trees and appends `_Copy_n` to the names of the duplicate copies.

Note: When you duplicate a segmentation tree, SAS Intelligent Decisioning creates a relationship between the original segmentation tree and the duplicate segmentation tree. If either object is changed, and later you copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the segmentation tree before you paste the new content.

Delete Segmentation Trees

.....
Note: You cannot delete a segmentation tree if it is open.
.....

In the Segmentation trees category view, select the check boxes for the segmentation trees that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the segmentation trees to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename Segmentation Trees

.....
Note: You cannot rename a segmentation tree if it is open.
.....

- 1 Click  to navigate to the Segmentation trees category view, and select the check box for the segmentation tree that you want to rename.
- 2 Click  and select **Rename**. The Rename window appears.
- 3 Enter a new name for the segmentation tree, and click **Rename**.

Move Segmentation Trees to a Different Folder

- 1 Click  to navigate to the Segmentation trees category view, and select the segmentation trees that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the segmentation trees, and click **OK**.

Managing Versions of Segmentation Trees

Set the Displayed Version

On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Segmentation Tree Version” on page 354](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.

- 3 Click **New version**. The New Version window appears.
- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the segmentation tree whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the segmentation tree whose contents you want to copy:

- a Select the version whose contents you want to copy.

- b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target segmentation tree into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target segmentation tree. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

TIP The input and output designations for a variable on the **Variables** tab for the new version are removed, and the variable is treated as a temporary variable in the following situations:

- The new version of the target object does not use a variable that is used in an earlier version.
- The source and target object originally had a variable of the same name, and you deleted the variable from the source object before you copied it into the target object.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see [“Default Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#) and [“Folder-Based Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#). By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of segmentation trees, the [sas.decisions.segmentationtrees.deleteVersions](#) configuration option must be turned on.

On the object's **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Upgrade Decisions to Use a New Version of a Segmentation Tree

If you create a new version of a segmentation tree that is already used in decisions, you can upgrade the decisions to use the new version of the segmentation tree.

- 1 On the **Versions** tab for the segmentation tree, click , and select **Upgrade decisions**. The Upgrade Decisions window appears. This window lists all of the decisions that include the segmentation tree.
- 2 In the **Version to upgrade to** field, select the version of the segmentation tree to which you want to upgrade the decisions.
- 3 Select **Automatically map variables** if you want SAS Intelligent Decisioning to automatically map new segmentation tree variables in the decisions where the segmentation tree is used.

For information about how SAS Intelligent Decisioning maps variables, see [“About Decision Variables and Mapping” on page 436](#).

- 4 Select the check boxes for the decisions that you want to upgrade, and click **Upgrade decisions**.

TIP To update all of the objects that are used in a decision, see [“Update Decisions to Use New Object Versions” on page 443](#).

Determine Which Objects Use a Segmentation Tree

To list the objects that use a specific segmentation tree:

- 1 On the **Segmentation trees** category page, select the check box for segmentation tree, click , and select **View used by report**. The All Objects that Use the Selected Item window appears. By default, this window lists the objects that use the latest version of the selected segmentation tree.
- 2 (Optional) Select the specific version for which you want to view the Used By report.

To view the list of all objects that use any version of the selected segmentation tree, clear the value in the **Version** field.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Segmentation Tree Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.decisions.segmentationtrees.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a segmentation tree version, the **Check Out** button for that segmentation tree is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Segmentation Tree Versions” on page 353](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Segmentation trees category view shows two segmentation trees with the same name, but the folders listed in the **Location** column differ for each object. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

Checking Out a Segmentation Tree from within a Decision

See [“Checking Out and Committing Objects from within A Decision” on page 464](#) for information.

Concurrently Checking Out and Committing Segmentation Tree Versions

The ability for multiple users to check out the same segmentation tree at the same time is controlled by the `sas.decisions.segmentationtrees.checkout.allowConcurrentCheckout` configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's **My Folder** location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Segmentation Tree Version

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 355](#).

- 4 On the **Versions** tab, click **Commit**. The Commit Segmentation Tree Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.

- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 7 (Optional) Enter information about the new version in the **Notes** field.
- 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

How you undo the checkout of an object depends on how the object was checked out.

If you [checked out an object from within a decision](#), click  on the object's node in the decision diagram, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Testing Segmentation Trees

Ways to Test a Segmentation Tree

There are two types of tests:

Basic test

For more information, see [“Create and Run a New Test” on page 356](#).

Scenario test

enables you to enter specific input values and the output values that you expect the test to generate. A scenario test identifies differences between the output that you expect to see and the actual output that is generated when the test is run. You can also compare the test definitions and test results of different scenarios. Scenario tests are also run in CAS. For more information, see [“Create and Run a Scenario Test” on page 359](#).

Test a Segmentation Tree

Create and Run a New Test

Testing a segmentation tree is optional, but doing so is a best practice. Testing enables you to discover any problems before the segmentation tree is incorporated into a production system.

- 1 On the **Scoring** tab, click the **Tests** tab.
- 2 Click **New test**. The New Test window appears.

- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 5 Click  for the **Location** field, and select the folder in which you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 6 Click  for the **Input table** field, select the input table for the test, and click **OK**.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name : "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name : "HMEQ". [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 7 Verify or change the variable mappings. To run a full test, map all of the input variables to columns in the input table that you selected for the test. To run a partial test, you can map only the input variables that are needed for the test.

SAS Intelligent Decisioning automatically maps the input variables in the segmentation tree to columns in the input table when the names and data types of the variables match those of the table columns.

If any input variables are not mapped to input columns or to static values, the application displays the warning message `Input variables must be mapped to table columns`. At run time, SAS Intelligent Decisioning assigns missing values to input variables that are not mapped.

You can change the automatic variable mappings in the Variable Mappings window.

To change variable mappings:

- a Click **Variables**. The Variable Mappings window appears.

TIP Click **Show unmapped variables** to filter the list to show only the variables that have not been mapped to an table column.

- b For each input variable, select the table column to which the variable should be mapped. If the input table contains more than 15 columns, click **More**

columns to display additional column names. Alternatively, for Decimal, Integer, and Character variables, you can select **Use value** for the table column, and specify a literal value in the **Value** column. When you are entering literal values, remember these rules:

- For character variables, input values are limited to 10 Mb. Output values longer than 32767 characters will be truncated.
- Do not enclose character strings in quotation marks.
- To specify a missing value for character variables, select **Use value** and leave the **Value** column empty. When SAS Intelligent Decisioning generates code, it generates an empty string (' '). For numeric values, enter a period (.).

For more information about entering variable values, see [“The Properties of a Variable” on page 307](#).

- c Click **OK** to close the Variable Mappings window.
- 8 (Optional) Click **Advanced** to display the advanced options.
- 9 (Optional) Click  for the **Output data library** field, and select the library to which you want to write the output of the test.
- 10 (Optional) Enter a name for the test results output table in the **Output table name** field if you do not want to use the default name. The default name is `test-name_object-name_timestamp_output`.
- 11 (Optional) For the **Target version**, select the version of the segmentation tree that you want to test. The default version is the latest version.
- 12 (Optional) Select the variable that you want to serve as an input debug variable. You can specify an input-only variable or an input-output variable.
For more information, see [“Debugging Segmentation Tree Tests” on page 359](#).
- 13 (Optional) Select **Preserve unmapped columns in the output table** if you want columns that are not mapped to an output variable to be written to the output table.
- 14 Click **Run** to save and run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 364](#).

- 15 Click  in the **Results** column to view the results of the test.
- 16 In the test results window, click **Test Results** in the navigation pane to display the URIs and other information for the test. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

Debugging Segmentation Tree Tests

When you create a test, you can specify a variable to use as a debugging variable in the **Input debug variable** field. You can specify an input-only variable or an input-output variable. For more information, see [Step 12 of “Create and Run a New Test” on page 356](#).

When you specify an input debug variable, SAS Intelligent Decisioning automatically sets the maximum number of threads that can be allocated for the test to 1. Setting the thread count to 1 ensures that the variable's values are written to the log in the correct order and are not affected by different threads completing at different times.

To capture variable values for input-only or temporary variables after the segmentation tree logic has executed for a specific record, you can specify that the variable is an output variable, and then re-run the test. Before incorporating the segmentation tree into a production system, return the input and output settings for the variable to their previous settings. For more information, see [“Input Variables, Output Variables, and Temporary Variables” on page 309](#) and [“Edit Variable Properties” on page 314](#).

Test a Scenario

Create and Run a Scenario Test

- 1 On the **Scoring** tab, click the **Scenarios** tab.
- 2 Click **New test**. The New Scenario Test window appears.

- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Click  for the **Location** field, and select the folder where you want to save the test definition.
- 5 Click  for the **Output table location** field, and select the data library where you want to save the test results.
- 6 (Optional) Select the version of the segmentation tree that you want to test. The default version is the latest version.

Note: When you change the version, variables for the new version are reloaded and any variable values that were entered for the test are cleared.

- 7 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.
- 8 Enter the values that you want to use for each input variable. You do not have to enter values for every input variable. At run time, SAS Intelligent Decisioning uses missing values for input variables for which you do not specify a value.

Note: For character variables, input and output values longer than 32767 characters will be truncated. You cannot enter input values for variables of type binary or of type varying-length binary.

To enter values for the columns in a data grid variable:

- a Click  in the **Value** field. The Edit Data Grid window appears. One row is automatically added to the input data grid.
- b Enter values for each column in the data grid.
- c (Optional) Click  to add a new row, and then enter the values for each column.

Repeat this step for each row of values that you want to add to the data grid.

TIP By default, data grid column names appear across the top of the data grid view, and row numbers appear down the left side. You can click  to change the view of the data grid so that row numbers appear across the top and data grid column names appear down the left side.

- d Click **OK** to save the data grid values and close the Edit Data Grid window.

For more information about entering variable values, see [“The Properties of a Variable” on page 307](#).

- 9 (Optional) For each output variable, select the **Include** check box and enter the expected output value. The **Include** check box controls whether a variable's expected value is used to determine the status of a scenario test. If you select

Include for a variable and the test does not return the expected value, the test status is set to Completed with warnings (⚠️). If you do not select the check box, the application ignores the expected value of that variable when it determines the status of the test.

To enter expected values for the columns in a data grid variable, click in the **Expected Output** field, and follow the instructions in [Step 8](#) for entering values for the columns in a data grid variable.

Note: A scenario test cannot verify issues with trailing spaces. For example, it cannot distinguish between a string that contains a single space ' ' and a string that contains three spaces ' '.

- 10 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 364](#).

- 11 Click  in the **Results** column to view the results of the test.
- 12 On the Test Results page, click **Scenario Test Results** in the navigation pane to display the URLs and other information for the test. Click **Input** to display the values of input variables and dynamic attributes. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

On the **Output** page, click **Show All** to display the expected and actual values of all output variables. Click **Show Differences** to display only the variables whose expected values do not match the actual values that were returned by the test.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Output** page, click **Export** to export the output table as a comma-separated values (CSV) file. On the **Log** page, click  to download the log file.

- 13 Click **x** to close test results and return to the **Scoring** tab.

Import Scenario Test Definitions

You can import scenario test values from a comma-delimited (CSV) file. Each line in the CSV file is imported as one scenario test. In the CSV file, add a column of values for each variable. In the header row, enter the names of the input variables and of the output variables with `_expected` appended to the name.

For example, suppose your scenario test has the input variables `policyholder`, `cscore`, and `claims`, and it has the output variables `eligible` and `policies`. An import file for this test might appear in a spreadsheet application as shown in the following figure.

	A	B	C	D	E	F	G	H	I	J
1	policyholder	cscore	claims	eligible_expected	policies_expected					
2	Smyth,Joe	784	0	TRUE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["4535					
3	Dupree, Marcel	803	2	FALSE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["9836					

The policies output variable in this example is a data grid variable. To enter values for data grid variables, use the JSON string format described in [“Introduction to Data Grids” in SAS Intelligent Decisioning: Using Data Grids](#).

The format for date and datetime variables depends on your locale. Use the same format that is created by the date and datetime pickers when you click  or  to enter initial values for custom variables on the **Variables** tab.

To import scenario test definitions:

- 1 On the **Scoring** tab, click the **Scenarios** subtab, and then click **Import scenarios**. The Import Scenarios window appears.
- 2 In the **Import from** field, click  and select the CSV file that contains the scenario test values.

Note: The import file is limited to 10 MB.

- 3 Select **Select encoding** to select the encoding of the CSV file from the predefined list, or select **Enter encoding** to enter a custom encoding.
- 4 Select or enter the encoding of the CSV file.
- 5 Enter a prefix for the scenario test definitions. SAS Intelligent Decisioning appends a number to this prefix for each test definition. The prefix can include double-byte characters and special characters, including single quotation marks.
- 6 (Optional) Click  for the **Folder location** field, and select the folder where you want to save the test definitions.
- 7 Click  for the **Output table location** field, and select the folder where you want to save the test results.
- 8 Click **Import**.

Exporting a Scenario Test Definition

On the **Scoring** tab, click the **Scenarios** subtab, select the tests that you want to export, and then click **Export**. SAS Intelligent Decisioning exports the test as a comma-separated value (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Compare Different Scenarios

You can display the scenario definitions or results of two or more tests side-by-side. On the **Scenarios** tab, select two or more tests, click , and select one of the following options:

Compare ⇒ **Definitions**

displays the input values and the expected output values that you entered for both tests. To edit the input values for a scenario test definition, click  next to the test name under **Input Table**. To edit the expected output values for a test, click  next to the test name under **Output Table**.

Compare ⇒ **Results**

displays the input values and the actual output values that were generated by the test. To display both the expected values and the actual values in the output table, select **Display expected values**. For each variable for which you selected **Include** in [Step 9 on page 360](#), the application highlights the variable if the actual and expected values do not match.

Click **Export** to export the results comparison as a comma-separated values (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Working with Test and Validation Test Output Data

Note: The actions available to you depend on the applications that are available at your site.

On the Test Results page, select the **Output** table in the navigation pane, click **Actions**, and select one of the following options:

Explore Lineage

opens SAS Lineage Viewer. SAS Lineage Viewer enables you to better understand the relationships between objects in your SAS Viya platform applications. These objects include data, transformation processes, reports, and visualizations. For more information, see [SAS Lineage: User's Guide](#).

Explore and Visualize

opens the output table in SAS Visual Analytics. SAS Visual Analytics enables you to create, test, and compare models based on the patterns discovered during exploration of the data. You can export the model before or after performing model comparison for use with other SAS products or to put the model into production. SAS Visual Analytics supports a range of visualization, discovery, and reporting features. For more information, see [Welcome to SAS Visual Analytics](#).

Access and Manage Data

opens the output table in SAS Data Hub. SAS Data Hub enables you to import and download tables, load tables into memory, unload tables, and more. For more information, see [SAS Data Hub: User's Guide](#).

Status Icons for Tests

Icon	Status
	The test is not ready to run. The test definition is not complete, or it might contain errors.
	The test has been defined and can be run.
	The test is defined correctly and is ready to run.
	The test is running.
	The test completed successfully.
	The test completed, but warnings were issued in the SAS log. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.
	The test did not run successfully. Check the SAS log for information. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.

Duplicate Tests

To duplicate a single test:

- 1 On the **Scoring** tab, click the **Tests** tab or the **Scenarios** tab, depending on the test that you want to duplicate.
- 2 Select the check box for the test that you want to duplicate, click  on the subtab, and select **Duplicate**.
- 3 In the Duplicate Tests window, enter a name for the duplicate test if you do not want to use the default name.
- 4 (Optional) Enter a description for the duplicate test.
- 5 Click **Duplicate**.

To duplicate multiple tests:

- 1 On the **Scoring** tab, click the **Tests** tab or the **Scenarios** tab, depending on the tests that you want to duplicate.
- 2 Select the check boxes for the tests that you want to duplicate, click  on the subtab, and select **Duplicate**.

SAS Intelligent Decisioning duplicates the tests and adds `_Copy` to each filename.

Manage Comments for a Segmentation Tree Test

You can associate comments and attachments with a segmentation tree test.

To open the Comments properties pane, select the segmentation tree test on the **Scoring** tab, and click  in the property pane.

To add a new comment, enter the comment in the text box and click **Post**.

To add an attachment to a comment, click , select the file that you want to attach, and click **Post**. (The attachment icon appears after you enter text in text box.) You cannot attach executable files such as BAT and EXE files.

To reply to a comment, click **Reply**, enter your reply in the text box, and click **Post**.

To edit an existing comment, click  for the comment, and select **Edit**. Edit the comment, and click **Post**.

To delete a comment, click  for the comment, and select **Delete**.

Working with REST API Definitions

About REST API Definitions	368
Create a REST API Definition	368
Define the REST API Request	370
Define Response Headers	372
API Authorization Types	372
Managing the Variables in a REST API Definition	373
The Properties of a Variable	373
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object	374
Create Custom Variables	375
Duplicate a Variable	376
Importing and Exporting Variables	376
Delete Variables	377
Edit Variable Properties	377
Determine Which Objects Use a Particular Variable	378
Managing REST API Definitions	378
Duplicate REST API Definitions	378
Delete REST API Definitions	379
Rename REST API Definitions	379
Move REST API Definitions to a Different Folder	379
Managing Versions of REST API Definitions	380
Set the Displayed Version	380
Create a New Version	380
Copy the Content of a Version	381
Delete a Version	382
Upgrade Decisions to Use a New Version of a REST API Definition	383
Determine Which Objects Use a REST API Definition	383
Checking Out and Committing REST API Definition Versions	384
About Checking Out and Committing Versions	384
Checking Out a REST API Definition from within a Decision	385
Concurrently Checking Out and Committing REST API Definition Versions	385
Check Out and Commit a REST API Definition Version	386
Determine Who Has a Version Checked Out	387

Opening the Original Object	387
Undoing a Check Out	387

About REST API Definitions

A REST API definition node enables a decision to send requests to and receive responses from an external API. These nodes enable you to integrate external services such as large language models (LLMs), credit bureaus, open banking services, email, and other services directly into your decision logic. When a decision that contains a REST API definition node is run, SAS Intelligent Decisioning executes the REST call that is defined in the associated REST API definition and maps the response values to output variables in the decision.

Note: Decisions that contain REST API definition nodes can be published to and run in to CAS and SAS Micro Analytic Service destinations. Publishing to other destination types is not supported.

IMPORTANT You are responsible for ensuring that only users who are authorized to do so can configure support for REST API definition nodes, and develop, test, and execute nodes that communicate with REST APIs that are external to your enterprise.

Create a REST API Definition

- 1 Click  to navigate to the REST API Definitions category view, and click **New definition**.
- 2 In the New REST API Definition window, enter a name for the REST API definition if you do not want to use the default name. Definition names are limited to 100 characters, must be unique within a folder, and are case-sensitive.
- 3 (Optional) Enter a description for the new definition. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 4 Select the HTTP method type for the request. The supported methods are GET, POST, and HEAD.

Note: The PUT and PATCH methods are not supported.

- 5 In the **Request path** field, enter the URL for the API request. The URL must start with either `http://` or `https://`. The URL that you enter in the New REST API Definition window cannot contain authorization tokens, query parameters, spaces, or variable names.

TIP You can add variable names to the request path on the **Definitions** tab of the new definition.

- 6 Click , and select the folder where you want to save the REST API definition.
- 7 Click **Save**. SAS Intelligent Decisioning opens the new definition, and displays the **Request** page of the **Definition** tab.

SAS Intelligent Decisioning automatically creates two variables for the new definition:

- `responseBody` is a character variable that is set to response body that is returned by the API when the definition is run.
- `responseStatusCode` is a numeric variable that is set to the HTTP status code that is returned by the API when the definition is run.

TIP After you add the REST API definition to a decision, make sure that the length of the decision variable to which the `responseBody` variable is mapped is large enough to handle the data that is returned by the request. For more information, see [“The Properties of a Variable” on page 395](#).

Note: The `responseBody` and `responseStatusCode` variables are required and cannot be renamed or deleted.

- 8 Add variables, define the REST API request, and define the response.
 - [“Managing the Variables in a REST API Definition” on page 373](#)
 - [“Define the REST API Request” on page 370](#)
 - [“Define Response Headers” on page 372](#)
- 9 Click .

Define the REST API Request

On the **Request** page of the definition, enter the details of the outbound HTTP call that the REST API definition makes when it is executed.

- 1 Verify the method type.
- 2 (Optional) Enter the JSON for the **Request body**. Enclose the entire request body in braces (`{}`), and separate each property with commas. Enclose property values that are character strings and property names in double quotation marks (`"`). To include a variable value in the request body, enclose the variable name in braces, for example, `{myVariable}`. For Boolean values, you can specify `1`, `0`, `true`, or `false`.

Note: The only request body format that is supported is JSON. Date, datetime, and data grid variable types are not supported in the request body.

- 3 Select the authorization type. For information about the authorization types, see [“API Authorization Types” on page 372](#).

SAS Intelligent Decisioning displays fields for the authorization information that is required for the selected authorization type.

- 4 Enter the authorization information for the selected authorization type. For information about these fields, see [Table 12.1 on page 370](#).

Table 12.1 Authorization Fields

Authorization Field	Description
Credential domain name	Select the domain that specifies the value of the authentication key for the API that you are accessing. The domain must be defined in SAS Environment Manager , and it must define a credential domain that is valid for the API that you are accessing. The credential must specify the authentication key for that API as a secret. The secret name must be <code>token</code> . The selection list for this field includes only the token-type domains to which your user ID has access. For more information, see “Define the Authentication Domain for External REST API Requests” in SAS Intelligent Decisioning: Administrator’s Guide .
Header name	Enter the name of the authorization header for the API that you are accessing, or select the variable that evaluates to this name. Refer to the API’s documentation for the name of the authorization header.

Authorization Field	Description
Parameter name	Enter the name of the authorization query parameter for the API that you are accessing, or select the variable that evaluates to this name. Refer to the API's documentation for the name of the authorization query parameter.

- 5 Verify that the **Request path** is correct. The URL that you enter in the **Request** page cannot contain authorization tokens, query parameters, or spaces, but it can include variable names.

To include a variable name in the request path, enclose the variable name in braces. For example, to include the value of a variable named `customerId`, enter `{customerId}` in the path.

Note: Do not include query parameters in the **Request path** field. Define them in the **Query parameters** table instead.

- 6 In the **HTTP request headers** table, add any HTTP headers that are required for your request. For each header, specify a key name and a value.

- a Click  to add a new row to the table.

TIP To delete an existing request header, select the table row for the header, and click .

- b Enter a literal string for the header key name, or select a variable that evaluates to the header's key name. The value of the header name is sent to the API as a quoted string, even if the you select a numeric variable.
 - c Enter the header value, or select a variable that evaluates to the header's value. To specify a variable for the value, enclose the variable name in braces, for example, `{myVariable}`.
 - d (Optional) Enter a description for the header.
- 7 In the **Query parameters** table, add any query parameters that are required for your request. For each parameter, specify a key and a value.
 - a Click  to add a new row to the table.
 - b Enter a literal string for the query parameter name, or select a variable that evaluates to the parameter's name. The value of the parameter name is sent to the API as a quoted string, even if the you select a numeric variable.
 - c Enter the parameter value, or select a variable that evaluates to the parameter's value. To specify a variable for the value, enclose the variable name in braces, for example, `{myVariable}`.
 - d (Optional) Enter a description for the parameter.

- 8 Click .

Define Response Headers

SAS Intelligent Decisioning automatically records the response body and response status codes in variables named `responseBody` and `responseStatusCode`. These variables are required and cannot be deleted or renamed.

To capture additional output from other HTTP response headers, map the additional HTTP response headers to output variables in the REST API definition.

- 1 On the **Variables** tab, define output-only variables for each additional response header whose value you want to record.
- 2 On the **Definition** tab, select the **Response** page.
- 3 In the **HTTP response headers** table, map the HTTP response headers to the output variables. For each response header, enter the header key name and select the output variable.
 - a Click  to add a new row to the table.
 - b Enter a literal string for the header key name, or select a variable that evaluates to the header's key name. The value of the header name is sent to the API as a quoted string, even if the you select a numeric variable.
 - c Select the output variable where you want to save the header's value.
 - d (Optional) Enter a description for the response header.
- 4 Click  to save the definition.

API Authorization Types

Refer to the documentation for the API that you want to access for information about the authorization type that is supported by that API that you are accessing.

For the **Bearer**, **API Key Header**, and **API Key Query** authentication types, consult the documentation for the API that you are accessing for information about generating an authentication key. In SAS Environment Manager, create a credential domain that specifies that authentication key, and specify that domain in the **Credential domain name** field for the new REST API definition. For more information, see [“Define the Authentication Domain for External REST API Requests” in SAS Intelligent Decisioning: Administrator's Guide](#) and [“Define the REST API Request” on page 370](#).

If you are logged into SAS Intelligent Decisioning, you can use the **SAS OAuth** authentication type to test a decision that uses a REST API definition node to access the [SAS Viya REST APIs](#). For more information, see [SAS Viya Platform: Authentication](#).

Managing the Variables in a REST API Definition

The Properties of a Variable

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a REST API definition. Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See "Reserved Words in the DS2 Language" in SAS DS2 Programmer's Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.
Data type	REST API definitions support the following data types: Boolean, character, decimal, and integer. Integer and decimal variables are converted to doubles when generated code is run.
Input and Output	A variable in a custom function signature can be an input variable, an output variable, or both, but it cannot be a temporary variable. Note: For REST API definitions, SAS Intelligent Decisioning automatically defines the variables named <code>responseBody</code> and <code>responseStatusCode</code>. These variables are read-only and cannot be deleted or renamed.
Length	The length for Boolean and numeric variable types is set automatically.

Property	Description
	For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	Initial values are used only for output-only variables. When you publish a REST API definition as part of a decision, the initial value that is specified for the corresponding decision variable is used, and the initial value that is specified in the REST API definition is ignored. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. To use the number sign (#) in literal values, specify the percent-encoded value %23. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name : "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name : "HMEQ". [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➤**. To import all of the variables in the table, click **➤➤**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See [“Input Variables, Output Variables, and Temporary Variables” on page 398](#) for more information.

Create Custom Variables

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click **Add variable** and select **Custom variable**. The Add Variables window appears.
- 2 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 373](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Maximum row count**, and **Length** fields.
- c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable's data type. See [“The Properties of a Variable” on page 373](#) for additional information.
- d Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- e Verify that the **Input** and **Output** check boxes are selected correctly for each variable.
 - Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.

- Clear the **Output** check box for any variable that you want to exclude from the output data.
- 3 (Optional) Modify the variable properties in the table of variables.
 - 4 Click **OK** to add the variables and close the Add Variables window.

Duplicate a Variable

- 1 On the **Variables** tab, select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 2 Enter a new name for the duplicate variable.
- 3 (Optional) Enter a description for the variable.
- 4 Click **Duplicate**.

Importing and Exporting Variables

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, "Import File Formats," on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the REST API definition into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.
- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

Note: If you choose **Replace all variables**, the required variables `responseBody` and `responseStatusCode` will not be replaced, but all other variables will be replaced.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Export Variables

Note: If you make changes to the variables, you must save the REST API definition before exporting variable data.

- 1 Open the REST API definition from which you want to export variables.
- 2 On the **Variables** tab, select the variables that you want to export.
- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

Note: You cannot delete a variable if it is used in the current version of an object. You can delete variables from the current version if they are used only in locked versions, but those variables are still included in the locked versions.

Note: The variables named `responseBody` and `responseStatusCode` are required and cannot be deleted or renamed.

On the **Variables** tab, select the check box for the variables that you want to delete, click , and select **Delete**.

Edit Variable Properties

On the **Variables** tab, click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

When you rename a variable, references to that variable within the same object change to use the new name. You cannot change the name or data type of a variable that is used in locked versions.

Note: The variables named `responseBody` and `responseStatusCode` are required and cannot be deleted or renamed.

See [“The Properties of a Variable” on page 373](#) for additional information.

Determine Which Objects Use a Particular Variable

On the **Variables** tab, select the check box for the variable, click , and select **View used by report**.

In the report, you can use the **Filter** field to filter the list of objects based on the object names. If the selected variable is a data grid variable, and if you are interested only in a specific column within the data grid, you can select the column in the **Column name** field, and then click **Apply**. SAS Intelligent Decisioning narrows the search results to only the objects that use the data grid with the selected column.

Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Managing REST API Definitions

Duplicate REST API Definitions

To duplicate a single REST API definition:

- 1 Click  to navigate to the REST API definitions category view, and select the check box for the REST API definition that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate REST API Definition window appears.
- 3 Enter a new name for the duplicate REST API definition.
- 4 (Optional) Enter a description for the REST API definition.
- 5 Select the version of the REST API definition that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate REST API definition.
- 7 Click **Duplicate**.

To duplicate multiple REST API definitions:

- 1 Click  to navigate to the REST API definitions category view, and, select the check boxes for the REST API definitions that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the REST API definitions and appends `_copy_n` to the names of the duplicate copies.

Note: When you duplicate a REST API definition, SAS Intelligent Decisioning creates a relationship between the original REST API definition and the duplicate REST API definition. If either object is changed, and later you copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the REST API definition before you paste the new content.

Delete REST API Definitions

Note: You cannot delete a REST API definition if it is open.

In the REST API definitions category view, select the check boxes for the REST API definitions that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the REST API definitions to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename REST API Definitions

Note: You cannot rename a REST API definition if it is open.

- 1 Click  to navigate to the REST API definitions category view, and select the check box for the REST API definition that you want to rename.
- 2 Click  and select **Rename**. The Rename window appears.
- 3 Enter a new name for the REST API definition, and click **Rename**.

Move REST API Definitions to a Different Folder

- 1 Click  to navigate to the REST API definitions category view, and select the REST API definitions that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.

- 3 Select the location to which you want to move the REST API definitions, and click **OK**.

Managing Versions of REST API Definitions

Set the Displayed Version

The displayed version is the version whose information is displayed on the other tabs, such as the **Properties**, **Definition**, and **Variables** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a REST API Definition Version” on page 386](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.
- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator's Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the REST API definition whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the REST API definition whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the target REST API definition into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target REST API definition. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see [“Default Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#) and [“Folder-Based Permissions” in SAS Intelligent Decisioning: Administrator’s Guide](#). By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of REST API definitions, the [sas.decisions.restapidefinitions.deleteVersions](#) configuration option must be turned on.

On the object's **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Upgrade Decisions to Use a New Version of a REST API Definition

If you create a new version of a REST API definition that is already used in decisions, you can upgrade the decisions to use the new version of the REST API definition.

- 1 On the **Versions** tab for the REST API definition, click , and select **Upgrade decisions**. The Upgrade Decisions window appears. This window lists all of the decisions that include the REST API definition.
- 2 In the **Version to upgrade to** field, select the version of the REST API definition to which you want to upgrade the decisions.
- 3 Select **Automatically map variables** if you want SAS Intelligent Decisioning to automatically map new REST API definition variables in the decisions where the definition is used.

For information about how SAS Intelligent Decisioning maps variables, see [“About Decision Variables and Mapping” on page 436](#).

- 4 Select the check boxes for the decisions that you want to upgrade, and click **Upgrade decisions**.

TIP To update all of the objects that are used in a decision, see [“Update Decisions to Use New Object Versions” on page 443](#).

Determine Which Objects Use a REST API Definition

To list the objects that use a specific REST API definition:

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click ⓘ next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing REST API Definition Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator’s Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.decisions.restapidefinitions.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a REST API definition version, the **Check Out** button for that definition is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing REST API Definition Versions” on page 385](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent

Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the REST API definitions category view shows two REST API definitions with the same name, but the folders listed in the **Location** column differ for each object. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

Checking Out a REST API Definition from within a Decision

See “[Checking Out and Committing Objects from within A Decision](#)” on page 464 for information.

Concurrently Checking Out and Committing REST API Definition Versions

The ability for multiple users to check out the same value list at the same time is controlled by the [sas.decisions.restapidefinitions.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user’s **My Folder** location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user’s changes are preserved in a new version when they commit their changes. One user’s changes do not overwrite another user’s changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a REST API Definition Version

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see [“Undoing a Check Out” on page 387](#).

- 4 On the **Versions** tab, click **Commit**. The Commit REST API Definition Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of

existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

-
- 7 (Optional) Enter information about the new version in the **Notes** field.
 - 8 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

If you [checked out an object from within a decision](#), click  on the object’s node in the decision diagram, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Working with Decisions

<i>What can I include in a decision?</i>	391
<i>Create a Decision</i>	392
<i>Views for Editing a Decision</i>	394
The Decision Flow Tab versus the Decision Tab	394
Controlling the Tab Display	394
<i>Searching for Objects in a Decision Diagram</i>	395
<i>Managing the Variables in a Decision</i>	395
About Variables	395
Add Variables from a Data Table or from a SAS Intelligent Decisioning Object ...	399
Add Global Variables to a Decision	400
Create Custom Variables	400
Duplicate a Variable	401
Importing and Exporting Variables	402
Delete Variables	403
Compare Node and Decision Variables	403
Edit Variable Properties	403
Edit Metadata for Data Grid Variables	404
Determine Which Objects Use a Particular Variable	404
<i>Adding Objects to a Decision</i>	405
Add an Existing Object	405
Create and Add a New Object	407
Add a Variable Assignment Node	408
Adding Record Contacts Nodes	410
Adding A/B Test Nodes	412
Adding Parallel Processing Nodes	415
Adding Branches to a Decision	420
Adding Cross-Branch Links	430
Add a Micro Analytic Module	432
<i>Display and Export the Included Object Report for a Decision</i>	434
<i>Specifying a Subject ID Variable for A Decision</i>	434
<i>Specifying a Python Version for a Decision</i>	435
<i>Mapping Variables within a Decision</i>	436
About Decision Variables and Mapping	436
Add Missing Variables	438
Scoring Rows in a Data Grid	438

Mapping Data Grid Variables	439
Map Object Variables to Decision Variables	439
Editing the Objects in a Decision	441
Open an Object from within a Decision	441
Edit the Properties or Variable Mappings of a Decision Node	442
Reorder Objects	442
Remove an Object from a Decision	442
Replace an Object	443
Update Decisions to Use New Object Versions	443
Validate a Decision in the Decision Editor	444
Copy a Decision URL	445
Compare Decision Content	445
Compare Decision Code	447
Generate PDF Documentation for a Decision	447
Generate a Report for a Published Decision	448
Managing Decisions	449
Duplicating Decisions	449
Delete Decisions	451
Rename a Decision	451
Move Decisions to a Different Folder	451
Managing Versions of Decisions	452
Set the Displayed Version	452
Create a New Version	452
Creating a New Version by Using Tags	453
Create a New Version Automatically When You Publish a Decision	459
Copy the Content of a Version	459
Delete a Version	460
Upgrade Decisions to Use a New Version of a Nested Decision	461
Determine Which Objects Use a Decision	461
Checking Out and Committing Decision Versions	462
About Checking Out and Committing Versions	462
Checking Out and Committing Included Objects Together with A Decision	463
Checking Out and Committing Objects from within A Decision	464
Concurrently Checking Out and Committing Decision Versions	465
Check Out and Commit a Decision Version	465
Determine Whether an Object Is Checked Out	467
Determine Who Has a Version Checked Out	467
Opening the Original Object	468
Undoing a Check Out	468
Publishing a Decision	468
Introduction to Publishing	468
Where Can I Publish Decisions?	470
Publishing Decisions That Include Analytic Store Models	472
Publish Decisions	473
Testing Decisions	476
Ways to Test a Decision	476
Test a Decision	477

Test a Scenario	482
Run a Rule-Fired Analysis	491
Run a Path-Tracking Analysis	494
Convert a Data Grid to Array Format	496
Content That Is Used by Tests and Scenarios for Decisions	496
Validate a Published Decision	497
Working with Test and Validation Test Output Data	499
Status Icons for Tests	499
Duplicate Tests	500
Manage Comments for Decision Nodes and Tests	501
Executing Published Content	501
Executing Content Published to SAS Micro Analytic Service Destinations	501
Executing Content That Has Been Published to SAS Cloud	
Analytic Services Destinations	502
Content Executed by Published Decisions	504
View All Published Decisions	505

What can I include in a decision?

A decision enables you to different objects and node types into a single process. You can add the following object types to a decision. You create these objects in the various categories in SAS Intelligent Decisioning.

- **Rule sets.** There are three types of rule sets: [filtering rule sets](#), [assignment rule sets](#), and [common rule sets](#).
- **Treatment groups.** Treatment groups define offers that you might want to present to customers.
- **Custom code files.** You can include three types of custom code files in a decision flow:
 - [DS2 code files](#)
 - [Python code files](#)
 - [Data query files](#)
- **Segmentation trees.** A segmentation tree is a hierarchical structure that consists of a root node, branches, and final leaf nodes called outcomes. Each branch in the tree specifies criteria that determine whether that particular branch is executed.
- **Decisions.** You can include a decision in other decisions.

You can include the following items, which are created outside of SAS Intelligent Decisioning.

- **Models** that are registered into the SAS Model Manager common model repository. For information about the model types that you can include in a decision, see [“Support for Models in Decisions”](#). For more information, see [SAS Model Manager: User’s Guide](#).

- Custom node types. You can create custom node types by using the Decision Management REST API. For more information, see [SAS Intelligent Decisioning: Using Custom Node Types](#) and <https://developer.sas.com/rest-apis/decisions#decision-node-types>.

You can also define the following node types within the decision in which they are used:

- [Variable assignment nodes](#) enable you to add multiple assignment statements directly into a decision.
- [Record contacts nodes](#), which write the results of the decision to the subject contacts database.
- [A/B test nodes](#), which direct input records into a champion path or a challenger path for testing.
- [Parallel processing nodes](#), which process multiple decisions in parallel.
- [Branches](#), which add conditional logic to the decision flow. There are four types of branches:
 - A [Yes/No branch](#) is evaluated as a Boolean expression.
 - An [Equals branch](#) compares the value of the branch expression to other variables or to literal values.
 - A [Range branch](#) compares the value of the branch expression against one or more ranges of values.
 - A [Like branch](#) compares the value of the branch expression against one or more strings by using the LIKE operator.

If a decision includes a branch, you can use [cross-branch links](#) to direct the flow of execution among objects in the branch's paths or among other objects in the decision.

- [Micro Analytic Module nodes](#), which call SAS Micro Analytic Service modules.

Create a Decision

- 1 Click  to navigate to the Decisions category view.
- 2 Click **New decision**. The New Decision window appears.
- 3 Enter a name for the decision if you do not want to use the default name. Decision names are limited to 100 characters and must be unique within a folder.

Note: Some publishing destinations restrict the characters that can be used in the published name of a decision. For more information, see [Table 13.6 on page 474](#).

- 4 (Optional) Enter a description for the new decision. Descriptions are limited to 1000 characters.

TIP You can edit the description later on the **Properties** tab.

- 5 Click , and select the folder in which you want to save the decision.
- 6 (Optional) Select the workflow that you want to use to manage the development of the new decision.

Note: The list of workflows includes only those workflows that have been assigned the SAS Intelligent Decisioning client identifier and that you have permission to start. You must have Read access to the workflow definition and Update access to the decision. This field is disabled if you do not have permission to start a workflow. If someone else has selected a workflow but you do not have Read access to the workflow, the **Workflow** field contains asterisks.

For more information, see [Chapter 14, “Using Custom Workflows with SAS Intelligent Decisioning,”](#) on page 507.

TIP Alternatively, you can select and start a workflow on the **Properties** tab for the decision. For more information, see [“Start a Workflow Instance for a Decision”](#) on page 515.

- 7 Click **Save**. SAS Intelligent Decisioning opens the new decision and displays the **Decision Flow** tab.

TIP Objects that are saved in a folder for which the check-out and commit feature is enabled, such as the **Decision Repository** folder, must be checked out before they can be edited.

- 8 (Optional) Start the workflow that you selected in [Step 6 on page 393](#). Click **Workflow**, and select **Start workflow**.

TIP After a workflow is started, you cannot change the selected workflow, and the **Tasks** menu replaces the **Workflow** menu. For more information, see [“Working with Tasks”](#) on page 516.

- 9 If the decision is in a folder for which the check-out and commit feature is enabled, click the **Versions** tab and check out the latest version of the decision. For more information, see [“Check Out and Commit a Decision Version”](#) on page 465.
- 10 Add objects, and if needed, add variables to the decision. For more information, see [“Adding Objects to a Decision”](#) on page 405 and [“Managing the Variables in a Decision”](#) on page 395.

- 11 Click  to save the decision.

Views for Editing a Decision

The Decision Flow Tab versus the Decision Tab

There are two tabs on which you can view and edit decisions.

- On the **Decision Flow** tab, you can edit a decision by using a graphical editor.
To add objects to a decision, right-click on a node to display a menu or click  or . To choose objects that have already been defined by navigating to their location, click . To add branches, cross-branch links, parallel process nodes, or record contacts nodes, or to create new objects by selecting the object type, click .
- On the **Decision** tab, you can edit the decision by using a tabular view.
To add objects to a decision, select **Edit** ⇒ **Add**.

Save your work before you switch tabs. Click  to refresh the view in a tab.

Controlling the Tab Display

On the **Decision Flow** tab:

- Click  to open or close the properties pane for the selected object.
- Click  to open or close the diagram overview. The diagram overview is a scaled-down version of the entire diagram. You can pan across the diagram and position the view over different sections of the diagram. The section that is currently visible on the screen is outlined. The overview is useful when a decision diagram is too large to display all of the nodes on one screen.
- Click  to switch between displaying cross-branch links as either dashed lines or as nodes. See [“Overview of Cross-Branch Links” on page 430](#) for more information.
- Click  to switch between displaying decision node titles on one line or on two lines.

On the **Decision** tab, click  or  to collapse or expand the branch nodes in a decision.

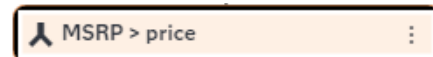
Searching for Objects in a Decision Diagram

Use the Search field and the object type drop-down list on the **Decision Flow** tab to search for specific objects or object types in decision diagrams. To search for a specific object, enter the object name in the Search field. To search for object types, select the object type from the drop-down list. Use both fields to narrow the search. For example, to search for all objects with the name “MSRP > price”, enter the following criteria:



The screenshot shows a search bar with the text "MSRP > price" and a magnifying glass icon. To the right is a dropdown menu labeled "Branches" with a downward arrow.

For any object that matches the search criteria, SAS Intelligent Decisioning highlights the object node with a thicker, darker outline.



The screenshot shows a single object node with a thick orange border. It contains a person icon, the text "MSRP > price", and a vertical ellipsis icon.

Managing the Variables in a Decision

About Variables

The Properties of a Variable

Note: For information about data grid variables, see [“Using Data Grids in SAS Intelligent Decisioning”](#) in *SAS Intelligent Decisioning: Using Data Grids*.

Property	Description
Name	Variable names must start with a letter or an underscore (_), and they can contain only alphanumeric characters and the underscore. Variable names can be up to 32 characters long. They must be unique within a decision.

Property	Description
Data type	Do not use any of these operators or keywords as variable names: AND, OR, IN, NOT, LIKE, TRUE, or FALSE. Do not use <code>_N_</code> or any DS2 reserved word as a variable name. Do not use the word <code>token</code> as a variable name. See “Reserved Words in the DS2 Language” in SAS DS2 Programmer’s Guide for information about reserved words in the DS2 language. SAS Intelligent Decisioning does not support double-byte character set (DBCS) characters in variable names.
Input and Output	SAS Intelligent Decisioning supports the following data types: Boolean, character, data grid, date, datetime, decimal, and integer. Binary and varying-length binary variables are supported only in decisions and only as input variables or temporary variables in order to support models that require binary data. It is recommended that you use varying-length variables where possible because they allow memory to be allocated more efficiently. Integer, decimal, date, and datetime variables are converted to doubles when generated code is run. A variable can be an input variable, an output variable, both, or neither (a temporary variable). See “Input Variables, Output Variables, and Temporary Variables” on page 398 for more information. Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.
Length	The length for Boolean and numeric variable types is set automatically. For output-only data grid variables, the length is set to the value that you specify. For output-only character variables in decisions that are published to container destinations or to SAS Micro Analytic Service destinations, the length that you specify is ignored unless the <code>sas.decisions.variable.length.honorOutputLengthInMAS</code> configuration option is turned on. For more information, see “sas.decisions.variable.length” in SAS Intelligent Decisioning: Administrator’s Guide. For character variables and data grid variables that are input-only or input-output variables, the <code>variableLengthOverridden</code> configuration property determines how the length is set. By default, this property is set to Off, and the length is set to the length in the input data. When the <code>variableLengthOverridden</code> property is set to On, the length of input-only and input-output character variables and data grid variables is set to the value that you specify. For more information,

Property	Description
	see "sas.decisions.variableLengthOverridden" in <i>SAS Intelligent Decisioning: Administrator's Guide</i>. Note: For scenario tests, the <code>variable.length.honorOutputLengthInMAS</code> and <code>variableLengthOverridden</code> configuration options are ignored. Scenario tests always use the length that you specify. In decisions that are published to SAS Micro Analytic Service destinations or to container destinations, the maximum length for character variables (outside of a data grid), for data grid variables, and for varying-length binary variables is 50 Mb. For other destination types, the maximum length for these variables is 10 Mb. The maximum length for character variables within a data grid is 32767 characters. The maximum length for binary variables (not varying-length binary variables) is 8387896 characters. For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.
Initial value	You can specify an initial value only for variables of type character, integer, decimal, or boolean. Initial values are used only for output-only variables. Initial values are used when you test a decision in SAS Intelligent Decisioning and when you publish a decision independently. When a decision is published as a subdecision (when it is published as part of another decision), the initial value that is specified for the corresponding decision variable is used, and the initial value that is specified in the subdecision is ignored. For Boolean variables, you can select True or False for the initial value. However, SAS Intelligent Decisioning represents Boolean values by using the numbers 1 and 0 in the code that it generates. When SAS Intelligent Decisioning is evaluating Boolean expressions, any non-zero number is considered True. When you are entering expressions in the expression editor, specify 1 for True and 0 for False. To use the number sign (#) in literal values, specify the percent-encoded value %23. For content that will be published to a SAS Micro Analytic Service destination, the values of date and datetime input variables must be numeric. They cannot be formatted values such as 18jul2019. Use a dot for the decimal point in decimal numbers. Decimal formats that use commas are not supported.
Description	Descriptions are limited to 256 characters.

Property	Description
Maximum row count	The maximum number of rows that are allowed in each column of a data grid that is generated from an input table in which the data grid column names use array syntax. If the input data contains more entries for a column than the specified row count allows, the input data for that column is truncated. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in SAS Intelligent Decisioning: Using Data Grids

Input Variables, Output Variables, and Temporary Variables

For each variable, you must specify whether the variable is an input variable, an output variable, both an input variable and an output variable, or a temporary variable.

- Input variables are variables that are present in the input table for the decision. When a decision is deployed in a production system, all of the decision's input variables must be mapped to table columns in input data. When you test a decision in SAS Intelligent Decisioning, for each input variable, you can either map it to a table column or specify a constant as its input value.

When you create or edit a decision variable, clear the **Input** check box for any variable that you does not receive its value from the input table.

- Output variables are variables that are written to the output table that is created when a decision is run. When you create or edit a variable, clear the **Output** check box for any variable that you want to exclude from the output data.

Note: It is strongly recommended that you use an assignment statement to initialize the value of output-only variables. This is especially important when the variable is used in a rule condition expression. Uninitialized output-only variables might produce incorrect results when a rule set or decision is run.

- Temporary variables are variables that are not present in the input data, and they are not written to the output table. To create a temporary variable for use only while a decision is executing, clear both the **Input** and **Output** check boxes.

Note: For character variables that are temporary variables, the default length is set to 100 characters, and values longer than 100 characters are truncated. It is strongly recommended that you explicitly set the length of temporary character variables.

When you create a new variable, it is created as an input-output variable by default.

Note: Binary and varying-length binary variables are supported in decisions only as input variables or temporary variables in order to support models that require binary data.

Add Variables from a Data Table or from a SAS Intelligent Decisioning Object

- 1 On the **Variables** tab, click the **Decision Variables** subtab, then click **Add variable** and select the object type from which you want to add variables. The Add Variables window appears.
- 2 Click . If you are importing variables from a data table, the Choose Data window appears. If you are importing from another SAS Intelligent Decisioning object type, the Choose an Item window appears.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name: "*string*" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name: "HMEQ". ["How to Enter Searches for Information Assets in SAS Data Governance" in SAS Data Governance: User's Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see ["Importing and Loading Data" on page 12](#).

- 3 Click , select the table or object from which you want to import variables, and click **OK** to return to the Add Variables window.
- 4 Select the variables that you want to import and click **➤**. To import all of the variables in the table, click **➤»**.
- 5 Click **Add** to add the selected variables, or click **Add and replace** to replace existing variables that have the same name.
- 6 On the **Variables** tab, select or clear the **Input** and **Output** check boxes as necessary. See ["Input Variables, Output Variables, and Temporary Variables" on page 398](#) for more information.

Add Global Variables to a Decision

In order to use a global variable in a decision, the global variable must be activated. Instructions for defining and activating global variables are in [“Create a New Global Variable” on page 174](#) and [“Activate a Global Variable” on page 182](#).

Note: You can add a global variable that has not been activated to a decision and publish the decision, but the variable’s value is set to missing until the global variable is activated.

If a global variable has not been activated, the warning icon  is displayed in the **Activated Value** column.

-
- 1 On the **Variables** tab, click the **Global Variables** subtab.
 - 2 Click **Select variables**. The Select Variables window appears.
 - 3 Select the check boxes for the variables that you want to add to the decision, and click **OK**.
 - 4 (Optional) Select the **Output** check box if you want the value of the variable to be written to the output table that is generated when the decision is run.

Create Custom Variables

Note: For information about data grid variables, see [“Defining Data Grid Variables” in SAS Intelligent Decisioning: Using Data Grids](#).

To create custom variables on the **Variables** tab:

- 1 Click the **Decision Variables** subtab.
- 2 Click **Add variable** and select **Custom variable**. The Add Variables window appears.
- 3 Complete these steps for each variable that you want to add:
 - a Enter the name of the new variable, and select the data type of the variable. See [“The Properties of a Variable” on page 395](#) for additional information.

TIP To re-add a variable that was used in a locked version, you must specify the same data type that was used in the previous version.

- b (Optional) Click **Optional** to display the **Description**, **Initial value**, **Maximum row count**, and **Length** fields.
- c (Optional) Enter any optional values needed for the new variable. Whether you can specify an initial value or length for the variable depends on the variable's data type. See [“The Properties of a Variable” on page 395](#) for additional information.

TIP You can specify an expression as the initial value only for variables of type character, integer, decimal, or boolean. To enter an expression, click  to open the expression editor. The expression can be up to 1998 characters long. For more information, see [“Using the Expression Editor” on page 535](#).

- d Click **Add**. SAS Intelligent Decisioning adds the new variable to the table of variables at the bottom of the window. By default, variables are added to the table as both input and output variables.
- e Verify that the **Input** and **Output** check boxes are selected correctly for each variable.
 - Clear the **Input** check box for any variable that you do not want to be mapped to a column in an input table or for which you do not want to specify a value.
 - Clear the **Output** check box for any variable that you want to exclude from the output data.
 - Clear both the **Input** and **Output** check boxes to create a temporary variable.
- 4 (Optional) Modify the variable properties in the table of variables.
- 5 Click **OK** to add the variables and close the Add Variables window.

Duplicate a Variable

- 1 On the **Variables** tab, click the **Decision Variables** subtab.
- 2 Select the variable that you want to duplicate, click , and select **Duplicate**. The Duplicate Variable window appears.
- 3 Enter a new name for the duplicate variable.
- 4 (Optional) Enter a description for the variable.
- 5 Click **Duplicate**.

Importing and Exporting Variables

Import Variables

You can import variables from either comma-delimited (CSV) files or from JavaScript Object Notation (JSON) files. These files must conform to formats described in [Appendix 2, “Import File Formats,” on page 553](#). These formats are the same formats that are created when you export variables.

- 1 Open the decision into which you want to import variables.
- 2 On the **Variables** tab, click **Import**, and select either **Comma-delimited (*.csv)** or **JSON (*.json)**. The Import File window appears.
- 3 Click  and select the file from which you want to import variables.
- 4 Select an encoding from the selection list, or enter a custom encoding.
- 5 Select **Add variables** to append the variables to the current list of variables, or select **Replace all variables** to replace the current list of variables with the imported variables.

TIP To add new columns to an existing data grid variable, select **Replace all variables**.

Note: If you replace the list of variables in a decision and a variable that is used in the decision is not included in the import file, the variable is removed from the list of variables. An error marker is added to the nodes that use that variable in the diagram on the **Decision Flow** tab.

- 6 Click **Import**, and then click  to import the variables and save your changes.

Export Variables

Note: If you make changes to the variables, you must save the decision before exporting variable data.

- 1 Open the decision from which you want to export variables.
- 2 On the **Decision Variables** tab, select the variables that you want to export.

- 3 Click **Export**, and select the file type to which you want to export the variables. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Delete Variables

Note: You cannot delete a variable if it is used in the current version of an object. You can delete variables from the current version if they are used only in locked versions, but those variables are still included in the locked versions.

- 1 On the **Variables** tab, click the **Decision Variables** subtab.
- 2 Select the check box for the variables that you want to delete, click **:**, and select **Delete**.

Compare Node and Decision Variables

To compare the input and output settings for a node's variables with the settings for the corresponding decision variables, open the decision, click the decision node, and select **Compare node variables**. The Compare Node Variables window lists the input and output settings for each of the node's variables and for the corresponding decision variables. To change the mappings for a decision variable, click **Go to Variables** tab to switch to the **Variables** tab for the decision. For more information, see ["Input Variables, Output Variables, and Temporary Variables" on page 398](#) and ["Mapping Variables within a Decision" on page 436](#).

Edit Variable Properties

On the **Variables** tab, click the **Decision Variables** subtab, then click on the name of the variable that you want to edit. The Edit Variable window appears. Edit the properties as needed, and then click **OK**.

When you rename a variable, references to that variable within the same object change to use the new name. You cannot change the name or data type of a variable that is used in locked versions.

See ["The Properties of a Variable" on page 395](#) for additional information.

Edit Metadata for Data Grid Variables

For information, see “Editing Data Grid Variable Metadata” in *SAS Intelligent Decisioning: Using Data Grids*.

Determine Which Objects Use a Particular Variable

- 1 On either the **Decision Variables** subtab or the **Global Variables** subtab, select the check box for the variable, click  on the subtab, and select **View used by report**. The Select the Objects to Include in the Report window appears.
- 2 Select the objects within which you want to search for the selected variable. Select **Search the current object and all objects included within it** to search only the current decision and the objects that are included in it. Select **Search all objects in the SAS Intelligent Decisioning environment** to search all of the objects that are defined in your current environment.
- 3 Click **Search**.

In the report, you can use the **Filter** field to filter the list of objects based on the object names. If the selected variable is a data grid variable, and if you are interested only in a specific column within the data grid, you can select the column in the **Column name** field, and then click **Apply**. SAS Intelligent Decisioning narrows the search results to only the objects that use the data grid with the selected column.

Click on an object name to open the object. Click  next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

If you selected **Search the current object and all objects included within it**, you can click  next to a decision name to display the list of objects within that decision that use the selected variable. Click  and  to open and close the list objects within the nested decision.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Adding Objects to a Decision

Add an Existing Object

TIP On the **Decision Flow** tab, if you drop the new object directly on top of an existing node, the new object is added immediately below the existing node. If no nodes are selected in the diagram, and you drop the new object anywhere except on top of another node, the new object is added as the last node in the decision. If an existing node is already selected, and you drop the new object anywhere except on top of another node, the new object is added immediately below the selected node. Also, if the selected node is a branch, the **Edit** menu changes to show an **Add to path** option for each outgoing branch path.

On the **Decision** tab, if no nodes are selected in the diagram, newly added nodes are added as the last node in the decision. If an existing node is already selected, newly added nodes are added immediately below the selected node.

- 1 On the **Decision Flow** tab, click  to open the Content Selection window, and then drag the object from the list onto the diagram where you want to add it.

Alternatively, on the **Decision** tab:

- a Select **Edit** ⇒ **Add**, and then select the object type that you want to add to the decision. The Choose an Item window appears.
- b Select the object that you want to add, and click **OK**.

Note: The objects that are included in a decision determine the destination types to which you can publish the decision. For more information, see [“Where Can I Publish Decisions?” on page 470](#).

IMPORTANT It is recommended that you do not add a decision as a nested decision within itself.

If SAS Intelligent Decisioning detects an error with an object that you add to a decision, it displays a red X  on the object. Click  to open the **Errors** pane for more information.

A decision that includes an object must define decision variables to which you can map the object's variables. If you are using the default settings, and you

have not already defined decision variables with the same name and data type as the object's input and output variables, SAS Intelligent Decisioning displays a red X  to indicate that decision variables need to be created. For more information, see [“About Decision Variables and Mapping” on page 436](#).

- 2 (Optional) Click  in the toolbar, and select **Add missing variables** to add any missing variables to the decision. Alternatively, you can manually add the decision variables on the **Variables** tab. For more information, see [“Add Missing Variables”](#).
- 3 (Optional) Reorder the objects in the decision. For more information, see [“Editing the Objects in a Decision” on page 441](#).
- 4 (Optional) For rule sets, code files, models, and subdecisions, select the version of the object that you want to include in the decision in the **Properties** pane for the object. For models, if you select a specific version, the decision always uses that version, even if newer versions of the model are deployed. If you select **latest**, the decision always uses the most recently deployed version of the model.

Note: Do not include multiple versions of the same analytic store model in the same decision if two or more versions of the model use the same ASTORE files.

TIP You can change the version of most objects at any time by changing the version on the **Properties** pane. See [“Edit the Properties or Variable Mappings of a Decision Node” on page 442](#) for more information. You cannot specify the version of a treatment group that is used in a decision. For more information, see [“Content That Is Used by Tests and Scenarios for Decisions” on page 496](#) and [“Content Executed by Published Decisions” on page 504](#).

- 5 (Optional) If the object that you add to the decision is a Python model or a subdecision that uses Python, verify the version of Python that is specified for the decision on the decision's **Properties** tab. For more information, see [“Specifying a Python Version for a Decision” on page 435](#).
- 6 (Optional) If you are adding an object that uses a function that is defined in a custom context file, specify the custom context file on the **Properties** tab. See [“Associating a Custom Context File with a Decision” on page 198](#) for more information.
- 7 (Optional) On the **Properties** tab, select the **Subject ID** variable and the **Subject level** variable. For more information, see [“Specifying a Subject ID Variable for A Decision” on page 434](#).
- 8 Verify that the object's variables are mapped to the correct decision variables. For more information, see [“Mapping Variables within a Decision” on page 436](#).

TIP When you add an object to a decision, SAS Intelligent Decisioning uses the object's ID, not its name, to add the object. If you delete an object and create a new object with the same name, you must delete and re-add the object to the decision.

Create and Add a New Object

- 1 On the **Decision Flow** tab, click , and drag the new object type onto the diagram where you want to add it. For models and Micro Analytic modules, opens a dialog for you to select the object that you want to add. For record contacts nodes and parallel process nodes, SAS Intelligent Decisioning adds the new object to the diagram and opens the properties pane for the new object. For all other object types, the **New object** window appears.

TIP If you drop the new object directly on top of an existing node, the new object is added immediately below the existing node. If an existing node is already selected, and you drop the new object anywhere except on top of another node, the new object is added immediately below the selected object. If no nodes are selected in the diagram, and you drop the new object anywhere except on top of another node, the new object is added as the last node in the decision.

- 2 Define the new object and its contents.
 - For models and [Micro Analytic modules](#), select the object that you want to add to the decision, and click **Select** or **OK**.
 - For [record contacts nodes](#) and [parallel process nodes on page 415](#), define the content for the new node in the **Properties** pane.
 - For [branches](#), select the branch type, optionally enter a name for the new branch, and click **OK**.
 - For [cross-branch links](#), select the target node for the link, and click **OK**.
 - For all other object types, enter a name for the object and any additional required information, and click **Save**. SAS Intelligent Decisioning creates a new object and adds it to the decision. To define the new object's content, click **Open** on the object's **Properties** pane or tab. For detailed information about creating new objects and defining their content, see the following topics:
 - ☐ [“Create a New Rule Set”](#)
 - ☐ [“Define a Treatment Group”](#)
 - ☐ [“Create a New Code File”](#)
 - ☐ [“Create a Segmentation Tree”](#)
 - ☐ [“Create a Decision”](#)

If SAS Intelligent Decisioning detects an error with an object that you add to a decision, it displays a red X  on the object. Click  to open the **Errors** pane for more information.

If you added new variables to the object, SAS Intelligent Decisioning displays a red X  on the object to indicate that decision variables need to be created.

- 3 (Optional) Click  in the toolbar, and select **Add missing variables** to add any missing variables to the decision. Alternatively, you can manually add the decision variables on the **Variables** tab. For more information, see [“About Decision Variables and Mapping”](#)
- 4 Click  to save the decision.

Add a Variable Assignment Node

A variable assignment node enables you to add multiple assignment statements directly into a decision instead of creating a rule set that contains only standalone assignment statements.

TIP To reuse a set of assignment statements, you can define the statements in one of two ways:

- Create a rule set that contains only assignment statements and include that rule set where you want to reuse the statements.
- Create a decision that contains only an assignment node and include that decision as a subdecision where you want to reuse the statements.

Note: You cannot set the value of global variables in a variable assignment node. Also, rule-fired data is not generated for assignment statements.

- 1 Do one of the following:
 - On the **Decision Flow** tab, select  and drag the **Variable assignment** object onto the diagram where you want it. Alternatively, right-click the node below which you want to add the variable assignment node and select **Add** ⇒ **Variable assignment**. SAS Intelligent Decisioning opens the **Properties** pane for the node.
 - On the **Decision** tab, select the node below which you want to add the variable assignment node, select **Edit** ⇒ **Add** ⇒ **Variable assignment**, and click  to open the node's **Properties** pane.
- 2 (Optional) Edit the node name if you do not want to use the default name. Click  in the **Name** field, enter a new name, and click **Rename**. Assignment node names are limited to 100 characters, must be unique within a decision, and cannot contain forward slashes (/), semicolons (;), or braces ({}).

- 3 (Optional) Enter a description for the node. Descriptions are limited to 1000 characters.
- 4 Click  above the **Assignments** table.
- 5 In the Edit Assignments window, add one or more assignment statements to the node.

For each assignment statement:

- a Click **Add assignment**. SAS Intelligent Decisioning adds a row to the list of assignment statements for the node.
- b Select the variable to which you want to assign a value.
- c Enter or select a value to assign to the variable. Each assignment statement must specify a value that matches the data type of the variable.
 - For Boolean variables, select either **True** or **False**.
 - For date or datetime variables, click  or , select a value, and click **OK**.
 - For data grid variables:
 - 1 Click . In the Edit Data Grid window, one row is automatically added to the input data grid.

Note: If SAS Intelligent Decisioning displays an error message stating that all columns are hidden, add columns to the data grid. For more information, see [“Editing Data Grid Variable Metadata” in SAS Intelligent Decisioning: Using Data Grids](#).

- 2 Enter values for the columns in the data grid. If you do not enter a value for a column, the column's value is defined as null.
- 3 (Optional) Click  to add a new row, and enter the values for the row.
Repeat this step for each row of values that you want to add to the data grid.

TIP By default, data grid column names appear across the top of the data grid view, and row numbers appear down the left side. Click  to change the view of the data grid so that row numbers appear across the top and data grid column names appear down the left side.

- 4 Click **OK** to save the data grid values and close the Edit Data Grid window.

TIP To delete an assignment statement from the variable assignment node, click  for that statement in the node's Assignments table.

- 6 SAS Intelligent Decisioning validates each assignment statement, and if they are all valid, enables the **OK** button. Click **OK** to create the assignment node.
- 7 Click  to save your changes to the decision.

Adding Record Contacts Nodes

About Recording Contacts

The primary purpose of record contacts nodes is to record the outcome of a decision. You can use record contacts nodes to record the values of specific variables in a decision. Record contacts nodes also generate response tracking codes that you can use to access or update the data generated by the record contacts node. For example, you can use record contacts nodes to record metadata for treatments or data about the paths in A/B test nodes.

A decision can contain multiple record contacts nodes. For example, your decision might have different paths for different channels, and you might want a record contacts node for each channel. If you are using an [A/B test node](#), it is recommended that you add a record contacts node below the output paths for the A/B test node.

Record contacts nodes record information when the decision in which they are included is published to and executed in a publishing destination that supports these node types. Depending on the destination type, the destination might need to be configured, or post-processing might be required to copy the information into the subject contacts database.

- SAS Micro Analytic Service destinations automatically support record contacts nodes. For decisions that are published to a SAS Micro Analytic Services destination, the record contacts node writes a record to the subject contact history. The record contains the information that you specified that you want to track, such as the data grid of treatments returned by the decision and the values of other variables. In addition, the record contacts node creates an output variable, the record contacts variable, that contains a response tracking code. An application, such as a customer service application, can use this response tracking code to add additional data to the subject contacts history.
- For decisions that are published to container destinations, you must configure an intermediate database. The record contacts node writes a record to your intermediate database. You must define a script to copy the records from your intermediate database to the subject contacts database. For more information, see [“Configuring Support for Record Contacts Nodes in Container Destinations” in SAS Intelligent Decisioning: Administrator’s Guide](#).
- For destinations of other types, the record contacts node does not write a record to the subject contact history. Instead, the information that you specified that you want to track is written to a record contacts variable. The name of this variable is based on the name of the record contacts node: `rt_node-name`. This

variable is a character string, and it also contains the response tracking code. You can issue a POST request to post the data to the subject contact history. The POST request type is `application/vnd.sas.decision.subject.contact+json`. For more information, see the [Subject Contacts API](#) documentation and “Update Subject Contact History” in *SAS Intelligent Decisioning: Decision Management REST API Examples*.

Add a Record Contacts Node

- 1 On the **Decision Flow** tab, select , and drag the **Record Contacts** object onto the diagram where you want to add it. SAS Intelligent Decisioning opens the **Properties** pane for the node.

Alternatively, on the **Decision** tab, select **Edit** ⇒ **Add** ⇒ **Record contacts**. If an object in the decision is selected, select **Edit** ⇒ **Add below** ⇒ **Record contacts**.

- 2 (Recommended) Rename the record contacts node. Record contacts node names must be unique within the decision. To rename the node:
 - a On the **Decision Flow** tab, click  for the node, and select **Rename**. Alternatively, on the **Decision** tab, select the node, and select **Edit** ⇒ **Rename**. The Rename window appears.
 - b Enter a new name and click **Rename**.

TIP When a decision is executed in a destination other than a SAS Micro Analytic Service destination, the record contacts node writes information to a variable. The name of this variable is based on the name of the record contacts node: `rt_node-name`. If you define a variable with this name on the **Variables** tab, the data in your decision variable will be overwritten by the record contacts node. To avoid this, rename either the record contacts node or the decision variable.

- 3 Click  to edit the properties of the record contacts node.
- 4 (Optional) Select the variable that contains the channel information to which you want to attribute the contact data that is recorded by the node.
- 5 (Optional) Click  on the **Variables to track** field, and select the variables whose values you want to record.

Note: You cannot track data grids in the **Variables to track** field because of the amount of space required to store the data. For character variables, SAS Intelligent Decisioning stores only the first 4000 characters of the value.

- 6 (Optional) Select **Track treatments** if you want to record which treatments are sent to the calling application. Then, select the data grid variable for the specific set of treatments that you want to track.

TIP If you specify a subject ID or subject level in the decision properties, this information is included in the records that are generated for the record contacts node. If you do not specify the subject ID or subject level, the subject ID or subject level is recorded as **NONE**.

- 7 (Optional) Clear the **Record rule-fired data** check box if you do not want to record the rule-fired information for all rules in the decision up to the point at which the record contacts node is included.

Note: This rule-fired data is recorded in the subject contact history. The rule-fired data that is recorded when you select **Record rule-fired data** when you publish a rule set or decision is recorded in the ruleFiredFlags column in the output table.

- 8 (Optional) Clear the **Record path tracking** check box if you do not want to record the path-tracking information for all nodes in the decision up to the point at which the record contacts node is included.

Note: This path-tracking data is recorded in the subject contact history. The path-tracking data that is recorded when you select **Record path tracking** when you publish a decision is recorded in the pathID column in the output table.

- 9 (Optional) Clear the **Include in contact policy** check box if you do not want this contact record included in aggregate reports for the channel.
- 10 Click **OK** to save your changes.

Adding A/B Test Nodes

Using A/B Test Nodes

You can use an A/B test node to direct input records into a champion path or a challenger path for testing. Each path includes one or more objects such as rule sets, treatment groups, models, or other objects.

There are two types of A/B test nodes:

- Champion/Challenger node types have one champion path and zero or more challenger paths. You specify the percentage of the input records that are processed by each challenger path. The champion path receives the remainder of the input records after the percentages for all of the challenger paths have been met. SAS Intelligent Decisioning calculates the percentage for the champion path. The percentage of the input records that are processed by the champion path is at least as large as the smallest challenger path. The total of all of the percentages, including the champion path, must equal 100%.

- Challenger/Challenger node types do not have a champion path. This node type has only challenger paths. For challenger/challenger node types, SAS Intelligent Decisioning automatically assigns the same percentage of the input records to each path.

You can specify a numeric input variable as a path key. Enabling a path key ensures that all of the input records for the same values of the key variable are processed consistently. When you specify a path key, the outgoing path for each input record is chosen deterministically by using the value of the path key as a seed. This calculation ensures that the same outgoing path is chosen for each record that has the same value of the path key variable. For example, if the path key variable is `Customer_ID`, then each input record that contains the same `Customer_ID` value is processed through the same outgoing path. If you do not specify a path key, the output path for each record is chosen randomly.

It is recommended that you add a record contacts node to the decision below the output paths for the A/B test node. When you add a record contacts node to a decision that includes an A/B test node, SAS Intelligent Decisioning adds an **abTests** field to the record contacts data. This field includes the ID of the A/B test node that generated the record contacts data, and the name and ID of the path that processed the input record, plus additional data. Record contacts nodes record this information as described in [“About Recording Contacts” on page 410](#).

Note: Record contacts nodes record all of the information that is available to them. The information that is tracked depends on the other objects in the decision and where in the decision the record contacts node occurs. For example, if the record contacts node appears in the decision below two A/B test nodes, the record contacts node records the information from both A/B test nodes.

For more information, see the following topics:

- [“Adding Record Contacts Nodes” on page 410](#)
- [Subject Contacts REST API](#)
- [“Update Subject Contact History” in SAS Intelligent Decisioning: Decision Management REST API Examples](#)

Add an A/B Test Node

- 1 On the **Decision Flow** tab, click , and then drag the **A/B Test** object type onto the diagram where you want to add it. The New A/B Test window appears.
- 2 Enter a name for the A/B test node if you do not want to use the default name. A/B test node names must be unique within the decision. Node names are limited to 255 characters.
- 3 Select the [A/B test node type](#), and click **OK**. SAS Intelligent Decisioning adds the node to the decision and opens the node's properties pane.

For Champion/Challenger node types, SAS Intelligent Decisioning adds two outgoing paths to the node: one path for a champion flow and one path for a

challenger flow. The first path listed in the **Output paths** table is assigned the role of champion. This path is identified by the  icon and is assigned a default percentage (sample size) of 50%. Challenger paths are identified by the  icon.

For Challenger/Challenger node types, SAS Intelligent Decisioning adds two outgoing paths. Each path is a challenger path. None of the paths are identified by icons or assigned sample size percentages.

TIP If you change the type of an existing A/B test node in the node's properties pane, any objects that have been added to an output path are removed.

- 4 (Optional) To specify an input variable as the path key, toggle the **Path key** switch in the **Properties** pane to enable the **Input variable** field, and then select a numeric input variable. For more information, see [“Using A/B Test Nodes” on page 412](#).
- 5 (Optional) Rename the output paths if you do not want to use the default names. Path names are limited to 255 characters.
- 6 For a Champion/Challenger node type, enter the percentage (sample size) that you want to assign to each challenger path. The sum of all of the percentages must equal 100%. You can run the [%DCM_CALCULATE_SAMPLE_SIZE macro](#) for each path to calculate an appropriate sample size.
- 7 (Optional) Click  for each additional output path that you want to add to the node. As needed, rename and assign percentages to the additional paths.
- 8 Add one or more objects to each path in the A/B test node. For instructions, see [“Add an Existing Object” on page 405](#) and [“Create and Add a New Object” on page 407](#).

If SAS Intelligent Decisioning detects an error with an object, a red X  is displayed on the object. Click  to open the **Errors** pane for more information.

If you are using the default settings, and you have not already defined decision variables with the same name and data type as the object's input and output variables, SAS Intelligent Decisioning displays the red X  to indicate that decision variables need to be created. For more information, see [“About Decision Variables and Mapping” on page 436](#).

- 9 Add any missing variables that are needed by the new objects in the A/B Test node. Either [add the variables manually](#), or use the [Add missing variables](#) option.
- 10 (Optional) [Add a record contacts node](#) below the output paths for the A/B test node in order to record the A/B test data.

It is recommended that you add a record contacts node to the decision below the output paths for the A/B test node. When you add a record contacts node to a decision that includes an A/B test node, SAS Intelligent Decisioning adds an **abTests** field to the record contacts data. This field includes the ID of the A/B test node that generated the record contacts data, and the name and ID of the path that processed the input record, plus additional data. Record contacts

nodes record this information as described in [“About Recording Contacts”](#) on page 410.

TIP To remove an object from an output path, right-click the object and select **Remove**. To open an object in the node, right-click the object and select **Open**.

Adding Parallel Processing Nodes

Using Parallel Process Nodes

You can use parallel process nodes to process multiple decisions in parallel. This ability is especially useful when you have many decisions that require a significant amount of input and output operations or that have a lot of complex logic, and all of these decisions need to be run before a final decision is reached. For example, you might need to process several hundred treatments before you decide which treatment to offer to a customer.

All of the decisions that are added to a parallel process node must return exactly one output variable of type data grid. SAS Intelligent Decisioning automatically adds several columns to the output data grids for each parallel process node in a decision. See [“Content of the Output Data Grids”](#) on page 416 for descriptions of these fields.

You can publish decisions that contain parallel process nodes to SAS Micro Analytic Service destinations, container destinations, and SAS Cloud Analytic Services (CAS) destinations. By default, decisions in a parallel process node are processed in parallel only in SAS Micro Analytic Service destinations and in container destinations. In CAS destinations, the decisions in the node are always processed sequentially.

In container destinations and in SAS Micro Analytic Service destinations, the configuration option `sas.decisions.masParallelProcessNodesRunSerially` controls whether the decisions in a parallel process node are run sequentially or in parallel. By default, this option is turned off, and the decisions are run in parallel. However, you might want to turn this option on for debugging purposes if a decision does not run correctly in parallel (in multiple threads). For more information, see [“`sas.decisions.masParallelProcessNodesRunSerially`”](#) in *SAS Intelligent Decisioning: Administrator's Guide*.

To successfully execute the decisions in parallel process nodes, you must configure the time-out limit and the maximum number of threads for the destination to which you publish decisions that contain the parallel process nodes. For more information, see [“Configuring Support for Parallel Process Nodes”](#) in *SAS Intelligent Decisioning: Administrator's Guide*.

Rules for Using Parallel Process Nodes

- A parallel process node can contain only decisions. You cannot add other object types such as rule sets or code files.
- The same decision cannot be added to the same parallel process node more than once.
- None of the decisions or their subdecisions that are added to a parallel process node can contain parallel process nodes.
- Parallel process nodes cannot be included in any subdecisions.
- All of the decisions that are added to a parallel process node must return exactly one output variable of type data grid. For all of the decisions that are included in the same parallel process node, input variables that have the same name must have the same data type, and the columns in the output data grids that have the same name must have the same data type. All of the input variables in the decisions and in their subdecisions must be input-only variables; they cannot be both input and output variables. For more information about variable data types and about input or output designations, see [“The Properties of a Variable” on page 395](#).

Content of the Output Data Grids

After you add at least one decision to a parallel process node, SAS Intelligent Decisioning adds the columns that are listed in [Table 13.1 on page 416](#) to the output data grid for the node.

The output data grid for a parallel process node contains the output records that are produced from each decision in the node that runs successfully and that generates output. If a decision runs successfully but does not generate output, the output data grid for the node does not contain any rows for that decision. If a decision fails to execute successfully, then the output data grid for the parallel process node contains at least one row for that decision, even if that decision did not generate output. The PPN_DECISION_ERROR_ORIGIN and PPN_DECISION_EXEC_STATUS columns in the row provide information about why the decision failed.

Table 13.1 Columns Automatically Added to Each Output Data Grid

Column Name	Data Type	Description
PPN_NODE_NAME	Character	The name of the parallel process node that generated the row in the data grid.
PPN_DECISION_NAME	Character	The name of the decision in the node that is specified in

Column Name	Data Type	Description
		the PPN_NODE_NAME column.
PPN_DECISION_VERSION	Character	The version of the decision that is specified in the PPN_DECISION_NAME column.
PPN_DECISION_URI	Character	The URI of the decision version.
PPN_DECISION_ERROR_ORIGIN	Character	Identifies when the error occurred. See Table 13.2 on page 417 for descriptions of the contents of this column.
PPN_DECISION_EXEC_STATUS	Decimal	The status or return code that was returned by the decision that is identified in the PPN_DECISION_NAME and PPN_DECISION_VERSION columns. This column contains a 0 if the decision executes successfully. This column contains any errors that were returned by the decision, a 404 return code if the results are missing, or a 500 return code for other errors. If the decision times out, this column contains 1 (timed out) or 2 (canceled).

Table 13.2 Error Origin Values

PPN_DECISION_ERROR_ORIGIN	Description
masparallelexec call setup	An error occurred while code was being generated for the node. (internal error)
masparallelexec execute	An error occurred while the generated code was executing.
masparallelexec processing results	An error occurred while code was being generated for the node. (internal error)

PPN_DECISION_ERROR_ORIGIN	Description
masparallelexec timeout	The time-out limit was reached before the code finished executing.
<i>empty</i>	The decision ran successfully.

Mapping Variables in Parallel Process Nodes

When you add a parallel process node to a decision, the default name of the node is PPN_OUT_*n*, where *n* is the number of the parallel process node. SAS Intelligent Decisioning creates a decision variable named PPN_OUT_*n*. Unless you created custom variables with the name PPN_OUT_*n*, the name of the decision variable matches the name of the parallel process node. If that name is not unique, then SAS Intelligent Decisioning increments the number in the decision variable name.

SAS Intelligent Decisioning also creates a placeholder output variable named PPN_OUT for the node. This variable is used to represent the collection of all of the data grids that are produced from all of the decisions in the parallel process node. SAS Intelligent Decisioning automatically maps the PPN_OUT node variable to the PPN_OUT_*n* decision variable.

You cannot modify or delete the node's PPN_OUT variable, but you can change the decision variable that is mapped to it.

Add a Parallel Processing Node

- 1 On the **Decision Flow** tab, select , and drag the **Parallel process** object onto the diagram where you want to add it. SAS Intelligent Decisioning opens the **Properties** pane for the node.

Alternatively, on the **Decision** tab, select **Edit** ⇒ **Add** ⇒ **Parallel process**.

SAS Intelligent Decisioning automatically defines an output variable named PPN_OUT for the object.

A decision that includes an object must define decision variables to which you can map the object's variables. If a decision does not already have a variable to which the object's variables can be mapped, then SAS Intelligent Decisioning displays a red X  on the object to indicate that decision variables need to be created. See [“Mapping Variables in Parallel Process Nodes” on page 418](#) and [“About Decision Variables and Mapping” on page 436](#) for more information.

- 2 You can manually add the decision variables on the **Variables** tab, or you can click  in the toolbar, and select **Add missing variables** to add any missing variables to the decision. This action creates a decision variable named PPN_OUT_*n* and maps the object's PPN_OUT variable to the decision's PPN_OUT_*n* variable. For more information, see [“Add Missing Variables”](#).

- 3 On the **Decision Flow** tab, click  to open the **Properties** pane
Alternatively, on the **Decision** tab, click  to open the Properties window, and select the **Properties** tab.
- 4 Add one or more decisions to the node. For each decision that you want to add:
 - a Click .
 - b In the Choose an Item window, select the decision that you want to add, and click **OK**.

Note: See [“Rules for Using Parallel Process Nodes” on page 416](#) for criteria for adding a decision to a parallel process node.

- c In the **Properties** pane, select the version of the decision that you want to use.

For each new decision that you add to the parallel process node, SAS Intelligent Decisioning displays a red X  to indicate that decision variables need to be created.

TIP To remove a decision from the parallel process node, select the decision in the list in the **Properties** pane, and click . To open a decision in the list, select the decision, and click . To check out or commit a decision that is stored in a folder for which the check-out feature is enabled, select the decision in the list, click , and select **Check out** or **Commit**.

- 5 You can manually add the decision variables on the **Variables** tab, or you can click  in the toolbar, and select **Add missing variables** to add any missing variables to the decision.
- 6 Click  to open the **Input Variables** property pane, and map the node's input variables to decision variables.
For information about mapping decision variables, see [“Mapping Variables within a Decision” on page 436](#).
- 7 Click  to open the **Output Variables** property pane, and map the node's output variables to decision variables.
- 8 If you opened the Properties window from the **Decision** tab, click **Close** to close the Properties window and return to the **Decision** tab.
- 9 Click  to save the decision.

Adding Branches to a Decision

Overview of Branches

Branches enable you to add conditional logic to a decision. Depending on the branch type, a branch can have multiple outgoing paths. You can add four types of branches to a decision:

- [“About Yes/No Branches” on page 420](#)
- [“About Equals Branches” on page 424](#)
- [“About Range Branches” on page 425](#)
- [“About Like Branches” on page 426](#)

The conditions for the branch paths are evaluated in the order in which you specify them. The first path whose condition evaluates to True is taken by the executing decision. The conditions for the remaining branches are not evaluated.

You can change the labels used for branch paths in Equals, Range, and Like branches. For more information, see [“Customize Branch Path Labels” on page 429](#).

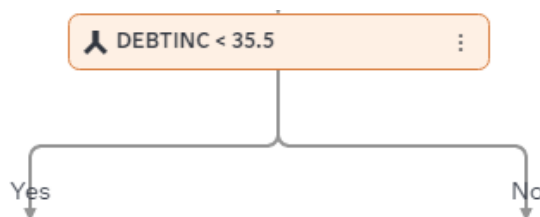
Adding Yes/No Branches

About Yes/No Branches

A Yes/No branch expression is evaluated as a Boolean expression. A Yes/No branch has a single condition and two outgoing paths: Yes (True) and No (False). The default name for a Yes/No branch is Yes/No, but you can specify a custom name.

You can enter the condition for a Yes/No branch in two ways. You can use the expression editor to enter an expression that uses multiple variables or complicated expressions. Alternatively, you can enter a simple condition by specifying a single variable, operator, and variable or value.

For example, the following branch tests whether the value of the DEBTINC variable is less than the constant 35.5. The name in the diagram is customized to display the condition that is being tested.



The following figure shows both types of property panes for this branch:

Yes/No (expression)

Name:

DEBTINC < 35.5

Branch type:

Yes/No

Branch expression: ?

DEBTINC < 35.5

Yes/No (variable)

Name:

DEBTINC < 35.5

Branch type:

Yes/No

Branch expression: ?

Variable:

DEBTINC

Operator:

<

Comparison mode:

Value

Variable or value: ?

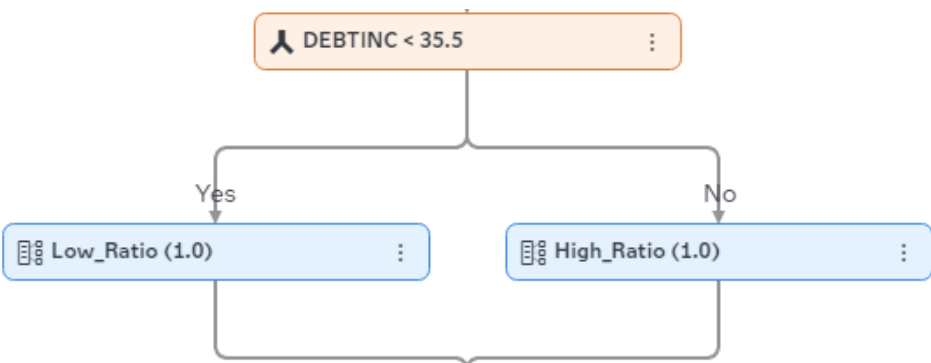
35.5

For more information, see [“Add a Yes/No Branch” on page 422](#) and [“Yes/No Branches and THEN/ELSE Clauses” on page 421](#).

Yes/No Branches and THEN/ELSE Clauses

Objects in a THEN clause on the **Decision** tab correspond to nodes in a **Yes** branch path on the **Decision Flow** tab. Objects in an ELSE clause correspond to nodes in a **No** branch path.

For example, suppose you have the following nodes on the **Decision Flow** tab:



The same nodes appear on the **Decision** tab as an IF-THEN-ELSE statement:



Add a Yes/No Branch

Note: When you add a Yes/No branch on the **Decision Flow** tab, any objects that follow the currently selected object become part of the No path. When you add a Yes/No branch on the **Decision** tab, any objects that follow the currently selected object become part of the ELSE clause.

1 Do one of the following:

- On the **Decision Flow** tab, select  and drag the **Branch** object from the list of objects onto the diagram where you want to add it.
- On the **Decision** tab, select **Edit** ⇒ **Add** ⇒ **Branch**.

The New Branch window appears.

2 (Optional) Enter a name for the branch if you do not want to use the default name. The default name is **Yes/No**.

3 Select one of the Yes/No options, and click **OK**.

- Select **Yes/No (variable)** to enter simple expressions that follow this format:

variable operator variable-or-value

- Select **Yes/No (expression)** if you want to use the expression editor to enter the branch condition. The expression editor is useful if the expression involves multiple variables or complicated expressions.

SAS Intelligent Decisioning adds the branch. On the **Decision Flow** tab, SAS Intelligent Decisioning opens the **Properties** pane. On the **Decision** tab, you must open the Properties window.

4 If you selected **Yes/No (variable)** in [Step 3](#) complete these steps:

- a If you are working on the **Decision** tab, click  to open the Properties window for the branch.
- b In the **Variable** field on the **Properties** pane or in the Properties window, select the first variable in the branch condition.

- c Select the expression operator.
 - d Select **Value** or **Variable** for the comparison mode.
 - Value**
compares the first variable to a literal value. The branch condition becomes *first-variable operator value*.
 - Variable**
compares the first variable to the value of a second variable. The branch condition becomes *first-variable operator second-variable*.
 - e In the **Variable or value** field, select a variable or enter a literal value, depending on what you selected in the previous step.
 - f If you are working on the **Decision** tab, click **Close** to close the Properties window.
- 5 If you selected **Yes/No (expression)** in [Step 3](#) complete these steps:
- a If you are working on the **Decision** tab, click  to open the Properties window for the branch.
 - b In the **Properties** pane or window, click . SAS Intelligent Decisioning opens the expression editor.
 - c Define the expression for the branch. Branch expressions can be up to 4000 characters long. See [“Using the Expression Editor” on page 535](#) for additional information.
 - d Click **Save** in the expression editor to save the expression and return to the decision editor.
 - e If you are working on the **Decision** tab, click **Close** to close the Properties window.
- 6 (Optional) Add objects to the Yes and No branch paths. Right-click on the branch node, and select either **Add to yes path** ⇒ **object** or **Add to no path** ⇒ **object**.

For more information, see the following topics:

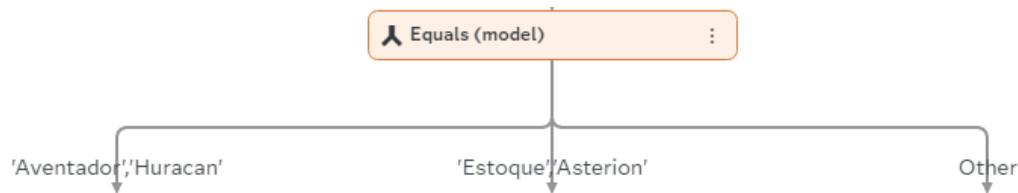
- [“Add an Existing Object” on page 405](#)
- [“Create and Add a New Object” on page 407](#)
- [“Adding Record Contacts Nodes” on page 410](#)
- [“Adding Parallel Processing Nodes” on page 415](#)
- [“Add a Micro Analytic Module” on page 432](#)

Adding Equals, Range, or Like Branches

About Equals Branches

An Equals branch compares the value of the branch expression to other variables or to literal values. By default, this branch has one outgoing path for each comparison variable or literal value, plus a branch labeled **Other** for any values that are not included in the branches that you create. However, you can combine an outgoing path with the path that immediately follows it by selecting the **OR** check box.

For example, the following branch compares the value of the character variable MODEL to the string 'Aventador', then to 'Huracan', and so on, in the order listed in the property pane. The first two comparison strings and the last two strings are combined into one path by using the OR operator.



The following figure shows the properties pane for this branch:

Properties

Name:

Equals (model)

Branch type:

Equals

Branch expression: ⓘ

model

Expression result type: ⓘ

Character

Branch Errors:

No items are available.

Paths:

+

↑

↓

🗑️

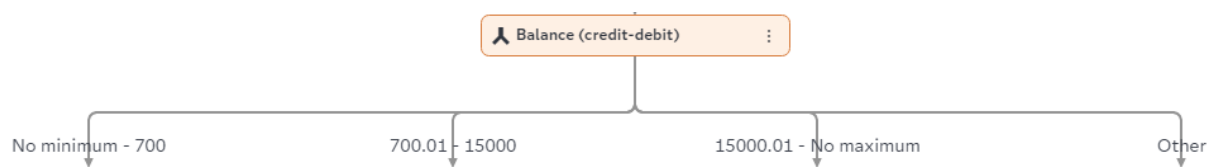
	Value	OR ⓘ
1	'Aventador' ▾	☒
	'Huracan' ▾	☐
2	'Estoque' ▾	☒
	'Asterion' ▾	☐

For more information, see [“Add Equals, Range, or Like Branches”](#) on page 427.

About Range Branches

A Range branch compares the value of the branch expression against one or more ranges of values. You can specify range values by using variables or constants. This branch has an outgoing path for each range, plus a branch labeled **Other** for any values that are not included in the branches that you create.

For example, the following branch compares the value of the expression `credit-balance` to three ranges. The first range has no minimum value, so it is treated as if the minimum is negative infinity. The third branch has no maximum value, so it is treated as if the maximum is positive infinity.



The following figure shows the properties pane for this branch:

Properties

Name:

Branch type: **Range**

Branch expression:

Expression result type:

Branch Errors: No items are available.

Paths: + ↑ ↓ 🗑️

	Minimum	Maximum
1	No mini	700
2	700.01	15000
3	15000.0	No max

[More...](#)

For more information, see [“Add Equals, Range, or Like Branches” on page 427](#).

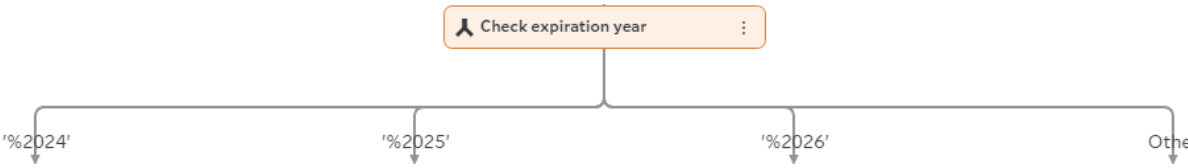
About Like Branches

A Like branch compares the value of the branch expression against one or more strings by using the LIKE operator. In LIKE expressions, you can use the underscore (.) and percent (%) characters as wildcards. For more information, see [“Using the LIKE Operator” on page 60](#).

By default, this branch has one outgoing path for each comparison string, plus a branch labeled **Other** for any values that are not included in the branches that you create. You can combine paths by selecting the **OR** check box.

For example, suppose your input data contains a character variable named EXPDATE whose value is a date. The variable values might be in any of several formats: 04/02/2022, April 2, 2022, or 02APR2022. You are interested in only the

year, not in the day or month. The following Like branch determines whether the last four characters of the variable EXPDATE end with “2022”, “2023”, or “2024”.



The following figure shows the properties pane for this branch:

Properties

Name:

Check expiration year

Branch type:

Like

Branch expression: ⓘ

expirationDate

Branch Errors:

No items are available.

Paths:

Value

OR ⓘ

1

'%2024'

☐

2

'%2025'

☐

3

'%2026'

☐

[More...](#)

For more information, see “Add Equals, Range, or Like Branches” on page 427.

Add Equals, Range, or Like Branches

- 1 Select , and drag the **Branch** object from the list of objects onto the diagram where you want to add it. The Create New Branch window appears.
- 2 (Optional) Enter a name for the branch.

- 3 Select **Equals**, **Range**, or **Like**, and click **OK**. SAS Intelligent Decisioning adds the branch and, if you are working on the **Decision Flow** tab, opens the **Properties** pane.
- 4 If you are working on the **Decision** tab, click  to open the Properties window. For more information, see [“The Decision Flow Tab versus the Decision Tab” on page 394](#).
- 5 In the **Properties** pane or window, click  to open the branch expression editor.
- 6 Define the expression for the branch. Branch expressions can be up to 4000 characters long. See [“Using the Expression Editor” on page 535](#) for additional information.
- 7 If you are defining an Equals branch or a Range branch, select the **Expression result type**.

Note: For Like branches, the result of the expression must be a character value.

- 8 For each outgoing path that you want to add to the branch node, complete the following steps:
 - a Click  to add the path.

Note: SAS Intelligent Decisioning automatically adds a path labeled **Other**.

- b Specify the values or expressions for the path.

Branch Type	Values or Expressions
Equals	Specify a comparison variable or constant of the branch variable for the path. See “About Equals Branches” on page 424 for more information.
Range	Specify a minimum value, a maximum value, or both for the branch variable. A minimum or maximum value is optional. If you do not specify a minimum value, the minimum is treated as negative infinity. If you do not specify a maximum value, the maximum is treated as positive infinity. If the minimum and maximum are the same value, the path is followed only if the branch variable is equal to that value. The minimum and maximum boundary values are inclusive. If the minimum or maximum values for any ranges are static and overlapping, SAS Intelligent Decisioning displays a warning when you validate the decision. See “About Range Branches” on page 425 for more information.
Like	Specify the LIKE expression for the path in single quotation marks. See “Using the LIKE Operator” on page 60 and “About Like Branches” on page 426 for more information.

- 9 (Optional) Click **↑** or **↓** to reorder the paths. Branch paths are evaluated in order. The decision follows the first path that evaluates to True.
- 10 (Optional) Combine branch paths. For Equals and Like branches, you can select the **OR** check box to combine a branch path with the path that immediately follows it. The application combines the conditions for the branch paths by using the OR operator.
- 11 If you are working on the **Decision** tab, click **OK** to add the branches to the decision and close the **Properties** pane.
- 12 In the decision diagram, add objects to the outgoing branch paths.

On the **Decision Flow** tab, right-click on the branch node, and select **Add to branch path** ⇒ **Branch label** ⇒ **Object type**.

On the **Decision** tab, select **Edit** ⇒ **Add to branch path** ⇒ **Branch label** ⇒ **Object type**.

SAS Intelligent Decisioning opens the appropriate window, depending on the object type.

For more information, see the following topics:

- [“Add an Existing Object” on page 405](#)
- [“Create and Add a New Object” on page 407](#)
- [“Adding Record Contacts Nodes” on page 410](#)
- [“Adding Parallel Processing Nodes” on page 415](#)
- [“Add a Micro Analytic Module” on page 432](#)

Customize Branch Path Labels

You can change the labels that are used for branch paths in Equals, Range, and Like branches.

- 1 Select the branch.
- 2 In the **Properties** pane for the branch, click **More**. The More Branching Properties window appears.
- 3 Edit the labels in the **Alternate Label** field for each branch path that you want to change, and click **Close** to close the More Branching Properties window.

TIP You can also change the branch variable and search branch path values in the More Branching Properties window.

Rename Branches

The default name for a branch is the branch type, such as "Equals" or "Yes/No". Multiple branches in the same decision can have the same name.

To rename any branch type:

- 1 Select the branch.
- 2 In the **Name field** on the **Properties** pane for the branch, click .
- 3 Enter the new name, and click **Rename**.

Alternatively, to rename an Equals, Range, or Like branch:

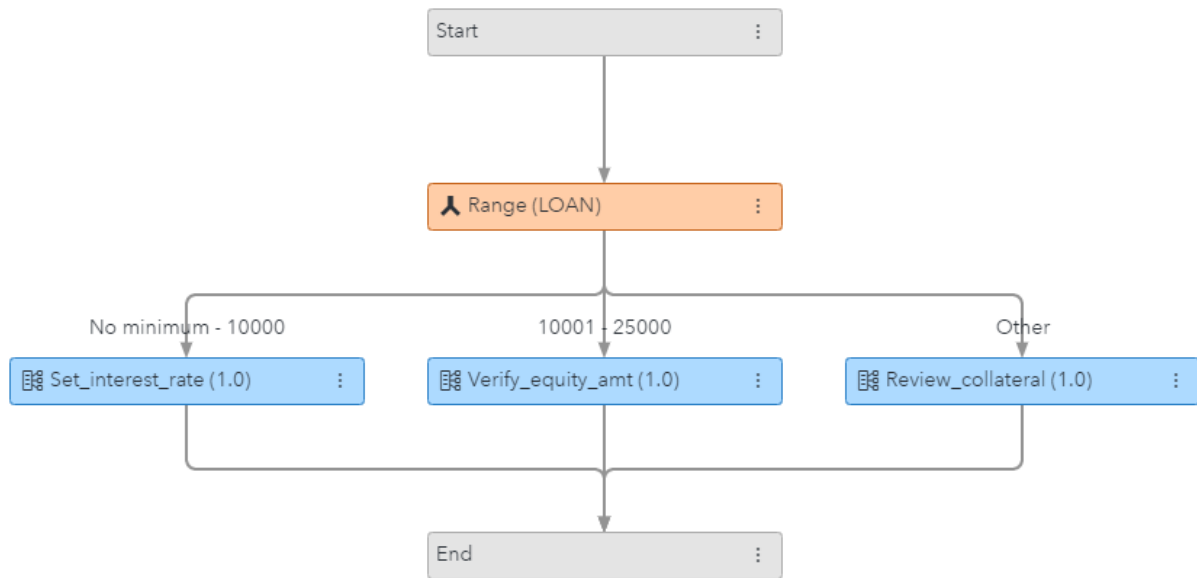
- 1 Right-click on the branch node, and select **Rename**. The Rename window appears.
- 2 Enter the new name, and click **Rename**. The maximum length is 100 characters.

Adding Cross-Branch Links

Overview of Cross-Branch Links

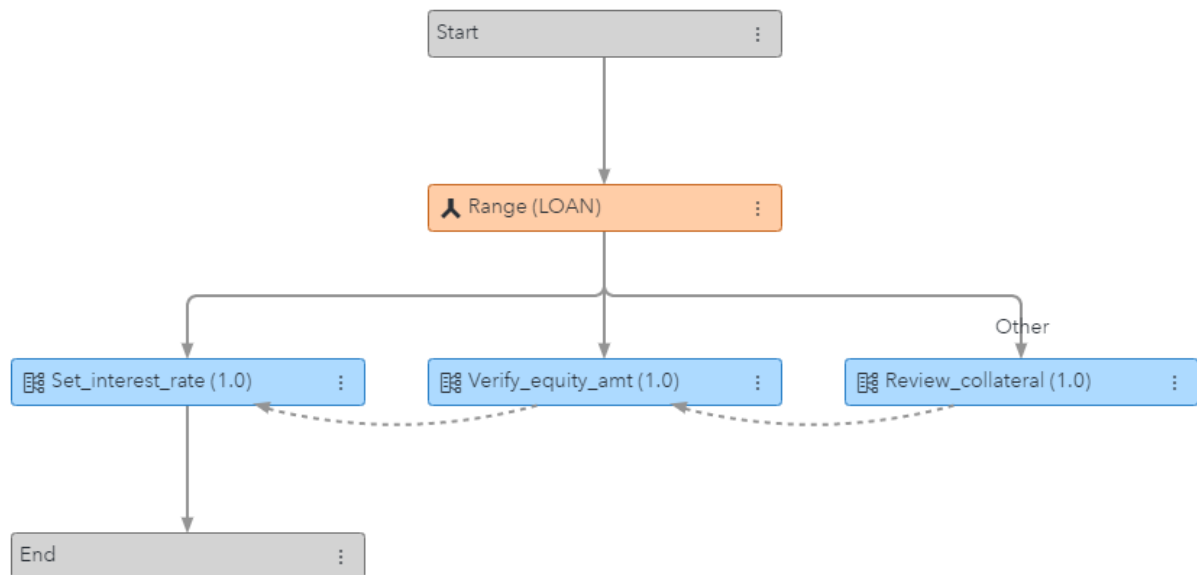
Cross-branch links enable you to easily reuse components in a decision. You can add a cross-branch link to a target node at a branch level that is the same as or lower than the level of the source node. You can add cross-branch links that ends at the End node, but you cannot add a cross-branch link that ends at the Start node or at any other node that might create an infinite loop in the decision logic.

For example, given the decision shown in the following figure, you can add cross-branch links between any of the rule sets.

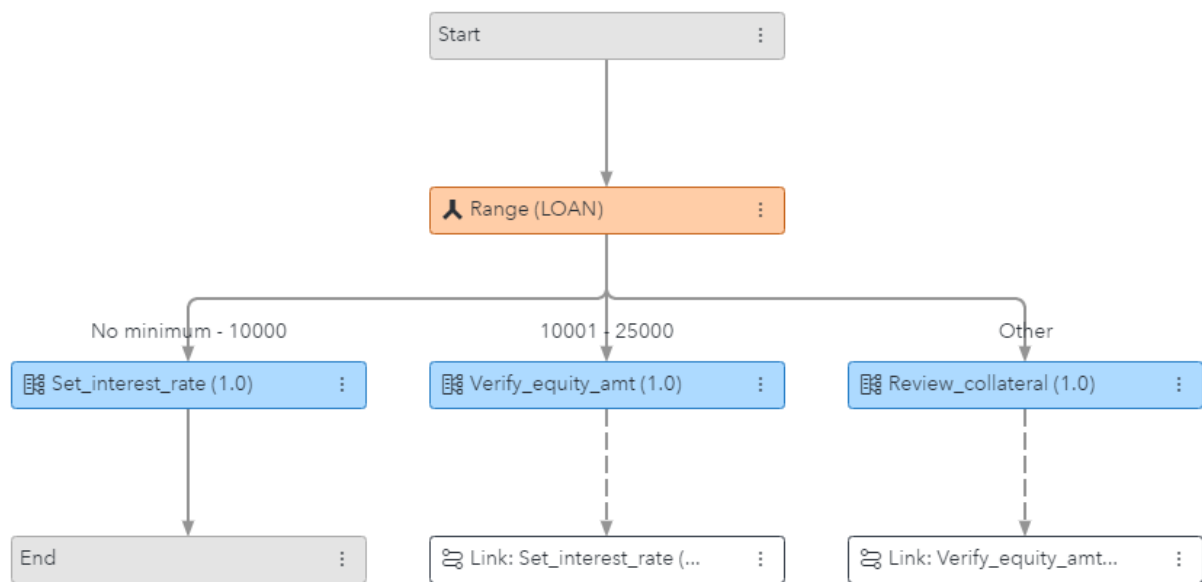


The Range branch has one path for each of three different ranges of loan values. Suppose you want all three paths to include the Set_interest_rate rule set, and you want both the mid-range and Other path to include the Verify_equity_amt rule set. You can add a cross-branch link from the Review_collateral node to the Verify_equity_amt node and a cross-branch link from the Verify_equity_amt node to the Set_inte.

Figure 13.1 Cross-Branch Links Displayed as Arcs



You can display cross-branch links as either arcs or nodes. By default, cross-branch links are displayed as arcs. To switch between the two views, click  in the upper right corner of the diagram.

Figure 13.2 Cross-Branch Links Displayed as Nodes

Add a Cross-Branch Link

Note: The source node for a cross-branch link is the node at which you want the link to start. The target node is the node where the cross-branch link ends.

- 1 Right-click the source node on the **Decision Flow** tab or select the source node on the **Decision** tab, then select **Add** ⇒ **Cross-branch link**. The New Cross-branch Path window appears. This window displays the nodes that you can link to from the current source node. (Any nodes that are not displayed would result in the possibility of an infinite loop in the decision logic if you linked to them.)
- 2 Select the target node to which you want to link, and click **OK**. SAS Intelligent Decisioning highlights the target node in the decision diagram.

TIP To change the target node of an existing cross-branch link, open the **Properties** pane for the link, click , and select a new target node.

Add a Micro Analytic Module

When you publish content to a SAS Micro Analytic Service destination, SAS Intelligent Decisioning generates code for the content and writes that code as a module in SAS Micro Analytic Service. You can use a Micro Analytic Module node in a decision to call a SAS Micro Analytic Service module. Micro Analytic Module

nodes use the MASCall package to call published modules. You can call the modules only for published decisions.

When you add a Micro Analytic Module node to a decision, the decision always uses the most recently published version of the selected module. You do not need to edit and republish the decision that includes the module in order to pick up newly published versions of the selected modules.

Note: To execute a decision that contains a Micro Analytic Module node, the setting of the SAS Intelligent Decisioning `sas.decisions.masnode.removeTrailingUnderscoresFromInput` configuration option must match the setting of the SAS Micro Analytic Service configuration option `service.removetrailingunderscoresfrominputs`.

IMPORTANT You can publish decisions that contain Micro Analytic Module nodes only to SAS Micro Analytic Service destinations and Git destinations that are compatible with SAS Micro Analytic Service.

Using the %DCM_EXECUTE_DECISION macro to execute in batch a decision that contains a Micro Analytic Module node is not supported. Execution times and performance might be unacceptable.

To add a Micro Analytic Module node to a decision:

- 1 On the **Decision Flow** tab, click , and drag the **Micro Analytic Module** object onto the diagram where you want to add it.

Alternatively, on the **Decision** tab, select **Edit** ⇒ **Add** ⇒ **Micro Analytic Module** ⇒ . If an object in the decision is selected, select **Edit** ⇒ **Add below** ⇒ **Micro Analytic Module**.

The Choose a SAS Micro Analytic Module window appears.

- 2 Click , and select the decision whose module you want to call.
- 3 Click **Browse**. SAS Intelligent Decisioning displays the list of published modules for the selected decision.
- 4 Select the module that you want to use, and then click **Select**.

For more information about MASCall packages, see [“Performing Calls between SAS Micro Analytic Service Modules”](#) in *SAS Micro Analytic Service: Programming and Administration Guide*.

Display and Export the Included Object Report for a Decision

To display the versions of the objects that are included in a decision version, click  in the **Version** column on the **Versions** tab for that decision version. The Included Objects window appears.

For assignment rule sets, filtering rule sets, nested decisions, segmentation trees, and code files, the Included Objects window lists each object and the version that is used in the decision. For treatment groups and for lookup tables, the window lists the active version. The report does not list common rule sets.

The report also includes the location of each object that is stored in a folder. Items that are not in checkout-enabled folders are identified by the icon . Items that are not found are identified by the icon . The items might have been deleted, or you might not have permission to access the items.

To export the report from the Included Objects window, click  in the **Export** button, and select either **CSV** or **PDF**. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Specifying a Subject ID Variable for A Decision

A subject ID serves two purposes:

- If you specify a subject ID variable in a decision's properties, then the application writes additional log messages if an error occurs when you run a decision test in CAS. If you publish the decision to a CAS destination and run the published decision, then the additional error messages are written to the log for the destination. If you publish the decision to a SAS Micro Analytic Service destination, and if you define the App.tk.SID.Error logger, then the additional messages are also written to the log for that destination when the published decision is run.

For the subject ID variable, specify a column whose values enable you to identify which input records are being processed when an error occurs. When an error occurs, the application writes the following string to the logs:

ERROR in package: *package_name*, SUBJECT_ID_NAME: *variable*, SUBJECT_ID_VALUE: *value*

For more information, see [“Enable Enhanced Error Reporting for Published Decisions” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- When you are working with treatments, a subject ID is a unique identifier for a specific customer. The subject ID is used to associate a specific customer with the customer’s contact history. You must select a subject ID and subject level in order to be able to retrieve or update the contact history for a customer. If you do not specify the subject ID or subject level, the subject ID or subject level is recorded as **NONE** in the subject contact history for treatments.

For more information about subjects and subject contact histories, see the following topics:

- [“About Treatments and Decisions” on page 112](#)
- [“Example: A Decision That Includes a Treatment Group” on page 113](#)
- [“Adding Record Contacts Nodes” on page 410](#)
- [“Predefined Lookup Tables” on page 146](#)

Specifying a Python Version for a Decision

You can specify that a decision that is published to a container destination uses a specific version of Python, or you can specify that the decision uses the latest version of Python that is supported by SAS Intelligent Decisioning. The version that you specify determines the Python distribution that is included in the published container. The specified distribution is used to run all Python code files and Python models that are included in the decision and its subdecisions. The Python version that is specified in the decision’s properties is not used by other destination types. For more information, see [“About Publishing Destinations and Python Distributions” on page 189](#) and [“Understanding Container Base Images” in SAS Container Runtime: Programming and Administration Guide](#).

To set the Python version for a decision, open the decision, click the **Properties** tab, and select a value in the **Python version** field. If you want the decision to always use the latest version of Python that is supported by SAS Intelligent Decisioning, select **Default version**. The number in parentheses is the current default version. The default version might change in future releases.

SAS Intelligent Decisioning displays warning messages in situations in which there is a mismatch between Python versions:

- You validate a decision that contains a subdecision or a model that specifies a different version of Python than the version that is specified for the decision. In this case, the Python version that is specified in the decision’s properties is used by the published decision. The version that is specified in the subdecisions or models is ignored.

Note: Python version numbers for decisions include only major and minor numbers. Python version numbers for models might also include patch versions. SAS Intelligent Decisioning compares only the major and minor version numbers.

- You run a scoring test for a decision that specifies a different version of Python than the version that is configured for the CAS publishing destination. In this case, scoring tests are run in CAS and use the Python version that is configured for CAS.
 - You publish a decision to a SAS Micro Analytic Service publishing destination that is configured to use a different version of Python than the version that is specified for the decision. In this case, the published decision uses the Python version that is configured for the destination instead of the version that is specified in the decision's properties.
-

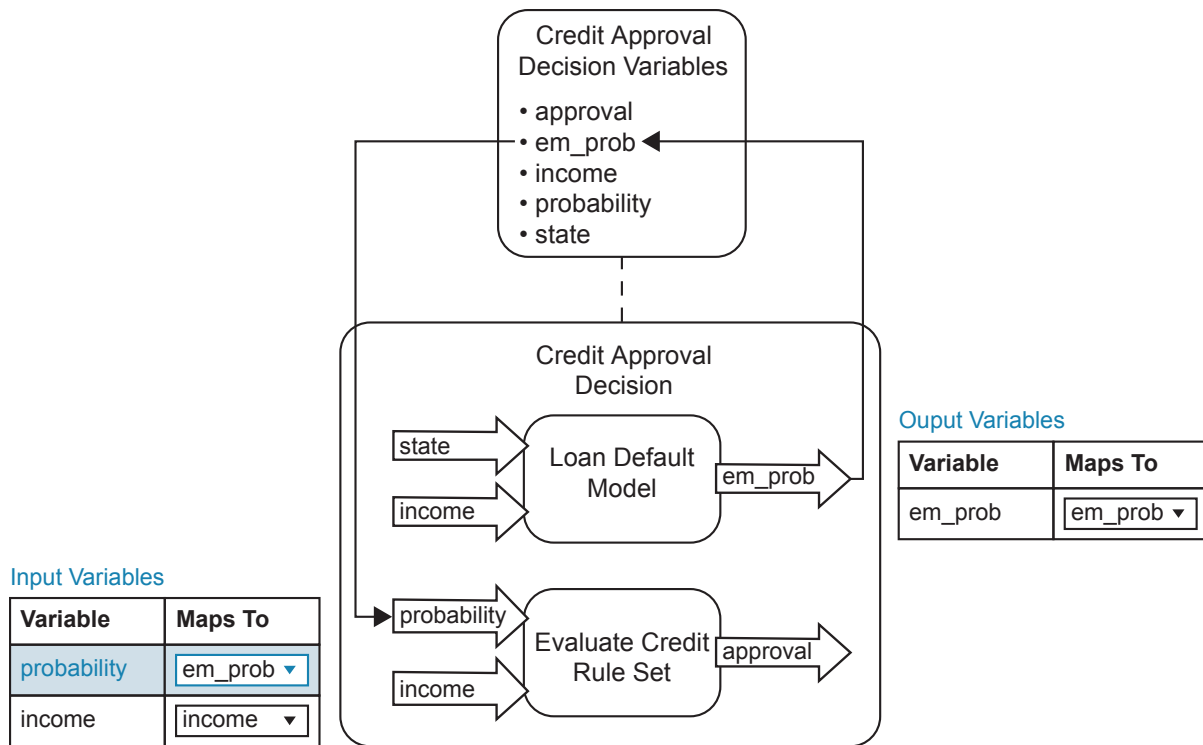
Note: These warnings are for your information only and do not affect your ability to publish the decision or to execute the published decision.

Mapping Variables within a Decision

About Decision Variables and Mapping

The objects that you add to a decision, such as models, treatment groups, and other decisions, each define their own variables. Each decision that includes an object must define decision variables to which you can map the object's variables.

For example, suppose you have a decision named Credit Approval, and this decision contains a model named Loan Default and a rule set named Evaluate Credit. The model has an output variable named `em_prob`. The value of this variable must be passed as input to the Evaluate Credit rule set, but the rule set is expecting a variable named `probability`. In order for the value to be passed to the rule set, you must map the `em_prob` output variable of the model to the `em_prob` decision variable, and you must map the `probability` input variable of the rule set to the `em_prob` decision variable.



You can create decision variables with different names, and then change the decision variable mappings on the **Input Variables** and **Output Variables** property panes for objects in the decision. For more information, see [“Input Variables, Output Variables, and Temporary Variables”](#) on page 398.

IMPORTANT When the decision is published and run in a production environment, the decision expects the input data to contain variables that have the same name and data type as the decision's input variables.

By default, if a decision does not already have a variable of the same name and data type as an object's variable, then when you add the object to a decision, SAS Intelligent Decisioning displays a red X ❌ to indicate that decision variables need to be created. You can use the **Add missing variables** option to add the missing decision variables, or you can turn on the **Create variables automatically in decisions** setting if you want SAS Intelligent Decisioning to automatically create decision variables. For information, see [“Add Missing Variables”](#) on page 438 and [“SAS Intelligent Decisioning Settings”](#) on page 6. If you turn on the **Create variables automatically in decisions** setting, SAS Intelligent Decisioning does not display the message.

Alternatively, you can create decision variables with different names, and then change the decision variable mappings in the **Input Variables** and **Output Variables** property panes for the objects in the decision.

When you add a treatment group to a decision, SAS Intelligent Decisioning creates a decision variable of type data grid with the name *group_name_out*. If the treatment group name is longer than 32 characters, then this name is truncated.

When you add a data query file to a decision, the decision variables that are created depend on whether the query selects data and on whether the file contains the

comment /* include sqlReturnInfo */. For more information, see [“Decision Variables for Data Query Files” on page 194](#).

Add Missing Variables

If the **Create variables automatically in decisions** setting is turned off, SAS Intelligent Decisioning does not automatically add decision variables when you add an object to the decision. In these cases, you can use the **Add missing variables** option to define decision variables with the same name and data type as the object's variables and add them to the decision.

- 1 Open the decision, click  and select **Add missing variables**.
- 2 In the Add Missing Variables window, select the variables that you want to add, and select whether each variable is an input variable, an output variable, both, or neither for the decision. The default input and output settings match the settings in the object, but you can change them to match how the object is used in the decision. For more information, see [“Input Variables, Output Variables, and Temporary Variables” on page 398](#)
- 3 Click **Add** to add the variables to the decision. SAS Intelligent Decisioning adds the decision variables and maps the object's variables to the decision variables.

TIP You can turn on the **Create variables automatically in decisions** setting if you want SAS Intelligent Decisioning to automatically create decision variables. For more information, see [“SAS Intelligent Decisioning Settings” on page 6](#) and [“Mapping Variables within a Decision” on page 436](#).

If you change a setting, you must sign out and sign back in to SAS Intelligent Decisioning in order for the changes to take effect.

Scoring Rows in a Data Grid

Data grid variables are processed like any other character variable unless you do one of the following:

- use data grid functions to process the individual data values within the data grid. For more information, see [“Using Data Grid Functions” in SAS Intelligent Decisioning: Using Data Grids](#).
- use the **Score rows in this data grid** option to process each row in the data grid.

IMPORTANT Do not mix these two methods for working with data grids within the same rule set. The application might return incorrect results.

To process the rows within a data grid, select the **Score rows in this data grid** option on the **Input Variables** property pane when you are mapping decision variables for the node. If you select this option, you can map a node object's variables to individual columns in the decision's data grid. SAS Intelligent Decisioning automatically changes the default variable mappings to columns in the data grid if columns exist that have the same names as the node object's variables. You can customize the variable mappings as needed. When you select this option, the decision node processes all of the rows of the data grid in the current row of the input data before execution moves to the next node in the decision. Each row of the data grid might have different values for the data grid column, so for one input record, the node object's variable might have multiple different values. For more information, see [“Mapping Data Grid Variables” on page 439](#).

If you do not select the **Score rows in this data grid** option, then you cannot map the node's data grid variable to individual columns in the decision's data grid. You must map the node's variable to the decision's data grid variable. In this case, for one input record, the node's data grid variable has a fixed value.

For an example of scoring data grid rows, see [Steps 28 and 29 of “Complete the Credit_Card_Offers Decision” in SAS Intelligent Decisioning: Tutorials](#).

IMPORTANT When the node object is a filtering rule set, and you select **Score rows in this data grid**, rows that do not meet the criteria defined by the rules are removed from the output of the data grid.

Mapping Data Grid Variables

When you add an object to a decision and the object contains a data grid variable, SAS Intelligent Decisioning creates a decision variable for the data grid in the same way that it creates decision variables for object variables of other data types. When you select **Score rows in this data grid** for an object that uses a data grid, you can choose to map the columns in the object's data grid variable either to columns in the decision's data grid variable or to other decision variables. In the lists of variables in the **Input Variables** and **Output Variables** property panes, the decision's scalar variables are identified by the  icon, and the decision's data grid columns are identified by the  icon.

For more information, see [“Map Object Variables to Decision Variables” on page 439](#) and [“Scoring Rows in a Data Grid” on page 438](#).

Map Object Variables to Decision Variables

On the **Decision Flow** tab, complete these steps:

- 1 Select the object whose variables you want to map.

- 2 (Optional) Map the object's input variables to decision variables.
 - a Click  to open the **Input Variables** property pane.
 - b (Optional) Select **Score rows in this data grid**, and select the name of the data grid.

Note: This option appears only for objects that process data grid variables. For more information, see [“Scoring Rows in a Data Grid” on page 438](#).

 - c For each object variable in the **Input Variable** column, select the decision variable that you want to map it to in the **Maps To** column.
- 3 (Optional) Map the object's output variables to decision variables.
 - a Click  to open the **Output Variables** property pane.
 - b If the selected node is a treatment group, select the data grid that contains the outcome of the treatment group as the value of the **Treatment outcome maps to** field.
 - c For each object variable in the **Output Variable** column, select the decision variable that you want to map it to in the **Maps To** column. For information about mapping variables for data query files, see [“Decision Variables for Data Query Files” on page 194](#).

On the **Decision** tab, complete these steps:

- 1 Select the object for which you want to map variables.
 - 2 Click  to open the Properties window.
 - 3 (Optional) Map the object's input variables to decision variables.
 - a Select the **Input Variables** tab.
 - b (Optional) Select **Score rows in this data grid**, and select the name of the data grid.

Note: This option appears only for rule sets, models, and decisions that process data grid variables. For more information, see [“Scoring Rows in a Data Grid” on page 438](#).

 - c For each object variable in the **Input Variable** column, select the decision variable that you want to map it to in the **Maps To** column.
- 4 (Optional) Map the object's output variables to decision variables.
 - a Select the **Output Variables** tab.
 - b If the selected node is a treatment group, select the data grid that contains the outcome of the treatment group as the value of the **Treatment outcome maps to** field.

- c For each object variable in the **Output Variable** column, select the decision variable that you want to map it to in the **Maps To** column. For information about mapping variables for data query files, see [“Decision Variables for Data Query Files” on page 194](#).
- 5 Click **Close** to close the Properties window.

Editing the Objects in a Decision

Open an Object from within a Decision

On a node in the **Decision Flow** diagram, click **Open** on the **Property** pane for the object. Alternatively, right-click the object that you want to open, and select **Open**.

On the **Decision** tab, click  for the object that you want to open. Alternatively, select the object, click **Edit**, and select **Open**.

The object opens in the appropriate editor. Models open in SAS Model Manager if you have access to that application.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

When you object an object on the **Decision Flow** tab, the decision name at the top of the window is replaced with a breadcrumb. When you close an object, SAS Intelligent Decisioning returns to the previous object in the breadcrumb. You can click on any object in the breadcrumb to switch to that object. Any object that appears later in the breadcrumb is automatically closed. For example, if you open version one of the credit_offers decision, the card_approval nested decision, and the get_credit_data code file, the breadcrumb appears as follows:

Decisions > credit_offers (1.0) > card_approval (1.0)

You can click **Decisions** in the breadcrumb to return to the Decisions category view.

Edit the Properties or Variable Mappings of a Decision Node

On the **Decision Flow** tab, select the decision node, and click one of the icons to the right of the decision diagram. The icons that are available and the properties that you can modify depend on the node's object type.

- To modify input variable mappings, click  to open the **Input Variables** pane.
- To modify output variable mappings, click  to display the **Output Variables** pane.
- To modify other properties, click  to display the **Properties** pane.
- To associate comments with the object, click .

Note: To edit the properties of a cross-branch link that is displayed as an dashed line, click on the dashed line to open the **Properties** pane.

On the **Decision** tab, click  for the node whose variable mappings or other properties you want to edit.

Reorder Objects

On the **Decision Flow** tab, you can drag rule sets, branches, models, treatment groups, nested decisions, and code files from one position to another. You cannot move the nodes for cross-branch links, but you can move the target node of a cross-branch link.

On the **Decision** tab, to move an object up or down, including into and out of conditions, select the object and click  or . You can also move the selected object by using Shift +  and Shift + .

Remove an Object from a Decision

On the **Decision Flow** tab, click  on the object that you want to remove, and select **Remove**. To delete a cross-branch link that is displayed as a dashed line, right-click on the dashed line and select **Remove**.

On the **Decision** tab, click  on the object that you want to remove.

Replace an Object

You can replace an object with another object of the same type.

On the **Decision Flow** tab, click  to open the **Properties** pane for the node object that you want to change. In the **Properties** pane, click , and select a replacement object.

On the **Decision** tab, click  for the node whose object you want to change. In the window that appears, select the new object, and click **OK**.

Update Decisions to Use New Object Versions

You can update the versions of any of the objects that are used in a decision.

- 1 Open the decision, click , and select **Update object versions**. For each object that is included in the decision, SAS Intelligent Decisioning determines whether newer versions of the objects are available, and it displays the results in the Update Versions window.
- 2 (Optional) In the Update Versions window, select the objects that you want to update, or select **Select all** to update all of the objects.
- 3 Click **Update** to update the object versions that are used in the decision.

TIP For information about upgrading multiple decisions that use the same object, see the following topics:

- [“Upgrade Decisions to Use a New Version of a Rule Set” on page 76](#)
- [“Upgrade Decisions to Use a New Version of a Code File” on page 216](#)
- [“Upgrade Decisions to Use a New Version of a Segmentation Tree” on page 351](#)
- [“Upgrade Decisions to Use a New Version of a Nested Decision” on page 461](#)

Validate a Decision in the Decision Editor

Note: If the `sas.intelligent.decisioning.preventCommitWithErrors` configuration option is turned on, you must correct any errors that are found by the validation process before you can commit a decision that has been checked out. For information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

The following list describes some of the items that SAS Intelligent Decisioning checks when you validate a decision:

- All of the rule sets that are used in the decision define one or more rules, and the rule sets do not contain errors, such as missing required values or branch values that overlap.
- [The model type of each model that is used in the decision is a supported type](#), SAS Intelligent Decisioning can retrieve the DS2 code for the model, and the model defines a method named SCORE.
- [Treatment groups](#), [lookup tables](#), and [global variables](#) that are used in the decision have been activated.
- The objects that are specified in each of the decision nodes exist, and the current user has permission to access the objects.
- [Object variables are mapped to decision variables on page 436](#).
- All output paths for A/B test nodes and decision branches have names, and the names are unique. All output paths for A/B test nodes specify a sample size percentage, and all output paths for decision branches specify an expression.
- For Range branches, the branch values evaluate to numeric values, each range specifies both a minimum value and a maximum value, and the ranges do not overlap.
- Parallel process nodes contain only decisions, each decision has only one output variable, and the output variable is a data grid. All decisions in the same parallel process node define the same columns in their output data grid.

To validate a decision in the decision editor, save any changes that you have made to the decision. Then, on the **Decision Flow** or **Decision** tab, click **Validate**. SAS Intelligent Decisioning validates everything that can be determined without executing the decision.

- 1 Open the decision, or if the decision is already open, save any changes that you have made to the decision.
- 2 On the **Decision Flow** or **Decision** tab, click **Validate**.

- 3 On the **Decision Flow** tab, click  to open the **Errors** pane to check for error messages. Errors are categorized by error type. For some errors, you can click **Save** as you fix them to save your changes and update the **Errors** pane. For other errors, such as errors in nested decisions or activation errors, you must click **Validate** again to clear the error message.

If the validation finds errors, you can click **Export** in the **Errors** pane to export the list of errors to a PDF file.

TIP If you routinely validate very large decisions, increasing the number of threads available to the validation process can improve performance. For more information, see the [sas.decisions.validation.validationCoreThreadPoolSize](#) configuration option.

Note: If the validation process returns an error that indicates that a license has expired, you can use the [Model Publish API](#) to publish the decision. See also “Publish a Decision to the maslocal Destination” in *SAS Intelligent Decisioning: Decision Management REST API Examples* for an example of using Python code to publish a decision.

Copy a Decision URL

To create a link for external documentation that automatically opens a decision in SAS Intelligent Decisioning, open the decision, click , and select **Copy decision URL**. SAS Intelligent Decisioning copies the URL into your paste buffer.

Paste the link into your documentation.

Compare Decision Content

You can compare the contents of two different decisions, or you can compare the contents of two different versions of the same decision.

- 1 Select the objects that you want to compare.
 - To compare the contents of two different decisions, select the decisions in the category view, click , and select **Compare object contents**.
 - To compare the contents of two versions of the same decision, open the decision, click  on the **Versions** tab, and select **Compare object contents**.

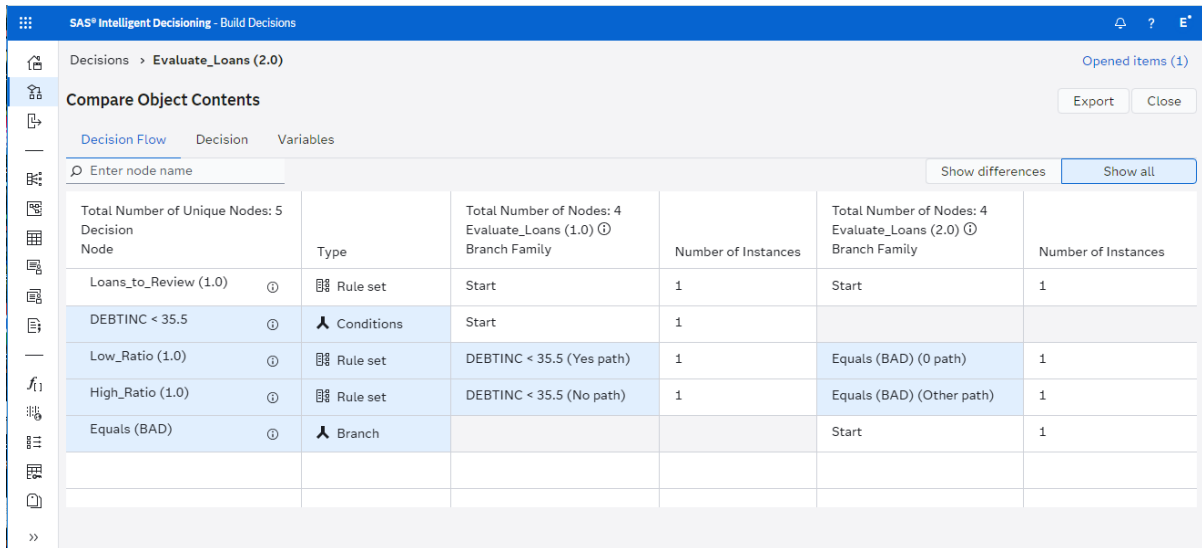
The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Object Contents window.

By default, the **Decision Flow** tab displays all of the nodes in each object and highlights the differences. Click **Show Differences** to limit the display to only the nodes that are different in each object. Click **Show All** to return to the default view.

For example, in the following figure, both version 1.0 and version 2.0 of the Evaluate_Loans decision contain the same rule sets, but the branches are different. Version 1.0 uses a Yes/No branch that is based on the value of the DEBTINC variable, and Version 2.0 uses an Equals branch that is based on the value of the BAD variable. In Version 1.0, the Low_Ratio rule set is in the Yes branch path, and in Version 2.0, the Low_Ratio rule set in the BAD=0 path.

TIP Click ⓘ beside a decision name to display its location.



Total Number of Unique Nodes: 5 Decision Node		Type	Total Number of Nodes: 4 Evaluate_Loans (1.0) ⓘ Branch Family		Number of Instances	Total Number of Nodes: 4 Evaluate_Loans (2.0) ⓘ Branch Family		Number of Instances
Loans_to_Review (1.0) ⓘ	ⓘ	Rule set	Start		1	Start		1
DEBTINC < 35.5 ⓘ	ⓘ	Conditions	Start		1			
Low_Ratio (1.0) ⓘ	ⓘ	Rule set	DEBTINC < 35.5 (Yes path)		1	Equals (BAD) (0 path)		1
High_Ratio (1.0) ⓘ	ⓘ	Rule set	DEBTINC < 35.5 (No path)		1	Equals (BAD) (Other path)		1
Equals (BAD) ⓘ	ⓘ	Branch				Start		1

Note: If both objects or both versions contain the same node, and if the **Score rows in this data grid** option is set for the node in one object or version but not for the node in the other, then the node is included in the list of differences.

- 3 (Optional) Click the **Variables** tab to display the properties of the variables in the two objects. This tab displays the name, type, initial value (if a value is defined), and length of each variable in both objects. The tab also indicates whether the variable is an input variable or an output variable. If the variable list for both objects is the same, this tab displays a message stating that the two objects are identical.

If either of the objects that you are comparing contain a data grid variable, you can click ⓘ beside the variable name to display the data type and length of each data grid column.

- 4 (Optional) Click **Export** to export the results of the comparison to a PDF file. The Export Comparison Results window appears.

- 5 (Optional) Select the information that you want to export, and click **Export**. To export the information on a specific tab, select the tab name. To export the data type and length of each column in data grid variables, select **Data grid metadata**. If you want the PDF file to display only the differences between the two objects, select **Show differences**. To display all of the objects' information, select **Show all**.

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Compare Decision Code

You can compare the generated code of two different decisions, or you can compare the generated code of two different versions of the same decision.

- 1 Select the objects that you want to compare.
 - To compare the generated code of two different decisions, select the decisions in the category view, click , and select **Compare code**.
 - To compare the generated code of two versions of the same decision, open the decision, click  on the **Versions** tab, and select **Compare code**.

The Select Versions window appears.

- 2 Select the versions that you want to compare, and click **Compare**. SAS Intelligent Decisioning opens the two objects side-by-side in the Compare Code window and highlights the differences.
- 3 (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Generate PDF Documentation for a Decision

You can generate detailed documentation for a decision as a PDF document. The PDF includes the decision properties, the properties and variable mappings for each node, and a table of the decision variables. You can download documents with

additional details about the rule sets, treatment groups, and nested decisions that are used in the decision.

- 1 Open the decision.
- 2 Click , and select **Create document**. The Create Document window appears.

TIP If this option is disabled, there might be unsaved changes. Click .

- 3 Enter a name for the document if you do not want to use the default name.
- 4 (Optional) Select **Show a diagram of the decision** to include the decision diagram in the PDF.
- 5 (Optional) Select **Include code from code nodes** to include the code from custom code files that are used in the decision.
- 6 (Optional) Select **Choose additional documents to download** to display a window from which you can download additional documents. You can download documents for any rule sets, treatment groups, and nested decisions that are used in the decision.
- 7 Click **Create**. SAS Intelligent Decisioning creates the PDF. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

If you selected the option **Choose additional documents to download**, the Download Additional Documents window appears.

- 8 (Optional) Click **Download** for each additional document that you want to download.
- 9 (Optional) Click **Close** to close the Download Additional Documents window.

Generate a Report for a Published Decision

A published version report documents the content of a published version of a decision. The report includes the decision properties, the properties of each object in the decision, and a table of the decision's variables. For each variable that is a data grid, the report includes a separate table that lists the properties of each data grid column. The report also includes the name and URI of the published object, the date on which the decision version was published, the user ID that published the version, and information about the publishing destination. The report is generated as a PDF file.

You can generate the report both on the **History** tab for a decision and in the Deployments category view.

- 1 Open the decision and click the History tab, or click  to navigate to the Deployments category view.
- 2 Select the decision, and click **Generate report**. The default file name is `decision-name_report.pdf`

The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Managing Decisions

Duplicating Decisions

About Duplicating Decisions in Folders for Which the Check-Out and Commit Feature Is Enabled

When you duplicate a decision, SAS Intelligent Decisioning first creates an empty decision, and then updates the decision to add references to the included objects. The default permissions for the Decision Repository enable users to create new objects, but the permissions do not include the ability to update the objects. In order to duplicate an object that is in the Decision Repository, you must check out the object.

If your site has created additional folders for which the check-out and commit feature is enabled, and if those folders have been assigned the same permissions as those assigned to the Decision Repository, you must also check out objects in those folders before you can duplicate them.

When you duplicate an object that is checked out, the duplicate object is not automatically checked out.

Duplicating Included Objects

When you duplicate a decision, you can choose whether to also duplicate the objects that are in the decision. If you choose not to duplicate the included objects, the duplicate decision uses the existing objects. If you choose to duplicate the included objects, the new copy of the decision uses the new copies of the included

objects. If the decision includes a nested decision (subdecision), SAS Intelligent Decisioning duplicates the objects that are in the nested decision.

Note: Any models, lookup tables, custom context files, or Micro Analytic modules that are used in the decision are not duplicated. The duplicated decision uses the original objects. If the decision includes treatment groups, the treatment groups are duplicated, but the individual treatments within the treatment groups are not duplicated. The duplicated treatment group uses the original treatments.

When you duplicate a decision and its included objects, you must enter a custom suffix that SAS Intelligent Decisioning appends to the names of the duplicated objects. See [“Duplicate Decisions” on page 450](#). All of the duplicated objects are placed in the same folder

Duplicate Decisions

To duplicate a single decision:

- 1 In the Decisions category view, select the decision that you want to duplicate.
- 2 Click  and select **Duplicate**. The Duplicate Decision window appears.
- 3 (Optional) Enter a new name for the duplicate decision. If you do not specify a new name or specify a suffix in [Step 7b](#) SAS Intelligent Decisioning appends `_Copy` to the name of the duplicate object.
- 4 (Optional) Enter a description for the decision.
- 5 Select the version of the decision that you want to duplicate.
- 6 Click  and select the location where you want to save the duplicate decision. If you select **Duplicate included objects** in the next step, the duplicated objects are also saved to the location that you choose.
- 7 (Optional) Click the **Duplicate included objects** toggle if you want to duplicate the objects that are included in the decision. Selecting this option enables the **View included objects** link and the **Name suffix** field. This option also disables the **Name** field at the top of the dialog. The duplicate decision name uses the suffix that you enter.
 - a (Optional) Click the **View included objects** link to open the **Included Objects** window. This window lists the objects that will be duplicated together with the decision. You can use the filter field to search for specific objects by name in order to verify which objects will be duplicated. Objects that are not listed will not be duplicated.

Click **Cancel** to close the window.
 - b In the **Name suffix** field, enter a custom string that you want SAS Intelligent Decisioning to append to the names of all of the duplicated objects.
- 8 Click **Duplicate**.

To duplicate multiple decisions:

- 1 In the Decisions view, select the decisions that you want to duplicate.
- 2 Click  and select **Duplicate**. SAS Intelligent Decisioning duplicates the latest version of the decisions and appends `_Copy_n` to the names of the duplicate copies.

Note: When you duplicate a decision, SAS Intelligent Decisioning creates a relationship between the original decision and the duplicate decision. If either object is changed, and you later copy the contents of one object into the other, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the decision before you paste the new content.

Delete Decisions

Note: You cannot delete a decision if it is open.

In the Decisions category view, select the decisions that you want to delete, click , and select **Delete**. SAS Intelligent Decisioning moves the decisions to the recycle bin. For more information, see [“Manage Folders and Folder Content” on page 18](#).

Rename a Decision

Note: You cannot rename a decision if it is open.

- 1 In the Decisions category view, select the decision that you want to rename.
- 2 Click  and select **Rename**. The Rename window appears.
- 3 Enter a new name for the decision, and click **Rename**.

Move Decisions to a Different Folder

- 1 In the Decisions category view, select the decisions that you want to move.
- 2 Click  and select **Move**. The Choose a Location window appears.
- 3 Select the location to which you want to move the decisions, and click **OK**.

Managing Versions of Decisions

Set the Displayed Version

The displayed version is the version whose information is displayed on the other tabs, such as the **Properties**, **Decision**, and **Variables** tabs. On the **Versions** tab, a ✓ in the **Displayed Version** column indicates the displayed version. To change the displayed version, click the version number for the version that you want to view. The displayed version is shown in the title bar.

Create a New Version

IMPORTANT The current version of an object is the version with the highest version number. When you create a new version, SAS Intelligent Decisioning locks the current version before it creates the new version. You cannot unlock or modify a locked version. However, you can [copy and paste the content of the older version](#), and modify the pasted content.

Note: For objects that are stored in locations for which the check-out and commit feature is enabled, you cannot manually create a new version. The only way to create a new version is to check out an existing version and commit a new version. For information, see [“Check Out and Commit a Decision Version” on page 465](#).

To manually create a new version:

- 1 Open the object for which you want to create a new version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 On the **Versions** tab, [click the version number](#) for the existing version that you want to use as the basis for the new version.
- 3 Click **New version**. The New Version window appears.

- 4 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented. If you select **Minor**, the number to the right of the period is incremented.
- 5 (Optional) For each version tag that you want to add, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- 6 (Optional) Enter information about the new version in the **Notes** field.

TIP You can edit these notes at any time on the **Versions** tab.

- 7 Click **Save**.

Creating a New Version by Using Tags

About Using Tags to Create a Version

If the objects in a decision are tagged with version tags, you can create a new version of the decision by using these object tags to select which version of the included objects to use in the new decision version. This feature is useful when, for example, multiple users are developing the objects that are used within a decision. Each user can assign tags to object versions that they have developed for a new decision version. The person who is creating the new decision version can then use these tags to easily create the new version.

When you use version tags to create a new decision version, the versions of the included objects are ignored. Suppose your current decision uses version 1 of a rule set that has four versions. Versions 2 and 3 of the rule set are tagged with Tag_1, but versions 1 and 4 are not tagged. If you create a new decision version by specifying the tag Tag_1, you will be able to select either version 2 or 3 of the rule set for use in the new decision version. You cannot select version 4 of the rule set because it is not tagged with Tag_1.

Note: You cannot use tags to create new versions of common rule sets. To create a new version of a common rule set, open the rule set and select **New Version** on the **Versions** tab.

Create a New Version by Using Tags

TIP Before you begin, you need to know the version tags of the objects that you want to include in the new decision.

- 1 Open the decision for which you want to create a new version.

TIP For decisions that are in folders for which the check-out feature has been enabled, your administrator must give you permission to create a new version by using tags. For more information, see [“Grant Permission to Create Versions in Check-Out Folders by Using Tags”](#) in *SAS Intelligent Decisioning: Administrator's Guide*.

- 2 On the **Versions** tab, click  and select **Create version from tags**. The Create Version From Tags wizard opens.
- 3 On the **Choose Strategy: Step 1** page of the wizard, enter or select up to two tags that are associated with object versions that you want to use to create the new decision version. Type the first letters of the existing tag to display the matching list of previously entered tags. Press Enter after each manually entered tag.
- 4 (Optional) If you want to select specific versions of each object to use in the new decision version, select **Let me choose the version of each object**.
- 5 (Optional) If you do not want to automatically map the object's variable to the decision's variables, deselect **Automatically map the object's variable to the decision's variables**.

If you select this option, new object variables are mapped to decision variables only if decision variables of the same name and type already exist. If a decision variable does not already exist, you must manually map the object variable to a decision variable. For additional information, see [“Mapping Variables within a Decision”](#) on page 436.

- 6 Click **Next**.

The node objects that have versions that are tagged with a tag that you specified are listed on the **Review and Confirm: Step 2** page of the wizard.

Note: Common rule sets and treatment groups are not listed in the wizard. To create a new version of a common rule set, open the rule set and select **New**

Version on the **Versions** tab. New decision versions use the active version of each treatment group.

-
- 7 (Optional) Select any node that you want to remove from the update process. Click , and then confirm the deletion. For any node that you remove from the update process, the existing node is used in the new decision version.

TIP You can filter the list of nodes by using the **Search** field or by clicking  to filter by more specific criteria in the Filter Nodes window.

- 8 (Optional) If you selected **Let me choose the version of each object** in Step 4 and any node object has multiple versions that are tagged with the same tag, select the version of those objects that you want to use in the new decision version. To use the latest version of all of the objects, click **Use latest version**.
- 9 Click **Next**.
- 10 On the **Create Version: Step 3** page of the wizard, do the following:
- a Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period increments. If you select **Minor**, the number to the right of the period increments.
 - b (Optional) Enter any version tags for the new version of the decision.
For each tag that you want to add, either enter a new custom tag or select the tag from the list of previously entered tags. Type the first letters of an existing tag to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').
-
- Note:** If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).
-
- c (Optional) Enter information about the new version in the **Notes** field.
 - d (Optional) To add the tags in the **Version tags** field to all of the objects that you are committing, select the **Apply tags to all selected decision components** check box.

- 11 Click **Create**.

The new version of the decision is listed on the **Versions** tab for the decision.

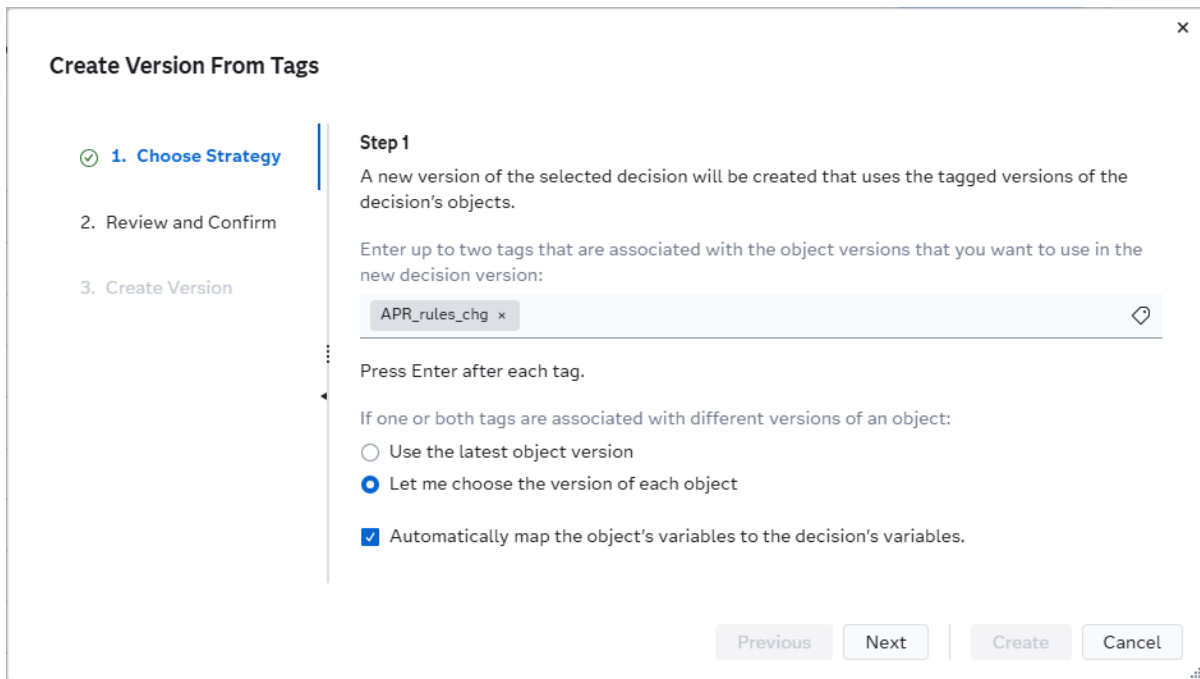
Example

Suppose you have the decision flow `Update_credit_history`. Version 1 of this decision contains the following objects:

- Version 2 of the rule set `Select_accounts`, which is not tagged. Version 1 of this rule set is tagged with `APR_rules_chg`, but this tag was added to the wrong version of this rule set. The tag should have been added to version 2 of the rule set.
- Version 3 of the decision `Calc_credit_chgs`. Both version 3 and version 4 of this decision are tagged with `APR_rules_chg`.
- Version 1 of the code file `Update_history`. This version is the only version of this code file and it is not tagged.

You need to create a new version of `Update_credit_history` that includes version 4 of the decision `Calc_credit_chgs` (the tagged version) and version 2 of the rule set `Select_accounts` (which is not tagged).

- 1 Open the decision `Update_credit_history`.
- 2 On the **Versions** tab, click  and select **Create version from tags**. The Create Version From Tags wizard opens.
- 3 On page 1 of the Create New Version wizard, specify the tag `APR_rules_chg`, and select the option **Let me choose the version of each object**.



Create Version From Tags

1. Choose Strategy

2. Review and Confirm

3. Create Version

Step 1

A new version of the selected decision will be created that uses the tagged versions of the decision's objects.

Enter up to two tags that are associated with the object versions that you want to use in the new decision version:

APR_rules_chg

Press Enter after each tag.

If one or both tags are associated with different versions of an object:

☐ Use the latest object version

☒ Let me choose the version of each object

☒ Automatically map the object's variables to the decision's variables.

Previous Next Create Cancel

- 4 Click **Next**.

Page 2 of the wizard displays a list of the objects in the decision that are tagged with the `APR_rules_chg` tag. If you do not make any changes in this list, then the

new version of the decision `Update_credit_history` will include the versions of the objects that are shown in the list. If multiple versions of an object are tagged with the same tag, the wizard includes each version in the Node Version column for that object. In this example, both versions 3 and 4 of the decision `Calc_credit_chgs` are tagged with `APR_rules_chg`. By default, the latest version, version 4, is automatically selected in the version list in the Node Version column.

The list also includes version 1 of the `Select_accounts` rule set. However, the tag `APR_rules_chg` was incorrectly added to this version of the rule set, so you do not want the new version of the `Update_credit_history` decision to use version 1 of this rule set.

- 5 Select the `Select_accounts` rule set, and click  to remove version 1 of this rule set from the list of objects that will be used in the new version of the decision `Update_credit_history` decision.

When you remove an object from the list, the original object that is used in the decision is still used in the new version of the decision.

Create Version From Tags

1. Choose Strategy

2. Review and Confirm

3. Create Version

Step 2

The new decision version will be updated to use the node versions listed below. If you do not want to update a node, remove the node from this list by selecting the node's check box and clicking the Delete icon.

Search



Use latest version

☐	Name	Type	Used ...	Node Version and...	Num...	Delete	st Modified
☒	Selec...	Rule ...	Upda...	1 .0: APR_rules_chg	1		Jul 2, 2025 ...
☐	Calc_...	Decis...	Upda...	4 .0: APR_rules_chg	2		Jul 2, 2025 ...

Previous

Next

Create

Cancel

- 6 Click **Next**.
- 7 On page 3 of the wizard, select **Major** for the version type, and then click **Create**.

1. Choose Strategy

2. Review and Confirm

3. Create Version

Step 3

Version:
2.0

Version type:
Major

Version tags:

Press Enter after each tag.

Notes:

☐ Apply tags to all selected decision components

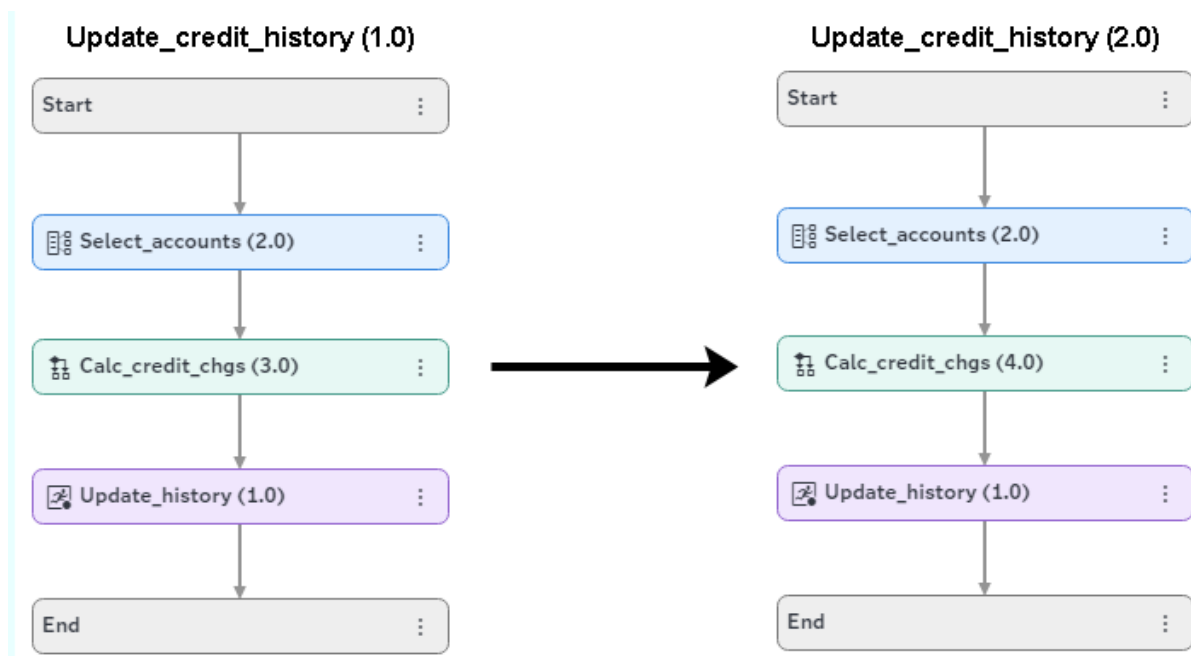
Previous

Next

Create

Cancel

SAS Intelligent Decisioning creates version 2 of the decision.



Create a New Version Automatically When You Publish a Decision

You can automatically create a new version of a decision when you publish the current version by selecting **Publish and lock**. For information, see [“Publish Decisions” on page 473](#).

Copy the Content of a Version

You can copy and paste the content of one version (the source version) into another version (the target version). The source version and the target version must be the same object type. When you paste the copied content, SAS Intelligent Decisioning creates a new version of the target object that contains only the pasted content. You can use this feature to copy and modify an older version that is locked.

You can copy the content of an object's version in the category view or on the **Version** tab for the object.

- 1 In the category view, complete these steps:
 - a Select the decision whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
 - c Select the version whose contents you want to copy.

Alternatively, on the **Versions** tab of the decision whose contents you want to copy:

- a Select the version whose contents you want to copy.
 - b Click , and select **Copy version**. The Copy Version window appears.
- 2 Click , and select the decision into which you want to paste the contents of the version.

When you paste the contents, SAS Intelligent Decisioning creates a new version of the target decision. The target object contains only the pasted content.

- 3 Select whether you want to create a new major or minor version.
- 4 (Optional) Modify the notes that will be associated with new version.
- 5 Click **Paste version**.

TIP The input and output designations for a variable on the **Variables** tab for the new version are removed, and the variable is treated as a temporary variable in the following situations:

- The new version of the target object does not use a variable that is used in an earlier version.
- The source and target object originally had a variable of the same name, and you deleted the variable from the source object before you copied it into the target object.

Note: When you copy the contents of a source object into a target object, SAS Intelligent Decisioning creates a relationship between the two objects. If the source object is modified after you copy its contents, and you later copy the contents of the target object back into the source object, SAS Intelligent Decisioning displays a warning message. Verify that you want to replace the current contents of the source object before you paste the new content.

Delete a Version

IMPORTANT When you delete a specific version, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

In order to be able to delete a specific version of an object, you must have permission to delete the object itself. For information on permissions for deleting objects, see “[Default Permissions](#)” in *SAS Intelligent Decisioning: Administrator's Guide* and “[Folder-Based Permissions](#)” in *SAS Intelligent Decisioning: Administrator's Guide*. By default, users have permission to delete objects that they create and any versions of those objects.

Note: To be able to delete specific versions of decisions, the `sas.decisions.deleteVersions` configuration option must be turned on.

On the object's **Versions** tab, select the version that you want to delete, click , and select **Delete**.

You cannot delete the current version.

Upgrade Decisions to Use a New Version of a Nested Decision

If you create a new version of a nested decision that is already used in other decisions, you can upgrade the other decisions to use the new version of the nested decision.

- 1 On the **Versions** tab for the nested decision, click , and select **Upgrade decisions**. The Upgrade Decisions window appears. This window lists all of the decisions that include the nested decision.
- 2 In the **Version to upgrade to** field, select the version of the nested decision to which you want to upgrade the decisions.
- 3 Select **Automatically map variables** if you want SAS Intelligent Decisioning to automatically map new nested decision variables in the decisions where the nested decision is used.

For information about how SAS Intelligent Decisioning maps variables, see [“About Decision Variables and Mapping” on page 436](#).

- 4 Select the check boxes for the decisions that you want to upgrade, and click **Upgrade decisions**.

TIP To update all of the objects that are used in a decision, see [“Update Decisions to Use New Object Versions” on page 443](#).

Determine Which Objects Use a Decision

To list the objects that use a specific decision:

- 1 On the **Decisions** category page, select the check box for decision, click , and select **View used by report**. The All Objects that Use the Selected Item window appears. By default, this window lists the objects that use the latest version of the selected decision.
- 2 (Optional) Select the specific version for which you want to view the Used By report.

To view the list of all objects that use any version of the selected decision, clear the value in the **Version** field.

Note: The **View used by report** option is also available from within the open object.

You can use the **Filter** field to filter the list of objects based on the object names. Click on an object name to open the object. Click ⓘ next to an object name to display the date on which the object was last modified and the ID of the user who modified the object. For objects that have been checked out, SAS Intelligent Decisioning also displays the IDs of the users that have checked out the objects. For decisions, SAS Intelligent Decisioning includes the decision's workflow status.

Click **Export** to export the Used By report to a PDF file. If you open the PDF file directly in Adobe Reader or Adobe Acrobat (not in a browser), you can click on an object name to open the object in a new browser tab.

Checking Out and Committing Decision Versions

About Checking Out and Committing Versions

Your administrator can enable the check-out and commit feature for a folder as described in [“Enable the Check-Out and Commit Feature” in SAS Intelligent Decisioning: Administrator's Guide](#).

By default, SAS Intelligent Decisioning defines a folder where you can store objects that must be checked out before they can be edited. This folder is the **Decision Repository** folder. The default permissions for this folder require that you check out a version and commit your changes to the checked-out version. You cannot edit the objects in **Decision Repository** without first checking them out.

If a version must be checked out before it is modified, the **Check Out** button appears at the top of the **Versions** tab for that object. You can check out only one copy of a version at a time. You can check out any version of an object, but you cannot commit changes to an existing version. When you commit your changes, SAS Intelligent Decisioning creates a new version with your changes.

TIP If the `sas.decisions.checkout.allowConcurrentCheckout` option is turned off, and a user has checked out a decision version, the **Check Out** button for that decision is disabled for all other users. For more information, see [“Concurrently Checking Out and Committing Decision Versions” on page 465](#).

When you check out a version, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. SAS Intelligent

Decisioning adds “(Checked Out)” to the name that is displayed at the top of the window.

While you have a version checked out, the Decisions category view shows two decisions with the same name, but the folders listed in the **Location** column differ for each decision. The original version is in the location from which you checked it out, and your checked-out copy is in your **My Folder** folder.

TIP If an object that you have checked out does not appear in the category view, click  to refresh the category view.

Note: When a decision that uses a treatment group is deployed to a production environment, the decision always uses the active version of a treatment group. When you edit a decision that uses a treatment group that you have checked out, the properties pane for the treatment group shows the location of the treatment group in the original folder. The pane does not list the location of the treatment group that is checked out because that copy is not the active version.

A **Commit** button appears at the top of the **Versions** tab for the checked-out version. When you are finished editing the checked-out version, you must commit your changes in order for other users to be able to see them. When you commit your changes, SAS Intelligent Decisioning creates a new version in your repository that includes your changes, and it deletes the working copy in **My Folder**. If you check out objects that are used in the decision and modify the checked-out decision to use the checked-out objects, SAS Intelligent Decisioning also commits the checked-out objects when you commit the decision. If the decision uses a treatment group that is checked out, and if the treatments are also checked out, then the treatment group and the treatments are all committed when you commit the decision.

If the parent object is deleted before you commit your changes, you will not be able to commit your changes.

You cannot publish the checked-out version that is in **My Folder**. To publish a version with your changes, you must commit your changes, and publish the committed version.

Checking Out and Committing Included Objects Together with A Decision

If you turn on the [Enable the Check Out Decision Contents window](#) user setting, then when you check out a decision version, SAS Intelligent Decisioning displays the list of object versions that are used in the decision in the Check Out Decision Contents window. You can select any object that you want to check out at the same time. For any object that you choose to check out, SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder, and updates the

checked-out decision so that it uses the checked-out versions of each object that you select.

To check out objects listed in the Check Out Decision Contents window, select the objects, click ➔ to move the objects to the **Selected items** list, and click **Check out**.

When you commit the decision, SAS Intelligent Decisioning also commits the objects. If the decision includes a treatment group that is checked out, and if the treatment group includes treatments that are checked out, then both the treatment group and its checked-out treatments are committed when you commit the decision. However, SAS Intelligent Decisioning does not automatically activate the newly committed version of the treatment group. See [“Activate a Treatment Group” on page 123](#) for instructions for activating a treatment group.

If you select an object that is used only in a nested decision, ensure that you also select the nested decision. If you did not check out the nested decision, the object that is used in the nested decision is not committed when you commit your work.

Your decision might use an older version of an object, or it might use different versions of the same object. The list of included objects has an entry for each object version that is used in the decision. If you select two versions of the same object, SAS Intelligent Decisioning checks out the oldest version. SAS Intelligent Decisioning checks out only one version of each selected object. If a selected object is an older version of the object, you must select the **Update the decision to use the latest version of the selected objects** check box in the Check Out Decision Contents window in order to successfully check out the selected objects. When you select this option, SAS Intelligent Decisioning asks you to confirm that you want to replace references to the selected versions of the objects with references to the latest version. References to any version of any object that you do not select in the Check Out Decision Contents window remain unchanged when you commit the decision.

Checking Out and Committing Objects from within A Decision

If a decision is checked out, and if an object that is included in the decision is stored in a folder for which the check-out feature is enabled, then you can check out the object from within the decision. When you check out an included object, the decision is modified to use the checked-out copy of the object and is saved. When you commit the decision, all checked-out objects that are used in the decision are also committed.

To check out or commit a single object from within a checked-out decision, click on the node for the object, and select **Check out** or **Commit**.

To check out or commit a decision that is used in a parallel process node, in the **Properties** pane or **Properties** window for the node, select the decision, click ⋮, and select **Check out** or **Commit**.

When you check out an object from within a decision, SAS Intelligent Decisioning adds an asterisk in front of the name of the object, and the decision is modified to use the checked-out copy of the object.

To open the checked-out object, click the node for the object, and select **Open**.

Commit the checked-out object from within the decision. When you commit the decision, all checked-out objects that are used in the decision are also committed.

Note: You cannot check out a nested decision's objects from within a decision. After you check out the nested decision, you can open the nested decision and check out its included objects from within the nested decision itself.

Concurrently Checking Out and Committing Decision Versions

The ability for multiple users to check out the same decision at the same time is controlled by the [sas.decisions.checkout.allowConcurrentCheckout](#) configuration option. This option is turned on by default.

When this option is turned on, different users can check out the same version of the same object at the same time. Because the objects that are checked out are saved in each user's `My Folder` location, the default permissions allow individual users to see only the copies that they have checked out.

When this option is turned off and a user has checked out an object, the **Check Out** button for that object is disabled for all other users.

If multiple users check out the same version of the same object at the same time, each user's changes are preserved in a new version when they commit their changes. One user's changes do not overwrite another user's changes.

IMPORTANT If two users attempt to commit changes to the same object simultaneously, the first user's attempt will succeed but the second user might see an error message that the commit has failed. If the second user subsequently commits their changes, the **Modified By** column on the **Versions** tab for both the version committed by the first user and the version committed by the second user displays the user ID of the second user.

Check Out and Commit a Decision Version

Note: If the `sas.intelligent.decisioning.preventCommitWithErrors` configuration option is turned on, you must validate decisions and correct any errors that are

found by the validation process before you can commit decisions. For information, see “[Validate a Decision in the Decision Editor](#)” on page 444 and “[SAS Intelligent Decisioning Service Properties](#)” in *SAS Intelligent Decisioning: Administrator's Guide*.

Note: If you are checking out an object from within a decision instead of from within the category view for that object, see “[Checking Out and Committing Objects from within A Decision](#)” on page 464 for information.

- 1 In the category view, open the object that you want to check out.
- 2 On the **Versions** tab, select the version that you want to check out, and click **Check Out**. SAS Intelligent Decisioning writes a working copy of the version into your **My Folder** folder and opens the working copy. The **Location** and **Original object link** fields on the **Properties** tab indicate that the version is checked out.
- 3 Modify the checked-out version as needed, and save it.

TIP To discard the changes and delete the checked-out version from **My Folder**, you can commit the object without saving it first. However, committing the object without saving creates a new version of the object whose contents match the contents of the previous version. For information on undoing a check out, see “[Undoing a Check Out](#)” on page 468.

- 4 On the **Versions** tab, click **Commit**. The Commit Decision Version window appears.
- 5 Select the version type: **Minor** or **Major**. Version numbers follow the format *major.minor*. If you select **Major**, the number to the left of the period is incremented, and the minor number is reset to zero. If you select **Minor**, the number to the right of the period is incremented.
- 6 (Optional) In the **Version tags** field, either enter a new custom tag or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Press Enter after each new custom tag. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see “[SAS Intelligent Decisioning Service Properties](#)” in *SAS Intelligent Decisioning: Administrator's Guide*.

- 7 (Optional) To add the tags in the **Version tags** field to all of the objects that you are committing, select the **Apply tags to all committed components** check box.
- 8 (Optional) Enter information about the new version in the **Notes** field.

- 9 (Optional) Click **Show included objects** to display the list of objects that will be committed at the same time that you commit the decision. This list includes the version number of each object that you have checked out, and it includes the latest version of each object. If a new version of an object was created after you checked out the object, the object is identified by the warning icon ⚠. Hover over the warning icon to display more information about the object. Before you commit the decision, review the content of the newer version and verify that any needed changes will not be lost. See [“Undoing a Check Out” on page 468](#) for information about undoing the check out of the older version. For more information, see [“Checking Out and Committing Included Objects Together with A Decision” on page 463](#).
- 10 Click **Commit**. SAS Intelligent Decisioning creates a new version with your changes, and deletes the working copy from your **My Folder** folder.

TIP If you or SAS Intelligent Decisioning have updated the decision to use any objects that you have checked out, then SAS Intelligent Decisioning also commits those objects. See [“Checking Out and Committing Objects from within A Decision” on page 464](#) for more information.

Determine Whether an Object Is Checked Out

The **Checked Out** column in the category view displays a check mark if the object is checked out.

If you are editing an object that is stored in a checkout-enabled folder, the object name in the editor is displayed with either (Read-Only) or (Checked Out) depending on whether it is checked out.

If an object in a decision is checked out, an asterisk appears before the object name in the diagram on the **Decision Flow** tab.



Also, the **Properties** pane for each object in the decision specifies whether the object is checked out.

Determine Who Has a Version Checked Out

If the current version of an object is checked out, the IDs of the users that checked it out and the timestamps when each user checked it out appear in the **Checked out by** field on the **Properties** tab for the original object. You can also display this

information by clicking  beside the version number on the **Versions** tab for the original object. (This information is not displayed on the **Properties** tab for the checked-out object. It appears only for the original object.)

Opening the Original Object

When you check out an object, SAS Intelligent Decisioning adds the field **Original object link** to the **Properties** tab for the checked-out object. This field contains a link to the original object that was checked out. You can use this link to verify that you have checked out the correct version and to compare the original content with the modified content in the checked-out version.

Undoing a Check Out

If you [checked out an object from within a decision](#), click  on the object's node in the decision diagram, and select **Cancel checkout**.

You can discard a checked-out version and any changes that you made by deleting the working copy of the version from your **My Folder** folder if the following conditions are true:

- You have not checked out a decision that uses the object.
- The object was not [checked out from within a decision](#) after the decision was checked out.

IMPORTANT When you delete the working copy, that version is deleted permanently. It is not moved into the recycle bin, and it cannot be restored.

Publishing a Decision

Introduction to Publishing

Publishing content makes it available to other applications. Publishing a decision creates an entity that can be managed and run in another environment. When you publish content, SAS Intelligent Decisioning generates code for that content and writes it to the destination. The following table describes what form the generated code takes for each destination type.

The publishing destinations that are available to you depend on what is configured at your site. See [SAS Viya Platform: Publishing Destinations](#) for more information.

Table 13.3 What SAS Intelligent Decisioning Publishes to Each Destination Type

Destination Type	What SAS Intelligent Decisioning Does
SAS Cloud Analytic Services (CAS), Teradata, Apache Hadoop, or Databricks	Adds a row to the model table for the destination. The rows in the model tables are callable REST API endpoints, independent of SAS Intelligent Decisioning.
SingleStore	Publishes the decision as an embedded process with the specified published name. You can run the published decision as a job in SAS Data and AI Studio and SAS Job Execution Web Application. Running the decision invokes SAS Embedded Process, which is installed in SingleStore.
SAS Micro Analytic Service	Writes a Micro Analytic Service module in the service. The Micro Analytic Service modules are callable REST API endpoints, independent of SAS Intelligent Decisioning.
Git	Creates a directory in the remote Git repository with the same name as the published object and writes the generated code to a file named <code>scoreResource.txt</code>. If the decision includes analytic store models, the model files are written as either <code>.sasast</code> files or <code>.astore</code> files, depending on how the model was built. You can use the SAS Intelligent Decisioning Git Deployment CLI to deploy content from the remote repository to destination of the type for which the Git repository was configured. The destination type is either a CAS destination or to a SAS Micro Analytic Service destination. The deployed content is a callable REST API endpoint. For more information, see “Configure Git Publishing Destinations” in SAS Viya Platform: Publishing Destinations.
Container destinations	Creates a SAS Container Runtime container and writes it to the destination. The SAS Container Runtime containers are callable REST API endpoints, independent of SAS Intelligent Decisioning. Note: The container destination types to which you can publish content from SAS Intelligent Decisioning are Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and private Docker repositories.

Where Can I Publish Decisions?

The destinations to which you can publish a decision depend on the destinations that are available and on the objects that are used in the decision. Your system administrator determines which publishing destinations are available. The following table shows the destination types to which you can publish a decision that contains various object types.

Table 13.4 *Decision Objects and the Destinations to Which Decisions Can Be Published*

Objects in the Decision	SAS Micro Analytic Service and Compatible Git Destinations	SAS Cloud Analytic Services (CAS) and Compatible Git Destinations	Teradata, Hadoop, Databricks, and SingleStore Destinations	Container Destinations
Assignment rule sets that do not use advanced lists, filtering rule sets, treatment groups, models, segmentation trees, nested decisions, or DS2 code files and custom context files that do not contain SQL queries ^{1, 2}	yes	yes	yes ³	yes
Assignment rule sets that use advanced lists ¹²	yes ⁶	yes ¹¹	yes ¹¹	yes ⁷
Filtering rule sets ¹⁴	yes	yes ⁸	yes	yes
Rule sets that use SAS Data Quality Functions	no	yes	no ⁹	yes
Python code files	yes	yes	no	yes
Data query files, DS2 code files that contain SQL queries, or custom context files that contain SQL queries	yes	no	no ¹⁰	yes ⁴
Record contacts nodes	yes	yes	yes	yes ¹³
Micro Analytic Module nodes	yes	no	no	no
Parallel process nodes	yes	yes ⁵	no	yes

Objects in the Decision	SAS Micro Analytic Service and Compatible Git Destinations	SAS Cloud Analytic Services (CAS) and Compatible Git Destinations	Teradata, Hadoop, Databricks, and SingleStore Destinations	Container Destinations
REST API definitions	yes	yes	no	no

- 1 Rule sets and nested decisions can use lookup tables, custom functions, and global variables.
- 2 The score code type of a model determines the destination types to which you can publish a decision that contains the model. See [Table 13.5 on page 471](#).
- 3 For SingleStore destinations, only the latest version of DS1 models and analytic store models are supported. Also, Python models included in decisions that are published to SingleStore destinations produce null values for all output variables.
- 4 In container destinations, only certain database types can be queried by using data query files, DS2 code files that contain SQL queries, or custom context files that contain SQL queries. For information, see [“Configuring a Database Connection” in SAS Container Runtime: Programming and Administration Guide](#).
- 5 You can publish decisions that contain parallel process nodes to CAS destinations, but the decisions in the parallel process nodes are run sequentially instead of in parallel. For more information, see [“Using Parallel Process Nodes” on page 415](#).
- 6 Redis instances that are configured with Transport Layer Security (TLS) are not supported.
- 7 The Redis instance must be configured with Transport Layer Security (TLS).
- 8 You can publish the decision to CAS destinations but not to GIT destinations.
- 9 Rule sets that use SAS Data Quality functions fail if they are run inside the database.
- 10 These items will fail if they are run inside the database.
- 11 The `sas.businessrules.inlineAdvanceListValues` configuration option must be set to Yes. For more information, see [“sas.businessrules” in SAS Intelligent Decisioning: Administrator’s Guide](#).
- 12 Rule sets and decisions that are published to SAS Micro Analytic Service destinations or to container destinations can reference multiple advanced lists. Rule sets and decisions that are published to other destination types can reference only one advanced list.
- 13 The `sas.decisions.subjectContactsSCRTTablePrefix` configuration option must be set to the table name prefix that is specified in the CATALOG= option in the database connection string.
- 14 Filtering rule sets behave differently in different destination types. For more information, see [“Using Filtering Rule Sets” on page 30](#).

The score code type of a model determines the destination types to which you can publish a decision that contains those models. The following table lists the score code types a model can have when it is used in a decision that is published to a specific destination type.

Table 13.5 Model Score Code Types and the Destinations to Which Decisions Can Be Published

Model Score Code Type	CAS and Compatible Git Destinations	SAS Micro Analytic Service and Compatible Git Destinations	Hadoop, Teradata, Databricks, and SingleStore Destinations	Container Destinations
DATA step	yes	yes	yes	yes
DS2 multi-type	yes	yes	yes	yes
DS2 package	no	yes	no	yes

Model Score Code Type	CAS and Compatible Git Destinations	SAS Micro Analytic Service and Compatible Git Destinations	Hadoop, Teradata, Databricks, and SingleStore Destinations	Container Destinations
Python	yes	yes	no	yes ^{1, 2}

- 1 You can publish decisions that include a model node for a Python model that is stored in the common model repository. The packages that are used by the model must be specified in the requirements.json file. For more information, see [“Scoring Python Models” in SAS Model Manager: User’s Guide](#).
- 2 Decisions that use models that are built using pipelines by combining SAS procedures, CAS actions, and open-source Python code cannot be published to a container destination.

Note: SAS Model Manager supports publishing some additional model types independently (not in a decision). For additional information, see [“Model Score Code Types and Publishing Destinations” in SAS Model Manager: User’s Guide](#).

For more information about publishing destinations, see [“Configuring Publishing Destinations” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Publishing Decisions That Include Analytic Store Models

If you are publishing a decision that includes an analytic store model, the model’s analytic store (ASTORE) file must be in the `/opt/sas/viya/config/data/modelsvr/astore` directory on the CAS server for the destination to which you are publishing the decision. The ASTORE file is copied to that location when you do any of the following:

- run a decision test for the decision that uses the analytic store model
- set the model as a project champion in SAS Model Manager
- publish the model to SAS Micro Analytic Service from SAS Model Manager

If you are publishing a decision that includes an analytic store model and the model has not been set as a project champion or published from SAS Model Manager, you must test the decision before you publish it. For more information, see [“Testing Decisions” on page 476](#).

Note: When you import an analytic store model into SAS Model Manager, the score code type is set to DS2 multi-type.

Note: Do not include multiple versions of the same analytic store model in the same decision if two or more versions of the model use the same ASTORE files.

Publish Decisions

Note: You must validate decisions and correct any errors that are found by the validation process before you publish the decisions. For information, see [“Validate a Decision in the Decision Editor” on page 444](#).

- 1 Open the decision. By default, the version that is published is the displayed version.

TIP If you do not have permission to modify an object, if the object is a locked version, or if the object must be checked out before it can be modified, SAS Intelligent Decisioning displays **(Read-Only)** in the title bar next to the object name, and the buttons for modifying and saving the object are disabled.

- 2 (Optional) On the **Versions** tab, [click the version number](#) to change the displayed version to the version that you want to publish.
- 3 Publish the selected version.

Note: By default, SAS Intelligent Decisioning displays a **Publish** button. If your administrator has turned on the [requirePublishAndLock configuration option](#), the **Publish and Lock** button replaces the **Publish** button, and the **Publish and lock** menu option is removed.

Note: If the decision is being developed in a workflow that controls whether the **Publish** or **Publish and lock** button is enabled, you might need to claim and complete a user task that enables the button before you can publish the decision. For more information, see [“Controlling Publishing from within a Workflow” on page 513](#).

Select the appropriate option:

- Click the **Publish** or **Publish and lock** button, depending on which button is available. The **Publish** button publishes the selected version without locking it or creating a new minor version. The **Publish and lock** button locks and publishes the selected version and creates a new minor version.
- If the requirePublishAndLock configuration option is turned off, you can use the **Publish and lock** menu option to lock and publish the selected version, and simultaneously create a new minor version. Click , and select **Publish and lock**.

- 4 In the Publish Decisions window, select the destination to which you want to publish. The publishing destinations that are available to you depend on what is

configured at your site. See [SAS Viya Platform: Publishing Destinations](#) for more information.

- 5 (Optional) Enter any tags that you want to associate with the published object. For each tag, either type a new custom tag and press Enter, or select the tag from the list of previously entered tags. Click  to display the list of previously entered tags. Tags are limited to 100 characters each and cannot contain single quotes (').

Note: If your administrator has turned on the `sas.intelligent.decisioning.requireVersionTags` configuration option, you must enter at least one version tag. If your administrator has turned on the `sas.intelligent.decisioning.requireTagFromSelection` configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags. For more information, see “[SAS Intelligent Decisioning Service Properties](#)” in *SAS Intelligent Decisioning: Administrator's Guide*.

- 6 (Optional) In the **Items to Publish** section, complete the following steps for each item that you are publishing:
 - a Edit the **Published name** if you do not want to use the default name for the published module. The maximum length for all published names is 100 characters. Character restrictions differ depending on your destination. See [Table 13.6](#).

Table 13.6 Requirements and Restrictions for Published Names

Destination	Requirements And Restrictions
Container destinations	For Amazon Web Services (AWS) destinations, the published name can contain only alphanumeric characters. For other container destinations, the published name must start with a letter, a number, or a period. The name can include dashes, a single underscore, or a double underscore. It cannot contain other special characters, spaces, or multi-byte characters. Note: The name that you enter is prepended with a forward slash (/) and assigned to the <code>SAS_SCR_APP_PATH</code> environment variable. The value of the environment variable determines the published module name. For more information, see “Changing the Endpoint Name for a Container” in <i>SAS Container Runtime: Programming and Administration Guide</i>.
Git	The published name cannot contain colons (:), forward slashes (/), single quotation marks ('), or double quotation marks (").
SAS Micro Analytic Service	The published name cannot contain the following characters: ! @ # \$ % ^ & * () = ~ ` \ / . { } " ' ; -

Destination	Requirements And Restrictions
SAS Cloud Analytic Services (CAS)	The published name cannot contain single or double quotation marks.
Databricks	The published name cannot contain the following characters: ! @ # \$ % ^ & * () = ~ ` \ / . { } " ' ; -
SingleStore	The published name cannot contain the following characters: ! @ # \$ % ^ & * () = ~ ` \ / . { } " ' ; -
Apache Hadoop	The published name cannot contain colons (:) or double quotation marks.
Teradata	The published name must start with a letter or an underscore. It cannot contain spaces, multi-byte characters, or special characters other than the underscore.

- b If you have previously published the decision, click the switch to turn on the **Replace item with the same name** option in order to replace the previously published decision of the same name in the same destination.
- c Select the **Path tracking** check box if you want the published decision to generate path tracking data.

TIP This path-tracking data is recorded in the pathID column in the output table. The path-tracking data that is recorded when you select **Record path tracking** for a record contacts node is recorded in the subject contact history.

This option is disabled if you are publishing content to a SAS Micro Analytic Service destination, a Git destination, or a container destination.

- d Select the **Rule-fired tracking** check box if you want the published decision to generate rule-fired data.

TIP This rule-fired data is recorded in the ruleFiredFlags column in the output table. The rule-fired data that is recorded when you select **Record rule-fired data** for a record contacts node is recorded in the subject contact history.

This option is disabled if you are publishing content to a SAS Micro Analytic Service destination, a Git destination, or a container destination.

- 7 Publish the decision.

The button that is available depends on the option that you selected in [Step 3](#).

Publish

publishes the decision.

Publish and lock

locks and publishes the decision and creates a new minor version.

The Publishing Results window appears. It displays the names of the published items, their status, and a link to the log that was generated during the publishing process.

Note: In order for content that is published to a SAS Cloud Analytic Services (CAS) destination to be available to other applications, the CAS destination table must be reloaded. For other destinations, the destination table does not need to be reloaded.

If you are publishing content to a SAS Cloud Analytic Services (CAS) destination, SAS Intelligent Decisioning automatically reloads the CAS destination table. If another user is executing the code for an item that was previously published to CAS while the destination table is being reloaded, reloading the table might cause temporary problems with accessing the table content. After the table is reloaded, all authorized users can access all items in the table.

- 8 After the status changes to **Published successfully**, click **Close** to close the Publishing Results window.

TIP To view the publishing history for a decision, click the **History** tab.

- 9 (Optional) Click **×** to close the decision.

Testing Decisions

Ways to Test a Decision

There are three types of tests:

Basic test

executes the decision in the SAS Cloud Analytic Services (CAS) destination using the input table that you specify. You can also specify a debugging variable and enable value tracing. For more information, see [“Create and Run a New Test” on page 477](#).

Scenario test

enables you to enter specific input values and the output values that you expect the test to generate. A scenario test identifies differences between the output that you expect to see and the actual output that is generated when the test is run. You can also compare the test definitions and test results of different scenarios. Scenario tests are also run in CAS. For more information, see [“Create and Run a Scenario Test” on page 482](#).

Publishing validation test

executes the decision in a publishing destination using the input table that you specify. When you publish the decision, a validation test is automatically defined for that decision in that destination. For more information, see [“Validate a Published Decision” on page 497](#).

Test a Decision

Create and Run a New Test

Testing a decision is optional, but doing so is a best practice. Testing enables you to discover any problems before the decision is published and incorporated into a production system.

IMPORTANT If you are testing a decision that uses functions that are defined in a custom context file, verify that the context file is specified in the **Custom context** field on the **Properties** before running the test.

If you are testing a decision that uses treatment groups and both the decision and the treatment group are checked out, the test uses the checked-out version of the treatment group. If the treatment group is not checked out, the test uses the activated version of the treatment group.

If the decision uses a rule set that uses a lookup table and all three objects (the decision, the treatment group, and the lookup table) are checked out, the test uses the checked-out versions of both the rule set and the lookup table. If the rule set or lookup table are not checked out, the test uses the activated version of the lookup table.

- 1 On the **Scoring** tab, click the **Tests** tab.
- 2 Click **New test**. The New Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.

- 5 Click  for the **Location** field, and select the folder in which you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 6 Click  for the **Input table** field, select the input table for the test, and click **OK**.

TIP In the Choose Data window, you can select **Available** in the **Show** field to display the list of data tables that are loaded into memory. You can search for data tables with a specific name by entering Name : "string" in the **Search** field. For example, to search for all tables whose names begin with HMEQ, enter Name : "HMEQ". [“How to Enter Searches for Information Assets in SAS Data Governance” in SAS Data Governance: User’s Guide](#) for more ways to search the list of available tables.

If the table that you need does not appear in the list of available tables in the Choose Data window, see [“Importing and Loading Data” on page 12](#).

- 7 Verify or change the variable mappings. To run a full test, map all of the input variables to columns in the input table that you selected for the test. To run a partial test, you can map only the input variables that are needed for the test.

SAS Intelligent Decisioning automatically maps the input variables in the decision to columns in the input table when the names and data types of the variables match those of the table columns.

If any input variables are not mapped to input columns or to static values, the application displays the warning message `Input variables must be mapped to table columns`. At run time, SAS Intelligent Decisioning assigns missing values to input variables that are not mapped.

You can change the automatic variable mappings in the Variable Mappings window.

To change variable mappings:

- a Click **Variables**. The Variable Mappings window appears. This window displays all of the decision variables and the input table columns to which the decision variables are mapped.

TIP Click **Show unmapped variables** to filter the list to show only the variables that have not been mapped to an table column.

- b For each input variable, select the table column to which the variable should be mapped. If the input table contains more than 15 columns, click **More columns** to display additional column names. Alternatively, for Decimal, Integer, and Character variables, you can select **Use value** for the table

column, and specify a literal value in the **Value** column. When you are entering literal values, remember these rules:

- For character variables, input values are limited to 10 Mb. Output values longer than 32767 characters will be truncated.
- Do not enclose character strings in quotation marks.
- To specify a missing value for character variables, select **Use value** and leave the **Value** column empty. When SAS Intelligent Decisioning generates code, it generates an empty string (' '). For numeric values, enter a period (.).

For more information about entering variable values, see [“The Properties of a Variable” on page 395](#).

- c Click **OK** to close the Variable Mappings window.
- 8 (Optional) Click **Advanced** to display the advanced options.
- 9 (Optional) Click  for the **Output data library** field, and select the library to which you want to write the output of the test.
- 10 (Optional) Enter a name for the test results output table in the **Output table name** field if you do not want to use the default name. The default name is `test-name_object-name_timestamp_output`.
- 11 (Optional) For the **Target version**, select the version of the decision that you want to test. The default version is the latest version.
- 12 (Optional) Select the variable that you want to serve as an input debug variable. You can specify an input-only variable or an input-output variable. The decision writes the name and value of this variable to the log for each input record that is processed. It writes the value just before the logic of the decision is executed for the input record.

For more information, see [“Debugging Decision Tests” on page 480](#).
- 13 (Optional) Select **Preserve unmapped columns in the output table** if you want columns that are not mapped to an output variable to be written to the output table.
- 14 (Optional) To generate a data grid variable of all decision variables and their values, select **Record variable values by node**. For more information, see [“Enabling Value Tracing” on page 481](#).
- 15 (Optional) Select **Input file uses array syntax for data grid variables** if you want SAS Intelligent Decisioning to convert input columns with names that use array syntax into standard JSON data grid syntax. Data grid column names in the input file must use the format `column_name[index]`, where `column_name` is identical for all of the columns that belong to the same data grid variable. SAS Intelligent Decisioning converts the column names and data in the input file into a data grid that uses standard data grid JSON strings. For more information, see [“Testing Decisions by Using Array Syntax for Data Grid Column Names” in SAS Intelligent Decisioning: Using Data Grids](#).
- 16 (Optional) Select **Keep filtered records in output table** to include records in the test output that would normally be excluded by any of the filtering rule sets in

the decision. When you select this option, SAS Intelligent Decisioning creates a new column in the output table named `_filter_`. If the value in this column is 0, then the record does not match any of the conditions in any filtering rule set, and the record is filtered out of the output when the decision is run in a production environment. If the value in this column is 1, then the record matches at least one filtering condition, and the record is included in the output.

Note: This option is ignored if the decision does not contain a filtering rule set.

- 17 Click **Run** to save and run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 499](#).

- 18 Click  in the **Results** column to view the results of the test.
- 19 In the test results window, click **Test Results** in the navigation pane to display the URIs and other information for the test. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

Debugging Decision Tests

When you create a test, you can specify a variable to use as a debugging variable in the **Input debug variable** field. You can specify an input-only variable or an input-output variable. The decision writes the name and value of this variable to the log for each input record that is processed. It writes the value just before the logic of the decision is executed for the input record. For more information, see [Step 12 of “Create and Run a New Test” on page 477](#).

When you specify an input debug variable, you can use the `sas.decisions.messageOrder` configuration option to control whether the log messages are written as they are produced or after the decision executes. For more

information, see [“sas.decisions.messageOrder” in SAS Intelligent Decisioning: Administrator’s Guide](#).

When you specify an input debug variable, SAS Intelligent Decisioning automatically sets the maximum number of threads that can be allocated for the test to 1. Setting the thread count to 1 ensures that the variable’s values are written to the log in the correct order and are not affected by different threads completing at different times.

To capture variable values for input-only or temporary variables after the decision logic has executed for a specific record, you can specify that the variable is an output variable, and then re-run the test. Before publishing the decision to a production environment, return the input and output settings for the variable to their previous settings. For more information, see [“Input Variables, Output Variables, and Temporary Variables” on page 398](#) and [“Edit Variable Properties” on page 403](#).

TIP To capture intermediate variable values that occur during the execution of the decision, you can use DS2 code files that include PUT statements for the variable. For more information, see [Chapter 7, “Using Custom Code Files,” on page 183](#).

Enabling Value Tracing

When you test a decision, you can select the **Record variable values by node** check box to trace how variable values change for each node in the decision flow. When you select this option, SAS Intelligent Decisioning creates an output data grid variable named `nodeTraceDataGrid`. This data grid contains a row for each rule set, model, nested decision, and code file node in the decision. It contains one column for every variable in the decision. Condition nodes, branch nodes, and record contact nodes are not included.

The first column in `nodeTraceDataGrid` contains the node name, and the remaining columns contain the values of the input and output variables, including temporary variables.

Within the `nodeTraceDataGrid` variable, data grids are written as string variables, and their length is limited to 32767 characters.

When a node sets a variable’s value, that value appears unchanged in all subsequent rows until another node changes the variable to a different value. In the following example, the `Code_1` node sets `score` variable to a value of 1. The `Code_2` and `Code_3` nodes do not change that value. The `Rule_set_1` node changes the value to 1.25. The `Code_1` node sets the value of the `scoreProbDataGrid` variable to null. The `Code_3` node sets the value of `scoreProbDataGrid` to a data grid with two columns: `score` and `probability`.

```
{
  "Data Grid": [{
    "metadata": [
      {"Node Name": "string"},
```

```

        {"additionalDiscount": "double"},
        {"age": "double"},
        {"cost": "double"},
        {"currentPhone": "double"},
        {"score": "double"},
        {"treatmentName": "double"},
        {"ServicePlans_out": "string"},
        {"newScore": "double"},
        {"probability": "double"},
        {"scoreProbDataGrid": "string"}
    ]
}, {
    "data": [
        ["Code_1", null, 42, 699.99, null, 1, null, null, 1.1, 0.625547, null],
        ["Code_2", null, 42, 699.99, null, 1, null, null, 1.1, 0.625547, null],
        ["Code_3", null, 42, 699.99, null, 1, null, null, 1.1, 0.625547,
            "{\"Data Grid\": [{\"metadata\": [{\"score\": \"double\",
                {\"probability\": \"double\"}], {\"data\": [[1.23,4.56]]}]]\"}],
        ["Rule_set_1", 99.99, 42, 699.99, null, 1.25, null, null, 1.1, 0.625547,
            "{\"Data Grid\": [{\"metadata\": [{\"score\": \"double\",
                {\"probability\": \"double\"}], {\"data\": [[1.23,4.56]]}]]\"}],
        ["Combined_Treatments", 99.99, 42, 699.99, null, 1.25, null, null, 1.1, 0.625547,
            "{\"Data Grid\": [{\"metadata\": [{\"score\": \"double\",
                {\"probability\": \"double\"}], {\"data\": [[1.23,4.56]]}]]\"}"]
    ]
}]
}

```

Note: Value tracing increases the amount of code that is generated for a decision and the time required to execute the decision. The magnitude of the increases depend on the number of variables and complexity of the decision.

Test a Scenario

Create and Run a Scenario Test

IMPORTANT If you are testing a decision that uses functions that are defined in a custom context file, verify that the context file is specified in the **Custom context** field on the **Properties** before running the test.

If you are testing a decision that uses treatment groups and both the decision and the treatment group are checked out, the test uses the checked-out version of the treatment group. If the treatment group is not checked out, the test uses the activated version of the treatment group.

If the decision includes a rule set that uses a lookup table and all three objects (the decision, the treatment group, and the lookup table) are checked out, the test uses the checked-out versions of both the rule set and

the lookup table. If the rule set or lookup table are not checked out, the test uses the activated version of the lookup table.

- 1 On the **Scoring** tab, click the **Scenarios** tab.
- 2 Click **New test**. The New Scenario Test window appears.
- 3 Enter a name for the test if you do not want to use the default name. The name cannot contain forward slashes (/) or curly braces ({}).
- 4 (Optional) Click  for the **Location** field, and select the folder where you want to save the test definition.
- 5 Click  for the **Output table location** field, and select the data library where you want to save the test results.
- 6 (Optional) Select **Enable variable change path logging** to record variable value information and variable mapping information for each node in the decision. The application writes this information to the CAS log. Selecting this option enables you to generate a change path report in Step 16. For more information, see [“Using Variable Change Path Logging” on page 487](#).

Note: If the decision is complicated or uses very large data grid variables, the change path file might be very large, and you might encounter memory issues. Do not select both the **Enable variable change path logging** option and the **Enable variable assignment logging** option for the same test.

TIP If you do not see the **Enable variable change path logging** check box, drag the horizontal divider ----▲ down.

- 7 (Optional) Select **Enable variable assignment logging** to include information in the log about how a variable's value changes as the decision executes. The application writes a log entry each time a variable is assigned a value. Selecting this option enables you to generate a variable assignment report in Step 17. For more information, see [“Using Variable Assignment Logging” on page 489](#).

Note: If the decision is complicated or uses very large data grid variables, the change path file might be very large, and you might encounter memory issues. Do not select both the **Enable variable change path logging** option and the **Enable variable assignment logging** option for the same test.

- 8 (Optional) Select the version of the decision that you want to test. The default version is the latest version.

Note: When you change the version, variables for the new version are reloaded and any variable values that were entered for the test are cleared.

- 9 (Optional) Enter a description for the test. Descriptions are limited to 1000 characters.

- 10 Enter the values that you want to use for each input variable. You do not have to enter values for every input variable. At run time, SAS Intelligent Decisioning uses missing values for input variables for which you do not specify a value.

Note: For character variables, input and output values longer than 32767 characters will be truncated. You cannot enter input values for variables of type binary or of type varying-length binary.

To enter values for the columns in a data grid variable:

- a Click  in the **Value** field. The Edit Data Grid window appears. One row is automatically added to the input data grid.
- b Enter values for each column in the data grid.
- c (Optional) Click  to add a new row, and then enter the values for each column.

Repeat this step for each row of values that you want to add to the data grid.

TIP By default, data grid column names appear across the top of the data grid view, and row numbers appear down the left side. You can click  to change the view of the data grid so that row numbers appear across the top and data grid column names appear down the left side.

- d Click **OK** to save the data grid values and close the Edit Data Grid window.

For more information about entering variable values, see [“The Properties of a Variable” on page 395](#).

- 11 (Optional) For each output variable, select the **Include** check box and enter the expected output value. The **Include** check box controls whether a variable's expected value is used to determine the status of a scenario test. If you select **Include** for a variable and the test does not return the expected value, the test status is set to Completed with warnings (). If you do not select the check box, the application ignores the expected value of that variable when it determines the status of the test.

To enter expected values for the columns in a data grid variable, click in the **Expected Output** field, and follow the instructions in [Step 10](#) for entering values for the columns in a data grid variable.

Note: A scenario test cannot verify issues with trailing spaces. For example, it cannot distinguish between a string that contains a single space ' ' and a string that contains three spaces ' ' '.

- 12 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 499](#).

- 13 Click  in the **Results** column to view the results of the test.
- 14 On the Test Results page, click **Scenario Test Results** in the navigation pane to display the URLs and other information for the test. Click **Input** to display the values of input variables and dynamic attributes. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

The **Log** tab contains two subtabs, **Diagram** and **Log**. The **Diagram** subtab displays both the decision diagram and the list of errors or warnings. Click  or  to display list the errors or warnings. In the diagram, a red X identifies a decision node that produced an error and a yellow triangle identifies a node that produced a warning. Click **View log** next to an error or warning in the list to jump to the line that generated that error on the **Log** subtab.

On the **Output** page, click **Show All** to display the expected and actual values of all output variables. Click **Show Differences** to display only the variables whose expected values do not match the actual values that were returned by the test.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Output** page, click **Export** to export the output table as a comma-separated values (CSV) file. On the **Log** page, click  to download the log file.

- 15 Click  to close test results and return to the **Scoring** tab.
- 16 (Optional) Generate a variable change path report. Select the scenario test, click , and select **Generate path report**. SAS Intelligent Decisioning generates the report, adds it as a comment attachment to the scoring test, and opens the **Comments** property pane.

Click on the report file name to open the report. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Note: If the decision is complicated or uses very large data grid variables, the report file might be very large, and you might encounter memory issues.

- 17 (Optional) Generate a variable assignment report. Select the scenario test, click , and select **Generate assignment report**. SAS Intelligent Decisioning generates the report, adds it as a comment attachment to the scoring test, and opens the **Comments** property pane.

Click on the report file name to open the report. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Import Scenario Test Definitions

You can import scenario test values from a comma-delimited (CSV) file. Each line in the CSV file is imported as one scenario test. In the CSV file, add a column of values for each variable. In the header row, enter the names of the input variables and of the output variables with `_expected` appended to the name.

For example, suppose your scenario test has the input variables `policyholder`, `cscore`, and `claims`, and it has the output variables `eligible` and `policies`. An import file for this test might appear in a spreadsheet application as shown in the following figure.

	A	B	C	D	E	F	G	H	I	J
1	policyholder	cscore	claims	eligible_expected	policies_expected					
2	Smyth, Joe	784	0	TRUE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["4535					
3	Dupree, Marcel	803	2	FALSE	[[{"metadata":{"policy":"string"},"premium":"decimal"}],{"data":["9836					

The policies output variable in this example is a data grid variable. To enter values for data grid variables, use the JSON string format described in [“Introduction to Data Grids” in SAS Intelligent Decisioning: Using Data Grids](#).

The format for date and datetime variables depends on your locale. Use the same format that is created by the date and datetime pickers when you click  or  to enter initial values for custom variables on the **Variables** tab.

To import scenario test definitions:

- 1 On the **Scoring** tab, click the **Scenarios** subtab, and then click **Import scenarios**. The Import Scenarios window appears.
- 2 In the **Import from** field, click  and select the CSV file that contains the scenario test values.

Note: The import file is limited to 10 MB.

- 3 Select **Select encoding** to select the encoding of the CSV file from the predefined list, or select **Enter encoding** to enter a custom encoding.
- 4 Select or enter the encoding of the CSV file.

- 5 Enter a prefix for the scenario test definitions. SAS Intelligent Decisioning appends a number to this prefix for each test definition. The prefix can include double-byte characters and special characters, including single quotation marks.
- 6 (Optional) Click  for the **Folder location** field, and select the folder where you want to save the test definitions.
- 7 Click  for the **Output table location** field, and select the folder where you want to save the test results.
- 8 (Optional) Select **Enable variable change path logging** to enable variable change path logging for the imported tests. For more information, see [“Using Variable Change Path Logging” on page 487](#).
- 9 Select **Enable variable assignment logging** to enable variable assignment logging for the imported tests. For more information, see [“Using Variable Assignment Logging” on page 489](#).
- 10 Click **Import**.

Compare Different Scenarios

You can display the scenario definitions or results of two or more tests side-by-side. On the **Scenarios** tab, select two or more tests, click , and select one of the following options:

Compare ⇨ **Definitions**

displays the input values and the expected output values that you entered for both tests. To edit the input values for a scenario test definition, click  next to the test name under **Input Table**. To edit the expected output values for a test, click  next to the test name under **Output Table**.

Compare ⇨ **Results**

displays the input values and the actual output values that were generated by the test. To display both the expected values and the actual values in the output table, select **Display expected values**. For each variable for which you selected **Include** in [Step 11 on page 484](#), the application highlights the variable if the actual and expected values do not match.

Click **Export** to export the results comparison as a comma-separated values (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Using Variable Change Path Logging

When you run a scenario test for a decision, you can select **Enable variable change path logging** to write information about node variables to a log as each node in the

decision runs. Each log entry records value and mapping information for the node's variables. After the scenario test has run, select the **Generate path report** menu option to generate a report from the log entries. SAS Intelligent Decisioning creates the report in a text file and adds this text file to the scenario test as a comment with an attachment.

Note: Subdecisions are not included in the report that is generated by **Enable variable change path logging** option.

For all node types except branch nodes and subdecisions, the report records the initial value, the final value, and the mapping for each of the node's variables. The information for each node is enclosed in lines that begin with IN and OUT. For example, if the scenario test executes the treatment group named `credit_card_offers` followed by the rule set named `calc_num_offers`, the trace file contains the following lines:

```
IN: decision - credit_card_offers
    IN: treatment group - card_offers
        ...<node variable values>...
    OUT: treatment group - card_offers
    IN: ruleset - calc_num_offers
        ...<node variable values>...
    OUT: ruleset - calc_num_offers
        ...<decision variable values>...
OUT: decision - credit_card_offers
```

Between the IN and OUT lines for each node, the report records the initial value, final value, and mapping for each of the node's variables. For example, if a node has an input-only variable named `RATE`, an output-only variable named `NUM_OFFERS`, and an input-output variable named `ACCTSCORE`, the report might contain the following lines:

```
*Setting inOut Variable - ACCTSCORE ( mapped to ACCTSCORE )
    *ValueReceived: 0.698
    *ValueReturned: 0.796
*Setting input Variable - RATE ( mapped to RETURN )
    *ValueReceived: 15.4
*Setting output Variable - num_offers ( mapped to num_offers )
    *ValueReceived: .
    *ValueReturned: 2
```

Note: A period (.) indicates a missing value. Boolean values are recorded as 0 (false) and 1 (true).

Between the last node of the decision and the END node, the report contains the same information for each decision variable. These lines include the name of the decision. For example:

```
*Setting credit_card_offers output Variable - zero_offers ( mapped to zero_offers )
    *ValueReceived: .
    *ValueReturned: 0
```

For branches, the report records the branch condition and the path that the test takes out of the branch. For example, suppose the decision has a range branch that

has different paths for the CLAGE variable. If the test follows the path for the range of values 101–300, the trace file contains the following line:

```
IN: branch - Range(CLAGE) 101 - 300==>GEN==>
```

In this case, the branch label is the default value 101 - 300. If you customize the branch label, then 101 - 300==>GEN==> is replaced with the custom label.

If the decision has a Yes/No branch with the condition zero_offers=True, and the test follows the No path, the trace file contains the following line:

```
IN: branch - zero_offers=True onFalse
```

If the **Score rows in this data grid** option is selected in the **Input Variables** property pane for a node, the IN line for that node lists the data grid that is being scored. For example, if the credit_card_offers decision contains a model named credit_limit and that model scores the data grid named card_offers_out, the trace file contains the following line:

```
IN: model - credit_limit scoring datagrid card_offers_out of credit_card_offers row 1
```

TIP If the log files that are generated by either the **Enable variable change path logging** option or the **Enable variable assignment logging** option are very large, you might need to increase the JVM heap size for the Subject Contact and Files services. Because of the storage required by these options, do not select both of these options in the same test. For more information, see [“Subject Contact Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Instructions for enabling change path logging and for generating the report are in Step 6 and Step 16 of [“Create and Run a Scenario Test” on page 482](#).

Using Variable Assignment Logging

When you run a scenario test for a decision, you can select **Enable variable assignment logging** to record how variable values change as a decision runs. This option adds the following [DS2_OPTIONS statement](#) to the generated code:

```
DS2_OPTIONS TRACEVARIABLES;
```

Note: To enable this option, your administrator must define the App.TableServices.DS2.Runtime.TraceVariables logger, and set other configuration options.

If you select this option and configure the logger, SAS Intelligent Decisioning writes a log entry each time a variable is assigned a value. Each log entry includes the following information:

- the line number in the generated code where the value is assigned
- the type of code block in which the value was assigned (package or thread)
- the name of the code block that assigned the value

- the line number in the individual code block
- the variable name and its assigned value

Note: Log messages are limited to 32000 bytes, including the log message header. In locales that use multi-byte character sets, log entries might be truncated to 8000 characters.

For example, if the decision includes a rule set named `rs_hmeq_loan` that sets the value of a variable named `risky`, the trace file might contain the following lines:

```
Line 1053 (package "56b88b7c_60c8_4c30_c8ec_82ea1b788180_rs_hmeq_loan" line 102):
    "risky"=0
Line 1058 (package "56b88b7c_60c8_4c30_c8ec_82ea1b788180_rs_hmeq_loan" line 107):
    "risky"=1
```

These entries indicate that on line 1053 of the generated code, the variable `risky` is set to 0. This line is line 102 of the code block that was generated for the rule set. Similarly, the variable was set to 1 on line 1058 of the generated code.

Note: A period (.) indicates a missing value. Boolean values are recorded as 0 (false) and 1 (true).

For data grid variables, SAS Intelligent Decisioning creates a log entry for any assignment that changes the value of the variable and that is performed by using data grid functions or methods. For example, if the generated code uses the SAS Micro Analytic Service `setValue` method to set the value of a cell in a data grid named `zero_offers_out_grid`, the log might include the following entry:

```
Line 556 (package "71a328a8_7336_4c26_70c7_b5901f39d692_credit_card_offers_zero_offers"
line 44):
Cell for column index 1 added for DataGrid "zero_offers_out_grid"
with value 8d9bee3d-45f0-4249-b4d7-9aa92b865a92
```

SAS Intelligent Decisioning does not write log entries for direct assignments from one data grid variable to another such as `gridVar1 = gridVar2`.

If the data grid variable is an output variable, the log contains the entire value of the variable that is written to the output table. The log entry includes both the column metadata and the values for each column. For example, if the decision includes a treatment group named `tg_hmeq`, the decision creates an output data grid variable for the treatment group named `tg_hmeq_out`. In the following log entry, the value of the `tg_hmeq_out` variable was set on line 2847 in the generated code:

```
Line 2847 (thread "56b88b7c_60c8_4c30_c8ec_82ea1b788180_decisionflowthread" line 205):
"tg_hmeq_out"=[{"metadata":[{"TREATMENT_DEFINITION_GROUP_ID":"string"},
{"TREATMENT_DEFINITION_GROUP_REVISION_ID":"string"},{"TREATMENT_DEFINITION_ID":"string"},
{"TREATMENT_DEFINITION_REVISION_ID":"string"},{"TREATMENT_DEFINITION_NAME":"string"},
{"CLNO":"decimal"}, {"DEBTINC":"decimal"}, {"DELINQ":"decimal"}, {"DEROG":"decimal"},
{"DISCOUNT":"decimal"}, {"OFFERCODE":"string"}, {"OBJECTNODEID":"string"}]},
{"data":["e4c3c86d-419f-41d8-8e7f-4ba4dfc01f80", "2389be10-0445-4647-bc63-0221a078431d",
"7ab57211-a67a-4e03-adee-dae8436255b5", "bbbbe7cd-e11b-49a4-b7f1-afa736503397",
"hmeq_online_marketing_services", 23.2, 33, 2, 234, 20, "OMS01",
"8b758d75-1e60-4787-9e44-3421faf1548e"]]}]
```

For information about the JSON strings that are used to represent data grid variables, see [“Introduction to Data Grids” in SAS Intelligent Decisioning: Using Data Grids](#).

If the

`sas.decisions.nodetraces.includeRuleFiredPathTrackInfoInVariableAssignmentLogging` configuration option is turned on, the log includes rule-fired information and path-tracking information. However, if this configuration option is turned off, you cannot run rule-fired analyses or path-tracking analyses for scenario tests. For more information about the configuration option, see [“Decisions Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

After the scenario test has run, select the **Generate assignment report** menu option to generate a report from the log entries. The application creates the report in a text file and adds this text file to the scenario test as a comment with an attachment.

TIP If the log files that are generated by either the **Enable variable change path logging** option or the **Enable variable assignment logging** option are very large, you might need to increase the JVM heap size for the Subject Contact and Files services. Because of the storage required by these options, do not select both of these options in the same test. For more information, see [“Subject Contact Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Instructions for enabling variable assignment logging are in Step 7 of [“Create and Run a Scenario Test” on page 482](#).

For more information about data grid variables, see [SAS Intelligent Decisioning: Using Data Grids](#).

For more information data grid packages and methods, see the following resources:

- [“DS2 Datagrid Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#)
- [“Using the Datagrid Package” in SAS DS2 Programmer’s Guide](#)

Run a Rule-Fired Analysis

If a rule’s conditions evaluate to true, then the rule is said to have *fired*. Rule-fired data includes summary information about how many times each rule fired and detailed information for each time that a rule evaluates to true. See [“How Rules Are Evaluated and When Rule-Fired Records Are Generated” on page 55](#) for more information.

Note: Rule-fired data is recorded only for rule sets, including rule sets in nested decisions and filtering rule sets that are included directly in a decision. Rule-fired data is not recorded for other objects, including filtering rule sets that are used as eligibility rule sets in treatments.

For filtering rule sets that are included directly in a decision, rule-fired data is recorded for an input record if the rule set does not filter out the record. When the rule set filters out an input record, rule-fired data is not recorded for that input record.

For rule sets that iterate over a data grid (in other words, the rule sets score the rows in the data grid), the rule-fired data indicates that the rules in the rule set fire once for the entire data grid instead of firing once for each row in the data grid.

Rule fired data is recorded for nested decisions only if the `sas.decisions.includeRuleFiredInformationForSubdecisions` configuration option is turned on. If the `sas.decisions.nodetraces.includeRuleFiredPathTrackInfoInVariableAssignmentLogging` configuration option is turned off, you cannot run rule-fired analyses for scenario tests. For more information, see [“Decisions Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

TIP This rule-fired analysis uses the data that is in the `ruleFiredFlags` column in the test results output table. To analyze rule-fired data that is in the subject contact history, use the `%DCM_GET_SUBJECTCONTACT_HISTORY` and `%DCM_RULEFIRE_DETAIL` macros. For more information, see [SAS Intelligent Decisioning: Macro Guide](#).

To run a rule-fired analysis:

- 1 In the test results window, click **Rule-Fired Analysis** in the navigation pane.
- 2 Click **Run Rule-Fired Analysis**. SAS Intelligent Decisioning analyzes the test results to determine which rules fired for each row in the input table, and displays the Analysis page.

The Analysis page displays the number of rules that fired for each output record that was generated by the decision. The number in the **Rules Fired Count** column is a link to more information. You can click on this link to display the rules that fired for that output row.

Click Rule-Fired Analysis to display information about the tests such as the URIs for the test analyses and the paths to the rule-fired tables.

For example, in the following display there is one output record for which two rules fired.

Evaluate_Loans_Test

Analysis
Output Records

Rules Fired Count	REJECT	REVIEW	BAD
1	0		1
1	0		0
2	0	1	1
1	0		0
1	0		0
1		1	0
1		1	0
1	0		0

- Click on a number in the **Rule Fired Count** column. SAS Intelligent Decisioning displays the Rule Fired Count window. This window shows which rules produced the selected output record.

Rule Fired Count

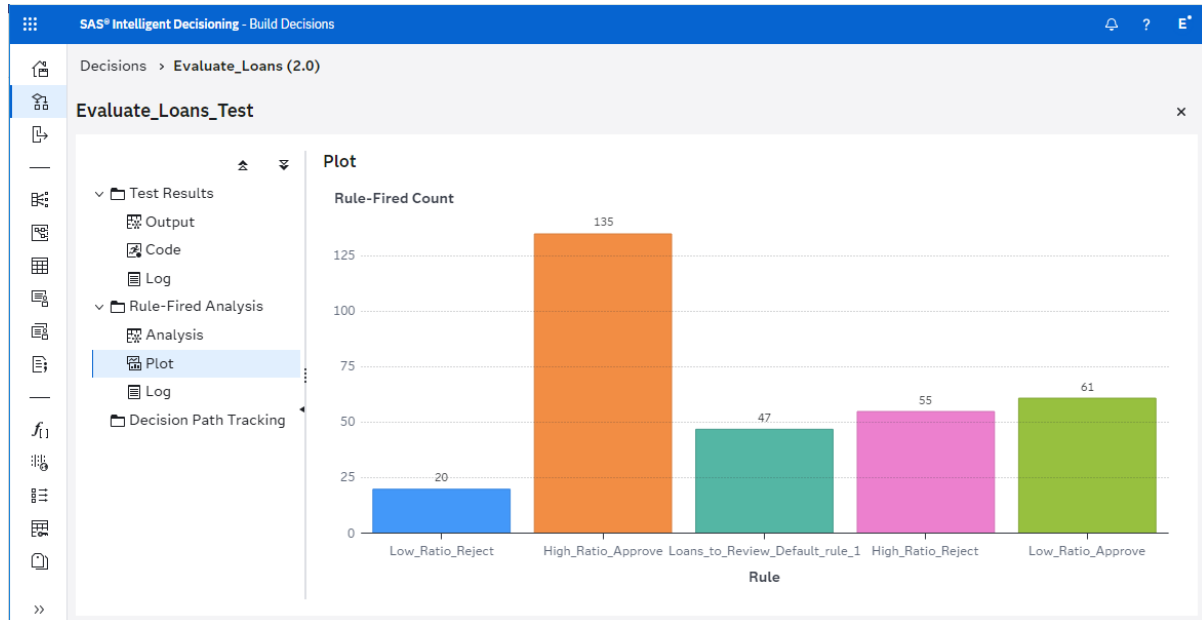
Details Analysis Output Columns

Rule Set Name	Rule Name	Order in Diagram	Rule Logic
Low_Ratio	Approve	2	IF (VALUE >= 120000) AND (REASON in ('HomeImp', 'DebtCon')) AND (CLNO < 15) THEN REJECT = 0
Loans_to_Review	Default_rule_1	1	IF (VALUE >= 120000) AND (DEBTINC < 35.5) AND (DEBTINC ^ = .) AND (DEROG = . OR DELINQ = .) THEN REVIEW = 1

Close

Note: The value in the **Rule Order** column is the ordinal number of the rule as it occurs in the decision diagram. The values in this column do not indicate the order in which the rules in the decision fired. For decisions that use cross-branch links, click  to switch the diagram between two different views of cross-branch links. It might be easier to correlate the rule order numbers with their location in the diagram by displaying the cross-branch links as nodes.

- 4 Click **Close** to close the Rule Fired Count window.
- 5 Click **Plot** in the navigation pane. SAS Intelligent Decisioning displays a bar chart that shows how many times each rule fired. Position your cursor over a bar to display the name of the rule and the number of times that the rule fired.



- 6 Click **Rule-Fired Analysis** in the navigation pane to display the URIs and other information for the rule-fired test.
- 7 Click **x** to close the decision.

Run a Path-Tracking Analysis

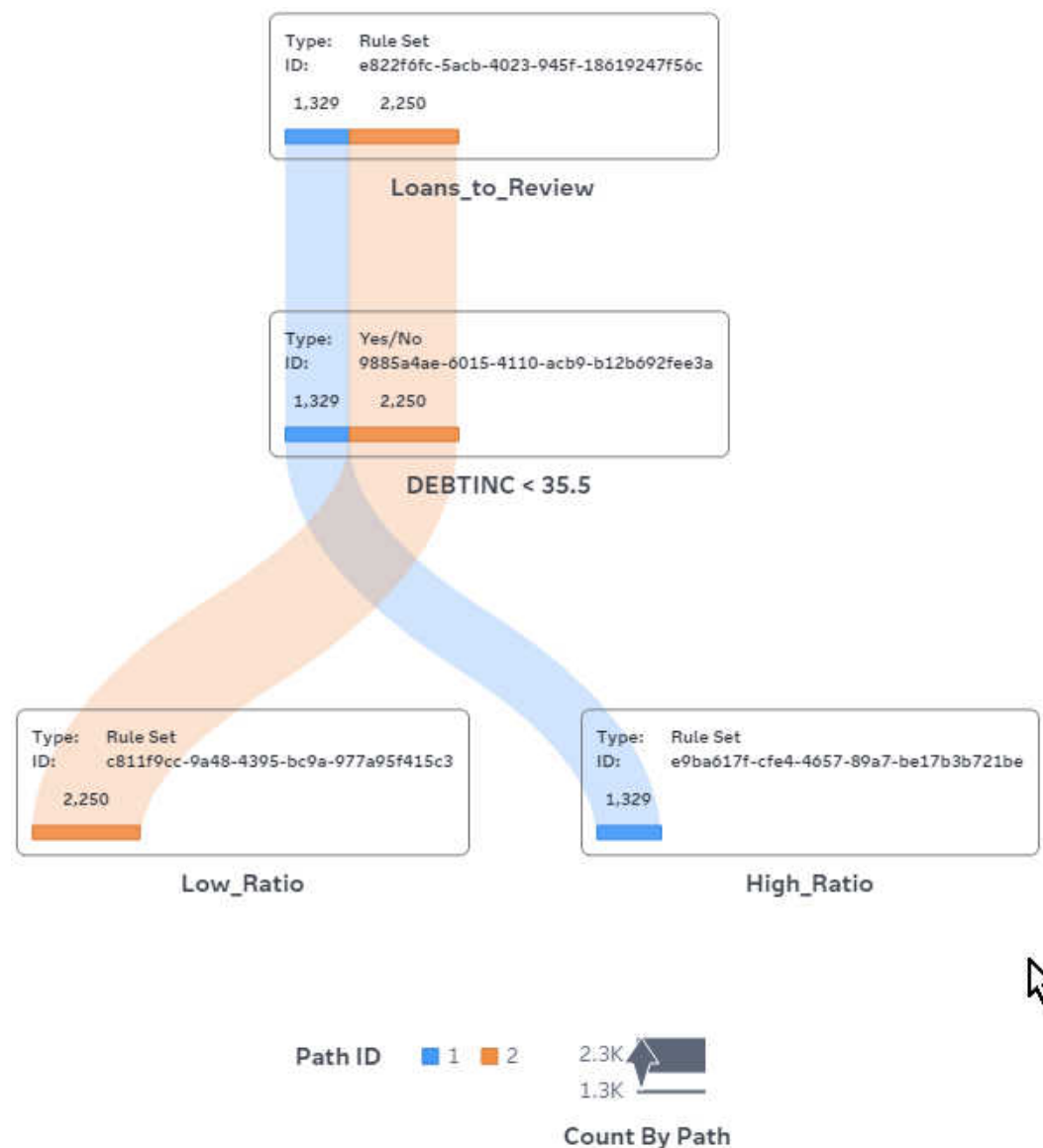
Decision path tracking shows you the route that input records take through the nodes in your decision.

Note: The path-tracking results do not include data for input records that are filtered out with a filtering rule set.

If the `sas.decisions.nodetraces.includeRuleFiredPathTrackInfoInVariableAssignmentLogging` configuration option is turned off, you cannot run path-tracking analyses for scenario tests.

TIP This path-tracking analysis uses the data that is in the pathID column in the test results output table. To analyze path-tracking data that is in the subject contact history, use the %DCM_DECISION_PATH_FREQUENCY, %DCM_DECISION_PATH_NODES, and %DCM_DECISION_NODES_COUNTS macros. For more information, see [SAS Intelligent Decisioning: Macro Guide](#).

- 1 In the test results window, click **Decision Path Tracking** in the navigation panel.
- 2 Click **Run Path Tracking** to run a decision path analysis.
- 3 Click **Analysis and Plot** to display a Sankey diagram that shows the flow of the input records through the nodes in the decision. The numbers in the diagram are the number of rows in the input table that followed each path.



Note: Nodes that are not executed are shown to the right of the Sankey diagram.

- 4 Click **Node Count** in the navigation panel to display a table showing the number of input records evaluated at each rule set, code file, and model node in the decision.
- 5 Click **x** to close the Test Results window.

Convert a Data Grid to Array Format

If you select the **Input file uses array syntax for data grid variables** option when you define the decision test, SAS Intelligent Decisioning creates an input data grid from specially named columns in your input data. When you run the test, SAS Intelligent Decisioning creates an output data grid. After the test is run, you can choose to convert the output data grid back into the original array format that is used in the input data table. For more information, see [“Testing Decisions by Using Array Syntax for Data Grid Column Names” in SAS Intelligent Decisioning: Using Data Grids](#).

- 1 In the test results window, click **Data Grid Array Results** in the navigation pane. This option appears only if you select the **Input file uses array syntax for data grid variables** option when you define the test.
- 2 Click **Convert a Data Grid to Array Format**.

TIP On the **Log** page, click  to download the log file.

Content That Is Used by Tests and Scenarios for Decisions

For each object in a decision, the content that is used when the decision is tested depends on the object type.

Object Type	Object Content that Is Used
Nested decision	The version that you select when you add the nested decision to the decision
Rule set	The version that you select when you add the rule set to the decision
Model	The version that you select when you add the model to the decision

Object Type	Object Content that Is Used
Segmentation tree	The version that you select when you add the segmentation tree to the decision
Treatment group	The active version
Lookup table	The active version
Global variable	See “Which Value Is Used for a Global Variable?” on page 181
Custom code file	The version that you select when you add the code file to the decision
Value list	The version that you select when you define a variable in a segmentation tree

Validate a Published Decision

You can test the published decision in the publishing destination. When you publish the decision to any destination type except Git, a validation test is automatically defined for that decision in that destination.

Note: When both the SAS Intelligent Decisioning `sas.decisions.masnode.removeTrailingUnderscoresFromInput` property and the SAS Micro Analytic Service `service.removeTrailingUnderscoresFromInputs` property are set to On, publishing validation tests will fail for content that has been published to a SAS Micro Analytic Service destination.

To run the publishing validation test:

- 1 On the **Scoring** tab, click the **Publishing Validation** tab. The  icon in the **Status** column indicates that the test is not ready to run. The  icon indicates that the test is ready to run.
- 2 Click on the test name. The Edit Publishing Validation Test window appears.

Note: To generate the name of the publishing validation test, SAS Intelligent Decisioning appends a timestamp to the decision name. The timestamp indicates when the decision was published.

- 3 (Optional) Click  in the **Location** field, and select the folder where you want to save the test definition and results.

TIP Selecting a location is optional, but it is highly recommended. Storing test definitions and test results in a folder simplifies the tasks of setting permissions and transferring the test files.

- 4 Click  in the **Input table** field, and select the input table for the test.

Note: If the input table contains a character column, and that column contains control characters in any row, do not use the table as input for publishing validation tests.

For character variables, output values longer than 32767 characters will be truncated.

If you are validating content that was published to SAS Micro Analytic Service, the time required to run the test depends on the number of worker threads on your system, the number of threads in the middle tier, and the network latency between CAS and the middle tier server. It is recommended that you select an input table with as few input records as needed to accurately test the published content. See [SAS Micro Analytic Service: Programming and Administration Guide](#) for more information.

- 5 (Optional) Expand the **Advanced** section, click  in the **Output data library** field, and select a library to store the validation test output data.
- 6 Click **Run** to run the test. Alternatively, click **Save** to save the test definition without running it.

The status of the test is indicated by the icon in the **Status** column. For explanations of each icon, see [“Status Icons for Tests” on page 499](#).

- 7 Click  in the **Results** column to view the test results.
- 8 In the test results window, click **Test Results** in the navigation pane to display the URIs and other information for the test. Click **Output**, **Code**, or **Log** to display the output data set, the code that was generated by SAS Intelligent Decisioning, or the SAS log that was generated when the code was run.

TIP You can click on the values of character variables to display the entire value in a separate window. For data grid variables, you can choose to view the variable value in three different formats:

- Click the **Data Grid** tab to view the data grid value as a table.
- Click the **Formatted** tab to view the data grid as a formatted JSON character string.
- Click the **Plain** tab to view the data grid as an unformatted character string.

TIP On the **Log** page, you can click  to download the log file.

- 9 Click **x** to close the test results.

Working with Test and Validation Test Output Data

After you run a test or a publishing validation test, you can work with the output table in other SAS applications to analyze the data, create and compare models, discover relationships hidden in the data, and generate reports based on the data.

Note: The actions available to you depend on the applications that are available at your site.

On the Test Results page, select the **Output** table in the navigation pane, click **Actions**, and select one of the following options:

Explore Lineage

opens SAS Lineage Viewer. SAS Lineage Viewer enables you to better understand the relationships between objects in your SAS Viya platform applications. These objects include data, transformation processes, reports, and visualizations. For more information, see [SAS Lineage: User's Guide](#).

Explore and Visualize

opens the output table in SAS Visual Analytics. SAS Visual Analytics enables you to create, test, and compare models based on the patterns discovered during exploration of the data. You can export the model before or after performing model comparison for use with other SAS products or to put the model into production. SAS Visual Analytics supports a range of visualization, discovery, and reporting features. For more information, see [Welcome to SAS Visual Analytics](#).

Access and Manage Data

opens the output table in SAS Data Hub. SAS Data Hub enables you to import and download tables, load tables into memory, unload tables, and more. For more information, see [SAS Data Hub: User's Guide](#).

Status Icons for Tests

Icon	Status
	The test is not ready to run. The test definition is not complete, or it might contain errors.
	The test has been defined and can be run. Some input variables have not been mapped or have not been assigned a value, so the test might execute only a single path through the decision flow.

Icon	Status
	The test is defined correctly and is ready to run.
	The test is running.
	The test completed successfully.
	The test completed, but warnings were issued in the SAS log. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.
	The test did not run successfully. Check the SAS log for information. The URI to the log file is shown on the Test Results page. On the Test Results page, click Test Results in the navigation pane to display the URI.

Duplicate Tests

To duplicate a single test:

- 1 On the **Scoring** tab, click the **Tests** tab, the **Scenarios** tab, or the **Publishing Validation** tab, depending on the test that you want to duplicate.
- 2 Select the check box for the test that you want to duplicate, click  on the subtab, and select **Duplicate**.
- 3 In the Duplicate Tests window, enter a name for the duplicate test if you do not want to use the default name.
- 4 (Optional) Enter a description for the duplicate test.
- 5 Click **Duplicate**.

To duplicate multiple tests:

- 1 On the **Scoring** tab, click the **Tests** tab, the **Scenarios** tab, or the **Publishing Validation** tab, depending on the test that you want to duplicate.
- 2 Select the check boxes for the tests that you want to duplicate, click  on the subtab, and select **Duplicate**.

SAS Intelligent Decisioning duplicates the tests and adds `_Copy` to each filename.

Manage Comments for Decision Nodes and Tests

You can associate comments and attachments with the Start and End nodes in a decision and with decision tests.

To open the Comments properties pane, open the decision, select a node or a decision test, and click  in the property pane.

To add a new comment, enter the comment in the text box and click **Post**.

To add an attachment to a comment, click , select the file that you want to attach, and click **Post**. (The attachment icon appears after you enter text in text box.) You cannot attach executable files such as BAT and EXE files.

To reply to a comment, click **Reply**, enter your reply in the text box, and click **Post**.

To edit an existing comment, click  for the comment, and select **Edit**. Edit the comment, and click **Post**.

To delete a comment, click  for the comment, and select **Delete**.

Executing Published Content

How you execute published content depends on the destination to which the content is published.

Executing Content Published to SAS Micro Analytic Service Destinations

The user who is executing the published content must be authenticated. In SAS Viya, authentication options vary, based on which interface and operating system are used in your environment. External mechanisms include direct LDAP authentication, host authentication, Kerberos, Security Assertion Markup Language (SAML), and OAuth 2.0 with OpenID Connect. For additional information, see [SAS Viya Platform: Authentication](#)

When a decision is published from SAS Intelligent Decisioning to a SAS Micro Analytic Service destination, an EXECUTE step is created in the published module. For information about the request and response data formats used in this step, see

Execute a step in the REST API documentation for the Micro Analytic Score API. For an example that uses Python to execute a published decision in the maslocal destination, see [“Execute a Published Decision” in SAS Intelligent Decisioning: Decision Management REST API Examples](#).

IMPORTANT If the SAS Micro Analytic Service configuration property `service.removetrailingunderscoresfrominputs` and the SAS Intelligent Decisioning property `sas.decisions.masnode.removeTrailingUnderscoresFromInput` are not defined or are set to **False**, add an underscore to the name of each input variable. If these options are set to **True**, do not add underscores. Your administrator can add the SAS Micro Analytic Service property to the `supplementalProperties` section in the `sas.microanalyticsservice.system` configuration definition in SAS Environment Manager. The default value for these options is **False**. For additional information, see [“sas.microanalyticsservice.system: supplementalProperties” in SAS Micro Analytic Service: Programming and Administration Guide](#) and [“sas.decisions.masnode.removeTrailingUnderscoresFromInput” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Executing Content That Has Been Published to SAS Cloud Analytic Services Destinations

To execute content that has been published to SAS Cloud Analytic Services (CAS), use the CAS Model Publishing and Scoring action set. For example, the following code runs a model named `Evaluate_Loans` on the local CAS server.

```
/* Start a CAS session named _mmcas_. */
cas _mmcas_;

/* Add the formats library for lookup */
/* tables to your search path. */
sessionProp.setFmtSearch / fmtLibNames="userformats3";

/* Create librefs for all existing caslibs so that they */
/* are visible in the SAS Data and AI Studio Libraries tree. */
caslib _all_ assign;

proc cas;
  /* Specify the session to use for the runModelLocal action. */
  session _mmcas_;

  /* Define the parameter list for the runModelLocal action. */
  /* Reload the destination model table ("targetCode") */
  /* before you execute the decision. */
  destination_model_table = "targetCode";
  destination_model_lib = "public";
```

```

destination_model = "Evaluate_Loans";

dp_inputTable="hmeq_test";
dp_inputCASLib="public";

dp_outputTable="hmeq_test_dm";
dp_outputCASLib="public";

parmlist = {
  modelTable={
    name=destination_model_table,
    caslib=destination_model_lib
  },
  modelName=destination_model,
  inTable={
    name=dp_inputTable,
    caslib=dp_inputCASLib
  },
  outTable={
    name=dp_outputTable,
    caslib=dp_outputCASLib
  }
};

/* Load the modelPublishing action set. */
loadactionset "modelPublishing";

/* Set the library search list for lookup tables. */
action setFmtSearch status=status / fmtLibNames="userformats3";
run;

/* (Optional) Display the list of libraries that will be searched.
*/
listFmtSearch result=r;
print "Format library search order:";
if (r.ListFmtSearch.nrows > 0) then
  do i=1 to r.ListFmtSearch.nrows ;
    print r.ListFmtSearch[i];
  end;
run;

/* Run the model locally on the CAS server. */
action runModelLocal submit result=r status=rc / parmlist;

/* Load the table action set. */
loadactionset "table";

/* Promote the output table */
table.promote /
  caslib=dp_outputCASLib
  name=dp_outputTable
  targetLib=dp_outputCASLib;
run;
quit;

```

```
/* Terminate the CAS session named _mmcas_. */
cas _mmcas_ terminate;
```

If SAS Data and AI Studio is licensed at your site, you can submit this code in SAS Data and AI Studio. To open SAS Data and AI Studio, click  and select **Develop Code and Flows**. For more information, see [SAS Data and AI Studio: User's Guide](#).

You can view additional examples of using this CAS action set to execute published content by viewing the test results that are generated by publishing validation tests. On the Test Results page for a decision, click **Code** to display the code that was generated by SAS Intelligent Decisioning. For information about running publishing validation tests and viewing the results, see [“Validate a Published Decision” on page 497](#).

For more information about CAS and the Model Publishing and Scoring action set, see the following documentation:

- [Getting Started with CASL Programming](#)
- [SAS Cloud Analytic Services: CASL Reference](#)
- “Model Publishing and Scoring Action Set” in [SAS Visual Analytics: Programming Guide](#)
- [SAS Cloud Analytic Services: User's Guide](#)

Content Executed by Published Decisions

When you execute a published decision, the version of the content that is executed depends on the publishing destination.

Destination Type	Content That Is Executed by Published Decisions
SAS Micro Analytic Service	Most of the published decision's content (the SAS Micro Analytic Service module) is locked. Updates to rule sets, models, custom functions, or custom code files that are used in the decision do not affect the published module. Treatment groups are embedded in decisions if the <code>masInlineTreatmentGroup</code> configuration option is turned on. If this option is off, treatment groups are not embedded, and published modules use newly activated versions of treatment groups. For more information, see “sas.decisions.masInlineTreatmentGroup” in SAS Intelligent Decisioning: Administrator's Guide. For information about the value of global variables that are used in published content, see “Which Value Is Used for a Global Variable?” on page 181. Lookup tables are embedded in decisions if the <code>lookupStaticBinding</code> configuration option is turned on. If this option is off, lookup tables are not embedded, and published modules use newly activated versions of lookup tables. For more information, see

Destination Type	Content That Is Executed by Published Decisions
	“sas.businessrules.lookupStaticBinding” in SAS Intelligent Decisioning: Administrator’s Guide. For Micro Analytic Module nodes, the decision always uses the most recently published version of the specified module.
SAS Cloud Analytic Services	Most of the published decision’s content is locked. Updates to rule sets, models, treatment groups, custom functions, or custom code files that are used in the decision do not affect the published decision. For information about the value of global variables that are used in published content, see “Which Value Is Used for a Global Variable?” on page 181. Lookup tables are locked if the lookupStaticBinding configuration option is turned on. If this option is off, lookup tables are not locked, and published modules use newly activated versions of lookup tables. For more information, see “sas.businessrules.lookupStaticBinding” in SAS Intelligent Decisioning: Administrator’s Guide.
Teradata, Hadoop, Git, Databricks, SingleStore, and Container Destinations	All of the published decision’s content, including treatment groups, custom functions, lookup tables, and global variables is included inline in the published module. Updates to the objects used in the decision are not used by the published module.

View All Published Decisions

The Deployments category view is a dashboard that provides a centralized view of information about all the decisions that are being used in your environment.

Click  to navigate to the Deployments category view.

The tiles at the top of this view visually display key metrics about the total number of decisions in your environment. For example, the tiles display information such as destination types, deployments to single destinations or to multiple destinations, and deployments with and without models.

TIP You can click  to hide the tiles.

The tabular view below the tiles displays details about each published decision in your environment. For example, details include information such as the decision name, the published destination, the published date, the number of nodes, and associated tags. By default, the decisions that were published within the last 6 months are included.

The table also includes the current version for each decision. You can click a decision version in the Decision (Version) column to see the details for that version. The decision version opens on the **Decision Flow** tab. For more information about viewing and editing decisions, see [“What can I include in a decision?” on page 391](#).

You can search for a decision by name by typing in the search field. Click **Advanced search** to search for deployed decisions that were published on a specific date or within a range of dates.

Using Custom Workflows with SAS Intelligent Decisioning

<i>About Custom Workflows and SAS Intelligent Decisioning</i>	507
<i>Defining Custom Workflows</i>	508
Requirements for Custom Workflows for Use with SAS Intelligent Decisioning ..	508
Data Objects That Are Required in a Custom Workflow for SAS Intelligent Decisioning	509
Defining Prompts in SAS Workflow Manager	510
Controlling Publishing from within a Workflow	513
<i>Start a Workflow Instance for a Decision</i>	515
<i>Working with Tasks</i>	516
Claim or Release a Task	516
Complete a Task	516
<i>View the Status Change History</i>	517
<i>Using the Sample Workflow</i>	517
About the Sample Workflow	517
Tasks and Status Values in the Sample Workflow	518

About Custom Workflows and SAS Intelligent Decisioning

You can use custom workflows that you develop in SAS Workflow Manager to manage the development, approval, and deployment of decisions in SAS Intelligent Decisioning. Your administrator must configure support for SAS Workflow in order to use custom workflows with SAS Intelligent Decisioning. For more information, see [“Configure SAS Workflow for Use with SAS Intelligent Decisioning”](#) in *SAS Intelligent Decisioning: Administrator’s Guide*.

Any user who has [permission to create workflow definitions](#) can define a custom workflow. The workflow must define the data objects and meet the other requirements that are described in “[Defining Custom Workflows](#)” on page 508.

If the following requirements are met, a workflow appears in the list of workflows that [can be started](#) for a decision in SAS Intelligent Decisioning:

- The workflow definition specifies **SAS Intelligent Decisioning** as its [client identifier](#).
- The definition has been [activated](#).
- The user who is starting a workflow for a decision has Read permission for the workflow and Update permission for the decision.

When a workflow is started, a new instance of the workflow is created for the decision. A workflow instance is associated with a specific version of a decision. Each decision version that is actively being worked on has its own workflow instance.

After a workflow is started, the active tasks that you have permission to claim appear on your SAS Intelligent Decisioning [home page](#). If you have permission to work with a task, you can also claim, release, and complete tasks from the [Tasks menu](#) in an open decision.

SAS Intelligent Decisioning provides a sample workflow named Sample Decision Approval v2 workflow. For more information, see “[Using the Sample Workflow](#)” on page 517.

Defining Custom Workflows

Requirements for Custom Workflows for Use with SAS Intelligent Decisioning

The following items are required for workflows that are developed in SAS Workflow Manager and are for use with SAS Intelligent Decisioning.

- Select **SAS Intelligent Decisioning** as the client identifier for the workflow. Select the identifier in the New Definition window when you create the workflow in SAS Workflow Manager. For more information, see “[Create and Activate a Workflow Definition](#)” in *SAS Workflow Manager: User’s Guide* and “[Specify a Client Identifier](#)” in *SAS Workflow Manager: User’s Guide*.
- Verify that the workflow definition defines the data objects that are described in “[Data Objects That Are Required in a Custom Workflow for SAS Intelligent Decisioning](#)” on page 509. If you select the **SAS Intelligent Decisioning** client identifier in the New Definition window, SAS Workflow Manager defines the data objects for you. If you select the client identifier in the Definition Properties window, you must define these data objects manually. For more

information, see [“Create and Manage Data Objects” in SAS Workflow Manager: User’s Guide](#).

Data Objects That Are Required in a Custom Workflow for SAS Intelligent Decisioning

A workflow that is used for developing a decision in SAS Intelligent Decisioning must define all of the data objects that are in the following table. If you select the **SAS Intelligent Decisioning** client identifier in the New Definition window in SAS Workflow Manager, all of these data objects are defined for you. For more information, see [“Create and Activate a Workflow Definition” in SAS Workflow Manager: User’s Guide](#) and [“Specify a Client Identifier” in SAS Workflow Manager: User’s Guide](#).

The data type for the `idIsPublishing` data type is Boolean. The data type for all other data objects is Character.

Table 14.1 Data Objects That Are Required in a Custom Workflow for SAS Intelligent Decisioning

Data Object Name	Description	Comments
<code>idDecisionState</code>	The current status of the decision’s workflow instance.	When a workflow with the SAS Intelligent Decisioning client identifier is started, its status value is automatically set to Developing. Custom workflows can store other status values to this data object.
<code>idComments</code>	Comments entered by the user if prompted.	These comments are written to the workflow history for the workflow instance.
<code>idDecisionId</code>	The ID of the decision that is running in the workflow instance.	SAS Intelligent Decisioning automatically sets the values of the following data objects when a new workflow instance is started: <code>idDecisionId</code> , <code>idDecisionRevisionId</code> , <code>idDecisionName</code> , <code>idDecisionMajorRevision</code> , and <code>idDecisionMinorRevision</code> .
<code>idDecisionRevisionId</code>	The ID of the decision revision that is running in the workflow instance.	
<code>idDecisionName</code>	The name of the decision that is running in the	

Data Object Name	Description	Comments
	workflow instance.	
idDecisionMajorRevision	The major revision number of the decision revision that is running in the workflow instance.	
idDecisionMinorRevision	The minor revision number of the decision revision that is running in the workflow instance.	
idsPublishing	Determines whether the workflow disables the Publish button when the workflow is started for a decision. To enable the Publish button, specify False. To disable the Publish button, specify True.	The default value for the Sample Asset Approval v2 workflow is False. For more information, see “Controlling Publishing from within a Workflow” on page 513.

Defining Prompts in SAS Workflow Manager

Using Attributes to Define Prompts

When you define prompts in a workflow that is for use with SAS Intelligent Decisioning, you can use attributes to control how the prompts are displayed to the user.

The [Attribute](#) field is described in [“Define a New Prompt” in SAS Workflow Manager: User’s Guide](#). This field provides two options for defining prompts:

- **Use custom prompt for client identifier (SAS Intelligent Decisioning) via attribute** This option enables you to define a prompt based on a pre-defined attribute. There are no pre-defined attributes for SAS Intelligent Decisioning. For more information, see [“Add Attributes to a User Task” in SAS Workflow Manager: User’s Guide](#).
- **Set prompt properties via attribute** enables you to use key-value pairs to set properties for the prompts. Key names must begin with a letter and can contain only alphanumeric characters and underscores.

The following key names are pre-defined for use with SAS Intelligent Decisioning:

order

Specifies the ordinal number of the prompt in the window. For example, if the order is 2, the prompt is the second prompt that is displayed in the resulting window.

lines

Specifies the number of lines to display for a multiline input field of type Character.

type

Specifies the type of button that is added to the field. [Table 14.2 on page 511](#) describes the available types.

Table 14.2 *Type Values and Button Icons*

Type	Button Icon	Description
date		Enables you to select a date.
time		Enables you to select a time.
datetime		Enables you to select a datetime value.
users		Enables you to select a user.
groups		Enables you to select a user group.
usersgroups		Enables you to select users or user groups.

Example: Defining Prompts

Suppose you want to display the following prompt window when the user completes the task named Test Decision.

Figure 14.1 Example Prompt Window

Complete - Test Decision

Date and time:

User:

Comments:

Results:

Ok **Cancel**

To define the **Date and time**, **User**, and **Description** prompts, you must use key-value pairs to define the properties for each prompt. In the New Prompt window, enter the key-value pairs for each prompt as shown in [Table 14.3](#) as follows:

- 1 For the **Attributes** field, select **Set prompt properties via attribute**.
- 2 In the **Properties** field, click .
- 3 In the Edit Prompt Properties window, add a new row for each of the key-value pairs that are shown in [Table 14.3](#).

Table 14.3 Example Key-Value Properties

Prompt Label	Key-Value Pairs
Date and time:	order: 1 type: datetime

Prompt Label	Key-Value Pairs
User:	order: 2 type: users
Description:	order: 3 lines: 3

The fourth prompt, **Results** is a selection list. To define this prompt in the New Prompt window, add a new row to the **Displayed Attributes** table for each entry in the selection list. For each entry shown in [Table 14.4](#), click  to add a new row to the table, and enter the name and value for the list entry.

Table 14.4 Selection List Values for the Result Prompt

Name	Value
Test ran successfully	success
Test completed with errors	completedWithErrors
Test failed	failed

For more information about defining and reusing prompts, see “[Managing Prompts](#)” in *SAS Workflow Manager: User's Guide*.

Controlling Publishing from within a Workflow

Note: By default, SAS Intelligent Decisioning displays the **Publish** button within open decisions. However, if your administrator has turned on the [sas.intelligent.decisioning.requirePublishAndLock](#) configuration option, this button is replaced with the **Publish and lock** button. The setting of the `idlsPublishing` data object that is described in “[Enabling and Disabling the “Publish” Button](#)” on page 514 affects whichever button is displayed.

By default, the **Publish** button or the **Publish and lock** button is always enabled within an open decision. However, custom workflows can control when this button is enabled. For example, your workflow might require that certain tasks be completed before the decision is published. In these cases, the workflow can disable the button when the workflow starts, and at the appropriate point in the workflow, you can define a user task that enables the button. The button is automatically re-enabled when the workflow completes or is terminated. For more information, see “[Enabling and Disabling the “Publish” Button](#)” on page 514 and

[“Example: Define a User Task That Enables the “Publish” or “Publish and Lock” Button” on page 514.](#)

Enabling and Disabling the “Publish” Button

The setting of the workflow's `idIsPublishing` data object determines whether the **Publish** button or the **Publish and lock** button is enabled.

`idIsPublishing = False`
enables the button. The default setting is False.

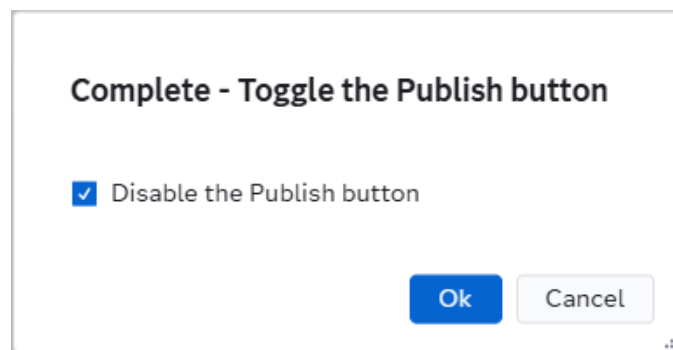
`idIsPublishing = True`
disables the button.

To ensure that the `idIsPublishing` data object is set when the workflow starts, specify its setting in the workflow's Definition Properties window. For more information, see [“Edit Definition Properties” in SAS Workflow Manager: User's Guide](#).

For information about all of the workflow data objects that are used by SAS Intelligent Decisioning, see [“Data Objects That Are Required in a Custom Workflow for SAS Intelligent Decisioning” on page 509](#).

Example: Define a User Task That Enables the “Publish” or “Publish and Lock” Button

If you have a workflow that sets the `idIsPublishing` data object to False in the workflow's Definition Properties window, you can use a user task to change this setting to True at an appropriate point in the workflow. Suppose you want to display the following prompt when a user completes the task named “Toggle the publish button”.



The `idIsPublishing` data object is a Boolean data object, so SAS Workflow displays only a check box. If the user selects the check box, the workflow sets the value of the `idIsPublishing` data object to True. If the user clears the check box, the workflow sets the value of the data object to False, which enables the **Publish** button.

In the New Prompt window for the user task in SAS Workflow Manager, define the prompt as follows:

- 1 In the workflow definition diagram, select the user task for which you want to define the prompt.
- 2 Click  to display the **Prompts** property pane.
- 3 Click **+** and select **Add new prompt**. The New Prompt window appears.
- 4 In the **Prompt label** field, enter **Disable the Publish button**.
- 5 In the **Data object** field, select **idIsPublishing**.
You do not need to enter any attributes or displayed values.
- 6 Click **OK**.

For additional information about defining a prompt, see [“Define a New Prompt” in SAS Workflow Manager: User’s Guide](#).

Start a Workflow Instance for a Decision

Note: After a workflow is started for a decision, that decision cannot be run in another workflow or in another instance of the same workflow.

- 1 Open the decision.

TIP If a workflow was selected when the decision was created, you can click **Workflow** on the **Decision Flow** tab, and select **Start workflow**. If the **Workflow** button does not appear, select the workflow on the **Properties** tab.

- 2 Click the **Properties** tab.
- 3 Select the workflow that you want to start in the **Workflow** field, and click .

Note: The list of workflows includes only those workflows that have been assigned the SAS Intelligent Decisioning client identifier and that you have permission to start. You must have Read access to the workflow definition and Update access to the decision. This field is disabled if you do not have permission to start a workflow. If someone else has selected a workflow but you do not have Read access to the workflow, the **Workflow** field contains asterisks.

- 4 Click **Start**. SAS Intelligent Decisioning starts a new workflow instance for the decision, and sets the workflow status to Developing.

If prompts are associated with the Start node in the selected workflow, SAS Intelligent Decisioning displays these prompts in the Start Workflow window.

- 5 In the Start Workflow window, address any prompts that are displayed by the workflow, and click **Start**.

Working with Tasks

Claim or Release a Task

You can claim or release a task either on the SAS Intelligent Decisioning [home page](#) or from within an open decision. To claim or release a task from within a decision:

- 1 Open the decision, click **Tasks**, and select the task that you want to claim or release.

.....

Note: The tasks that appear in the **Tasks** menu are determined by the participants that are associated with the user tasks in the workflow. For more information, see “[Managing Participants](#)” in *SAS Workflow Manager: User’s Guide*.

.....

- 2 In the Task window, click **Claim** or **Release**.
- 3 Click **Close**.

Complete a Task

You can complete a task either on the SAS Intelligent Decisioning [home page](#) or from within an open decision. To complete a task from within a decision:

- 1 Open the decision, click **Tasks**, and select the task that you want to complete.
- 2 In the Tasks window, click **Complete**.
- 3 In the Complete window, address any prompts that are defined by the workflow for the task that you are completing.

If you are using the sample workflow that is provided by SAS Intelligent Decisioning, select the new status value for the decision, and, optionally, enter any comments that you want to associate with the task in the workflow history.

- 4 Click **OK**.

The final task in the [sample workflow](#) is "Deploy decision". When you complete this task, the workflow status is set to Deployed. Workflow execution then proceeds immediately to the End node, and the workflow status is set to Completed.

View the Status Change History

You must have permission to view the status history. For more information, see ["Configure SAS Workflow for Use with SAS Intelligent Decisioning" in SAS Intelligent Decisioning: Administrator's Guide](#).

To view the history of workflow status changes for a decision:

- 1 Open the decision, click , and select **View status change history**. The Status Change History Range window appears.
- 2 Select the dates or versions for which you want to view the history:
 - Select **Dates**, select a start date, and, if needed, select an end date.
 - Select **Versions**, and, if needed, select a different start version and end version.
- 3 Click **OK**. The Status Change History window appears.

To download a status history report in the Status Change History window, click **Download**. SAS Intelligent Decisioning downloads the report as a comma-separated values (CSV) file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

Using the Sample Workflow

About the Sample Workflow

SAS Intelligent Decisioning provides a sample workflow named Sample Decision Approval v2. You can use this sample workflow for developing decisions, or you can design and use your own custom workflows.

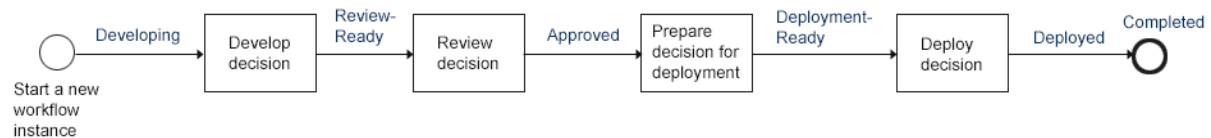
As with any workflow, in order to put a decision into this sample workflow, you must have Read permission for the workflow and Update permission for the decision. To claim and complete a task in the sample workflow, you must be added

as a participant on the task. Before you change the permissions or participants for the sample workflow, it is recommended that you duplicate the workflow and modify it. For more information, see [“Administering the Sample Decision Approval v2 Workflow” in SAS Intelligent Decisioning: Administrator’s Guide](#).

As each of the users who are responsible for developing, testing, reviewing, approving, and deploying a particular version of a decision complete their tasks, SAS Intelligent Decisioning sets the status of the workflow to the appropriate value.

Tasks and Status Values in the Sample Workflow

The movement of a decision version through the Sample Decision Approval v2 workflow is shown in the following figure. The initial status value of a new decision version is Developing. The tasks and the associated status value are described in [Table 14.5 on page 518](#).



Note: This figure does not show the actual workflow. It shows only the tasks and status values as a decision moves through the Sample Decision Approval v2 workflow.

Table 14.5 Tasks and Status Values

Task	Default Workflow Status When Task Is Completed	Comments
Start workflow	Developing	When you start the workflow, SAS Intelligent Decisioning automatically sets the workflow's status to Developing.
Develop decision	ReviewReady	When you complete any of these tasks, the Tasks menu shows the next task that you have permission to claim.
Review decision	Approved	
Prepare decision for deployment	Deployment-ready	

Task	Default Workflow Status When Task Is Completed	Comments
Deploy decision	Deployed	When you complete this task, the workflow status changes to Deployed, and workflow execution immediately proceeds to the workflow's End node.
n/a	Completed	The workflow's End node automatically sets the workflow's status to Completed, and the workflow instance ends. Until the workflow's status is set to Completed, the workflow can be moved back to a previous status value.

Note: A task or a workflow instance can be terminated by an administrator in SAS Workflow Manager. If an administrator terminates a task or a workflow instance, the task or workflow status is set to Cancelled.

If a version's status is not set to Developing, the lock  icon appears beside the version number in the Version column. You can edit or delete a decision version only if its status is set to Developing. You can delete a version only if its status is blank, or if its status is set to Developing and the version's status has never been set to Deployed. After a version's status has been set to Deployed, you cannot delete the version or change its status. To make additional changes to the decision, you must create a new version.

The **Status** column in the Decisions category view shows the current workflow status of the latest version of each decision. The **Status** column on the **Versions** tab for a decision displays the status value for each version of a decision. If a workflow has not been selected for the version, the **Status** column displays **Workflow not selected**. If a workflow has been selected but it has not been started, the **Status** displays **Workflow not started**.

The  icon also appears next to the version number for any version that is not the latest version. You can edit the content of only the latest version of a decision, and you can edit its content only if its status is Developing.

Managing and Using Tags

<i>About Tags</i>	521
<i>Configuring and Administering Tags</i>	522
<i>Managing the List of Tags</i>	522
Create a Tag	522
Change a Tag Name or Description	523
Delete a Tag	523
<i>Managing Tags That Are Associated with Object Versions</i>	523
Add a Tag to an Object Version	523
Remove a Version Tag	524
Modify a Version Tag	524
<i>Determine Which Objects Are Associated with a Tag</i>	525
<i>Compare the Tags That Are Associated with Objects</i>	525

About Tags

Tags enable you to better organize and group your content. For example, you can assign the same tag to all of the rule sets, treatment groups, decisions, and other object that are used for a specific campaign or version of a campaign. You can use tags to identify a set of objects that require updates or additional development. In this case, your tag might match a ticket number in the system that you use to track development work.

If the objects in a decision are associated with tags for a specific version, you can create a new version of the decision by using these object tags to select which version of the included objects to use in the new decision version. For more information, see [“Creating a New Version by Using Tags” on page 453](#).

Configuring and Administering Tags

Using a predefined set of tags reduces the possibility of data entry errors when users manually enter tags. You can designate a tag administrator who can define a specific set of tags for use at your site. You can then set configuration options and permissions to require the use of tags, to restrict the tags to the set that is defined by your administrator, or both. For more information about configuration options, see [“SAS Intelligent Decisioning Service Properties” in SAS Intelligent Decisioning: Administrator’s Guide](#).

Tag administrators must have the appropriate permissions in order to be able to create and manage tags. Authenticated users must have permission to view tags and associate tags with object versions. For information about setting permissions for tag administrators and setting general permissions for users, see [“Configuring Support for Administering and Using Tags” in SAS Intelligent Decisioning: Administrator’s Guide](#).

See [“Grant Permission to Create Versions in Check-Out Folders by Using Tags” in SAS Intelligent Decisioning: Administrator’s Guide](#) for information about setting permissions for using tags to create new object versions.

Managing the List of Tags

Create a Tag

- 1 Click  to navigate to the Tag Management category view.
- 2 Click **New tag**. The New Tag window appears.
- 3 Enter a name for the tag. Tag names must be unique within SAS Intelligent Decisioning.
- 4 (Optional) Enter a description for the new tag. Descriptions are limited to 1000 characters.
- 5 Click **Save** to create the tag.

Change a Tag Name or Description

Note: When you rename a tag, the tag name is changed for all of the items that are associated with the tag.

- 1 In the Tag Management category view, select the tag that you want to change, and click  to open the Properties pane for the tag
- 2 (Optional) Click  next to the **Name** field, enter a new name for the tag, and click **Rename**.
- 3 (Optional) Click  next to the **Description** field, enter a new description for the tag, and click **Save**.

Delete a Tag

When you delete a tag, it is removed from all of the items to which it has been added.

In the Tag Management category view, select the tags that you want to delete, and click .

Managing Tags That Are Associated with Object Versions

Add a Tag to an Object Version

Tags are associated with specific versions of an object and not with the entire object. You can add the same tag to any version of any object.

To add a tag to a version of an object:

- 1 On the **Versions** tab for the object, position your cursor in the **Version Tags** column for the version that you want to tag. If the version is not tagged,  appears. If the version has at least one tag,  appears.

TIP For some object types, you can filter the list of object versions based on the tags. To filter the list, right-click on the **Version Tags** column heading, enter a filter string in the **Filter** box, and press Enter. To clear the filter, delete the string from the **Filter** box, and press Enter.

- 2 Click **+** to open the New Version Tags window, or click  to open the Edit Version Tags window.
- 3 Do one of the following:
 - Select a tag from the list of previously entered tags. To display the list of existing tags, either click  and select the tag, or type the first letters of the tag, select the tag, then press Enter.
 - Enter a new custom tag. Tags are limited to 100 characters each and cannot contain single quotes ('). Press Enter after each new custom tag.

Note: If your administrator has turned on the [sas.intelligent.decisioning.requireTagFromSelection](#) configuration option, you cannot manually enter new custom tags. You must select a tag from the list of existing tags.

- 4 Click **Close** to close the window.

Remove a Version Tag

- 1 On the **Versions** tab, position your cursor in the **Version Tags** column for the version whose tag you want to remove, and click .
- 2 Click **x** beside the tag that you want to remove.
- 3 Click **Close** to close the window. The tag remains in the list of previously entered tags that is displayed when you add a tag, but the tag is no longer associated with version.

Modify a Version Tag

You cannot modify a version tag that already exists. To change the content of an existing tag, delete the tag as described in [“Remove a Version Tag” on page 524](#), and then add the tag again as described in [“Add a Tag to an Object Version” on page 523](#).

Determine Which Objects Are Associated with a Tag

To display the list of objects that is associated with a tag, click the number in the **Number of Items Tagged** column for that tag in the Tag Management category view. SAS Intelligent Decisioning opens the Associated Items window, which displays each object name, its type and version, plus additional information.

To export a PDF that contains the list of objects that are associated with a tag, select the tag, click , and select **Export to PDF**. For each object, the PDF includes the name, version, type, and URI. It also includes the user ID and date on which the object was created and the user ID and date on which the object was last modified.

Compare the Tags That Are Associated with Objects

- 1 Click  to navigate to the Tags Management category view.
- 2 Select the two tags that you want to compare, click , and select **Compare tags**. The Compare Tags window appears.

By default, the Compare Tags window displays tag cards at the top of the window and the table of objects associated with the tags at the bottom.

The tag cards display metadata for each tag such as the total number of items tagged, the date on which the tag was last modified, and the user IDs that created and modified the tag. Click **Hide summary** or **Show summary** to hide or display these tag cards.

By default, the table lists each object version that is associated with one of the tags but not with both of the tags, and it highlights the differences. In the column for a specific tag, a check mark in a specific row indicates that the tag is associated with the object version for that row. Click **Show all** to include object versions that are associated with both of the tags. Click **Show differences** to return to the default view.

For example, in the following figure, the credit_check_v3 tag is associated only with version 1.2 of the credit_card_offers decision. The credit_check_v2 tag is associated with all other objects.

SAS® Intelligent Decisioning - Build Decisions

< Tag Management

Compare Tags

Hide summary Show differences Show all Export Close

credit_check_v2
 3
 Total Number of Items ...

Date modified: Jul 2, 2025 8:07:02 PM
 Modified by: userID
 Created by: userID

credit_check_v3
 1
 Total Number of Items ...

Date modified: Jul 2, 2025 8:07:32 PM
 Modified by: userID
 Created by: userID

Filter items

Items	Version	credit_check_v2	credit_check_v3
credit_card_offers	1.1	✓	
credit_card_offers	1.2		✓
Evaluate_Loans	2.0	✓	
Evaluate_Loans	1.0	✓	

- (Optional) Click **Export** to export the results of the comparison to a PDF file. The result depends on which browser you are using and the browser's default download actions. The browser might automatically download the file and display a notification. Alternatively, the browser might prompt you to save the file or open it with a specific application.

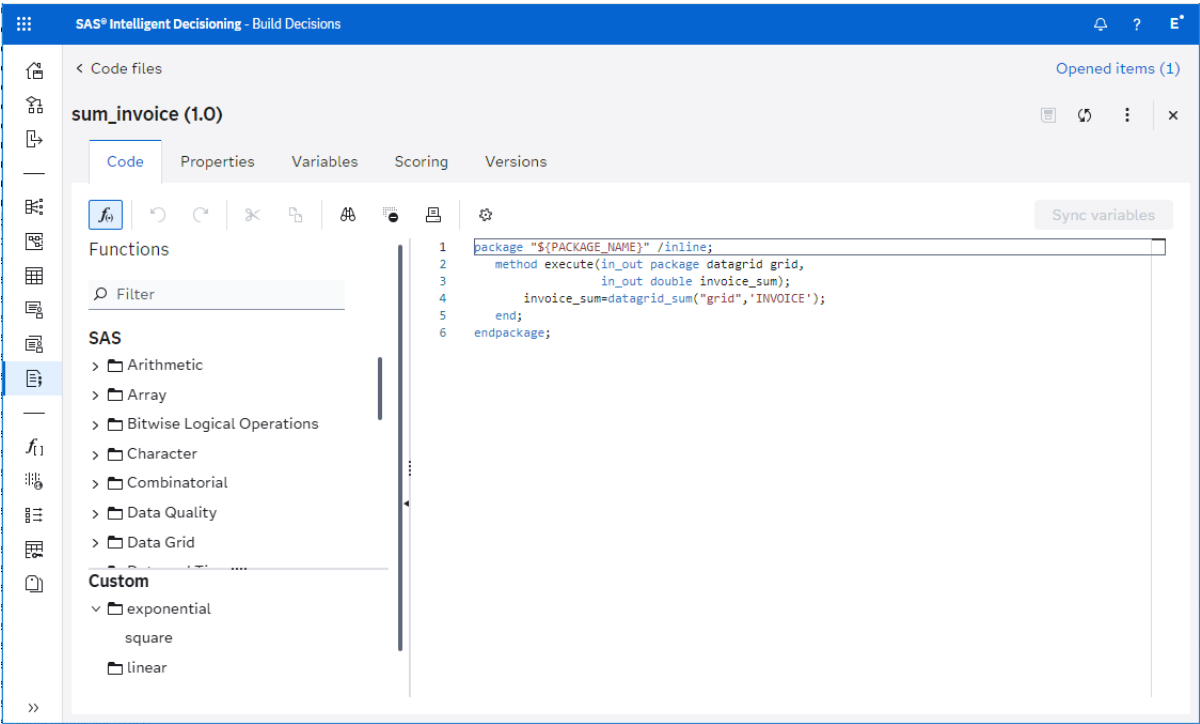
Using the Code Editor and Expression Editor

<i>Using the Code Editor</i>	527
<i>Code Editor Settings</i>	528
General Editor Settings	528
Settings for Brackets and Quotation Marks	530
Find and Replace Settings	532
Minimap Settings	533
Autocompletion and Suggestion Settings	533
Theme Settings	534
Settings for Syntax Highlighting	535
<i>Using the Expression Editor</i>	535

Using the Code Editor

SAS Intelligent Decisioning includes a color-coded, syntax-checking editor for editing new or existing code. The editor includes a variety of features such as autocompletion, automatic formatting, and pop-up syntax help. The code editor automatically opens when you create DS2 code files, Python code files, data query files for which you select **SQL Editor**, and custom context files.

Figure 16.1 DS2 Code File in the Code Editor



When you are editing a DS2 code file, custom context file, or custom function file, you can click  to toggle the display of the **Functions** pane. To display a brief description and syntax information about a DS2 function, hover over the function name.

When you are editing a DS2 code file, you can use the **Sync variables** tab to propagate changes between the **Code** tab and the **Variables** tab. For more information, see [“Syncing Variables and Code” on page 239](#).

You can change the code editor settings to customize the look and behavior of the code editor. To change the code editor settings, click . For more information, see [“Code Editor Settings” on page 528](#).

Code Editor Settings

General Editor Settings

Option	Description
Auto-indent	Specifies whether the current line's indentation is automatically adjusted when you type, paste, move, or

Option	Description
	indent lines. Click ⓘ for a description of each of the auto-indent settings that are available. ■ Never does not automatically insert any indentation. ■ Keep maintains the indentation of the current line. This option does not add indentation to a new block of code. ■ Brackets maintains the indentation of the current line, and adds indentation for any brackets properties that are defined for the language. ■ Advanced maintains the indentation of the current line, and adds indentation that is specified for any brackets and onEnterRules properties that are defined for the language. ■ Full maintains the indentation of the current line and adds indentation that is specified for any brackets, onEnterRules, and indentationRules properties that are defined for the language. This is the default value and the only option that adds indentation for SAS code.
Enable code folding	Enables you to collapse and expand regions of code in the code editor.
Print line numbers	Includes line numbers in the leftmost column when you print a program.
Show control characters	Displays control characters in the editor.
Show indent guides	Displays vertical lines that indicate where an indented section of code begins and ends.
Show line highlight	Highlights the specified portion of the line where the cursor is currently located. ■ Never does not highlight the current line. ■ Gutter highlights the area to the left of the code on the current line, including the line number if line numbers are displayed. ■ Current line highlights the code in the line where the cursor is located. ■ Gutter and current line highlights the entire line where the cursor is located, including the gutter and the code.
Show line numbers	Displays line numbers in the leftmost column in the editor. Click ⓘ for a description of each line number settings that are available. ■ Never turns off line numbers.

Option	Description
	■ Always displays all line numbers in ascending order. This is the default value. ■ Relative line numbers displays the current line number. The numbering of the other lines indicates their distance from the current line. For example, if the current line number is 10, the lines immediately above and below are numbered 1. ■ Every 10 lines displays every tenth line number as well as the current line number. For example, line numbers are displayed for the current line and lines 1, 10, 20, and 30.
Show whitespace characters	Displays a tiny dot for characters such as spaces and carriage returns in the editor. ■ Never displays spaces for whitespace characters. ■ Always displays small gray dots for all whitespace characters. ■ All except single characters between words displays small gray dots for all whitespace characters except for single whitespace characters between words. ■ Whitespace characters in selection displays small gray dots for all whitespace characters in the selected code. ■ Trailing whitespace characters displays small gray dots only for whitespace characters that occur past the end of each line's text.
Tab size	Specifies the number of spaces that are inserted into text when you insert a tab character. The default value is four spaces for each tab character.

Settings for Brackets and Quotation Marks

Option	Description
Auto-close brackets	Specifies whether a closing bracket is automatically added to an opening bracket. ■ Never does not add a closing bracket. ■ Always always adds a closing bracket. ■ Language-defined adds the closing brackets that are required by your coding language.

Option	Description
	■ When cursor is before whitespace character adds a closing bracket when the cursor is positioned before a whitespace character.
Auto-close quotation marks	Specifies whether a closing quotation mark is automatically added to an opening quotation mark. ■ Never does not add a closing quotation mark. ■ Always always adds a closing quotation mark. ■ Language-defined adds the closing quotation marks that are required by your coding language. ■ When cursor is before whitespace character adds a closing quotation mark when the cursor is positioned before a whitespace character.
Auto-remove matching characters	Specifies whether adjacent closing quotation marks or brackets are removed when the opening characters are deleted. ■ Never does not automatically remove any matching characters. ■ Always always removes matching characters. ■ For auto-inserted characters removes only matching characters that were automatically inserted.
Overtyping closing characters	Specifies whether to type over closing quotation marks or brackets. ■ Never does not type over any closing characters. ■ Always always types over closing characters. ■ For auto-inserted characters types over any closing characters that were automatically inserted.
Auto-surround	Specifies whether selected text is automatically enclosed by quotation marks or brackets when those characters are typed. ■ Never automatically surrounds the selected text. ■ Language-defined surrounds the text when it is required by your coding language. ■ Quotation marks automatically surrounds the text only when you enter an opening quotation mark. ■ Brackets automatically surrounds the text only when you enter an opening bracket.
Highlight matching brackets	Specifies whether matching brackets are automatically highlighted. Click ⓘ for a description of each setting that is available.

Option	Description
	■ Never does not highlight the matching bracket. ■ Always highlights matching brackets when you position the cursor adjacent to a bracket or between two brackets. This is the default value. ■ Near highlights matching brackets only when you position the cursor adjacent to a bracket.
Colorize matching brackets	Specifies whether to colorize matching brackets so that pairings are more easily distinguishable. If this setting is enabled, then changes to the syntax highlighting colors are not applied to the brackets.

Find and Replace Settings

Option	Description
Add blank space at the top	Specifies whether to add extra space at the top of the editor when the Find bar is displayed. This space moves the code down so that it is not overlaid by the Find bar.
Auto-find in selection	Specifies whether to limit your search to only the currently selected code rather than the entire file. ■ Never searches the entire file. ■ Always searches only the selected code. ■ In multiline selections only searches only selections with more than one line of code.
Move cursor to matches	Specifies whether the cursor moves to matches as you type.
Loop search	Specifies whether the search automatically restarts from the beginning (or the end) when no further matches can be found.
Prepopulate search string	Specifies whether the Find field is prepopulated with a search string. ■ Never does not prepopulate the Find box. ■ From selection or word at cursor position prepopulates the Find box with the word or selected code that starts at the current cursor position.

Option	Description
	■ From selection prepopulates the Find box with the selected code.

Minimap Settings

Option	Description
Show minimap	Specifies whether to display a small, zoomed-out preview of your code. This option enables you to easily view the structure of your code file and quickly navigate to different sections of your code.

Autocompletion and Suggestion Settings

The autocomplete feature can predict the next keyword before you type it completely.

Option	Description
Show auto-complete suggestions for comments	Specifies whether to display autocomplete suggestions for code comments. ■ Never offers no autocomplete suggestions. This is the default value. ■ Suggestions list displays a list of suggestions from which you can make a selection. ■ Inline auto-complete makes suggestions in the line in which you are typing.
Show auto-complete suggestions for strings	Specifies whether to display autocomplete suggestions for strings. Strings are text that is typically displayed in quotation marks. ■ Never offers no autocomplete suggestions. This is the default value. ■ Suggestions list displays a list of suggestions from which you can make a selection. ■ Inline auto-complete makes suggestions in the line in which you are typing.

Option	Description
Show auto-complete suggestions for other code	Specifies whether to display autocomplete suggestions for other code. ■ Never offers no autocomplete suggestions. ■ Suggestions list displays a list of suggestions from which you can make a selection. ■ Inline auto-complete makes suggestions in the line in which you are typing.
Show suggestions when a special (trigger) character is typed	Specifies whether to display suggestions when you enter a special character that triggers an action, such as @, which is used to insert a code snippet.

Theme Settings

The **Preview** section at the bottom of the Code Editor Settings window shows you how your current theme selections will look in the code editor. Your selections take effect after you close the Code Editor Settings window.

Options	Description
Syntax theme	Enables you to change the appearance of the code in the editor. The theme specifies the collection of colors and fonts that appear in the application. You can choose from these SAS themes: High Contrast Presents a dark background with high-contrast foreground elements to meet the needs of users with low vision. Dark Presents a dark background that enables code elements to stand out. Light Includes a clean and uncomplicated color palette that is easy to use.
Font	Specifies the font name and font size for the code.
Editor background	Specifies the background color for the code. This setting is disabled unless you select a theme for the Syntax theme setting.

Settings for Syntax Highlighting

You can specify highlighting options only if you have selected a theme in the **Syntax theme** field and selected the **Enable syntax highlighting** check box. The **Preview** section at the bottom of the Code Editor Settings window shows you how your current highlighting selections will look in the code editor. Your selections take effect after you close the Code Editor Settings window.

Options	Description
Enable syntax highlighting	Enables you to specify the text style and background color of specific file elements in the code editor. You can select different style and color attributes for different elements in different coding languages. This option is selected by default.
Language	Specifies the language to which the selections in the Element and Text style field apply. You can select SAS , Python , or XML .
Element	Specifies the text element to which the selections in the Text style field apply. You can select various code elements such as Normal text , Comments , Brackets , and Numeric constants .
Text style	Specifies the style and color of the element that is selected in the Element field. Click B to select bold text, I to select italics, and U to underline the text. Click  to change the text color.

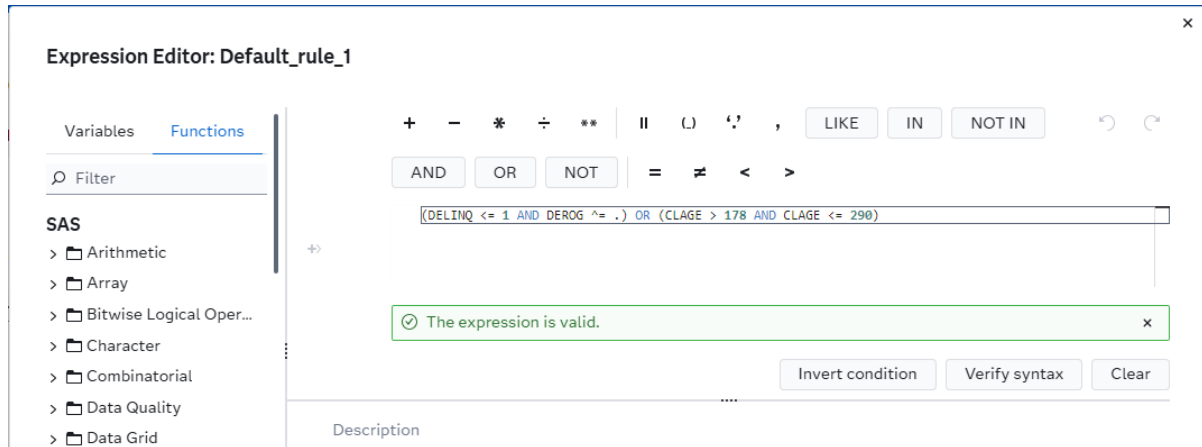
Using the Expression Editor

The expression editor is available for editing rule expressions in rule sets, for editing branch expressions in decisions, and for editing Boolean expressions in segmentation trees. To open the expression editor, save any changes that you have made to the object, and click  for the expression that you want to edit.

For expression examples and information about the functions and operators that are available in the expression editor, see [“Defining Expressions” on page 56](#).

You must use the expression editor to enter expressions that use the concatenation (||) operator or the exponent operator (**). In a rule set, you can use the expression editor to enter expressions that do not use the LOOKUP or LOOKUPVALUE functions. Y

Note: When you use the expression editor to enter an expression in a rule set, the expression field becomes a read-only field. You cannot enter changes to the expression directly in expression field. You must use the expression editor to additional changes to the expression. To reset the expression field back to an editable field, click . However, if you reset the expression field, all work in the expression field is lost.



You can enter expressions directly into the expression field, or you can use the lists of operators, function names, and variable names to add them to the expression.

- To add a variable, click the **Variables** tab, and double-click the variable name.
- To add a function call, click the **Functions** tab, expand the appropriate function category, select a function name, and click . The expression editor adds a basic syntax template for the function to the expression field. Hover over the function name to display a brief description of the function, its syntax, and descriptions of each of its arguments.

The icon beside each function name indicates the return type of the function. Functions that return character data are displayed with the  icon, and functions that return numeric data are displayed with the  icon.

- To invert an entire condition expression, click **Invert Condition**. SAS Intelligent Decisioning wraps the entire expression in parentheses and adds the keyword NOT:

`NOT(condition)`

TIP The **Invert Condition** button is a toggle. If the entire condition is already enclosed in NOT(), clicking this button removes the keyword NOT and the parentheses surrounding the expression. This button does not affect NOT conditions that are embedded within the expression. (This button is not available action expressions in rule sets or for variable initial values.)

- To save the expression, click **Save**.

IMPORTANT After you save a rule set expression in the expression editor, the expression field on the **Rule Set** tab becomes a read-only field. You can edit it only by launching the expression editor again.

- Click **Verify syntax** to check the syntax of the expression that you are building.

.....
Note: If you have made any changes to the object's variables, save the object before you open the expression editor and click **Verify syntax**. Also, if you invert the condition and click **Verify syntax**, the validation process might return incorrect results.
.....

- To clear the expression field, click **Clear**.

TIP For rule set expressions, when SAS Intelligent Decisioning generates code for the rule set, it adds a semicolon (;) to the end of each line that you enter in the expression editor, except for lines that end with an opening parenthesis, a closing parenthesis, a comma (,), or a semicolon (;).

TIP

In the autocomplete list, SAS Intelligent Decisioning displays all function names and variable names that match the text that you enter. Double-click an item to add it to the code field. Single-click an item to display information about the item, such as whether it is a function or a variable. For functions, the information includes the returned data type and category. For variables, the information includes the data type and direction (input, output, or both).

The autocomplete list does not include functions that are defined in a custom context file.

Appendix 1

Querying the Subject Contact Database

<i>About the Subject Contact Database</i>	539
<i>How to Query the Subject Contacts Database</i>	540
<i>Using Views Defined by SAS Intelligent Decisioning</i>	541
<i>Columns in the dcm_trt_fixed_attr_vw and dcm_trt_dynamic_attr_vw Views</i>	542
<i>Columns in the dcm_treatments_contacts_vw View</i>	545
Examples:	549
Determine Whether Any Treatments Were Sent to a Subject	549
Determine Whether a Treatment was Sent to a Subject within a Specific Time Frame	549
Determine Whether a Treatment with a Specific Attribute Value Was Sent to a Subject	549
Determine Whether a Treatment Was Presented to a Subject within a Specified Time Frame	550
Determine Whether a Subject Responded to a Treatment within a Specified Time Frame	550
Determine Whether a Subject Responded to a Treatment with a Specific Attribute Value within a Specified Time Frame	551
Determine Whether a Subject Responded to Any Treatment within a Specified Time Frame	551

About the Subject Contact Database

The subject contact database contains the records that are written by record contacts nodes when a decision is run. In SAS Micro Analytic Service destinations, each record contacts node writes a record to the subject contact database. In all other destinations, these nodes create output variables that contain the contact

information in JSON syntax. You can use these variables and the SAS Decision Management REST API to update the subject contact database. For example, your enterprise might have a batch job that runs at specific times and updates the subject contact database.

When you add a record contacts node to a decision, you specify the data grid variable and the decision variables that you want to track in the subject contacts record. When the decision contains a treatment, you usually specify the output data grid that is produced by the treatment as the data grid to track. This data grid contains information about the treatments for which the subject qualifies. You usually also track the decision variables that are mapped to dynamic attributes. The record contacts node also generates a response tracking code that you can use in a REST API request to update the subject contact database with information about the subject's response to the treatments.

For more information about treatments, attributes, and record contacts nodes, see [Chapter 4, "Working with Treatments and Treatment Groups," on page 111](#) and ["Adding Record Contacts Nodes" on page 410](#). Information about the Subject Contacts REST API is available at <https://developer.sas.com>.

You can query the subject contact records and treatment definitions to determine which treatments a subject qualifies for, whether a treatment has been presented to a subject, which treatments a subject has responded to, and so on. SAS Intelligent Decisioning provides database views that you can use to query the database.

How to Query the Subject Contacts Database

If you are using the PostgreSQL instance that was installed with SAS Viya for your database, the recommended way to query the database is to use the SAS Decision Management REST API. Complete reference information for the Decision Management API is available at <http://developer.sas.com>. For examples of using the Subject Contacts API, see the [examples on GitHub](#). For additional information and examples, see [SAS Intelligent Decisioning: Decision Management REST API Examples](#) and [SAS Intelligent Decisioning: Using Custom Node Types](#).

If you are using a third-party database, you can use the database views that are provided by SAS Intelligent Decisioning. You can create a data query file in the SQL editor that uses these views and include the data query file in a decision. For more information, see ["Using Views Defined by SAS Intelligent Decisioning" on page 541](#) and ["Data Query Files" on page 192](#).

Using Views Defined by SAS Intelligent Decisioning

SAS Intelligent Decisioning defines views that you can use to query the subject contacts records and treatment definitions. These views are created automatically when the subject contacts service and the decision-treatment service are started.

SAS Intelligent Decisioning defines the following three views:

`dcm_trt_fixed_attr_vw`

contains fixed treatment attributes for treatments that are members of a treatment definition group. This view enables you to retrieve the list of fixed attributes for a specific version of a treatment group. Treatments can be used in multiple treatment groups, so the table might contain multiple records for the same attribute. This view includes the values that were assigned to the attributes. The attribute values for the same treatment version are the same, but the attribute values might differ between treatment versions.

`dcm_trt_dynamic_attr_vw`

contains dynamic treatment attributes for treatments that are members of a treatment definition group. This view enables you to retrieve the list of dynamic attributes for a specific version of a treatment group. Treatments can be used in multiple treatment groups, so the table might contain multiple records for the same attribute.

These attributes can be assigned either a static value in the treatment group, or they can be mapped to a decision variable and receive their value from the variable when the decision executes.

For attributes that are assigned a static value in a group, this view includes the values that were assigned to them. The values of these attributes might be different for each treatment group in which the treatment that defines the attribute appears.

For dynamic attributes that are mapped to a decision variable, the attribute values are not present in this view. To trace the values of these attributes, the decision variables must be tracked by the record contacts node. For more information, see [“Adding Record Contacts Nodes” on page 410](#).

`dcm_treatment_contacts_vw`

contains detailed contact information for each treatment that was sent to a subject. This table enables you to determine which treatments were offered to a subject either for a specific response (identified by the response tracking code) or for the entire set of history records that are retained in the database. The number of treatments associated with a specific response tracking code or subject ID depends on how many treatments were returned by the decisions that created the contact records.

Note: To query both treatment definitions and subject contact records, you must join an attribute view with a treatment view. To join the views, both the subject contacts records and the treatment definitions must reside in the same database.

These views assume that the input and output values of the treatment groups are not modified before they are recorded by a record contacts node.

Columns in the dcm_trt_fixed_attr_vw and dcm_trt_dynamic_attr_vw Views

Both attribute tables contain the same columns. The dcm_trt_fixed_attr_vw view contains information about fixed attributes, and the dcm_trt_dynamic_attr_vw view contains information about dynamic attributes. For information about treatment attributes, see [“Attributes and Attribute Aliases” on page 116](#).

The following table describes each column in these views and lists the associated schema property in the Decision Management REST API.

Column	Description	Schema Property
attribute_nm	Name of the attribute.	name in the Treatment Definition Attribute object of the Treatment Definition Collection. For the details, see the Treatment Definitions API .
attribute_val_fixed_flg	A flag that indicates whether the attribute is fixed or dynamic. For dynamic attributes, this column is set to <code>false</code> . For fixed attributes, this column is set to <code>true</code> .	readOnly in the Treatment Definition Attribute object of the Treatment Definition Collection. For the details, see the Treatment Definitions API .
mapping_type_cd	A code that indicates whether dynamic attributes get their value from the decision flow or are set to a static value. For dynamic attributes, this column is set to <code>constant</code> if the attribute is assigned a static value in the treatment group. This column is set to <code>variable</code> if the attribute gets its value from the decision flow. For fixed attributes, this column is set to <code>null</code> .	mappingType in the Treatment Definition Group Attribute Value Mapping object of the Treatment Definition Collection. For the details, see the Treatment Definitions API .

Column	Description	Schema Property
attribute_default_val	The default value that was set for the attribute when the attribute was defined in the treatment. For dynamic variables, this value might be overridden if the attribute is assigned a static value in the treatment group or if the attribute gets its value from the decision flow. For fixed attributes, this value is the value that is used in the decision flow.	defaultValue in the Treatment Definition Attribute object of the Treatment Definition Collection. For the details, see the Treatment Definitions API.
attribute_val	The actual value or the variable mapping for dynamic attributes. For dynamic attributes that get their value from the decision flow, this column is set to the name of the variable to which the attribute is mapped in the decision flow. For dynamic attributes that are assigned a static value in the treatment group, this column is set to the static value. For fixed attributes, this column is set to the name of the variable to which the attribute is mapped in the decision flow.	value in the Treatment Definition Group Attribute Value Mapping object of the Treatment Definition Collection. For the details, see the Treatment Definitions API.
treatment_nm	Name of the treatment definition that contains the attribute.	name in the Treatment Definition object of the Treatment Definition Collection. For the details, see the Treatment Definitions API.
treatment_id	ID of the treatment definition.	id in the Treatment Definition object of the Treatment Definition Collection. For the details, see the Treatment Definitions API.
treatment_revision_id	ID of the treatment definition version.	definitionRevisionId in the Treatment Definition Group Member object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.
treatment_version_no	Version number of the treatment definition.	definitionRevisionName in the Treatment Definition Group Member object of the Treatment Definition Group Collection. For

Column	Description	Schema Property
		example, treatment_version_no could be 2.14, where 2 is the treatment definition major revision number and 14 is the treatment definition minor version number. For the details, see the Treatment Definitions API.
treatment_grp_nm	Name of the treatment definition group that contains the treatment.	name in the Treatment Definition Group object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.
treatment_grp_id	ID of the treatment definition group.	id in the Treatment Definition Group object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.
treatment_grp_revision_id	Version ID of the treatment definition group.	definitionRevisionId in the Treatment Definition Group Member object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.
treatment_grp_version_major_no	Major version number of the treatment definition group. If the treatment definition group is 2.5, then this column is set to 2.	majorRevision in the Treatment Definition Group object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.
treatment_grp_version_minor_no	Minor version number of the treatment definition group. If the treatment definition group is 2.5, then this column is set to 5.	minorRevision in the Treatment Definition Group object of the Treatment Definition Group Collection. For the details, see the Treatment Definitions API.

Columns in the dcm_treatments_contacts_vw View

The dcm_treatments_contacts_vw view contains detailed information about subject contact records. The following table describes each column in the view and lists the associated schema property in the Decision Management REST API.

Column	Description	Schema Property
contact_id	A unique identifier for the contact record. A contact record with a new contact_id is created when a decision determines that a subject is eligible for a specific treatment and writes the contact record to the database.	id in the contact record. For the details, see the Subject Contacts API .
subject_id	The ID of the subject if a subject ID variable is defined in the properties for the decision flow.	subjectId in the contact record. For the details, see the Subject Contacts API .
subject_level	The subject level if a subject level variable is defined in the decision flow. For more information, see “Predefined Lookup Tables” on page 146 .	subjectLevel in the contact record. For the details, see the Subject Contacts API .
created_by_nm	The ID of the user who ran the decision that created contact record.	createdBy in the contact record. For the details, see the Subject Contacts API .
modified_by_nm	The ID of the user who last modified the contact record. For example, subject contact records can be modified to include a recording of the treatment presentation or to include information about the subject's response to the treatment.	modifiedBy in the contact record. For the details, see the Subject Contacts API .
creation_dttm	A timestamp that indicates when the contact record was created.	creationTimeStamp in the contact record. For the details, see the Subject Contacts API .
modified_dttm	A timestamp that indicates when the contact record was last modified.	modifiedTimeStamp in the contact record. For the details, see the Subject Contacts API .

Column	Description	Schema Property
contact_channel_txt	The channel through which the contact occurred. For more information, see “About Channels” on page 118 .	channel in the contact record. For the details, see the Subject Contacts API .
response_tracking_cd	The response tracking code that was generated by the record contacts node.	responseTrackingCode in the contact record. For the details, see the Subject Contacts API .
conclusion_response_txt	The subject’s response to the results of the decision. When the decision includes treatments, this value might represent an overall response to all of the treatments in the subject contact record.	conclusionResponseValue in the contact record. For the details, see the Subject Contacts API .
conclusion_response_type_txt	A summary value, category name, or other description for the response that is specified in the conclusion_response_txt column. For example, this value might be Evaluating, No response, or Closed-Lost.	conclusionResponseType in the contact record. For the details, see the Subject Contacts API .
object_type_cd	A string that identifies the type of resource that created the contact record. Currently, the only value in this column is decision.	objectType in the contact record. For the details, see the Subject Contacts API .
object_revision_id	The ID for the version of the object that created the contact record.	objectRevisionId in the contact record. For the details, see the Subject Contacts API .
object_url_txt	The URI for the decision that created the contact record (for example, /decisions/flows/545783f1-4fb3-45aa-be7d-d9e5a13ed943).	objectUri in the contact record. For the details, see the Subject Contacts API .
exclude_from_contact_rule_txt	Specifies whether the contact record is included in aggregate reports for the channel. Column values are true or false.	excludeFromContactRule in the contact the record. For details, see the Subject Contacts API .
rule_fired_txt	Rule-fired information for the decision if that information was recorded by the record contacts node.	ruleFired in the contact record. For the details, see Subject Contacts API .
traversed_rule_path_list_txt	Path-tracking information for the decision if that information was recorded by the record contacts node.	pathTraversed in the contact record. For the details, see the Subject Contacts API .

Column	Description	Schema Property
treatment_receiver_id	A string that identifies the entity that received the treatment. If this entity is the subject, then the treatment_receiver_id column contains the same value as the subject_id column. Note: This column is present in the table, but it is not currently used.	receiverId in the contact record. For the details, see the Subject Contacts API.
treatment_receiver_role_txt	A string that identifies the role of the entity that received the treatment. For example, the receiver might be the subject or an agent. Note: This column is present in the table, but it is not currently used.	receiverRole in the contact record. For the details, see the Subject Contacts API.
treatment_revision_id	The ID of the treatment definition version.	treatmentRevisionId in the treatmentsForConsideration table. For the details, see the Subject Contacts API.
treatment_id	The ID of the treatment definition.	treatmentId in the treatmentsForConsideration table. For the details, see the Subject Contacts API.
presented_txt	Specifies whether the treatment was presented to the subject. The values in this column are a character string representation of a Boolean value: true or false.	presented in the treatmentsForConsideration table. For the details, see the Subject Contacts API.
presentation_dttm	A timestamp that indicates when the treatment was presented to the subject. The timestamp format is Coordinated Universal Time (UTC): <i>yyyy-mm-ddThh:mm:ss.sssZ</i>	presentedTimeStamp in the treatmentsForConsideration table. For the details, see the Subject Contacts API.
response_dttm	A UTC timestamp that indicates when the subject responded to the treatment.	respondedTimeStamp in the treatmentsForConsideration table. For the details, see the Subject Contacts API.
response_txt	The subject's response to the treatment. For example, this value might be a value or code such	responseValue in the treatmentsForConsideration table. For the details,

Column	Description	Schema Property
	as Purchased, Rejected, Requested quote, Email opened, or rtl.	see the Subject Contacts API .
response_type_txt	A summary value, category name, or other description for the response that is specified by the value of the response_txt column. For example, this value might be Positive, Negative, or Neutral.	responseType in the treatmentsForConsideration table. For the details, see the Subject Contacts API .
response_channel_txt	The channel through which the user responded to the treatment. The response channel might be different from the contact channel. For more information, see “About Channels” on page 118 .	responseChannel in the treatmentsForConsideration table. For the details, see the Subject Contacts API .
treatment_group_id	The ID of the treatment group that contains the treatment.	treatmentGroupId in the treatmentsForConsideration table. For the details, see the Subject Contacts API .
consider_id	The primary key of the dcm_treatment_consideration table.	An internal ID for each treatment that was sent to a subject. This ID is unique within the view. This column is in not in the schema.
treatment_group_revision_id	The ID of the treatment definition group version.	treatmentGroupRevisionId in the treatmentsForConsideration table. For the details, see the Subject Contacts API .
object_node_id	The ID of the node in the decision whose URL is specified in the object_url_txt column. For example, if the same treatment group is included in a decision multiple times, this ID enables you to determine which node generated an offer.	objectNodeId in the treatmentsForConsideration table. For the details, see the Subject Contacts API .

Examples

Example 1: Determine Whether Any Treatments Were Sent to a Subject

The following query determines whether any treatments were sent to the subject with the ID 98.

```
select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw
where subject_id='98';
```

Example 2: Determine Whether a Treatment was Sent to a Subject within a Specific Time Frame

The following query determines whether the specified treatment (the treatment with the ID fcdfabaf-9932-430e-8cf3-149b212881d7) was sent to the subject with the ID 98 between March 1, 2022 and March 31, 2022.

```
select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw
where creation_dttm >='2022-03-01 00:00:00.000'
and creation_dttm <= '2022-03-31 23:59:59.999'
and subject_id='98'
and treatment_ID = 'fcdfabaf-9932-430e-8cf3-149b212881d7';
```

Example 3: Determine Whether a Treatment with a Specific Attribute Value Was Sent to a Subject

The following query determines whether the specified treatment was sent to subject 98 at a price of 50.00 euros between January 1, 2022 and March 31, 2022. The price is a dynamic attribute that has been assigned a static value in the treatment group, so the query joins the dcm_treatment_contacts_vw view and the dcm_trt_dynamic_attr_vw view.

```

select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw as sub,
treatmentdefinitions.dcm_trt_dynamic_attr_vw as treat
where sub.treatment_id = treat.treatment_id
and sub.subject_id='98' and sub.creation_dttm >='2022-01-01
00:00:00.000'

and sub.creation_dttm <= '2022-03-31 23:59:59.999'
and treat.treatment_ID = 'fcdfabaf-9932-430e-8cf3-149b212881d7'
and treat.attribute_nm='price' and treat.attribute_val='50.00';

```

Example 4: Determine Whether a Treatment Was Presented to a Subject within a Specified Time Frame

The presence of a timestamp in the presentation_dttm column indicates that the treatment was presented to the subject. The following query determines whether the specified treatment was presented to subject 98 between May 15, 2022 and June 30, 2022.

```

select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw
where creation_dttm >='2022-05-15 00:00:00.000'
and creation_dttm <= '2022-06-30 23:59:59.999'
and subject_id='98'
and treatment_ID = 'fcdfabaf-9932-430e-8cf3-149b212881d7'
and presentation_dttm is not null;

```

Example 5: Determine Whether a Subject Responded to a Treatment within a Specified Time Frame

The presence of a timestamp in the response_dttm column indicates that the subject responded to the treatment. The following query determines whether subject 98 responded to the specified treatment between May 15, 2022 and June 30, 2022.

```

select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw
where creation_dttm >='2022-05-15 00:00:00.000'
and creation_dttm <= '2022-06-30 23:59:59.999'
and subject_id='98'

```

```
and treatment_ID = 'fcdfabaf-9932-430e-8cf3-149b212881d7'
and response_dttm is not null;
```

Example 6: Determine Whether a Subject Responded to a Treatment with a Specific Attribute Value within a Specified Time Frame

The following query determines whether subject 98 responded to the specified treatment at a price of 100.00 euros between January 1, 2022 and March 15, 2022. The price is a dynamic attribute that has been assigned a static value in the treatment group, so the query joins the dcm_treatment_contacts_vs view and the dcm_trt_dynamic_attr_vw view.

```
select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw sub,
treatmentdefinitions.dcm_trt_dynamic_attr_vw treat
where creation_dttm >='2022-01-01 00:00:00.000'
and creation_dttm <= '2022-03-15 23:59:59.999'
and subject_id='98'
and treat.treatment_ID = 'fcdfabaf-9932-430e-8cf3-149b212881d7'
and attribute_nm='price' and attribute_val='100.00'
and response_dttm is not null;
```

Example 7: Determine Whether a Subject Responded to Any Treatment within a Specified Time Frame

The following query determines whether subject 98 responded to any treatment in the first six months of 2022.

```
select count(distinct contact_id)

from subjectcontacts.dcm_treatment_contacts_vw
where creation_dttm >='2022-01-01 00:00:00.000'
and creation_dttm <= '2022-06-30 23:59:59.999'
and subject_id='98' and response_dttm is not null;
```


Appendix 2

Import File Formats

<i>Introduction</i>	553
<i>CSV Format of the Variable Import File</i>	553
<i>JSON Format of the Variable Import File</i>	556

Introduction

You can import variables from either comma-delimited (CSV) files or JavaScript Object Notation (JSON) files. These files must conform to formats described in [“CSV Format of the Variable Import File” on page 553](#) and [“JSON Format of the Variable Import File” on page 556](#). For more information, see [“Importing and Exporting Variables” on page 402](#).

CSV Format of the Variable Import File

Each row of the CSV input file contains the name of either a variable or a data grid column (if one of the variables is a data grid). The first line of the input file must be a header row. The CSV file must contain all of the columns listed in the following table, in the order listed. The value for some columns can be blank, as noted in the table. To create a blank column for a numeric variable in the CSV file, specify two comma separators without any content between them. To create a blank column for a character variable, specify an empty string (" ").

Table A3.1 CSV Format of the Variable Import File

CSV Column	Description
id	The ID of the variable specified in the name column.
name	The name of a variable or data grid column. If one of the rule set or decision variables is a data grid variable, then specify the name of a column within the data grid.
dataGridName	If the name column contains the name of a data grid column, then the dataGridName column contains the name of the variable that contains the data grid.
dataType	The data type of the variable. You can specify <i>decimal</i> , <i>integer</i> , <i>date</i> , <i>datetime</i> , <i>boolean</i> , or <i>datagrid</i> . For character variables, specify <i>string</i> .
direction	Specifies whether the variable is an input-only variable (<i>input</i>), an output-only variable (<i>output</i>), both (<i>inOut</i>). Do not specify a direction for data grid columns. If you do not specify a direction for a rule set or decision variable, the direction is imported as <i>inOut</i> . To designate a variable as a temporary variable, edit the variable properties on the Variables tab.
length	The length of the variable. Specify lengths for character variables only (strings or data grids).
generateDataGrid Columns	Specifies whether the JSON character strings for each row in the input data grid (named in the dataGridName column above) contains metadata that defines the name and data type for each column in the row. Specify TRUE or FALSE. This column in the CSV file corresponds to the Data does not contain column metadata check box in the Edit Columns window in the user interface. For more information, see “Add Columns to a Data Grid” in SAS Intelligent Decisioning: Using Data Grids.
description	A description of the variable. Descriptions are limited to 256 characters.
defaultValue	An initial (default) value for the variable.
dataGridMaxRowC ount	The maximum number of rows that are allowed in each column of a data grid when you use an input table with special column names to test a decision. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in SAS Intelligent Decisioning: Using Data Grids .
autoPopulate	Specifies whether SAS Intelligent Decisioning creates newly added data grid columns at run time. When you add a new column to the metadata for a data grid, you can choose whether the column is automatically created at run time. At run time, if a column already exists in the input data grid, SAS Intelligent Decisioning does not add a new column or overwrite the existing column. Specify TRUE or FALSE .

CSV Column	Description
	This column in the CSV file corresponds to the Create Column check box in the Edit Columns window in the user interface. For more information, see “Add Columns to a Data Grid” in SAS Intelligent Decisioning: Using Data Grids .
valueListId	The ID of the value list that is associated with the variable.
valueListContentId	The ID of the value list version.
valueListName	The name of the value list that is associated with the variable.

The following example specifies ten variables. The variable card_offers_out is a data grid. The variables annual_fee and card_type are columns in the data grid. The variable score is associated with the value list named cscore_levels. The variable i is a temporary variable that has an initial value of zero.

Note: In this example, IDs are abbreviated as *ID*. Exported IDs appear similar to 0d625b40-344b-4aed-969e-996ca67b07e1.

```
id,name,dataGridName,dataType,direction,length,generateDataGridColumns,description,default
tValue,dataGridMaxRowCount,autoPopulate,valueListId,valueListContentId,valueListName
ID,ACCOUNT,,string,inOut,13,FALSE,,,,,
ID,ACCTSCORE,,decimal,inOut,,FALSE,,,,,
ID,card_offers_out,,dataGrid,output,,FALSE,,,,,
,annual_fee,card_offers_out,decimal,,,,,FALSE,,,
,card_type,card_offers_out,string,,,,,FALSE,,,
ID,DELINQ,,decimal,inOut,,FALSE,,,,,
ID,REASON,,string,inOut,7,FALSE,,,,,
ID,zero_offers,,boolean,output,,FALSE,,,,,
ID,cscore,,decimal,inOut,,FALSE,,,,ID,ID,cscore_levels
ID,i,,decimal,inOut,,false,"temporary",0,,
```

In a spreadsheet application, this CSV file appears as shown in the following figure.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	id	name	dataGridName	dataType	direction	length	generateDataGridColumns	description	defaultVa	dataGridM	autoPopu	valueListI	valueListC	valueListName
2	0d625b40-344b-4	ACCOUNT		string	inOut	13	FALSE							
3	320dbe8a-d606-4	ACCTSCORE		decimal	inOut		FALSE							
4	377b0835-a70c-4	card_offers_out		dataGrid	output		FALSE							
5		annual_fee	card_offers_out	decimal							FALSE			
6		card_type	card_offers_out	string							FALSE			
7	f5c9807a-23c8-45	DELINQ		decimal	inOut		FALSE							
8	a13970cb-061a-4	REASON		string	inOut	7	FALSE							
9	090cc654-7ef6-42	zero_offers		boolean	output		FALSE							
10	d6a4f1e4-c56f-43	cscore		decimal	inOut		FALSE					ed9029d2-b954aaf4-	cscore_levels	
11	c50ccbd0-55ea-4	i		decimal	inOut		FALSE	temporary	0					

Note: Set the encoding value of the spreadsheet to the same value as the CSV file.

JSON Format of the Variable Import File

The JSON file contains a set of name-value pairs for each variable. The following table lists the fields that can be included for each variable.

Table A3.2 Fields in the JSON Variable Input File

Field	Description
id	The ID of the variable specified in the name column.
name	The name of a variable or data grid column. If one of the variables is a data grid variable, then specify the name of a column within the data grid.
dataType	The data type of the variable. You can specify <code>decimal</code> , <code>integer</code> , <code>date</code> , <code>datetime</code> , or <code>datagrid</code> . For Boolean variables, specify <code>integer</code> . For character variables, specify <code>string</code> .
direction	Whether the variable is an input-only variable (<code>input</code>), an output-only variable (<code>output</code>), both (<code>inOut</code>). Do not specify a direction for data grid columns. If you do not specify a direction for a rule set or decision variable, the direction is imported as <code>inOut</code> . To designate a variable as a temporary variable, edit the variable properties on the Variables tab.
description	A description of the variable. Descriptions are limited to 256 characters.
length	The length of the variable. Specify lengths for character variables only (strings or data grids).
dataGridExtension	The columns in the data grid, if the variable specified in the name field is a data grid. Specify the name and data type for each column in the data grid. If the column is a character column, also specify the length.
defaultValue	An initial (default) value for the variable.
generateDataGrid Columns	Specifies whether the JSON character strings for each row in the input data grid (named in the <code>dataGridName</code> column above) contains metadata that defines the name and data type for each column in the row. Specify TRUE or FALSE. This column in the CSV file corresponds to the Data does not contain column metadata check box in the Edit Columns window in the user interface. For more information, see “Add Columns to a Data Grid” in SAS Intelligent Decisioning: Using Data Grids.

Field	Description
dataGridMaxRowCount	Specifies the maximum number of rows that are allowed in each column of a data grid when you use an input table with special column names to test a decision. For more information, see “Testing Decisions by Using Array Syntax for Data Grid Column Names” in <i>SAS Intelligent Decisioning: Using Data Grids</i> .
autoPopulate	Specifies whether SAS Intelligent Decisioning creates newly added data grid columns at run time. When you add a new column to the metadata for a data grid, you can choose whether the column is automatically created at run time. At run time, if a column already exists in the input data grid, SAS Intelligent Decisioning does not add a new column or overwrite the existing column. Specify TRUE or FALSE. This column in the CSV file corresponds to the Create Column check box in the Edit Columns window in the user interface. For more information, see “Add Columns to a Data Grid” in <i>SAS Intelligent Decisioning: Using Data Grids</i>.
valueListId	The ID of the value list that is associated with the variable.
valueListContentId	The ID of the value list version.
valueListName	The name of the value list that is associated with the variable.

The following example specifies ten variables. The variable `card_offers_out` is a data grid. The variables `annual_fee` and `card_type` are columns in the data grid. The variable `score` is associated with the value list named `cscore_levels`. The variable `i` is a temporary variable that has an initial value of zero.

```
[
  {
    "creationTimeStamp": "2024-12-10T06:21:54.158Z",
    "modifiedTimeStamp": "2024-12-10T06:21:54.158Z",
    "createdBy": "edmdev",
    "modifiedBy": "edmdev",
    "id": "0d625b40-344b-4aed-969e-996ca67b07e1",
    "direction": "inOut",
    "name": "ACCOUNT",
    "dataType": "string",
    "length": 13,
    "generateDataGridColumns": false,
    "legacy": false
  },
  {
    "creationTimeStamp": "2024-12-10T06:21:54.160Z",
    "modifiedTimeStamp": "2024-12-10T06:21:54.160Z",
    "createdBy": "edmdev",
    "modifiedBy": "edmdev",
    "id": "320dbe8a-d606-4c98-bb55-d0a4dec80096",
    "direction": "inOut",
    "name": "ACCTSCORE",
    "dataType": "decimal",
```

```

        "generateDataGridColumns": false,
        "legacy": false
    },
    {
        "creationTimeStamp": "2024-12-10T06:21:54.162Z",
        "modifiedTimeStamp": "2024-12-10T06:21:54.162Z",
        "createdBy": "edmdev",
        "modifiedBy": "edmdev",
        "id": "377b0835-a70c-4c0f-8d51-d6f0e4412440",
        "direction": "output",
        "name": "card_offers_out",
        "dataType": "dataGrid",
        "dataGridExtension": [
            {
                "id": "a80fd79f-6296-47be-9f40-e9c86de55ac9",
                "name": "annual_fee",
                "dataType": "decimal",
                "autoPopulate": false
            },
            {
                "id": "aafa84c0-9593-4255-ba0f-f425202a69d2",
                "name": "card_type",
                "dataType": "string",
                "autoPopulate": false
            }
        ],
        "generateDataGridColumns": false,
        "legacy": false
    },
    {
        "creationTimeStamp": "2024-12-10T06:21:54.160Z",
        "modifiedTimeStamp": "2024-12-10T06:21:54.160Z",
        "createdBy": "edmdev",
        "modifiedBy": "edmdev",
        "id": "f5c9807a-23c8-45b8-9c5a-32ca41c6d76a",
        "direction": "inOut",
        "name": "DELINQ",
        "dataType": "decimal",
        "generateDataGridColumns": false,
        "legacy": false
    },
    {
        "creationTimeStamp": "2024-12-10T06:21:54.161Z",
        "modifiedTimeStamp": "2024-12-10T06:21:54.161Z",
        "createdBy": "edmdev",
        "modifiedBy": "edmdev",
        "id": "a13970cb-061a-4832-92e7-6ba148948f38",
        "direction": "inOut",
        "name": "REASON",
        "dataType": "string",
        "length": 7,
        "generateDataGridColumns": false,
        "legacy": false
    },
    {
        "creationTimeStamp": "2024-12-10T06:21:54.162Z",

```

```

    "modifiedTimeStamp": "2024-12-10T06:21:54.162Z",
    "createdBy": "edmdev",
    "modifiedBy": "edmdev",
    "id": "090cc654-7ef6-42a7-a853-36e16e69e2b5",
    "direction": "output",
    "name": "zero_offers",
    "dataType": "boolean",
    "generateDataGridColumns": false,
    "legacy": false
  },
  {
    "id": "d6a4f1e4-c56f-43a5-acdf-776b6f64ab80",
    "name": "cscore",
    "description": "",
    "dataType": "decimal",
    "direction": "inOut",
    "valueListId": "ed9029d2-13bb-42a4-b568-4ff81940414f",
    "valueListContentId": "b954aaf4-da52-42fb-86a7-193d48302903",
    "valueListName": "cscore_levels"
  },
  {
    "id": "c50ccbd0-55ea-41ee-ba99-2a7e4efa39ba",
    "name": "i",
    "direction": "inOut",
    "dataType": "decimal",
    "description": "temporary",
    "defaultValue": 0,
    "generateDataGridColumns": false
  }
]

```

